



GMD (GOWIN MCU Designer) Software

User Guide

SUG549-2.0E, 2025-07-04

Copyright © 2025 Guangdong Gowin Semiconductor Corporation. All Rights Reserved.

GOWIN, GOWIN, and LittleBee are trademarks of Guangdong Gowin Semiconductor Corporation and are registered in China, the U.S. Patent and Trademark Office, and other countries. All other words and logos identified as trademarks or service marks are the property of their respective holders. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior written consent of GOWINSEMI.

Disclaimer

GOWINSEMI assumes no liability and provides no warranty (either expressed or implied) and is not responsible for any damage incurred to your hardware, software, data, or property resulting from usage of the materials or intellectual property except as outlined in the GOWINSEMI Terms and Conditions of Sale. GOWINSEMI may make changes to this document at any time without prior notice. Anyone relying on this documentation should contact GOWINSEMI for the current documentation and errata.

Revision History

Date	Version	Description
07/29/2019	1.0E	Initial version published.
09/12/2019	1.1E	● Integrated GNU ARM and RISC-V MCU compilation tool chain. ● Updated the Eclipse to the latest version at website. ● Updated the GNU ARM MCU compilation tool chain to the latest version at website. ● Updated and optimized Interface Configuration.
11/12/2019	1.2E	● Updated the GNU RISC-V MCU compilation tool chain to the latest version at GNU website. ● IDE integrated with Java Development Kit and IDE installation flow simplified. ● IDE integrated with Gowin Programmer. ● Gowin Customized IDE interface and simplified options to improve the ease of use. ● Known issues of IDE installation and usage fixed.
12/10/2019	1.3E	● License Management. ● IDE Interface optimized.
06/01/2020	1.4E	● Gowin_EMPU (GW1NS-4C) and Gowin_EMPU_M3 programming design, compilation, downloading and online debugging supported. ● Gowin_PicoRV32 software online debugging supported. ● The debugging software OpenOCD integrated. ● Olimex debugging emulator driver software integrated. ● The download tool called Programmer updated and download of Gowin_EMPU (GW1NS-4C) supported. ● Gowin_EMPU (GW1NS-2C), Gowin_EMPU_M1, Gowin_EMPU (GW1NS-4C), Gowin_EMPU_M3, and Gowin_PicoRV32 software programming reference design example updated. ● Help system and online viewing of GMD and Gowin MCU user manuals supported.
10/28/2022	1.4.1E	Gowin_EMPU(GW1NS-2C) removed.
04/20/2023	1.5E	● GW5AT-138/GW5AST-138 Version B added. ● Help manuals updated. ● Programmer updated. ● Installation path and software toolchain configurations added. ● MCU software development library, application cases, and user manuals added. ● RTOS application cases added.

Date	Version	Description
07/04/2025	2.0E	● The base version of Eclipse updated to 2024-12, which is developed based on Eclipse CDT 2024-12. ● The base version of the ARM GCC/GDB Toolchain updated to 13.3.1-1.1. ● The base version of the RISC-V GCC/GDB Toolchain updated to 14.2.0-3. ● The base version of OpenOCD to updated 0.12.0-4. ● OpenOCD supports debuggers, such as GWU2X, GWUSB, and WinUSB JLink. ● OpenOCD supports MCU debugging, such as Gowin_EMPU_M1, Gowin_EMPU (GW1NS-4C), and Gowin_PicoRV32. ● The base version of Windows build tools updated to 4.4.1-2. ● ARM QEMU tool integrated (base version 8.2.6-1). ● RISC-V QEMU tool integrated (base version 8.2.6-1). ● ARM QEMU supports emulation & debugging for Gowin_EMPU (GW1NS-4C) MCU. ● RISC-V QEMU supports emulation & debugging for Gowin_PicoRV32 MCU. ● The Gprof software integrated to support Profiling. ● The Gcov software integrated to support the Code Coverage. ● Parse HexDump Data function supported, used in conjunction with Gprof and Gcov to collect and analyze runtime data. ● The base version of the Programmer software updated to 1.9.11.02. ● The GoBridgeDriver software integrated, supporting downloading and debugging based on the WinUSB drivers.

Contents

Contents	i
List of Figures	v
List of Tables	xi
1 About This Guide	1
1.1 Purpose	1
1.2 Related Documents	1
1.3 Terminology and Abbreviations	2
1.4 Support and Feedback	2
2 Overview	3
2.1 Reference Documents	4
2.2 MCU Support	4
3 Platform Support	5
3.1 Platform Support	5
3.2 Operating Environment	5
4 Download and Installation	6
4.1 Software Download	6
4.2 IDE Installation	6
4.3 Driver Installation	7
4.3.1 GWU2X	8
4.3.2 GWUSB V3.3	9
4.3.3 GWUSB V4.1	10
4.3.4 Olimex	11
4.3.5 J-Link WinUSB	11
4.3.6 Segger J-Link	12

5 User Interface.....	13
5.1 Title Bar.....	14
5.2 Menu Bar	14
5.2.1 File Menu Items	14
5.2.2 Edit Menu Items.....	19
5.2.3 Source Menu Items.....	21
5.2.4 Refactor Menu Items	22
5.2.5 Navigate Menu Items.....	22
5.2.6 Search Menu Items.....	25
5.2.7 Project Menu Items.....	25
5.2.8 Run Menu Items	26
5.2.9 Gowin Menu Items.....	29
5.2.10 Window Menu Items	30
5.2.11 Help Menu Items.....	33
5.3 Toolbar	34
5.4 Project Explorer	36
5.5 Code Editor Area	37
5.6 Console.....	38
5.6.1 Console View	38
5.6.2 Problems View	38
5.6.3 Terminal View.....	39
6 Create a Project	41
6.1 Create a New Project.....	41
6.1.1 Create a New Project (ARM MCU).....	42
6.1.2 Create a New Project (RISC-V MCU).....	44
6.2 Import an Existing Project.....	47
7 Import Projects Created in Legacy GMD Versions	50
7.1 Modify Imported ARM MCU Projects.....	50
7.2 Modify Imported RISC-V MCU Projects	52
8 Build Projects	56

8.1 Toolchains	56
8.2 Project Configuration	58
8.2.1 ARM MCU Project Configuration	60
8.2.2 RISC-V MCU Project Configuration.....	79
8.3 Build Projects.....	100
8.3.1 Build Methods	100
8.3.2 Build the Project.....	101
9 Download Projects	103
9.1 Downloading Tool	103
9.2 Downloading Methods	104
9.3 Download the Project.....	104
10 Debug and Run Projects	110
10.1 Debugging Tools	110
10.2 Notes	110
10.3 Debugging Modes.....	110
10.4 Debugging Methods.....	111
10.5 Debugging Settings	112
10.5.1 ARM MCU Debugging Setting	113
10.5.2 RISC-V MCU Debugging Setting.....	120
10.6 Run the Project	123
10.6.1 Launch Debug View.....	123
10.6.2 Debugging Options.....	124
10.7 Debug View	125
10.7.1 Memory View	126
10.7.2 Memory Browser View.....	133
10.7.3 Registers View	136
10.7.4 Variables View	137
10.7.5 Expressions View	137
10.7.6 Breakpoints View	138
10.7.7 Disassembly View.....	138

10.7.8 Console View	139
11 QEMU Emulation & Debugging.....	140
11.1 QEMU Tool.....	140
11.2 Use QEMU	140
11.2.1 ARM MCU QEMU Configuration.....	141
11.2.2 RISC-V MCU QEMU	145
11.3 Emulation Debugging.....	149
12 Code Coverage	150
12.1 About Code Coverage	150
12.2 Use Code Coverage	151
12.2.1 Create a Project.....	151
12.2.2 Set Relevant Options.....	151
12.2.3 Build the Project.....	155
12.2.4 Run the Project and Use Code Coverage	155
13 Profiling	163
13.1 About Profiling.....	163
13.2 Use Profiling	163
13.2.1 Create a Project.....	164
13.2.2 Set Relevant Options.....	164
13.2.3 Run the Project and Use Profiling	166

List of Figures

Figure 4-1 Select Gowin > GoBridgeDriver	8
Figure 4-2 GoBridgeDriver	8
Figure 4-3 Select Options > List All Devices	9
Figure 4-4 Install GWU2X Software Driver	9
Figure 4-5 Install GWUSB V3.3 Software Driver	10
Figure 4-6 Install GWUSB V4.1 Software Driver	10
Figure 4-7 Install Olimex Software Driver	11
Figure 4-8 Install J-Link WinUSB Software Driver	12
Figure 5-1 User Interface	13
Figure 5-2 New C/C++ Project	16
Figure 5-3 C Project	17
Figure 5-4 Import	18
Figure 5-5 Import Projects	19
Figure 5-6 Debug Configurations	28
Figure 5-7 Programmer	30
Figure 5-8 GoBridgeDriver	30
Figure 5-9 About GMD	34
Figure 5-10 Project Explorer	37
Figure 5-11 Code Editor Area	37
Figure 5-12 Console View	38
Figure 5-13 Problems View	39
Figure 5-14 Launch Terminal	39
Figure 5-15 Serial Terminal	40
Figure 5-16 Serial Terminal Output	40
Figure 6-1 Create a New Project	41

Figure 6-2 Select Project Type and Toolchain	42
Figure 6-3 Select Platform and Configuration.....	43
Figure 6-4 Select Software Toolchain and Path.....	44
Figure 6-5 Select Project Type and Toolchain	45
Figure 6-6 Select Platform and Configuration.....	46
Figure 6-7 Select Software Toolchain and Path.....	47
Figure 6-8 Select an Import Wizard	48
Figure 6-9 Select the Existing Project to Import	49
Figure 7-1 Build Error After Importing ARM Project.....	51
Figure 7-2 Toolchain Settings	52
Figure 7-3 Build Error After Importing RISC-V Project.....	53
Figure 7-4 RISC-V Toolchain Settings	54
Figure 7-5 Build Option Settings	55
Figure 8-1 ARM Toolchain.....	57
Figure 8-2 RISC-V Toolchain	58
Figure 8-3 ARM MCU Project Configuration.....	59
Figure 8-4 RISC-V MCU Project Configuration	60
Figure 8-5 Target Processor	61
Figure 8-6 Optimization.....	62
Figure 8-7 Warnings.....	64
Figure 8-8 Debugging	65
Figure 8-9 GNU ARM Cross Assembler > Preprocessor.....	66
Figure 8-10 GNU ARM Cross Assembler > Includes	67
Figure 8-11 GNU ARM Cross Assembler > Warnings	68
Figure 8-12 GNU ARM Cross Assembler > Miscellaneous	69
Figure 8-13 GNU ARM Cross C Compiler > Preprocessor.....	70
Figure 8-14 GNU ARM Cross C Compiler > Includes.....	71
Figure 8-15 GNU ARM Cross C Compiler > Optimization	72
Figure 8-16 GNU ARM Cross C Compiler > Warnings	73
Figure 8-17 GNU ARM Cross C Compiler > Miscellaneous	74

Figure 8-18 GNU ARM Cross C Linker > General	75
Figure 8-19 GNU ARM Cross C Linker > Libraries	76
Figure 8-20 GNU ARM Cross C Linker > Miscellaneous	77
Figure 8-21 GNU ARM Cross Create Flash Image.....	78
Figure 8-22 GNU ARM Cross Print Size	79
Figure 8-23 Target Processor	80
Figure 8-24 Optimization.....	81
Figure 8-25 Warnings.....	83
Figure 8-26 Debugging	84
Figure 8-27 GNU RISC-V Cross Assembler > Preprocessor	85
Figure 8-28 GNU RISC-V Cross Assembler > Includes	86
Figure 8-29 GNU RISC-V Cross Assembler > Warnings.....	87
Figure 8-30 GNU RISC-V Cross Assembler > Miscellaneous	88
Figure 8-31 GNU RISC-V Cross C Compiler > Preprocessor	89
Figure 8-32 GNU RISC-V Cross C Compiler > Includes	90
Figure 8-33 GNU RISC-V Cross C Compiler > Optimization.....	91
Figure 8-34 GNU RISC-V Cross C Compiler > Warnings.....	92
Figure 8-35 GNU RISC-V Cross C Compiler > Miscellaneous	93
Figure 8-36 GNU RISC-V Cross C Linker > General	94
Figure 8-37 GNU RISC-V Cross C Linker > Libraries	95
Figure 8-38 GNU RISC-V Cross C Linker > Miscellaneous	96
Figure 8-39 GNU RISC-V Cross Create Flash Image	97
Figure 8-40 GNU RISC-V Cross Create Listing.....	98
Figure 8-41 GNU RISC-V Cross Print Size	99
Figure 8-42 Build.....	100
Figure 8-43 ARM MCU Project Build Example	101
Figure 8-44 RISC-V MCU Project Build Example.....	102
Figure 9-1 Download Tool	103
Figure 9-2 Programmer.....	104
Figure 9-3 Download Settings.....	105

Figure 9-4 Download Settings.....	106
Figure 9-5 Download Settings.....	107
Figure 9-6 Download Settings.....	108
Figure 9-7 Download Settings.....	109
Figure 10-1 Debug	112
Figure 10-2 Create a Debug View	112
Figure 10-3 Main View	113
Figure 10-4 Debugger View	114
Figure 10-5 Startup View	115
Figure 10-6 Main View.....	116
Figure 10-7 Debugger View	117
Figure 10-8 Startup View	119
Figure 10-9 Main View	120
Figure 10-10 Debugger View	121
Figure 10-11 Startup View.....	122
Figure 10-12 Launch Debug View	124
Figure 10-13 Window > Show View > Other... ..	125
Figure 10-14 Debug View	126
Figure 10-15 Memory View.....	126
Figure 10-16 Select a Thread or Stack to Monitor.....	127
Figure 10-17 Add Memory Monitor	127
Figure 10-18 Monitor Memory.....	128
Figure 10-19 Memory Renderer in Traditional Format.....	128
Figure 10-20 Create a Memory Renderer in Floating-Point Format.....	129
Figure 10-21 Memory Renderer in Floating-Point Format.....	129
Figure 10-22 Set Byte Order.....	130
Figure 10-23 Memory Content Display	130
Figure 10-24 Set Rendering Preferences	131
Figure 10-25 Configure Memory Rendering Appearance	132
Figure 10-26 Import Memory	132

Figure 10-27 Export Memory	133
Figure 10-28 Memory Browser View.....	133
Figure 10-29 Memory Renderer in Traditional Format.....	134
Figure 10-30 Create a Memory Renderer in Floating-Point Format.....	134
Figure 10-31 Memory Renderer in Floating-Point Format.....	135
Figure 10-32 Set Byte Order.....	135
Figure 10-33 Find/Replace	136
Figure 10-34 Registers View.....	136
Figure 10-35 Variables View	137
Figure 10-36 Expressions View	137
Figure 10-37 Breakpoints View.....	138
Figure 10-38 Disassembly View	138
Figure 10-39 Console View.....	139
Figure 11-1 GDB QEMU arm Debugging.....	141
Figure 11-2 Main View	142
Figure 11-3 Debugger View	143
Figure 11-4 Startup View.....	144
Figure 11-5 GDB QEMU riscv32 Debugging	145
Figure 11-6 Main View	146
Figure 11-7 Debugger View	147
Figure 11-8 Startup View.....	148
Figure 11-9 Launch QEMU Emulation Debugging.....	149
Figure 12-1 Apply it to All Source Files	152
Figure 12-2 Apply it to a Specific Source File	153
Figure 12-3 Set Linux Tools Path.....	154
Figure 12-4 Set Linux Tools Path	154
Figure 12-5 Build the Project	155
Figure 12-6 Run the Project and View Profiling Information.....	156
Figure 12-7 Select All.....	157
Figure 12-8 Parse HexDump Data.....	158

Figure 12-9 Parse Profiling Data and Generate Files	158
Figure 12-10 View Generated Files	159
Figure 12-11 Open .gcda File	159
Figure 12-12 Gcov	160
Figure 12-13 Create chat... ..	161
Figure 12-14 Bar Chart	162
Figure 12-15 Pie Chart.....	162
Figure 13-1 Set Build Options.....	164
Figure 13-2 Set Linux Tools Path.....	165
Figure 13-3 Set Linux Tools Path	166
Figure 13-4 Run the Project and View Profiling Information.....	167
Figure 13-5 Select All.....	168
Figure 13-6 Parse HexDump Data.....	168
Figure 13-7 Parse Profiling Data and Generate Files.....	169
Figure 13-8 View Generated gmon.out File	169
Figure 13-9 Open gmon.out File	170
Figure 13-10 Gprof.....	170
Figure 13-11View Profiling Information	171
Figure 13-12 Create chat... ..	172
Figure 13-13 Bar Chart	173
Figure 13-14 Pie Chart.....	173

List of Tables

Table 1-1 Terminology and Abbreviations	2
Table 2-1 MCU Support	4
Table 4-1 File Descriptions of GMD Software	7
Table 5-1 File Menu Items	14
Table 5-2 Project Type	17
Table 5-3 Toolchains	17
Table 5-4 Edit Menu Items	19
Table 5-5 Source Menu Items	21
Table 5-6 Refactor Menu Items	22
Table 5-7 Navigate Menu Items	22
Table 5-8 Search Menu Items	25
Table 5-9 Project Menu Items	25
Table 5-10 Run Menu Items	26
Table 5-11 Debugging Methods	28
Table 5-12 Gowin Menu Items	29
Table 5-13 Window Menu Items	30
Table 5-14 Help Menu Items	33
Table 5-15 Toolbar Option	35
Table 8-1 Target Processor	61
Table 8-2 Optimization	62
Table 8-3 Debugging	65
Table 8-4 GNU ARM Cross Assembler > Preprocessor	66
Table 8-5 GNU ARM Cross C Compiler > Includes	71
Table 8-6 GNU ARM Cross C Compiler > Optimization	72

Table 8-7 GNU ARM Cross C Linker > General	75
Table 8-8 GNU ARM Cross Create Flash Image	78
Table 8-9 GNU ARM Cross Print Size	79
Table 8-10 Target Processor	80
Table 8-11 Optimization.....	82
Table 8-12 Debugging	84
Table 8-13 GNU RISC-V Cross Assembler > Miscellaneous.....	88
Table 8-14 GNU RISC-V Cross C Compiler > Includes	90
Table 8-15 GNU RISC-V Cross C Compiler > Optimization	91
Table 8-16 GNU RISC-V Cross C Compiler > Miscellaneous	93
Table 8-17 GNU RISC-V Cross C Linker > General	94
Table 8-18 GNU RISC-V Cross C Linker > Miscellaneous	96
Table 8-19 GNU RISC-V Cross Create Flash Image	97
Table 8-20 GNU RISC-V Cross Create Listing	98
Table 8-21 GNU RISC-V Cross Print Size	100
Table 9-1 Download Settings	105
Table 9-2 Download Settings	106
Table 9-3 Download Settings	107
Table 9-4 Download Settings	108
Table 9-5 Download Settings	109
Table 10-1 Debugging Modes	111
Table 10-2 Debugger View.....	114
Table 10-3 OpenOCD Configuration Files	114
Table 10-4 Startup View	115
Table 10-5 Debugger View.....	117
Table 10-6 ARM MCU Part Number.....	118
Table 10-7 Startup View	119
Table 10-8 Debugger View.....	121
Table 10-9 OpenOCD Configuration File	122
Table 10-10 Startup View	123

Table 10-11 Debugging Options.....	124
Table 11-1 Supported Peripheral Functions.....	140
Table 11-2 Debugger View	143
Table 11-3 ARM MCU QEMU.....	144
Table 11-4 Startup View	144
Table 11-5 Debugger View	147
Table 11-6 RISC-V MCU QEMU	148
Table 11-7 Startup View	148
Table 12-1 Gcov Functions	160
Table 13-1 Gprof Functions.....	171

1 About This Guide

1.1 Purpose

This manual primarily describes how to use Gowin GMD (GOWIN MCU Designer) software, including platform support, downloading and installation, user interface, project creation, importing legacy projects from earlier GMD versions, project building, downloading, and debugging, QEMU simulation, code coverage analysis, and profiling. It aims to help users quickly become familiar with GMD software workflow and improve development efficiency. The software screenshots in this manual are based on GMD 2025.01. As the software is subject to change without notice, some information may not remain relevant and may need to be adjusted according to the software that is in use.

1.2 Related Documents

The latest user guides are available on GOWINSEMI Website. You can find the related documents at www.gowinsemi.com:

- [IPUG928, Gowin EMPU\(GW1NS-4C\) IDE Software Reference Manual](#)
- [MUG1186, Gowin EMPU\(GW5AS-25\) Quick Start User Reference](#)
- [IPUG536, Gowin EMPU M1 IDE Software Reference Manual](#)
- [IPUG910, Gowin PicoRV32 IDE Software Reference Manual](#)
- [IPUG919, Gowin EMPU M3 IDE Software Reference Manual](#)

1.3 Terminology and Abbreviations

Table 1-1 shows the abbreviations and terminology used in this manual.

Table 1-1 Terminology and Abbreviations

Terminology and Abbreviations	Meaning
GMD	GOWIN MCU Designer
ARM	Advanced RISC Machine
RISC-V	Reduced Instruction Set Computer Five
IDE	Integrated Development Environment
MCU	Microcontroller Unit
FPGA	Field Programmable Gate Array
SoC	System on Chip
GNU	GNU's Not Unix
GCC	GNU Compiler Collection
GDB	GNU Debug
OpenOCD	Open On-Chip Debugger
QEMU	Quick Emulator
CDT	C/C++ Development Tooling
EPL	Eclipse Public License
ABI	Application Binary Interface
UART	Universal Asynchronous Receiver/Transmitter
GPIO	General Purpose Input/Output

1.4 Support and Feedback

Gowin Semiconductor provides customers with comprehensive technical support. If you have any questions, comments, or suggestions, please feel free to contact us directly by the following ways.

Website: www.gowinsemi.com

E-mail: support@gowinsemi.com

2Overview

GMD (Gowin MCU Designer) is a new-generation MCU software development environment independently developed by GOWINSEMI. It is designed based on Gowin's FPGA + MCU SoC architecture and built on the open-source GCC toolchain and Eclipse framework. The software supports general C/C++ embedded software development, enabling developers to efficiently complete code compilation, linking, executable generation, downloading, and debugging.

GMD integrates debugging and simulation tools such as OpenOCD and QEMU, allowing developers to debug and run projects easily, and quickly locate and analyze software programming issues.

GMD also integrates Gcov and Gprof tools to support Code Coverage and Profiling functions. Gcov is a test tool for measuring C/C++ code coverage, working in conjunction with GCC to provide statement, function, and branch coverage analysis of C/C++ programs. Gprof is a performance analysis tool that helps developers understand the runtime behavior of C/C++ programs. It provides information about each function in the program, including invocation information, call counts, and execution times, which are crucial for program optimization and performance improvement.

GMD supports Gowin MCU products based on both ARM and RISC-V architectures, including Gowin_EMPU (GW1NS-4C), Gowin_EMPU_M1, Gowin_EMPU (GW5AS-25), Gowin_EMPU_M3, and Gowin_PicoRV32.

GMD provides a graphical user interface that allows quick code editing, build result viewing, online debugging, and simulation. It is also integrated with Gowin Programmer tool to quickly launch the download function, supporting a complete user development workflow.

2.1 Reference Documents

The Eclipse platform operates under an open-source model and is distributed under the EPL (Eclipse Public License), which is both free and open-source, with global distribution rights. Eclipse itself is a modular framework. Except for the runtime kernel, all other features are provided through separate plug-ins. Developers can extend Eclipse by installing various plug-ins or by developing their own custom plug-ins. One plug-in can also be embedded into another, allowing maximum flexibility and integration.

Since Eclipse is widely adopted by the developer community, many common usage patterns are already included in the Eclipse IDE. In general cases (not tied to specific hardware), users can refer to official Eclipse documentation or third-party tutorials. Recommended resources include:

- <https://help.eclipse.org/latest/index.jsp>
- <https://eclipse-embed-cdt.github.io/>
- <https://mcuoneclipse.com/>

2.2 MCU Support

The MCUs currently supported by GMD software, along with their core architectures and corresponding FPGAs, are listed in Table 2-1.

Table 2-1 MCU Support

MCU	Core Architecture	FPGA
Gowin_EMPU(GW1NS-4C)	ARM Cortex-M3	GW1NS/R/ER-4C
Gowin_EMPU_M1	ARM Cortex-M1	GW1N/R series GW2A/N/R/NR series GW3A/R series GW5A/NT/NRT/R/RT/S/ST/T series
Gowin_EMPU(GW5AS-25)	ARM Cortex-M4	GW5AS-25
Gowin_EMPU_M3	ARM Cortex-M3	GW2A/N/R/NR series
Gowin_PicoRV32	RISC-V	GW1N/R series GW2A/N/R/NR series GW3A/R series GW5A/NT/NRT/R/RT/S/ST/T series

3 Platform Support

3.1 Platform Support

- Windows 10/11 (64-bit)
- Ubuntu 20.04/22.04/24.04 LTS

3.2 Operating Environment

- 4 GB of RAM is recommended
- 6 GB of disk space is recommended

4 Download and Installation

4.1 Software Download

GMD software is available as compressed packages for both Windows and Linux operating systems:

- GMD_20yy.xx_win.zip
- GMD_20yy.xx_linux.tar.gz

Note !

- Before downloading, users must register and log in to Gowin official website.
- In the software package name "20yy.xx", "yy" represents the release year, and "xx" denotes the version number within that year.

Users can download the corresponding GMD package for Windows or Linux from Gowin website, depending on their local system environment.

The software has been validated on Windows 10/11 64-bit and Ubuntu 20.04 LTS. It is recommended to use one of these operating systems.

4.2 IDE Installation

After downloading the GMD software package, simply extract it locally to use, and no additional IDE installation or environment variable configuration is required. Note that the extraction path must not contain any Chinese characters.

GMD software does not require a license and can be launched directly.

For example, after extracting the Windows package GMD_20yy.xx_win.zip, the directory will contain several folders and files as shown in Table 4-1.

Table 4-1 File Descriptions of GMD Software

GMD Software File	Description
configuration	Local temporary configuration files generated during runtime
dropins	User-installed plugins
features	Function-related modules
plugins	All installed plug-in files
tool	Software tools, such as Programmer, GoBridgeDriver
toolchain	Software toolchains, such as GCC, OpenOCD, QEMU
GMD.exe	Executable file for GMD software; double-click to launch
GMD.ini	Initialization file for GMD software
ver.20yy.xx.txt	Version information file

4.3 Driver Installation

GMD supports using OpenOCD to debug both ARM and RISC-V MCUs, and Segger J-Link to debug ARM MCUs. When using OpenOCD, the software supports Gowin-developed or third-party debuggers such as GWU2X, GWUSB, Olimex, and J-Link. On the Windows system, WinUSB drivers must be installed. When Segger J-Link is used, the Segger J-Link driver software must be installed.

On the Windows system, GMD supports the installation of the relevant WinUSB software drivers using the GoBridgeDriver driver software. Launch GMD, select “Gowin > GoBridgeDriver” from the menu bar to start the GoBridgeDriver driver software (administrator privileges on the PC are required), as shown in Figure 4-1 and Figure 4-2.

Figure 4-1 Select Gowin > GoBridgeDriver

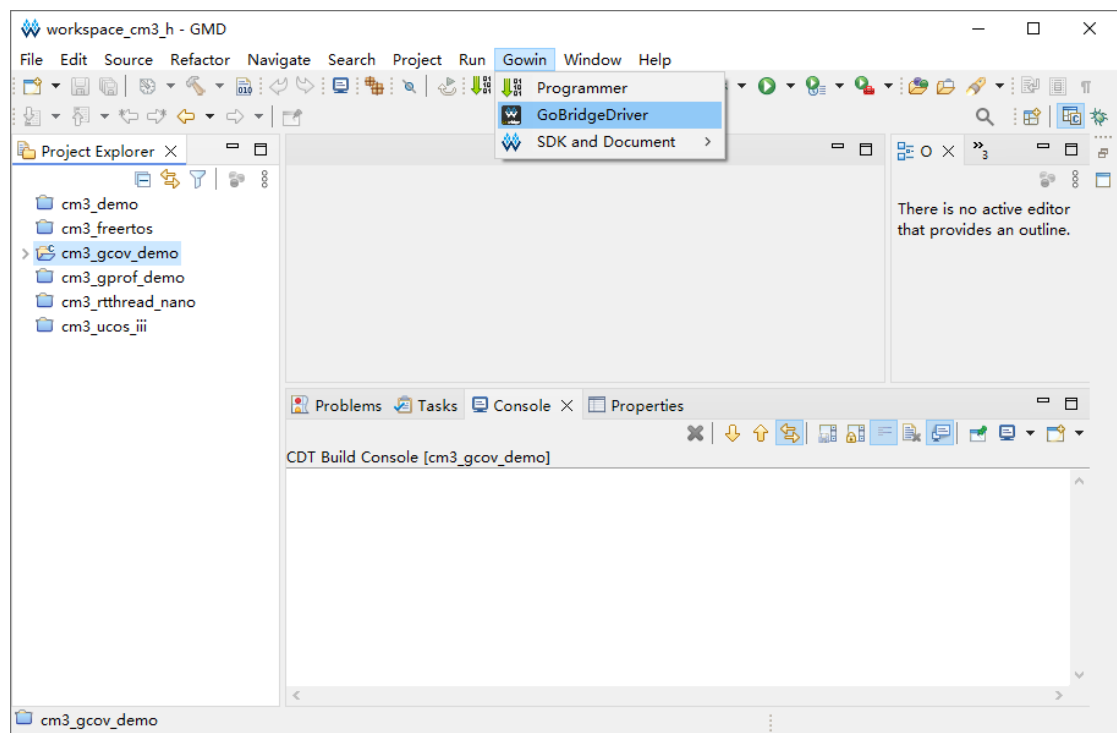
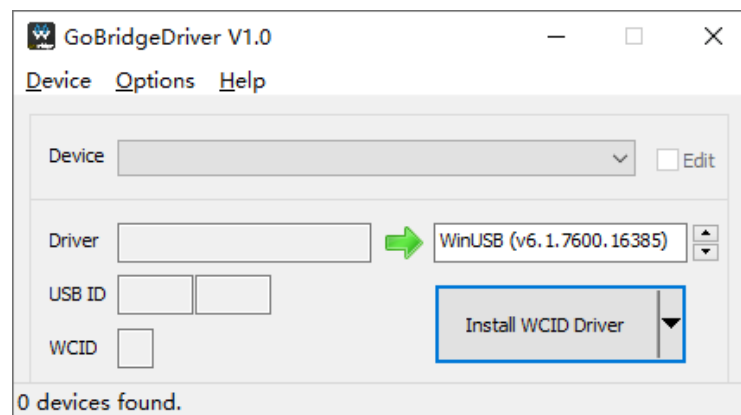


Figure 4-2 GoBridgeDriver



The installation methods for the software drivers of the aforementioned debuggers are described as follows.

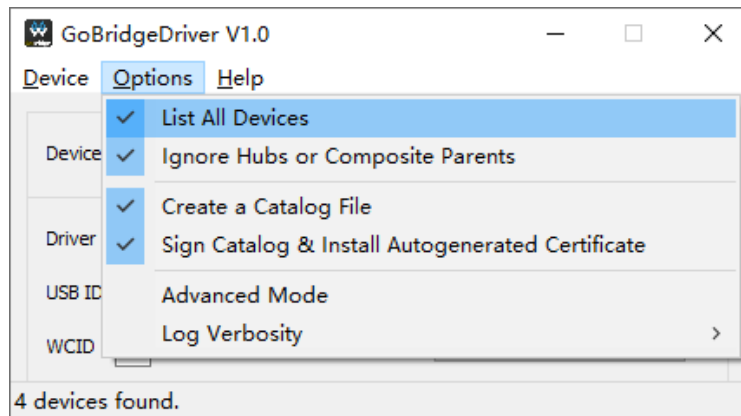
4.3.1 GWU2X

GWU2X downloader/debugger is equipped with Gowin's USB-to-JTAG chip. Please follow the steps below to install the GWU2X software driver:

1. Connect the GWU2X device to the PC host.

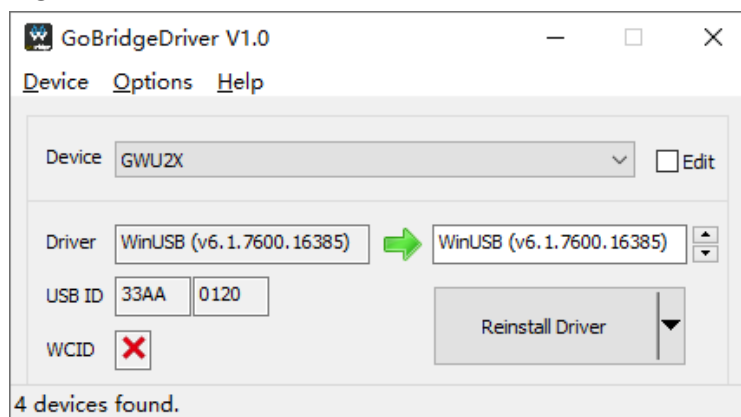
- From the menu bar, select "Options > List All Devices", and all USB devices currently connected to the PC are listed, as shown in Figure 4-3.

Figure 4-3 Select Options > List All Devices



- Select the device "GWU2X", choose the driver type "WinUSB", then click "Install Driver" to install the software driver, as shown in Figure 4-4. The installation will complete after a short moment.

Figure 4-4 Install GWU2X Software Driver



Note !

- If no driver has been previously installed for the device, the option will show "Install Driver".
- If the same driver has already been installed, the option will show "Reinstall Driver".
- If a different driver is currently installed, the option will show "Replace Driver".

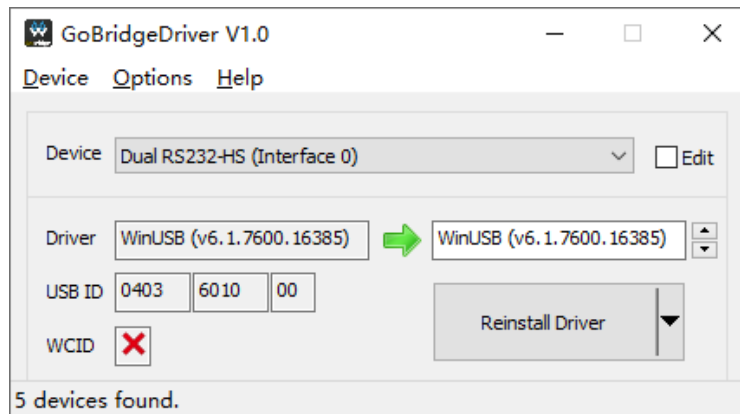
4.3.2 GWUSB V3.3

GWUSB V3.3 downloader/debugger is equipped with a dual-channel FTDI 232H chip. Please follow the steps below to install the GWUSB V3.3 software driver.

- Connect the GWUSB V3.3 device to the PC host.

2. Select "Options > List All Devices" from the menu bar, and all USB devices currently connected to the PC are listed.
3. Select the devices "Dual RS232-HS (Interface 0)" and "Dual RS232-HS (Interface 1)" respectively. Choose the driver "WinUSB", then click "Install Driver" to install the software driver. As shown in Figure 4-5, the driver installation will complete after a short moment.

Figure 4-5 Install GWUSB V3.3 Software Driver

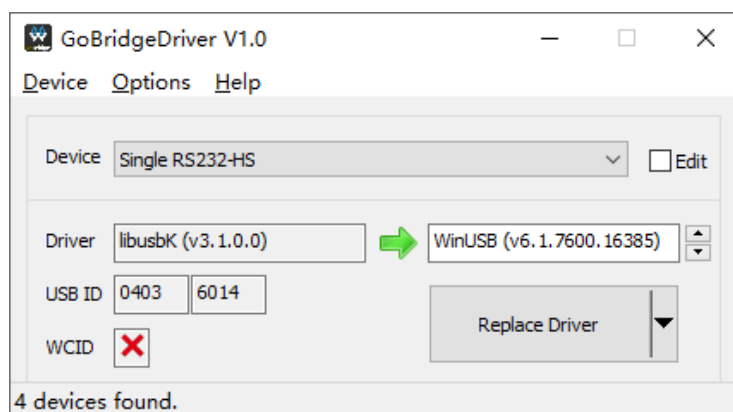


4.3.3 GWUSB V4.1

GWUSB V4.1 downloader/debugger is equipped with a single-channel FTDI 232H chip. Please follow the steps below to install the GWUSB V4.1 driver:

1. Connect the GWUSB V4.1 device to the PC.
2. Select "Options > List All Devices" from the menu bar, and all USB devices currently connected to the PC are listed.
3. Select the device "Single RS232-HS", and choose the driver "WinUSB", then click "install Driver" to install the driver, as shown in Figure 4-6. The driver installation will complete after a short moment.

Figure 4-6 Install GWUSB V4.1 Software Driver

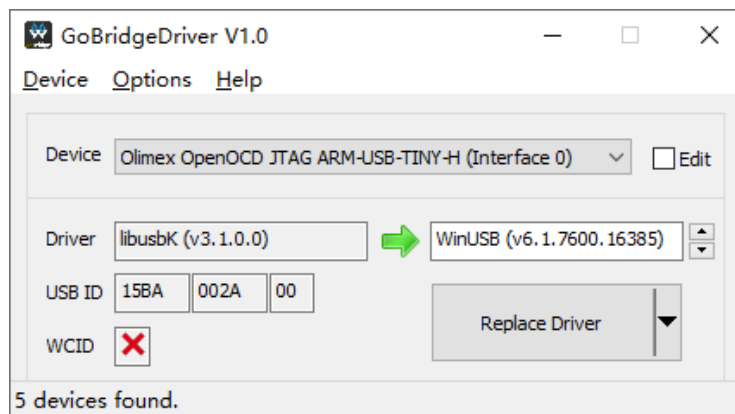


4.3.4 Olimex

ARM-USB-TINY-H Debugger (Olimex) is equipped with an FTDI FT2232H chip. Please follow the steps below to install the Olimex driver.

1. Connect the Olimex device to the PC host.
2. Select "Options > List All Devices" from the menu bar, and all USB devices currently connected to the PC are listed.
3. Select the devices "Olimex OpenOCD JTAG ARM-USB-TINY-H (Interface 0)" and "Olimex OpenOCD JTAG ARM-USB-TINY-H (Interface 1)" respectively. Choose the driver "WinUSB", then click "Install Driver" to install the driver, as shown in Figure 4-7. The installation will complete after a short moment.

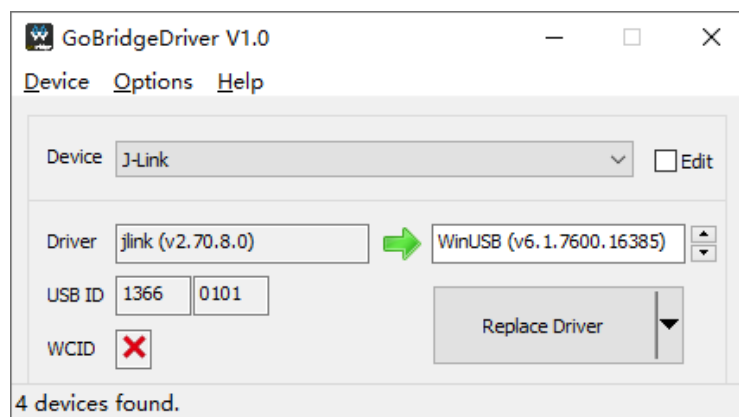
Figure 4-7 Install Olimex Software Driver



4.3.5 J-Link WinUSB

Please follow the steps below to install the WinUSB driver for the J-Link debugger.

1. Connect the J-Link device to the PC host.
2. Select "Options > List All Devices" from the menu bar, and all USB devices currently connected to the PC are listed.
3. Select the device "J-Link", choose the driver "WinUSB", and click "Install Driver" to install the software driver, as shown in Figure 4-8. The installation will complete after a short moment.

Figure 4-8 Install J-Link WinUSB Software Driver

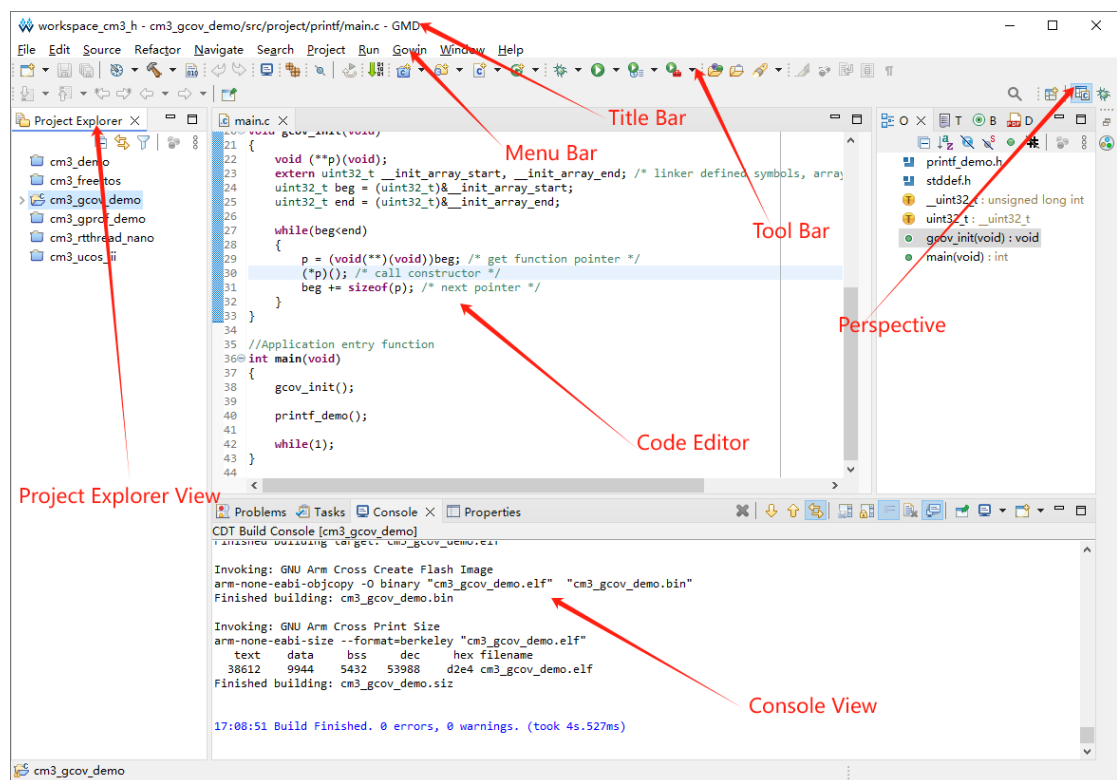
4.3.6 Segger J-Link

Please refer to the official Segger website (<https://www.segger.com/downloads/jlink/>) for downloading and installing the Segger J-Link driver. Detailed installation steps are not described here.

5 User Interface

Double-click the GMD executable file "GMD\GMD.exe" to launch the GMD software. The user interface mainly includes the Title Bar, Menu Bar, Tool Bar, Project Explorer, Code Editor, Console View, and Perspective, as shown in Figure 5-1.

Figure 5-1 User Interface



The main components of the user interface are described as follows:

5.1 Title Bar

Displays information about the current workspace, project name, and opened files.

5.2 Menu Bar

Provides access to commonly used menu items, including File, Edit, Source, Refactor, Navigate, Search, Project, Run, Gowin, Window, and Help.

Commonly used menu items are described below.

5.2.1 File Menu Items

The File menu items are listed in Table 5-1.

Table 5-1 File Menu Items

Menu Item	Submenu Item	Description
New	Makefile Project with Existing Code	Create a C/C++ project including Makefile
	C/C++ Project	Create a C/C++ project
	Project...	Create a C, C++, or C/C++ project
	Convert to a C/C++ Project (Adds C/C++ Nature)	Add C/C++ nature to a non-C/C++ project to enable the C/C++ toolchain
	Source Folder	Create a source folder
	Folder	Create a folder
	Source File	Create a C/C++ source file
	Header File	Create a C/C++ header file
	File from Template	Create a C/C++ template file
	Class	Create a C++ class
	Task	Track and manage development tasks or issues
	Example...	Edit and validate XML

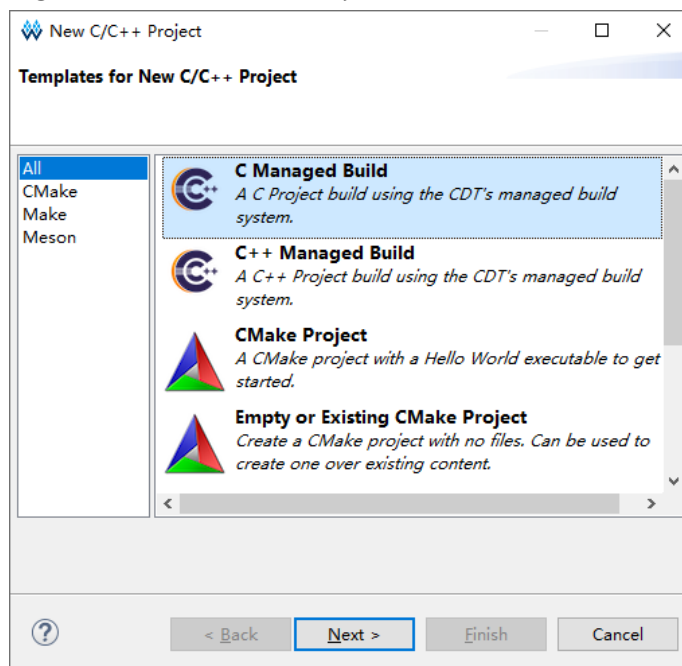
Menu Item	Submenu Item	Description
		files
	Other...	Open the item selector showing all available options
Open File...	-	Open an existing file
Open Projects from File System...	-	Import and open an existing project from the file system
Recent Files		List of recently opened files
Close Editor	-	Close the current editor
Close All Editors	-	Close all editors
Save	-	Save the content of the current editor
Save As...	-	Save the content of current editor with a new name
Save All	-	Save the content of all editors and unsaved changes
Revert	-	Revert the content of the current editor to the last saved version
Move...	-	Move resources
Rename...	-	Rename resources
Refresh	-	Refresh and reload project resources
Convert Line Delimiters To	Windows	Windows system format
	Unix	Unix system format
Print...	-	Print the content of the current editor
Import...	-	Import project wizard
Export...	-	Export project wizard
Properties	-	Configure project properties
Switch Workspace		Switch to a different workspace and restart

Menu Item	Submenu Item	Description
Restart	-	Restart GMD software
Exit	-	Exit GMD software

C/C++ Project

The option "New > C/C++ Project", as shown in Figure 5-2, is used to create a C/C++ project.

Figure 5-2 New C/C++ Project



Commonly used options include "C Managed Build" or "C++ Managed Build". For example, the "C Managed Build" option is shown in Figure 5-3. The available project types are listed in Table 5-2, and supported toolchains are listed in Table 5-3.

Figure 5-3 C Project

Table 5-2 Project Type

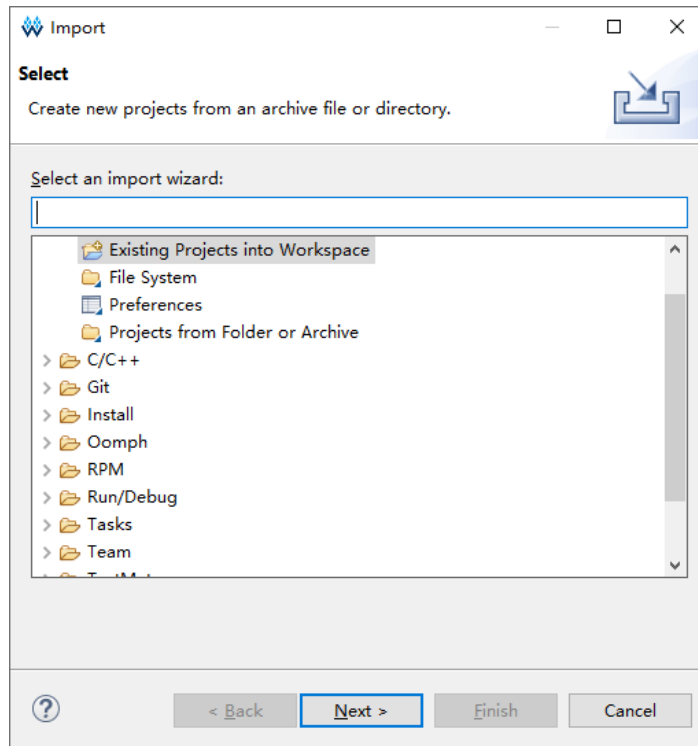
Project Type	Description
Executable	Create a executable project
Shared Library	Create a shared library project
Static Library	Create a static library project
Makefile project	Create a project based on Makefile

Table 5-3 Toolchains

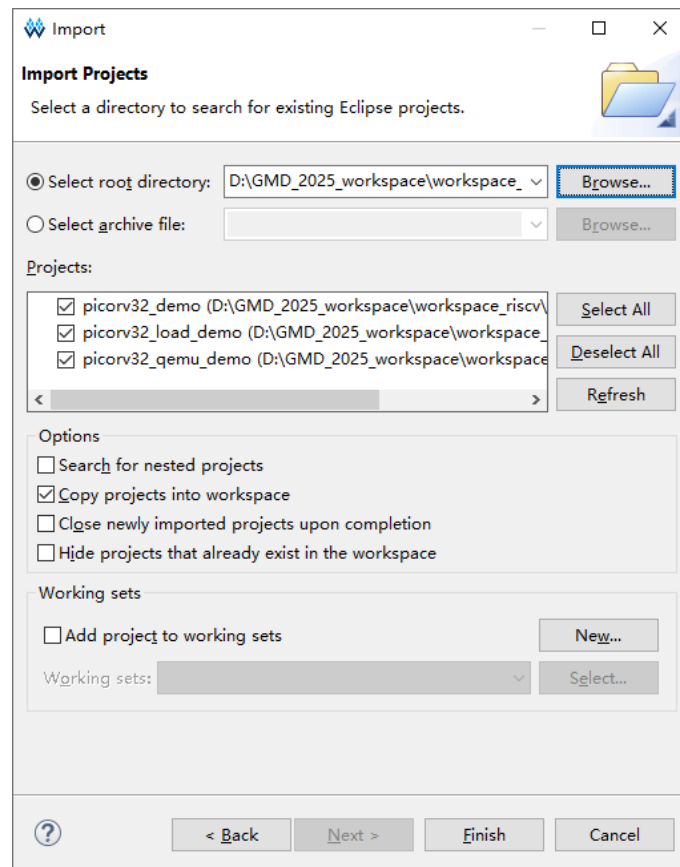
Toolchains	Description
ARM Cross GCC	ARM GCC cross-compilation toolchain
RISC-V Cross GCC	RISC-V GCC cross-compilation toolchain

Import

The "New > Import..." option, as shown in Figure 5-4, is used to import existing projects.

Figure 5-4 Import

A commonly used option is "Existing Projects into Workspace", as shown in Figure 5-5.

Figure 5-5 Import Projects

5.2.2 Edit Menu Items

The Edit menu items are listed in Table 5-4.

Table 5-4 Edit Menu Items

Menu Items	Submenu Items	Description
Undo	-	Undo the last change made in the editor
Redo	-	Redo the previously undone change
Cut	-	Cut selected text or elements
Copy	-	Copy selected text or elements
Paste	-	Paste copied or cut content
Delete	-	Delete selected text or elements
Select All	-	Select all content in the editor
Expand Selection to	Enclosing Element	Expand the selection to the outermost enclosing code element
	Next Element	Expand the selection to the next logical code element

Menu Items	Submenu Items	Description
	Previous Element	Expand the selection to the previous logical code element
	Restore Last Selection	Restore the last selection
Toggle Block Selection	-	Toggle block selection mode
To multi-selection	-	Select multiple positions or code segments simultaneously
Find/Replace...	-	Find or replace
Find Word	-	Find the currently selected word
Find Next	-	Find the next occurrence of the selected word
Find Previous	-	Find the previous occurrence of the selected word
Incremental Find Next	-	Start incremental search to find the next match
Incremental Find Previous	-	Start incremental search to find the previous match
Bookmark...	-	Add a bookmark to the selected text or element
Task...	-	Add a user-defined task to the selected text or element
Smart Insert Mode	-	Enable smart cursor insertion mode in the text editor
Insert ChangeLog entry	-	Quickly insert a changelog entry in the code
Show Tooltip Description	-	Show a tooltip description at the cursor position
Word Completion	-	Automatically complete words based on typed characters
Quick Fix	-	Generate fix suggestions or code fixes automatically based on the errors and warnings provided by the compiler or IDE
Content Assist	-	Provide intelligent suggestions and auto-completion
Parameter Hints	-	Automatically display the parameter

Menu Items	Submenu Items	Description
		list of a method when it is being called
Set Encoding...	-	Set the character encoding for the current file

5.2.3 Source Menu Items

Source menu items are as listed in Table 5-5.

Table 5-5 Source Menu Items

Menu Items	Description
Toggle Comment	Add or remove single-line comments at the beginning of the selected lines.
Add Block Comment	Comment the selected block of code using block comment format (<code>/* ... */</code>)
Remove Block Comment	Remove the selected block comments
Shift Right	Indent the selected code lines to the right
Shift Left	Indent the selected code lines to the left
Align Const	Align constants (typically static constants) in the selected region
Correct Indentation	Adjust the indentation of the selected code to conform to project coding standards
Format	Automatically format the entire file or the selected code.
Copy Qualified Name	Copy the fully qualified name of the current class, method, variable, or other elements
Add Include	Add an <code>#include</code> directive for a header file.
Organize Includes	Organize <code>#include</code> statements in the source file by removing unused entries and sorting them alphabetically
Sort Lines	Sort the selected code lines
Override Methods	Quickly override one or more methods inherited from a parent class or interface in the current class
Implement Method...	Implement unimplemented methods from interfaces/abstract classes in the current class
Generate Getters and Setters...	Automatically generate getter and setter methods based on the class fields
Surround With	Surround the selected code with a structure such as

Menu Items	Description
	try-catch, if, or for

5.2.4 Refactor Menu Items

Refactor menu items are listed in Table 5-6.

Table 5-6 Refactor Menu Items

Menu Items	Description
Rename...	Rename the selected element and update all its references
Extract Local Variable	Create a new variable assigned to the currently selected expression, and replace the selection with a reference to the new variable.
Extract Constant...	Create a static final field from the selected expression, replace field references, and rewrite other occurrences of the same expression.
Extract Function...	Create a new function assigned to the currently selected statements, and replace the selection with a reference to the new function.
Toggle Function	Toggle the selected function definition between header (inside/outside class definition) and implementation files as needed.
Hide Method...	Make the selected method private
Apply Script...	Apply a saved refactoring script
Create Script...	Export completed refactoring steps as a reusable script
History...	Show refactoring history

5.2.5 Navigate Menu Items

Navigate menu items are listed in Table 5-7.

Table 5-7 Navigate Menu Items

Menu Items	Submenu Items	Description
Go Into	-	Focus the current view on the root directory of the selected resource
Go To	Back	Return to the previously displayed hierarchy level
	Forward	Return to the level viewed after the last "Back" operation

Menu Items	Submenu Items	Description
	Up One Level	Navigate up one level in the hierarchy
	Resource...	Quickly navigate to a specific resource
	Next Member	Jump to the next member in the current class/interface
	Previous Member	Jump to the previous member in the current class/interface
	Go to Matching Bracket	Jump to the matching bracket/brace/parenthesis
	Next Bookmark	Jump to the next bookmark
Open Declaration	-	Jump to the declaration of the element at cursor position
Open Type Hierarchy	-	View inheritance hierarchy of current type (class/interface)
Open Call Hierarchy	-	View call hierarchy of the current method
Open Include Browser	-	View include relationships for the current file
Toggle Source/Header	-	Switch between source file (.c/.cpp) and corresponding header file (.h)
Open Element...	-	Open detailed view of element at cursor
Open Type in Hierarchy...	-	Open current type in Type Hierarchy view
Open Element in Call Hierarchy...	-	Display current element (method/class) in Call Hierarchy view
Open Resource...	-	Show all resources for selection/opening
Open Discovered Type...	-	Display and open discovered types
Open Task...	-	Open task list in current workspace
Activate Task...	-	Activate and highlight selected task
Deactivate Task	-	Deactivate selected task and remove it from the active state
Open Setup	User	View/modify user-specific settings

Menu Items	Submenu Items	Description
	Installation	View current installation and configuration
	Workspace	View/modify workspace settings
	Self Products	View product settings for current instance
	Parent Models	View and manage parent model settings for workspace/project
Open Setup Log	-	View setup log for project/workspace
Show In	Include Browser	Display Include Browser view
	C/C++ Projects	Display C/C++ Projects view
	Project Explorer	Display Project Explorer view
	Problem Details	Display Problem Details view
	Terminal	Display Terminal view
	Outline	Display Outline view
	System Explorer	Display System Explorer view
	Properties	View properties of selected file/project/resource
Quick Outline	-	Display structural outline of current editor (classes/methods)
Next	-	Navigate to next item in active view
Previous	-	Navigate to previous item in active view
Previous Edit Location	-	Move cursor to last edit location
Next Edit Location	-	Move cursor to next edit location
Go to Line...	-	Jump to specified line number
Back	-	Return to previously viewed resource in editor
Forward	-	Advance to next viewed resource in editor history

5.2.6 Search Menu Items

Search menu items are listed in Table 5-8.

Table 5-8 Search Menu Items

Menu Items	Submenu Items	Description
Search...	-	Search
File...	-	Search for specific text or patterns within files
C/C++...	-	Search within C or C++ source code
Remote...	-	Perform search operations on remote systems
Text	Workspace	Search across the entire workspace
	Project	Search within the current project
	File	Search within a single file
	Working Set...	Search within the currently selected working set
Quick Search	-	Perform a quick search

5.2.7 Project Menu Items

Project menu items are listed in Table 5-9.

Table 5-9 Project Menu Items

Menu Items	Submenu Items	Description
Open Project	-	Open the current project
Close Project	-	Close the current project
Build All	-	Build all projects
Build Configurations	Set Active	Set "Release" or "Debug" to the active
	Manage...	Manage configurations
	Build by Working Set	Build projects by working set
	Set Active by Working Set	Set a working set to the active
	Manage Working Sets...	Manage working sets
Build Project	-	Incrementally build the currently selected project

Menu Items	Submenu Items	Description
Build Working Set	Select Working Set	Incrementally build all projects within the selected working set
Clean...	-	Clean build results
Build Automatically	-	Automatically build all projects in the workspace
Build Targets	Create...	Create targets
	Build...	Build targets
	Rebuild Last Target	Rebuild the most recently target
C/C++ Index	Rebuild	Rebuild the C/C++ index for the entire project
	Freshen All Files	Refresh all files in the project
	Update with Modified Files	Update indexes only for modified files
	Re-resolve Unresolved Includes	Re-solve unresolved header file (#include) references
	Search for Unresolved Includes	Search for all unresolved header files in project
	Create Parser Log File	Generate parser log file
Properties	-	Property configuration

5.2.8 Run Menu Items

Run menu items are listed in Table 5-10.

Table 5-10 Run Menu Items

Menu Items	Submenu Items	Description
Run Last Launched	-	Run the most recently launched program
Debug Last Launched	-	Debug the most recently launched program
Profile Last Launched	-	Profile the most recently launched program
Profile History	-	Use the last profiling configuration from the history
Profile As	Local C/C++ Application	Profile the application as a local C/C++ application directly on the host system using the locally installed compiler and debugger

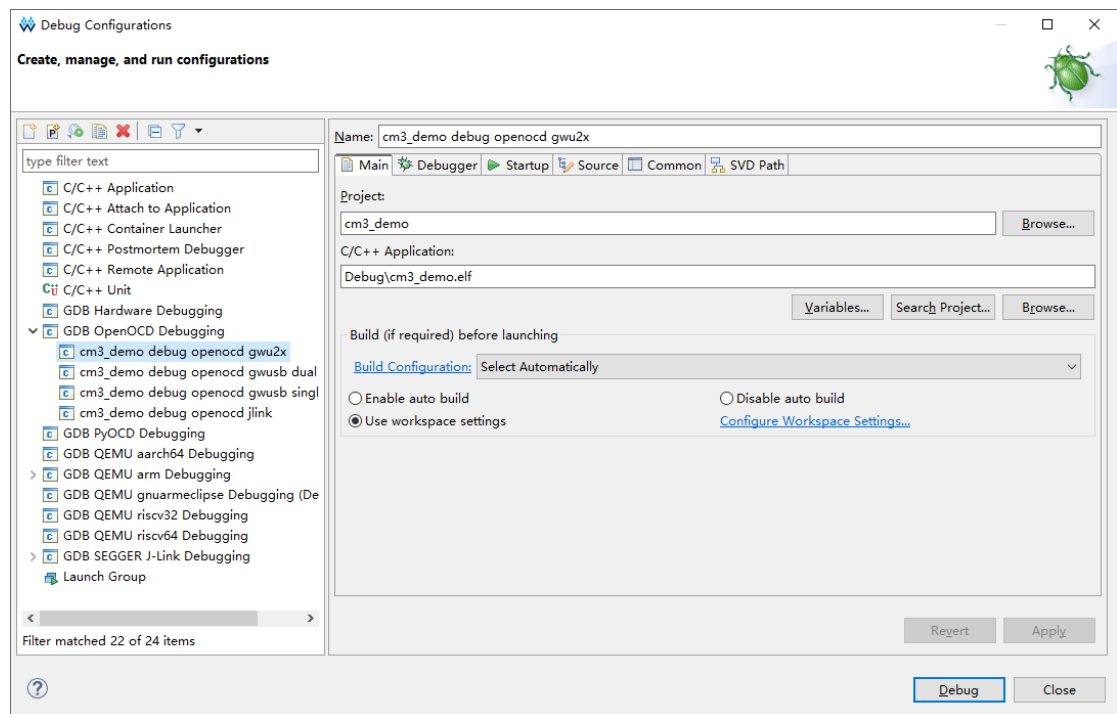
Menu Items	Submenu Items	Description
Profile Configurations...	-	Configure and manage profiling settings
Run History	-	Use the last debug configuration from the history
Run As	C/C++ Container Application	Run the application as a C/C++ container application by mounting the executable into a specified Docker container, and execute it within the isolated container environment
	Local C/C++ Application	Run the application as a local C/C++ application directly in the host environment, using the locally installed compiler and debugger.
Run Configurations...	-	Configure and manage run configurations
Debug History	-	Use the last debug configuration from the history
Debug As	C/C++ Container Application	Debug the application as a C/C++ container application by mounting the executable into a specified Docker container, and debug it within the isolated container environment
	Local C/C++ Application	Debug the application as a local C/C++ application directly in the host environment, using the locally installed compiler and debugger.
Debug Configuration...	-	Configure and manage debug configurations
Breakpoint Types	-	Configure breakpoint types (C/C++ Breakpoint or C/C++ Dynamic Printf)
Toggle Breakpoint	-	Add or remove a breakpoint at the current line
Toggle Line Breakpoint	-	Set a line breakpoint in the source code
Toggle Watchpoint	-	Add or remove a watchpoint for a field
Toggle Method Breakpoint	-	Set a breakpoint at method entry or exit
Skip All Breakpoints	-	Skip all breakpoints in the workspace

Menu Items	Submenu Items	Description
Remove All Breakpoint	-	Remove all breakpoints from the workspace
External Tools	-	Run and configure external tools (e.g., build scripts, CLI tools)

Debug Configurations

"Run > Debug Configurations..." option, as shown in Figure 5-6, is used to configure debug settings and launch the debugging.

Figure 5-6 Debug Configurations



Common debugging methods are listed in Table 5-11.

Table 5-11 Debugging Methods

Debugging Method	Description
GDB OpenOCD Debugging	Debug ARM and RISC-V MCUs using OpenOCD
GDB QEMU arm Debugging	Debug ARM MCUs using QEMU
GDB QEMU riscv32 Debugging	Debug 32-bit RISC-V MCUs using QEMU
GDB QEMU riscv64 Debugging	Debug 64-bit RISC-V MCUs using QEMU
GEB SEGGER J-Link	Debug ARM MCUs using SEGGER official

Debugging Method	Description
Debugging	software driver and J-Link debugger

Run Configurations

The "Run > Run Configurations..." option functions the same as the "Debug Configurations..." option and is not further elaborated here.

5.2.9 Gowin Menu Items

Gowin menu items are listed in Table 5-12, which include some of Gowin's software tools such as Programmer and GoBridgeDriver.

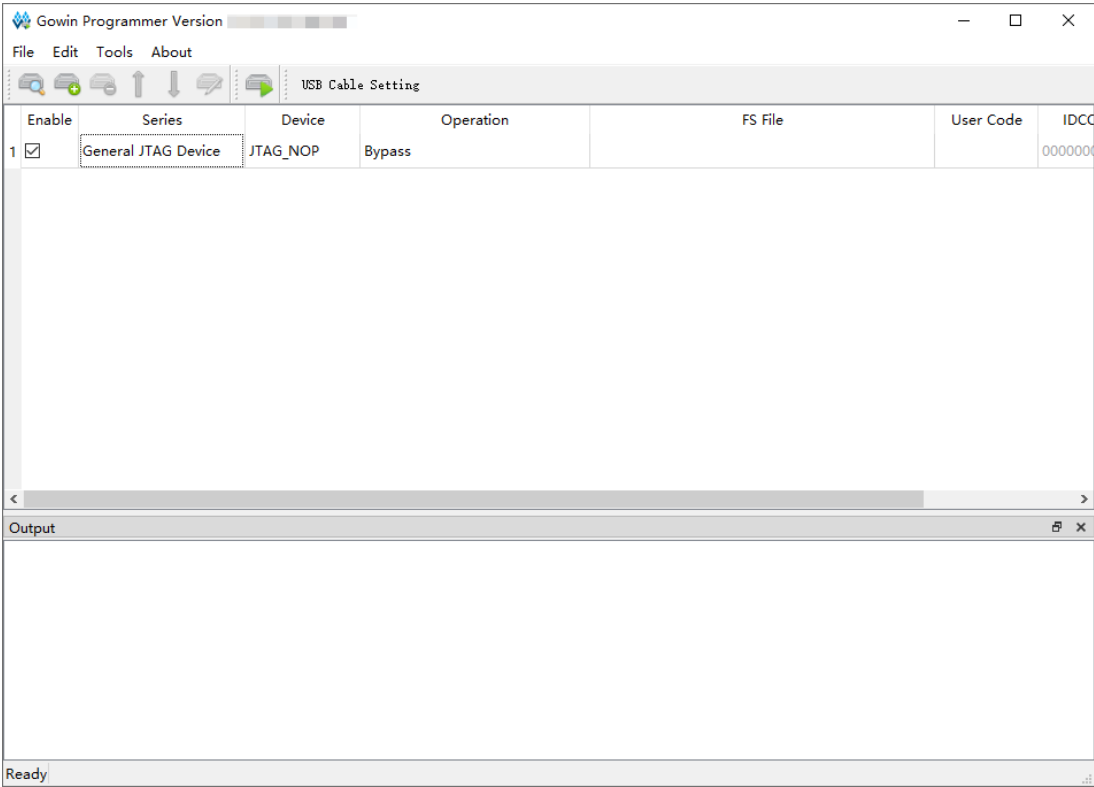
Table 5-12 Gowin Menu Items

Menu Items	Description
Programmer	Launch the download software Programmer
GoBridgeDriver	Launch the driver software GoBridgeDriver
SDK and Document	Official links to Gowin MCU-related SDK and user manuals

Programmer

"Gowin > Programmer" option, as shown in Figure 5-7, is used to launch the download software Programmer.

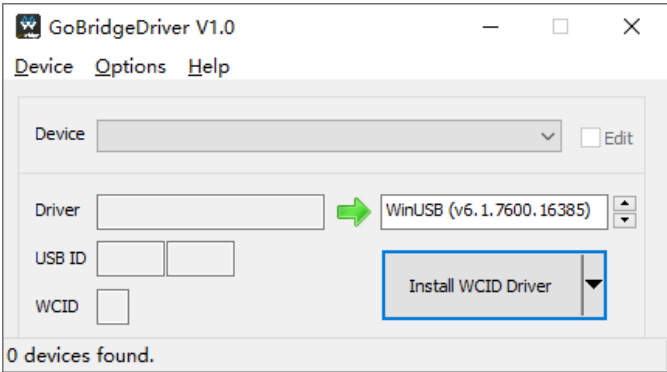
Figure 5-7 Programmer



GoBridgeDriver

"Gowin > GoBridgeDriver" option, as shown in Figure 5-8, is used to launch the driver software GoBridgeDriver.

Figure 5-8 GoBridgeDriver



5.2.10 Window Menu Items

Window menu items are listed in Table 5-13.

Table 5-13 Window Menu Items

Menu Items	Submenu Items	Description
New Window	-	Open a new workspace window

Menu Items	Submenu Items	Description
		using the current perspective, allowing simultaneous viewing and editing of multiple projects.
Editor	Toggle Split Editor (Horizontal)	Open a new editor instance horizontally within the current window
	Toggle Split Editor (Vertical)	Open a new editor instance vertically within the current window
	Clone	Duplicate a current editor instance in the same window
Appearance	Hide Toolbar	Hide the toolbar.
	Hide Status Bar	Hide the status bar.
	Toggle Full Screen	Switch to full-screen mode
	Maximize Active View or Editor	Maximize the currently active view or editor
	Minimize Active View or Editor	Minimize the currently active view or editor
Show View	Build Targets	Display the Build Targets view in the current perspective
	C/C++ Projects	Display the C/C++ Projects view in the current perspective
	Console	Display the Console view in the current perspective
	Documents	Display the Documents view in the current perspective
	Include Browser	Display the Include Browser view in the current perspective
	Navigator	Display the Navigator view in the current perspective
	Outline	Display the Outline view in the current perspective
	Problem Details	Display the Problem Details view in the current perspective
	Problems	Display the Problems view in the current perspective
	Project Explorer	Display the Project Explorer view in the current perspective

Menu Items	Submenu Items	Description
	Properties	Display the Properties view in the current perspective
	Search	Display the Search view in the current perspective
	Task List	Display the Task List view in the current perspective
	Tasks	Display the Tasks view in the current perspective
	Terminal	Display the Terminal view in the current perspective
	Other...	Display a list of all available view in the current perspective.
Perspective	Open Perspective	Open a new perspective
	Customize Perspective ...	Customize the visibility of menus, toolbars, and views in the current perspective.
	Save Perspective As...	Save the current layout and settings as a new perspective
	Reset Perspective...	Reset the current perspective to its default layout, discarding any customizations.
	Close Perspective	Close the current perspective and returns to the last active one
	Close All Perspectives	Close all opened perspectives and clear the workspace layout.
Navigation	Show System Menu	Display the system menu for the currently active view or editor.
	Show View Menu	Display the view menu for the currently active view
	Active Editor	Activate the editor and focus on the current active editor.
	Next Editor	Switch to the next opened editor
	Previous Editor	Switch to the previous opened editor.
	Switch to Editor...	List all opened editors and allow switching among them
	Next View	Switch to the next opened view

Menu Items	Submenu Items	Description
	Previous View	Switch to the previous opened view
	Next Perspective	Switch to the next opened perspective.
	Previous Perspective	Switch to the previous opened perspective.
Preferences	–	The Preferences settings for the current workspace, where users can configure the appearance, behavior, and plugin settings of the GMD software.

5.2.11 Help Menu Items

Help menu items are listed in Table 5-14.

Table 5-14 Help Menu Items

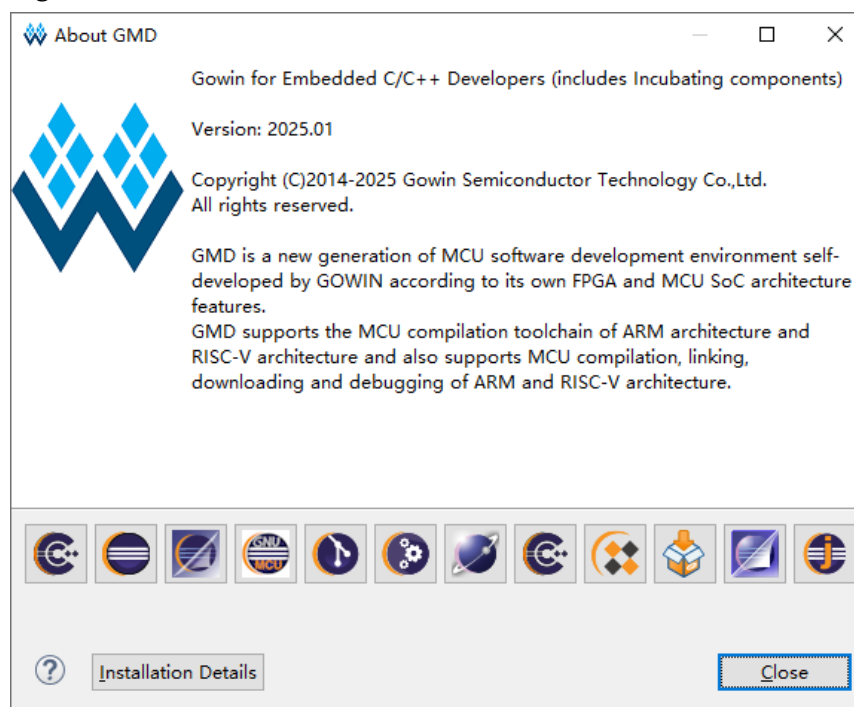
Menu Items	Description
Welcome	Open the Welcome page to help users understand the basic features of Eclipse.
Help Contents	Open the Eclipse help documentation with detailed descriptions of features and usage instructions.
Search	Searches for keywords within the help documentation
Show Context Help	Display context help
Show Active Keybindings...	Display a list of currently active and available keyboard shortcuts.
Tips and Tricks...	Provide helpful tips and best practices for using Eclipse efficiently.
Report Bug and Enhancement...	Guide users to report bugs or submit enhancement requests.
Cheat Sheets...	Offer interactive step-by-step guides to help users complete complex tasks
Eclipse User Storage	Allow secure, encrypted storage of sensitive information.
Performance Setup Tasks...	Tools for optimizing the performance of the IDE.
Check for Updates	Check for and install updates for Eclipse or installed plugins

Menu Items	Description
Install New Software...	Install new plugins or additional functions
Eclipse Marketplace...	Open the Eclipse Marketplace for easy browsing and installation of plugins.
About GMD	Display information about the current version of the GMD software

About GMD

The "Help > About GMD" option is as shown in Figure 5-9.

Figure 5-9 About GMD



5.3 Toolbar

The toolbar provides quick access to commonly used functions, including creating files or projects (New), saving files (Save, Save All), undo (Undo), redo (Redo), managing configurations (Manage Configurations), building projects (Build, Build All), opening terminals (Open a Terminal), programming (Programmer), debugging (Debug), running (Run), profiling (Profile), searching (Search), etc., as shown in Table 5-15.

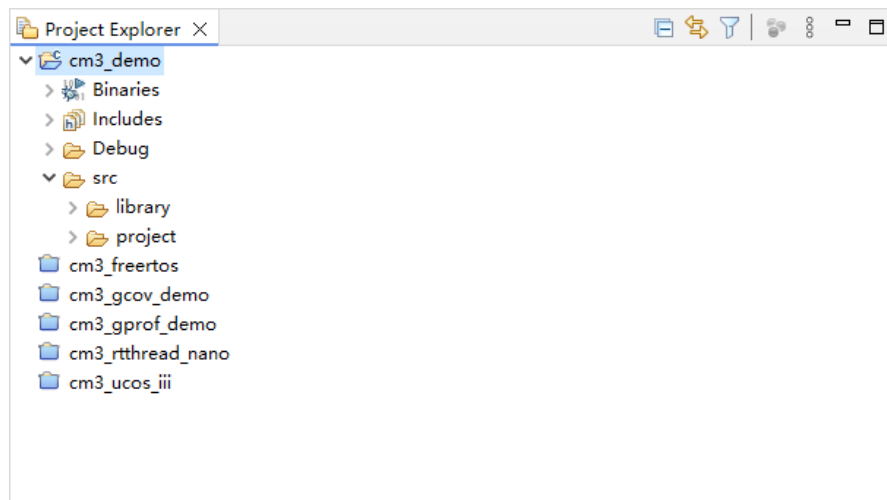
Table 5-15 Toolbar Option

Option	Icon	Description
New		Quickly create a new file or project
Save		Save the current file
Save All		Save all opened files
Manage configurations		Manage configurations (e.g., Debug and Release)
Build		Build the currently selected project
Build All		Build all opened projects in the workspace
Undo		Undo the most recent editing operation
Redo		Redo the most recently editing operation
Open a Terminal		Open a terminal (e.g., serial terminal)
Skip All Breakpoints		During debugging, all breakpoints will be marked as skipped, and the program will not pause at these breakpoints during execution
Restart		Restart the current method without terminating the entire debugging session
Debug		Launch a debugging session for the application
Run		Launch a run session for the application
Profile		Launch a performance profiling session for the application
Programmer		Launch the Programmer tool for downloading
Open Element		Open the detailed information of the element at the current cursor position
Open Task		Open the task list in the current workspace

Option	Icon	Description
Search		Search
Toggle Mark Occurrences		Highlight all occurrences of the selected element (e.g., variable, method and class)
Toggle Work Wrap		Enable or disable word wrap in the editor
Toggle Block Selection Mode		Select and edit text in a matrix mode
Show Whitespace Characters		Display whitespace characters in the editor
Next Annotation		Jump to the next annotation in the source code
Previous Annotation		Jump to the previous annotation in the source code
Last Edit Location		Quickly jump to the previously edited code location
Next Edit Location		Quickly jump to the next edited code location
Back		Navigate back to the previously viewed resource or view
Forward		Return to the resource or view that is last accessed after the "Back" operation
Pin Editor		Automatically close the editor
Open Perspective		Open a new perspective
C/C++		Switch to the C/C++ perspective
Debug		Switch to the Debug perspective.

5.4 Project Explorer

It mainly provides management and display functions for software engineering projects and their associated files, as shown in Figure 5-10.

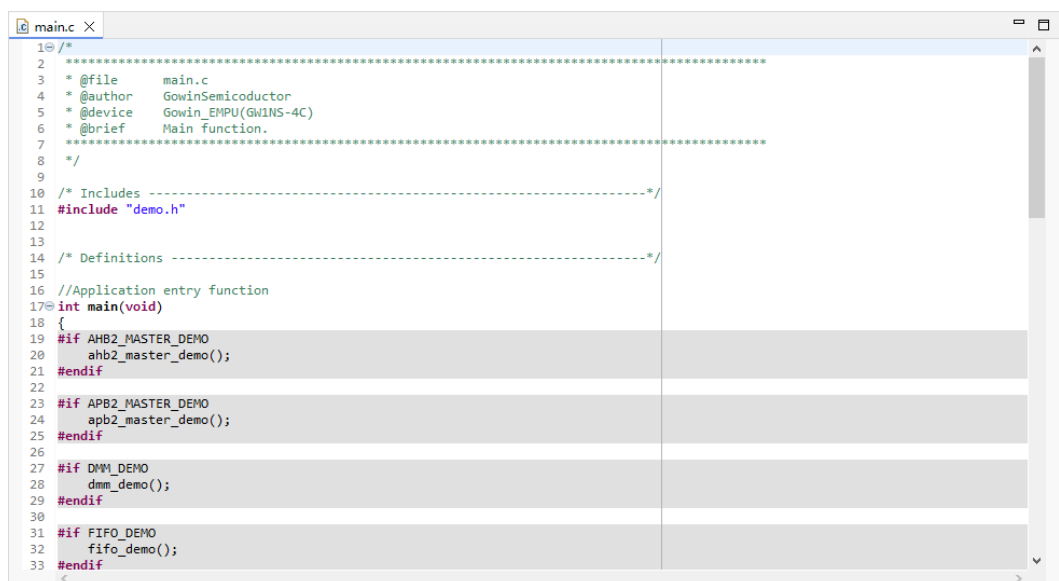
Figure 5-10 Project Explorer

5.5 Code Editor Area

It provides basic file editing and viewing functions, as shown in Figure 5-11.

Use the menu option "File > New > Source File" or "File > New > Header File" to create new source files (.c/.cpp) or header files (.h), which will then be displayed in the editor area.

Use "File > Close Editor" to close the currently displayed file in the editor, and "File > Close All Editors" is used to close all opened files in the editor.

Figure 5-11 Code Editor Area

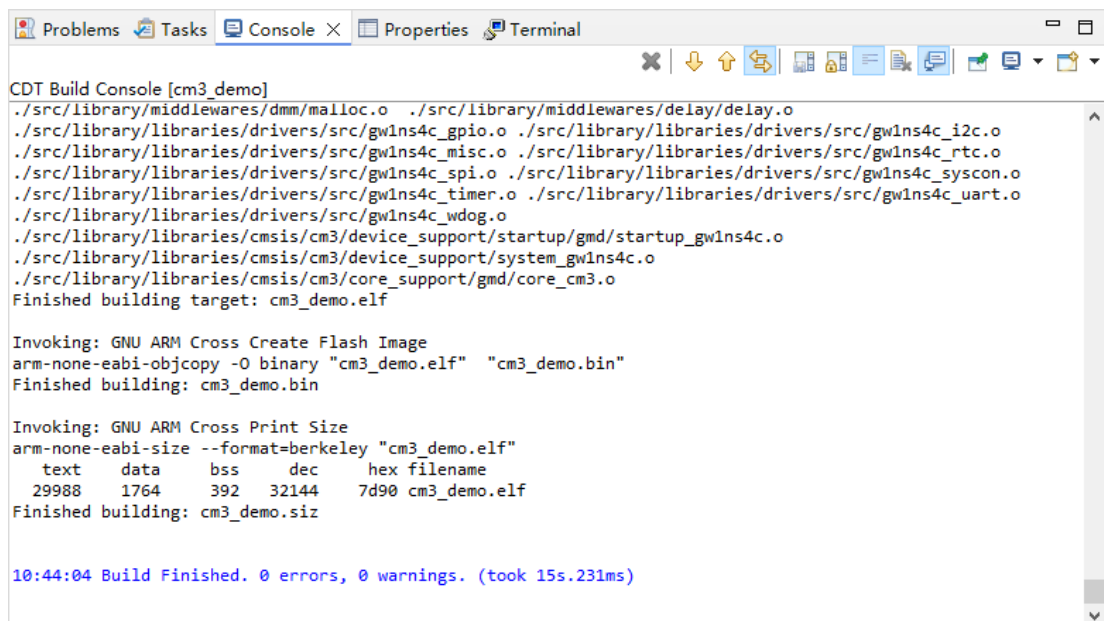
5.6 Console

It displays various processing messages generated during software execution. Users can manually switch between different views to inspect various output types, such as Console, Problems, Tasks, and Terminal, etc.

5.6.1 Console View

The console view, as shown in Figure 5-12, is used to display information such as building messages.

Figure 5-12 Console View



```

CDT Build Console [cm3_demo]
./src/library/middlewares/dmm/malloc.o ./src/library/middlewares/delay/delay.o
./src/library/libraries/drivers/src/gw1ns4c_gpio.o ./src/library/libraries/drivers/src/gw1ns4c_i2c.o
./src/library/libraries/drivers/src/gw1ns4c_misc.o ./src/library/libraries/drivers/src/gw1ns4c_rtc.o
./src/library/libraries/drivers/src/gw1ns4c_spi.o ./src/library/libraries/drivers/src/gw1ns4c_syscon.o
./src/library/libraries/drivers/src/gw1ns4c_timer.o ./src/library/libraries/drivers/src/gw1ns4c_uart.o
./src/library/libraries/drivers/src/gw1ns4c_wdog.o
./src/library/libraries/cmsis/cm3/device_support/startup/gmd/startup_gw1ns4c.o
./src/library/libraries/cmsis/cm3/device_support/system_gw1ns4c.o
./src/library/libraries/cmsis/cm3/core_support/gmd/core_cm3.o
Finished building target: cm3_demo.elf

Invoking: GNU ARM Cross Create Flash Image
arm-none-eabi-objcopy -O binary "cm3_demo.elf" "cm3_demo.bin"
Finished building: cm3_demo.bin

Invoking: GNU ARM Cross Print Size
arm-none-eabi-size --format=berkeley "cm3_demo.elf"
   text    data     bss     dec     hex filename
 29988    1764     392   32144   7d90 cm3_demo.elf
Finished building: cm3_demo.siz

10:44:04 Build Finished. 0 errors, 0 warnings. (took 15s.231ms)
  
```

5.6.2 Problems View

The Problems view, as shown in Figure 5-13, is used to display warnings and error messages reported during the building process.

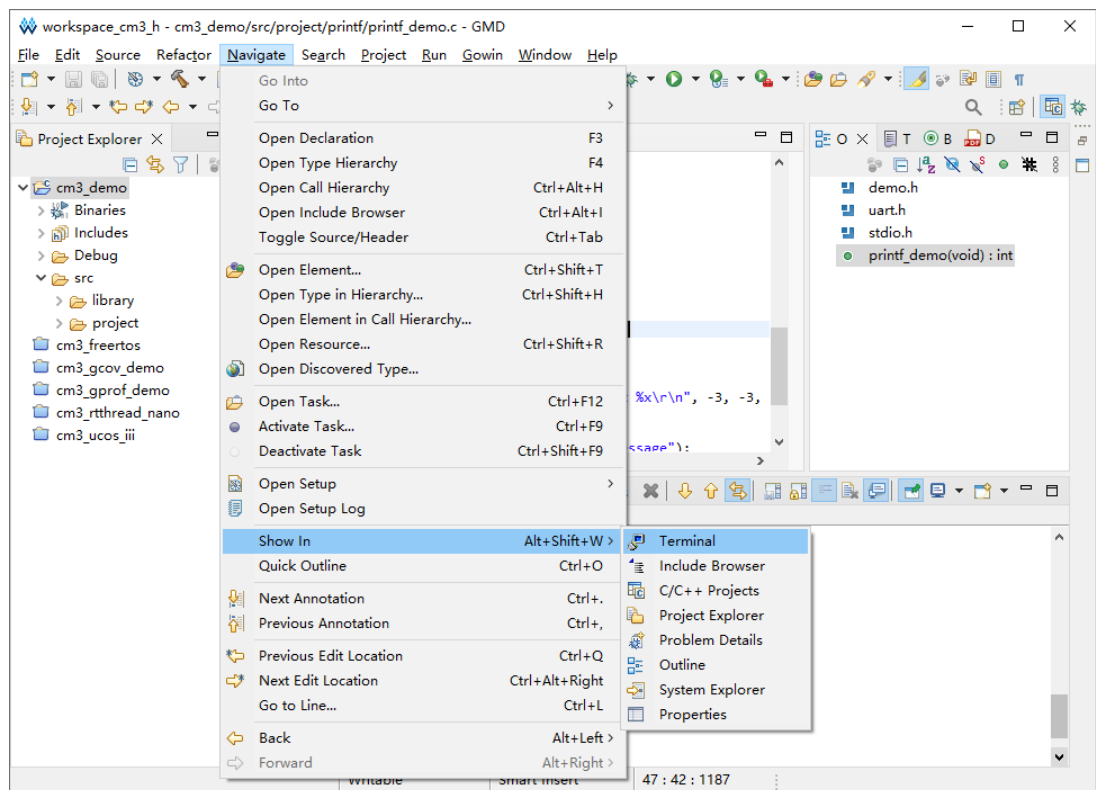
Figure 5-13 Problems View

Description	Resource	Path	Location	Type
✖ Errors (3 items)				
✖ 'mi' undeclared (first use in this function);	printf_demo.c	/cm3_demo/src/p...	line 43	C/C++ Prob...
✖ expected '=', ',', ';', 'asm' or '__attribute__'	printf_demo.c	/cm3_demo/src/p...	line 43	C/C++ Prob...
✖ make: *** [src/project/printf/subdir.mk:20:	cm3_demo			C/C++ Prob...
ℹ Infos (1 item)				
ℹ each undeclared identifier is reported onl	printf_demo.c	/cm3_demo/src/p...	line 43	C/C++ Prob...

5.6.3 Terminal View

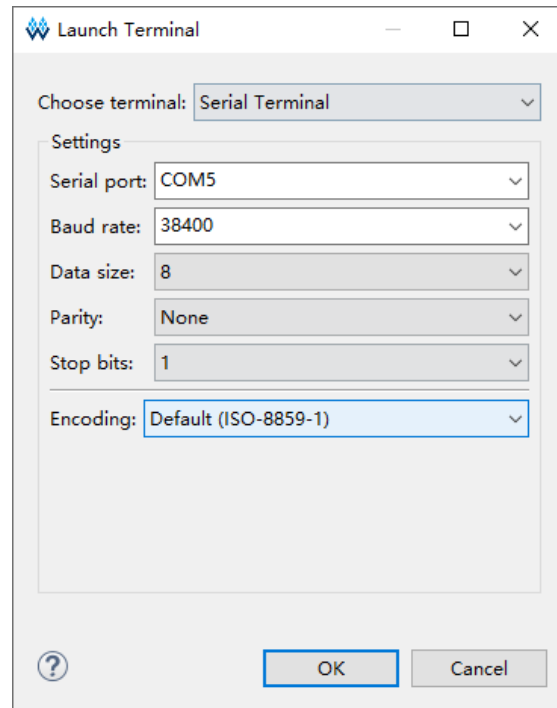
The "Navigate > Show In > Terminal" is used to launch the terminal, as shown in Figure 5-14.

Figure 5-14 Launch Terminal



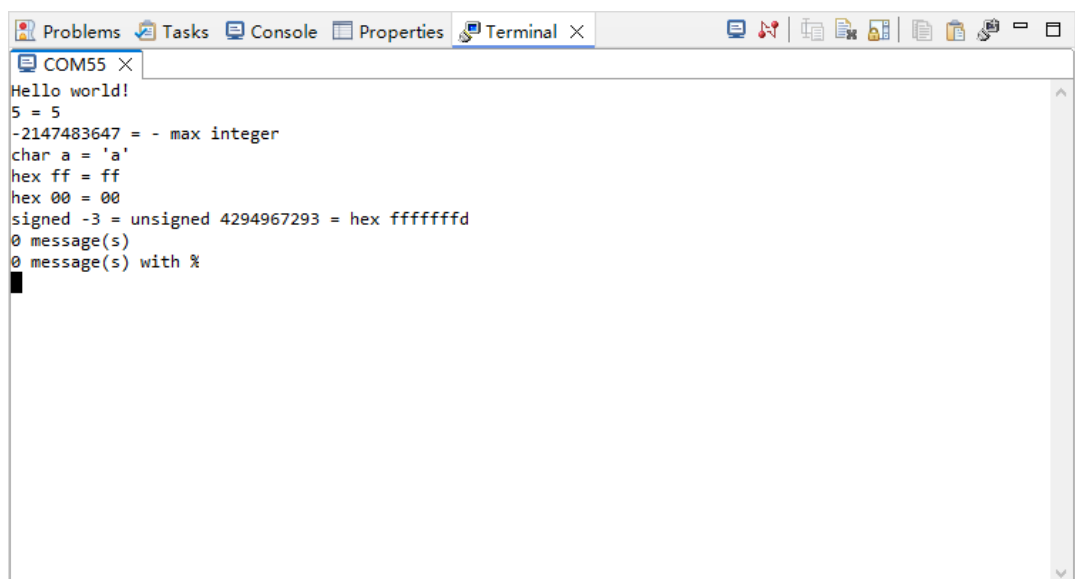
You can configure various types of terminals such as Local Terminal, SSH Terminal, Serial Terminal, and Telnet Terminal. A commonly used Serial Terminal is as shown in Figure 5-15.

Figure 5-15 Serial Terminal



The serial terminal outputs serial debug print messages, as illustrated in Figure 5-16.

Figure 5-16 Serial Terminal Output

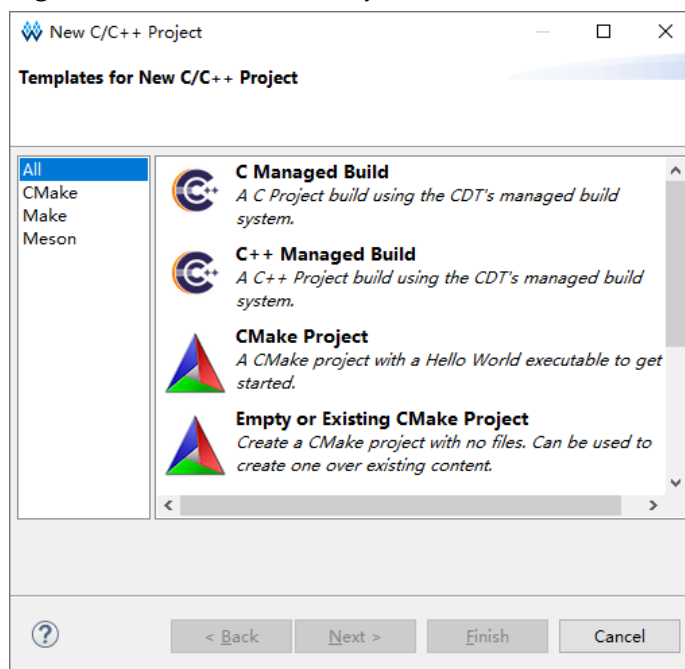


6 Create a Project

6.1 Create a New Project

To create a new C/C++ project, you can select "File > New > C/C++ Project" from the menu bar or "New > C/C++ Project" () from the toolbar, as shown in Figure 6-1.

Figure 6-1 Create a New Project



Commonly used project types include "C Managed Build" and "C++ Managed Build", which are used to create C and C++ projects, respectively. Please choose based on your actual development needs.

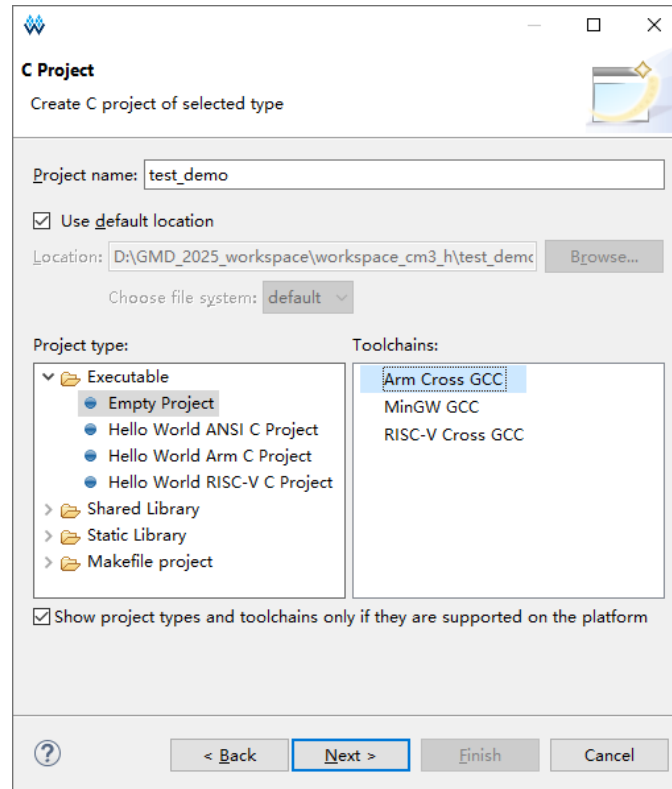
For example, when creating a C project, the steps for creating a new project for ARM and RISC-V MCUs are described separately below.

6.1.1 Create a New Project (ARM MCU)

1. Select project type and toolchain

Set the project name and location, then choose the project type and software toolchain, as shown in Figure 6-2.

Figure 6-2 Select Project Type and Toolchain

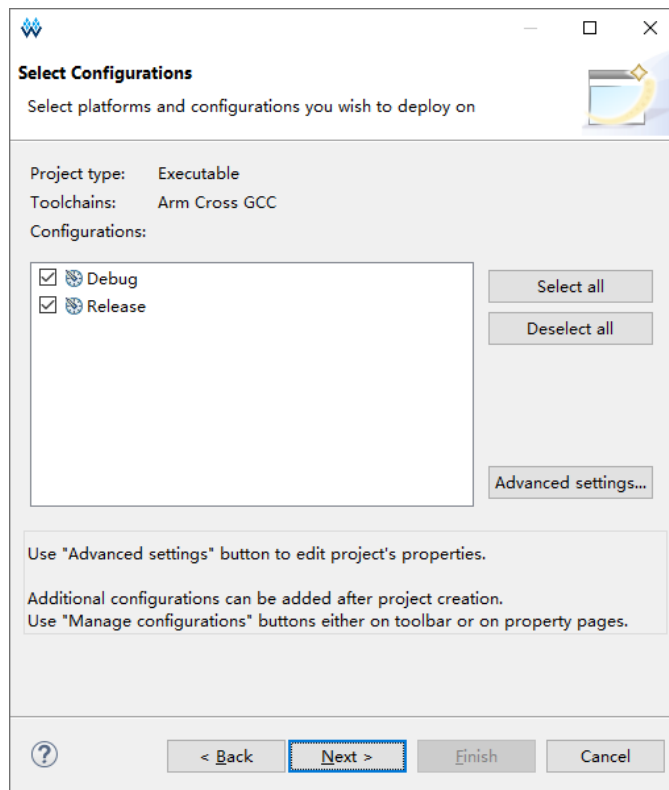


For example:

- Project name: test_demo
- Location: Use default location, (i.e., the current workspace)
- Project type: Empty Project
- Toolchains: ARM Cross GCC

2. Select platform and configuration

Choose the build platform and configurations, including "Debug" and "Release", as shown in Figure 6-3.

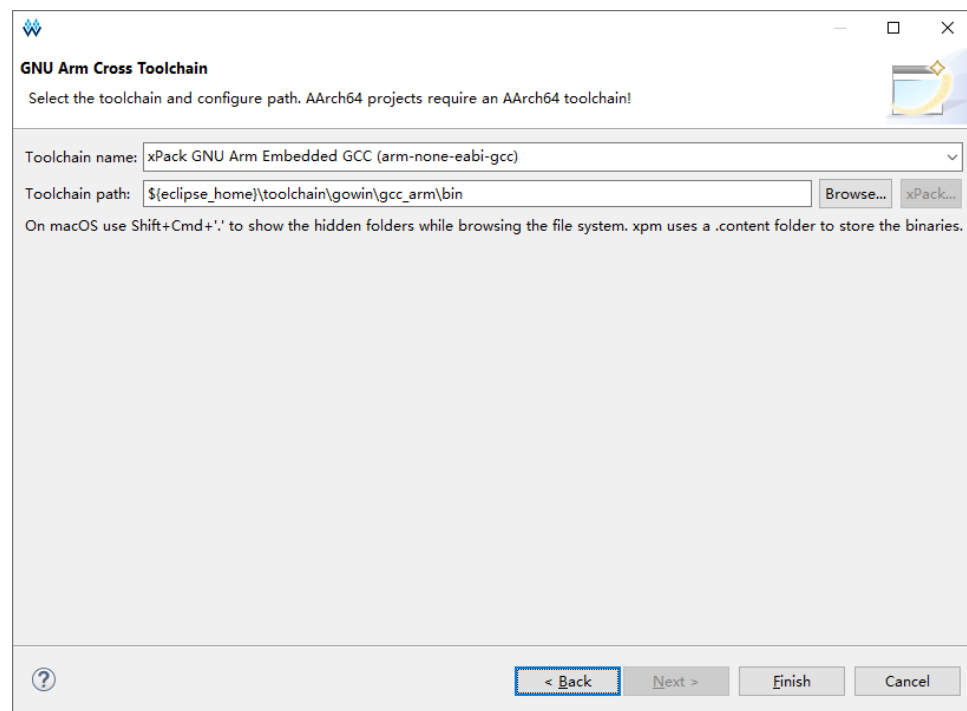
Figure 6-3 Select Platform and Configuration

For example:

Configurations: Check both Debug and Release.

3. Select software toolchain and path

Select the specific ARM cross-compiler toolchain and its installation path, as shown in Figure 6-4. GMD software comes with the toolchain pre-integrated and the path pre-configured, so in most cases, the default selection can be used.

Figure 6-4 Select Software Toolchain and Path

For example:

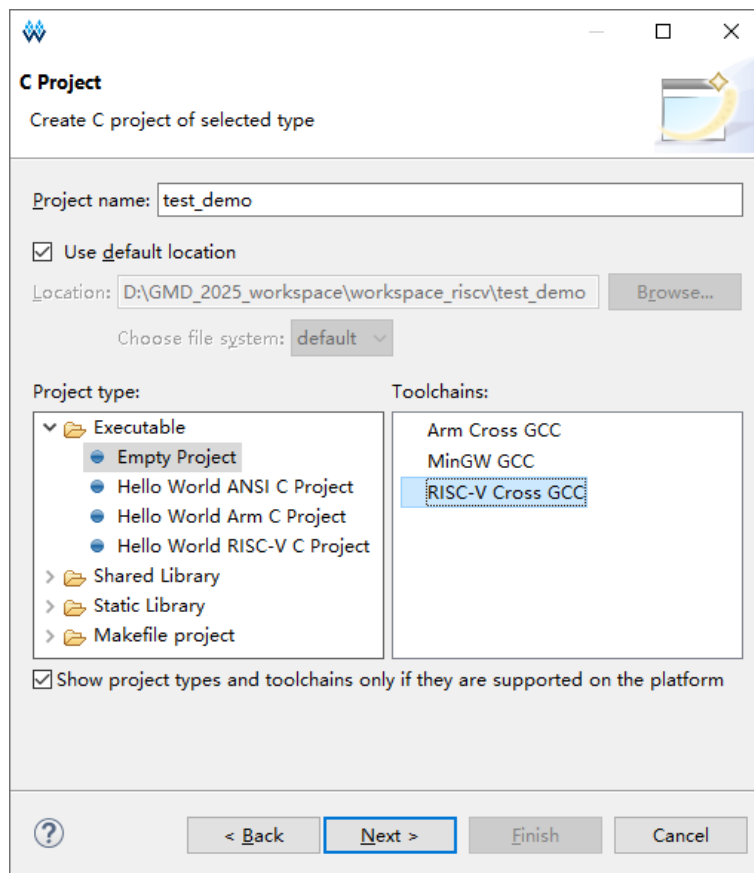
- Toolchain name: xPack GNU ARM Embedded GCC (arm-none-eabi-gcc)
- Toolchain path: \${eclipse_home}\toolchain\gowin\gcc_arm\bin

6.1.2 Create a New Project (RISC-V MCU)

1. Select Project Type and Toolchain

Set the project name and location, then select the project type and software toolchain, as shown in Figure 6-5.

Figure 6-5 Select Project Type and Toolchain



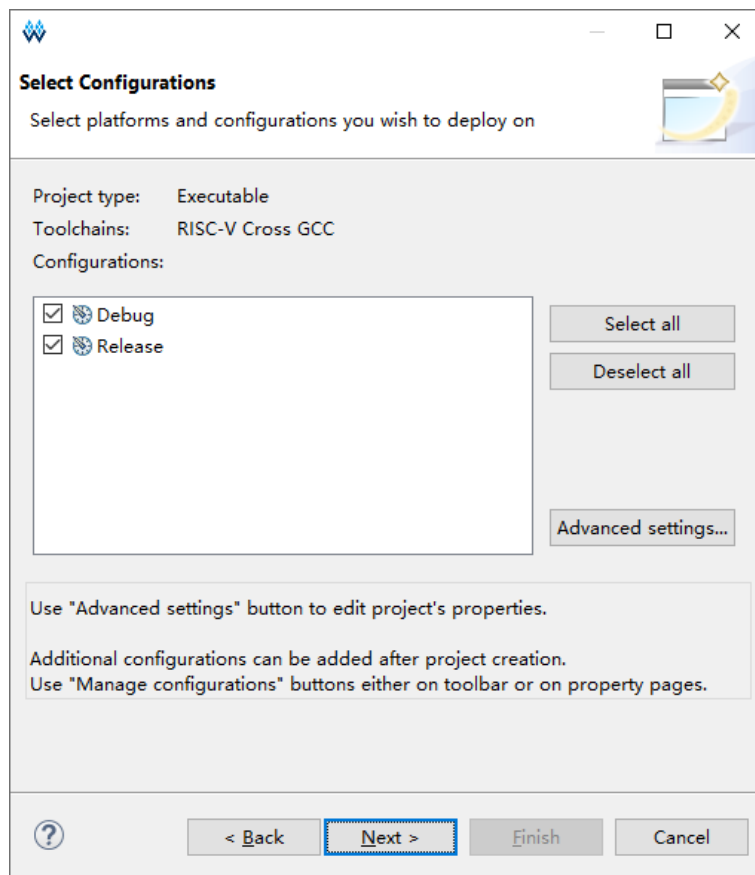
For example:

- Project name: test_demo
- Location: Use default location, (i.e., use the current workspace)
- Project type: Empty Project
- Toolchains: RISC-V Cross GCC

2. Select platform and configuration

Choose the platform and configurations, including "Debug" and "Release", as shown in Figure 6-6.

Figure 6-6 Select Platform and Configuration

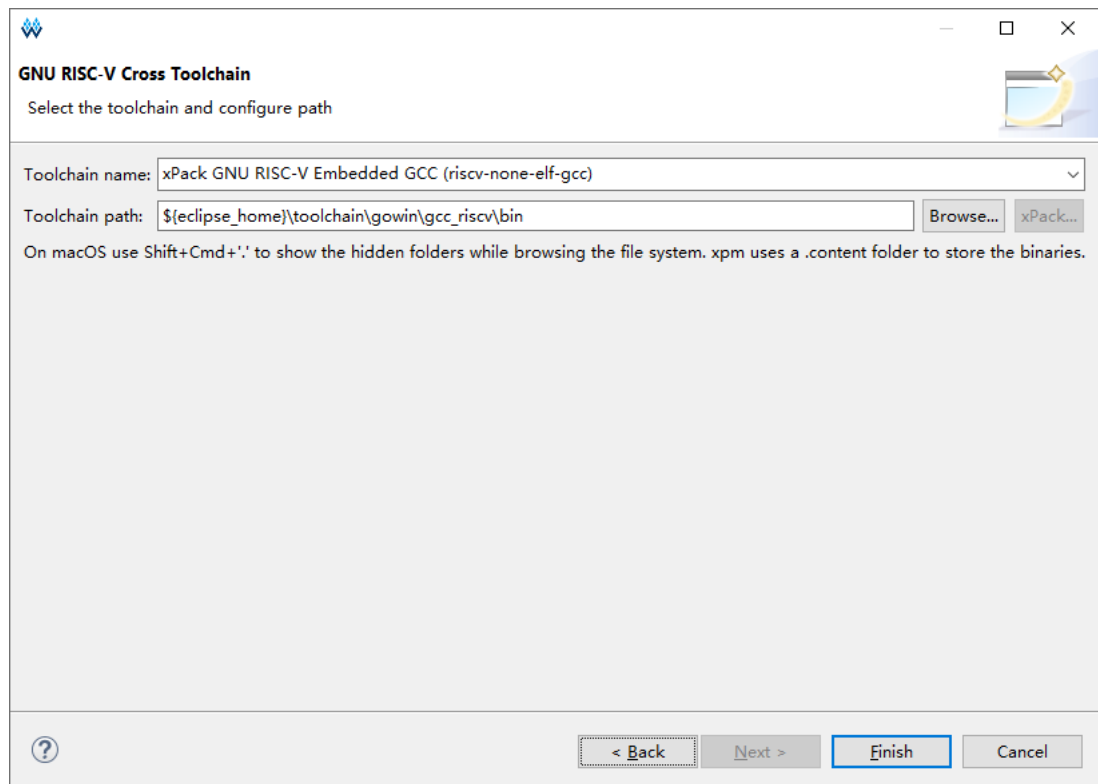


For example:

Configurations: Check both Debug and Release.

3. Select software toolchain and path

Select the specific RISC-V cross-compiler toolchain and its installation path, as shown in Figure 6-7. GMD software comes with the toolchain pre-integrated and the path pre-configured, so in most cases, the default selection can be used.

Figure 6-7 Select Software Toolchain and Path

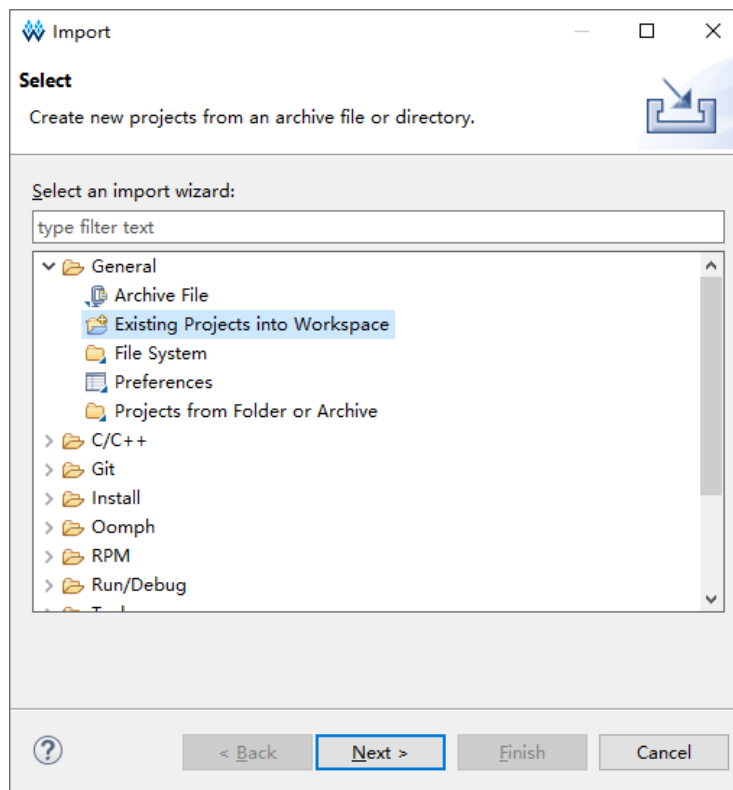
For example:

- Toolchain name: xPack GNU RISC-V Embedded GCC (riscv-none-elf-gcc)
- Toolchain path: `$(eclipse_home)\toolchain\gowin\gcc_riscv\bin`

6.2 Import an Existing Project

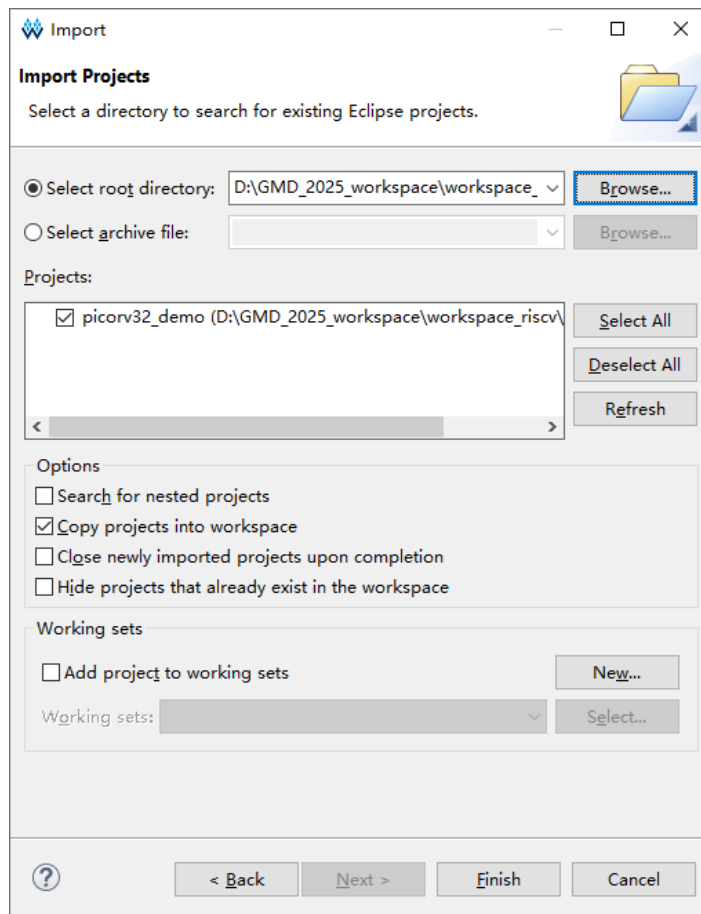
Gowin provides reference designs for all MCU products. Users can directly import and use them. The steps to import an existing project are as follows:

1. Select "File > Import..." to begin importing an existing project.
2. Choose "Existing Projects into Workspace", as shown in Figure 6-8.

Figure 6-8 Select an Import Wizard

3. Select the existing project to import and configure the import options

Click "Browse" to select the path of the project you want to import, as shown in Figure 6-9. Once the project is successfully recognized by the IDE, you can check the option "Copy projects into workspace" to copy the imported project into the current workspace.

Figure 6-9 Select the Existing Project to Import

7 Import Projects Created in Legacy GMD Versions

Starting from GMD 2025.01, the software toolchain has undergone significant updates. If users want to use projects created with GMD V1.2 or earlier, or use older versions of Gowin MCU SDKs, modifications are required as described in this section.

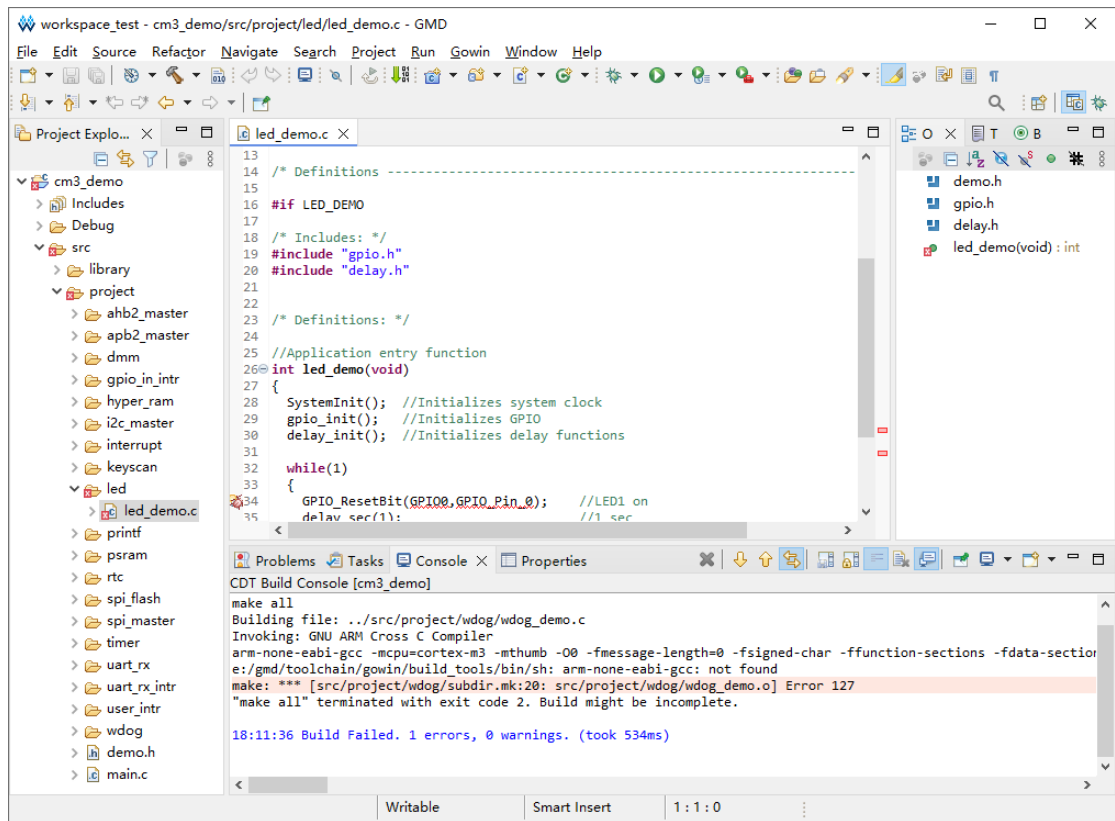
When importing projects created using older versions of GMD into GMD 2025.01 or later, or when using legacy MCU SDKs (i.e., SDKs released prior to GMD 2025.01), build errors may occur due to the project's dependency on outdated GCC toolchains. As a result, the software project may fail to build or function properly.

Therefore, follow the instructions below to reconfigure the toolchains for imported ARM MCU and RISC-V MCU projects.

7.1 Modify Imported ARM MCU Projects

1. Follow the instructions in Section [6.2 Import an Existing Project](#) to import an ARM MCU project created with an earlier version of GMD.
2. Upon building the project, build errors may occur as shown in Figure 7-1, because the old toolchain cannot be found.

Figure 7-1 Build Error After Importing ARM Project

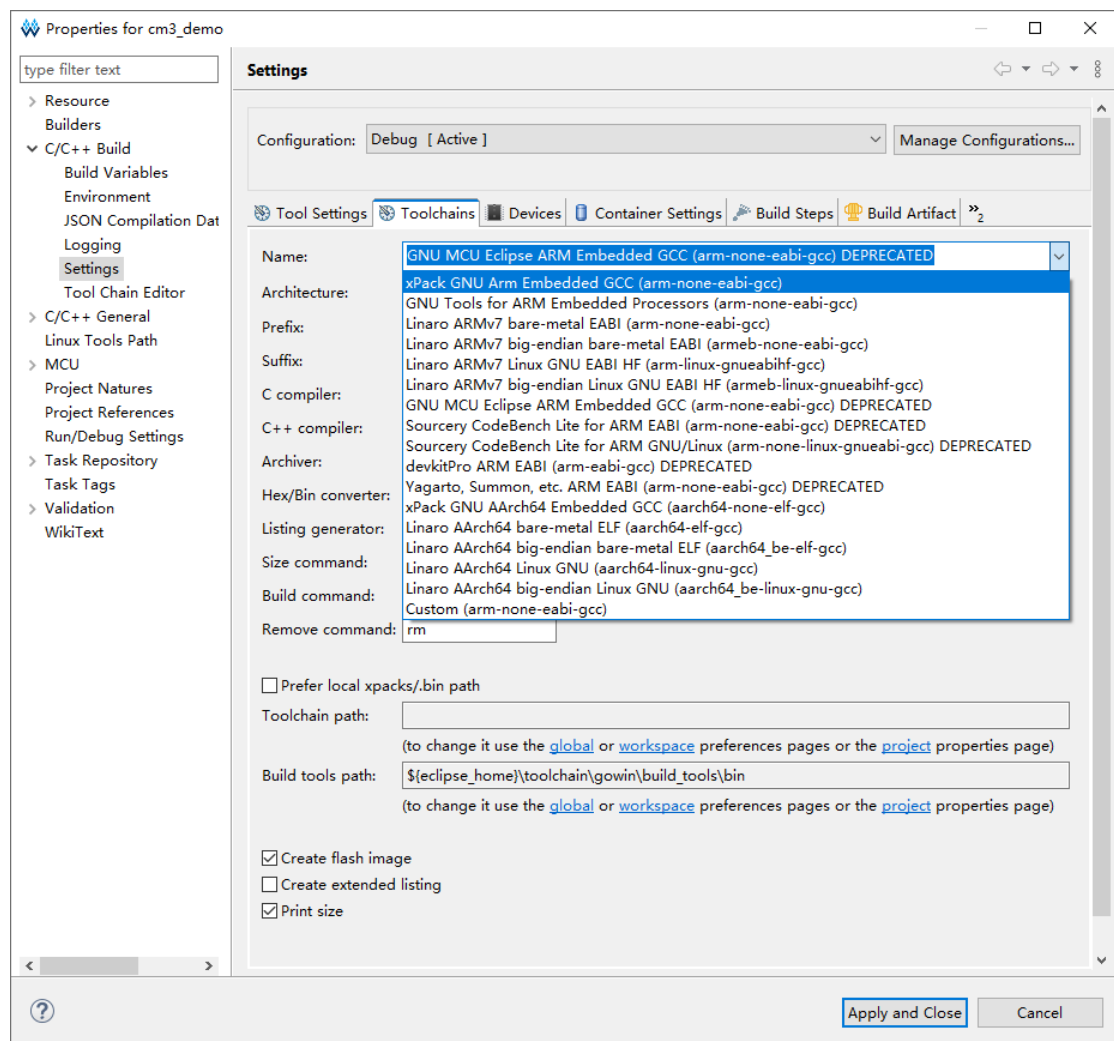


3. Update the toolchain configuration and select a valid ARM toolchain.

Right-click the current project and choose "Properties > C/C++ Build > Settings > Toolchains", then change the toolchain.

As shown in Figure 7-2, select "xPack GNU ARM Embedded GCC (arm-none-eabi-gcc)" from the "Name" dropdown list.

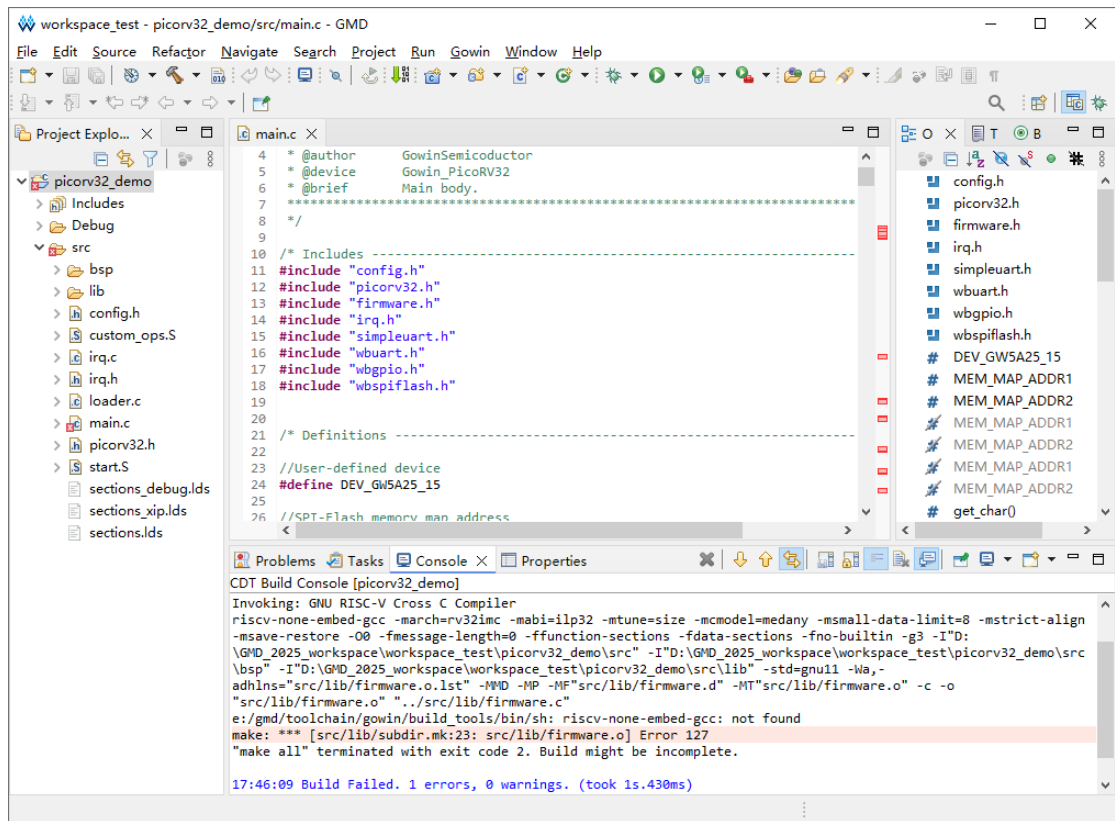
Figure 7-2 Toolchain Settings



7.2 Modify Imported RISC-V MCU Projects

1. Follow the instructions in Section [6.2 Import an Existing Project](#) to import a RISC-V MCU created with an earlier version of GMD.
2. Upon building the project, build errors may occur as shown in Figure 7-3, because the old toolchain cannot be found.

Figure 7-3 Build Error After Importing RISC-V Project

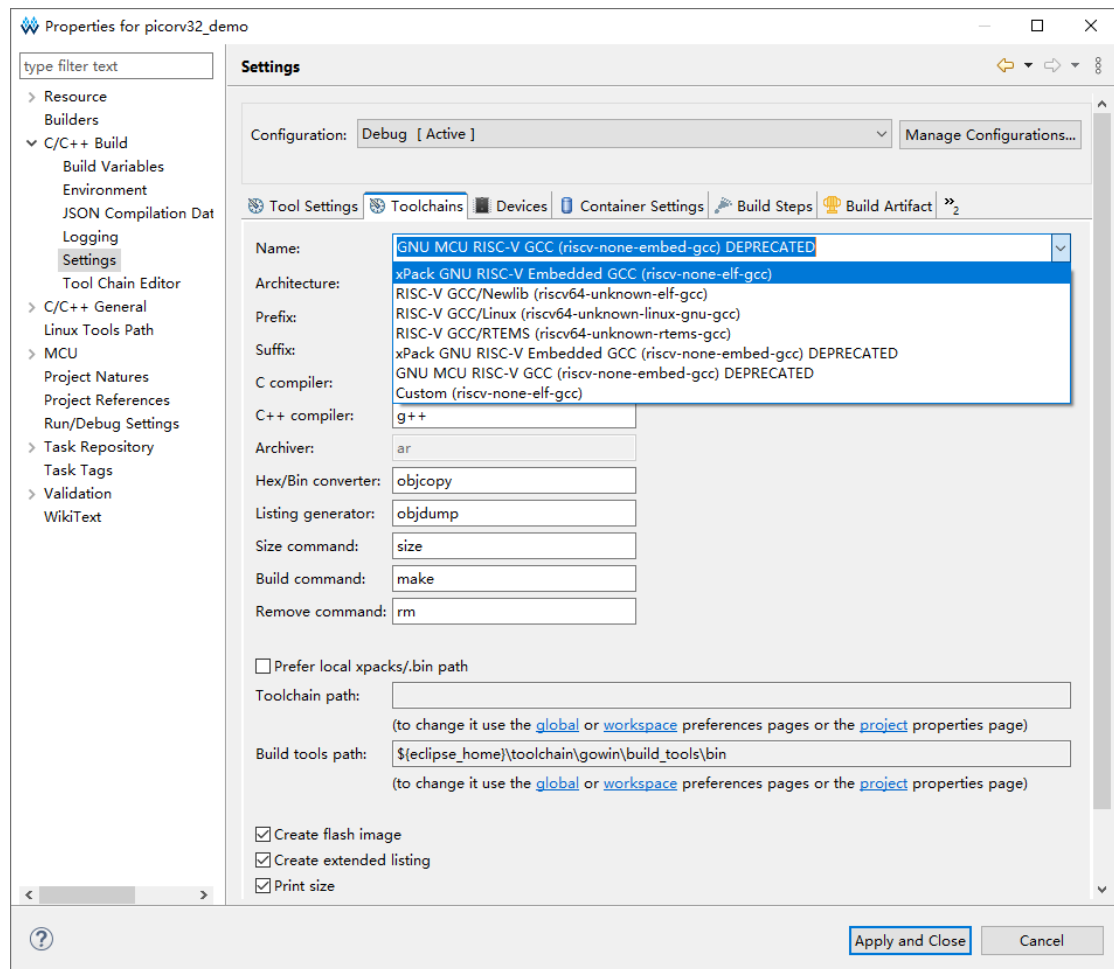


3. Update the toolchain configuration and select a valid RISC-V toolchain.

Right-click the current project and choose "Properties > C/C++ Build > Settings > Toolchains", then change the toolchain.

As shown in Figure 7-4, select "xPack GNU RISC-V Embedded GCC (riscv-none-elf-gcc)" from the "Name" dropdown list.

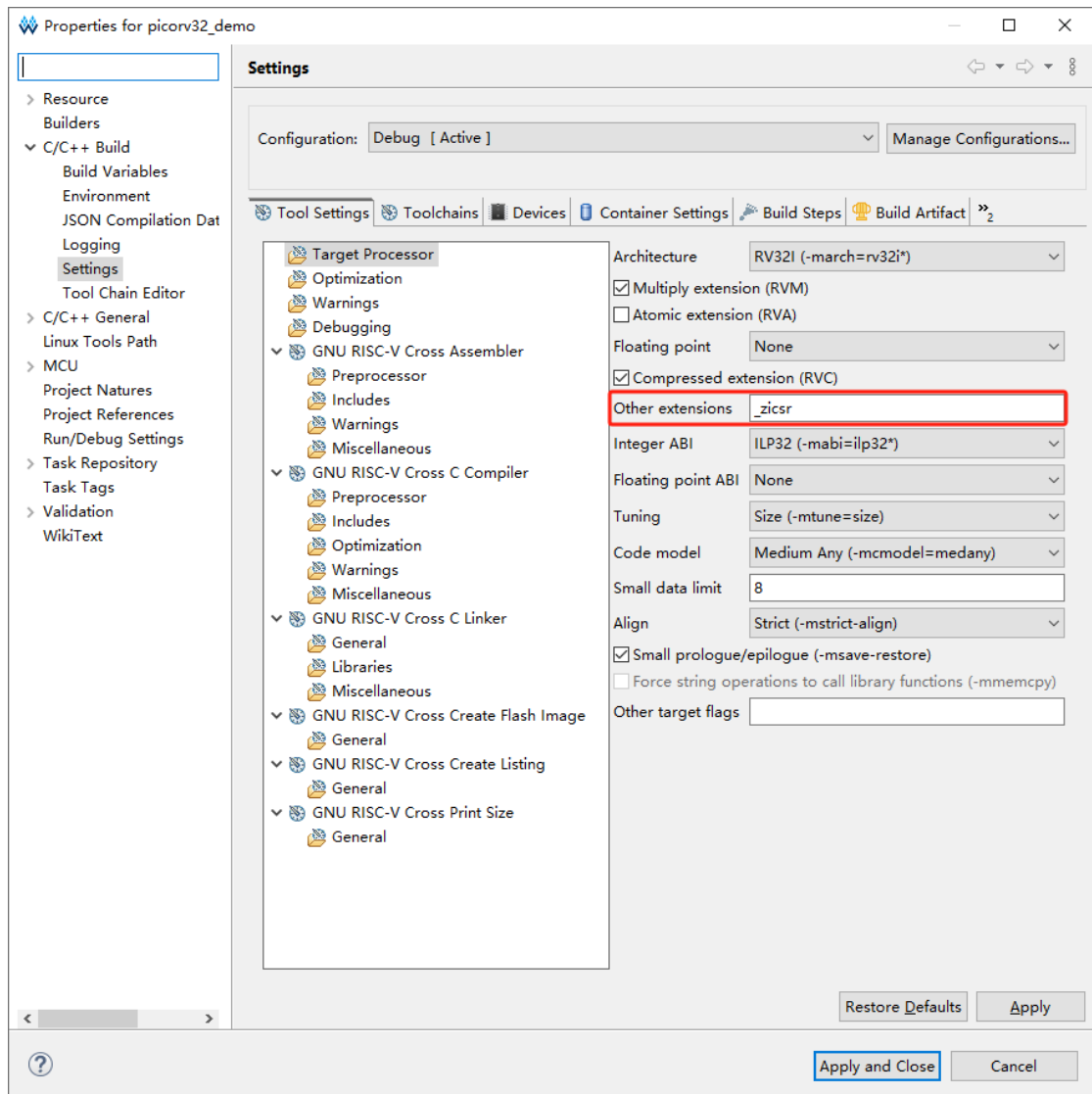
Figure 7-4 RISC-V Toolchain Settings



4. Modify the build settings and enter the "_zicsr" in "Other extensions" field.

Right-click the current project and choose "Properties > C/C++ Build > Settings > Tool Settings > Target Processor". In the "Other extensions" field, enter "_zicsr", as shown in Figure 7-5.

Figure 7-5 Build Option Settings



8 Build Projects

GMD software integrates multiple toolchains and supports project building for both ARM and RISC-V MCUs.

8.1 Toolchains

GMD is integrated with Gowin GNU toolchains, which are custom-developed by Gowin based on the standard GNU releases to better align with the characteristics of Gowin's MCU products. The GNU toolchain is a comprehensive software development suite provided by the GNU Project. It includes compilers, debuggers, linkers, libraries, and other essential tools for software development and build processes. Renowned for being open-source, cross-platform, highly customizable, and powerful, the GNU toolchain is widely adopted by the global developer community.

When creating a project in GMD, If the developer selects an ARM MCU project, the toolchain "xPack GNU ARM Embedded GCC (arm-none-eabi-gcc)" can be used, as shown in Figure 8-1. If the developer selects a RISC-V MCU project, the toolchain "xPack GNU RISC-V Embedded GCC (riscv-none-elf-gcc)" can be used, as shown in Figure 8-2.

Figure 8-1 ARM Toolchain

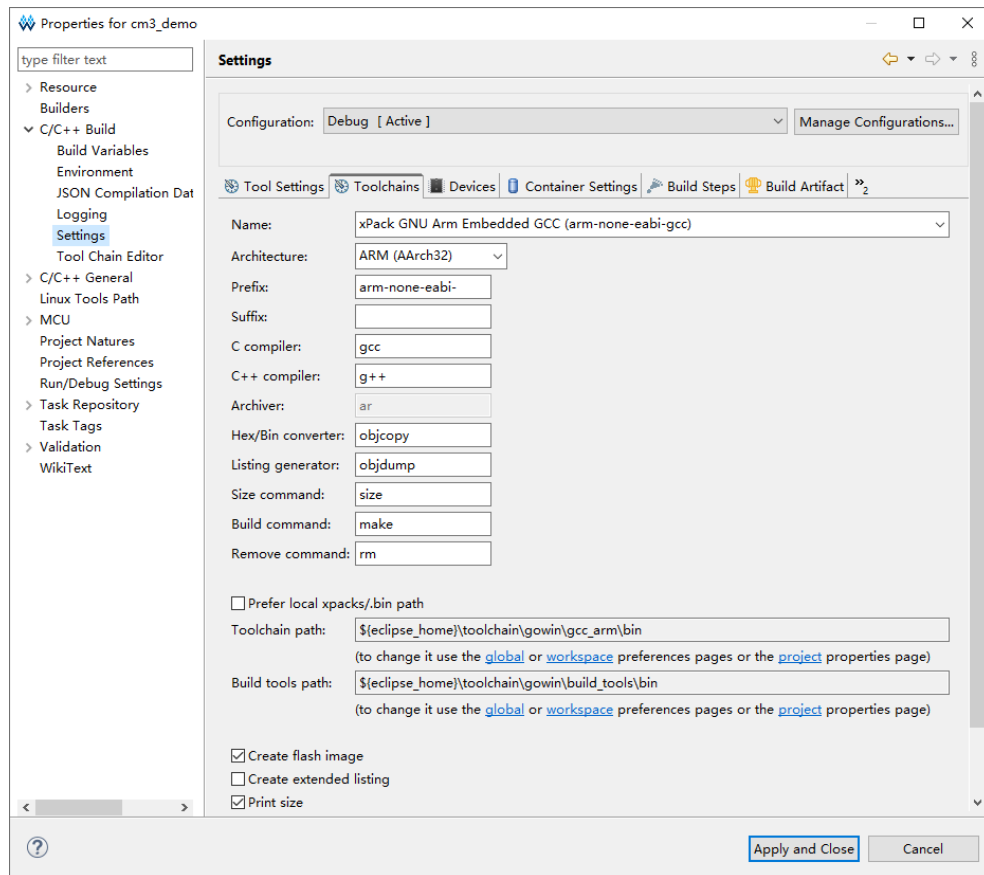
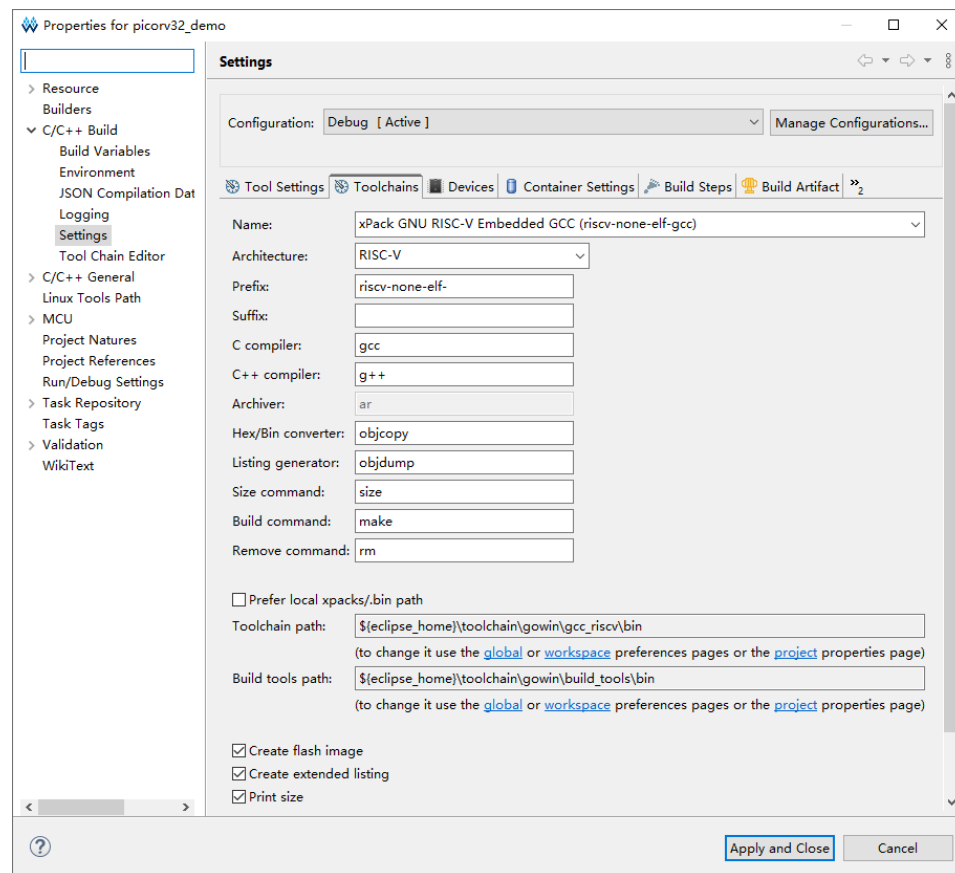


Figure 8-2 RISC-V Toolchain



8.2 Project Configuration

After creating a project and before building it, you need to configure building and linking options based on your application requirements.

Right-click the current project and choose "Properties > C/C++ Build > Settings > Tool Settings" to configure the project. For example, Figure 8-3 shows the configuration for an ARM MCU project. Figure 8-4 shows the configuration for a RISC-V MCU project.

Figure 8-3 ARM MCU Project Configuration

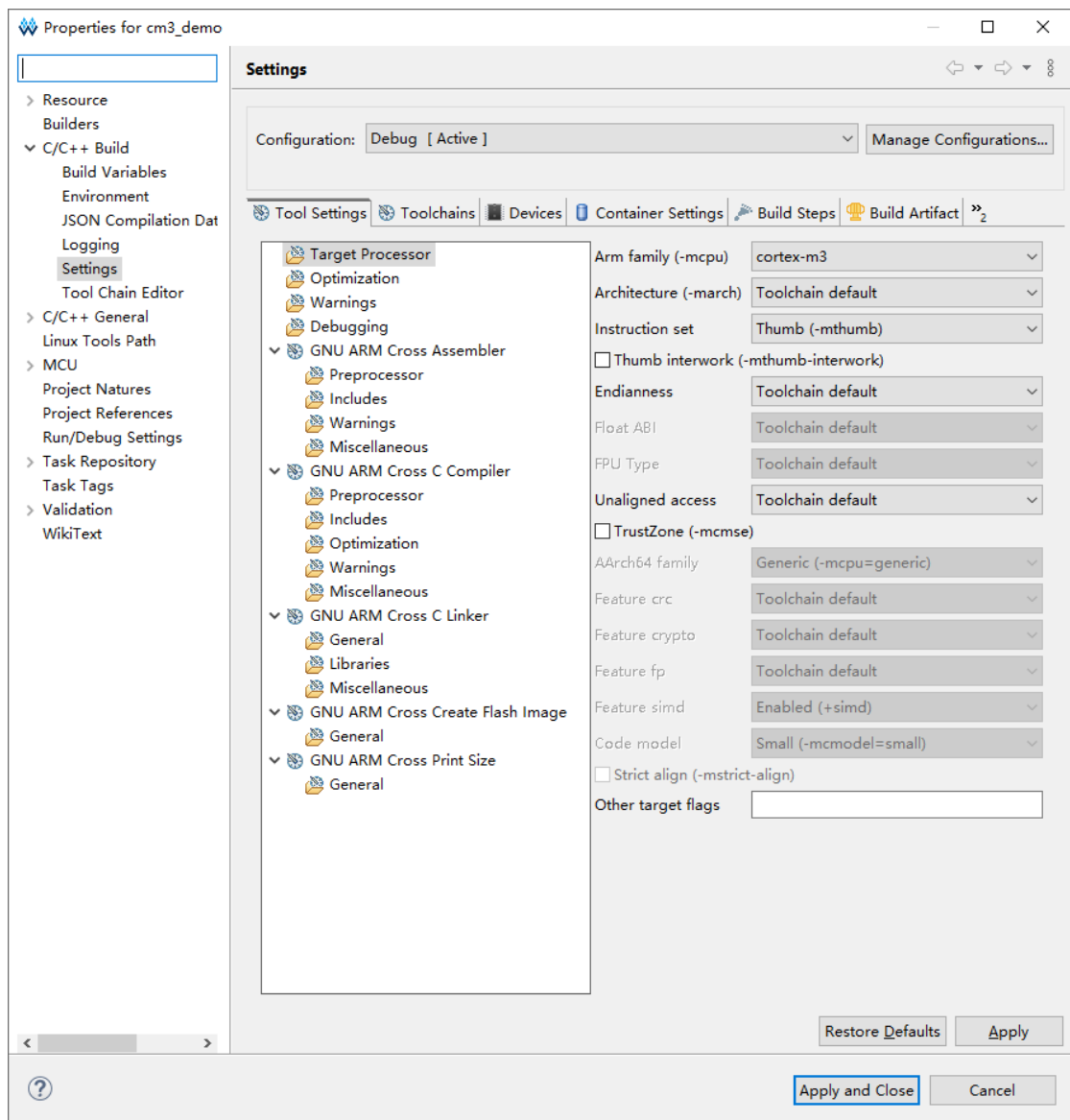
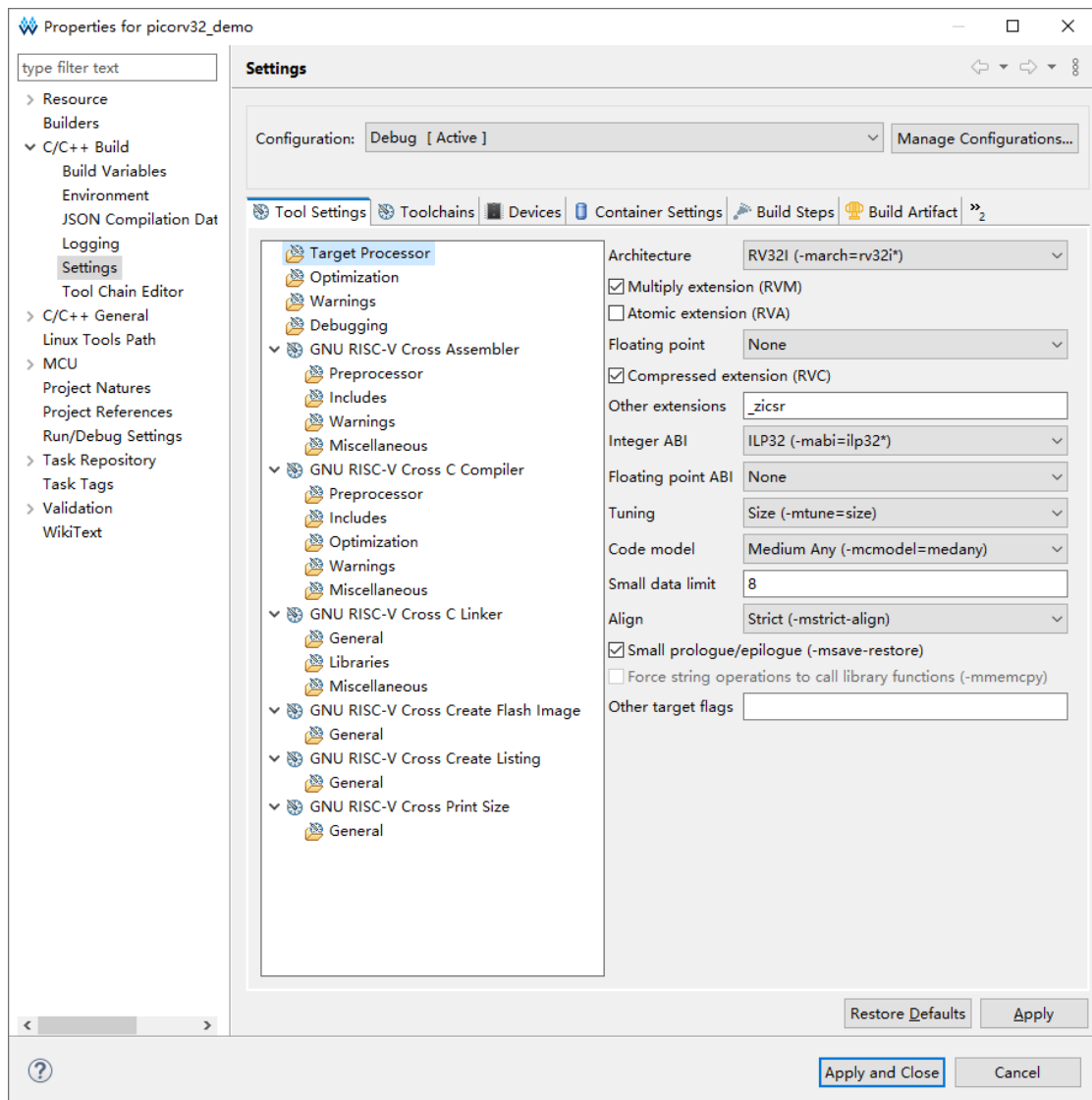


Figure 8-4 RISC-V MCU Project Configuration

The configuration steps for ARM MCU and RISC-V MCU projects are described below.

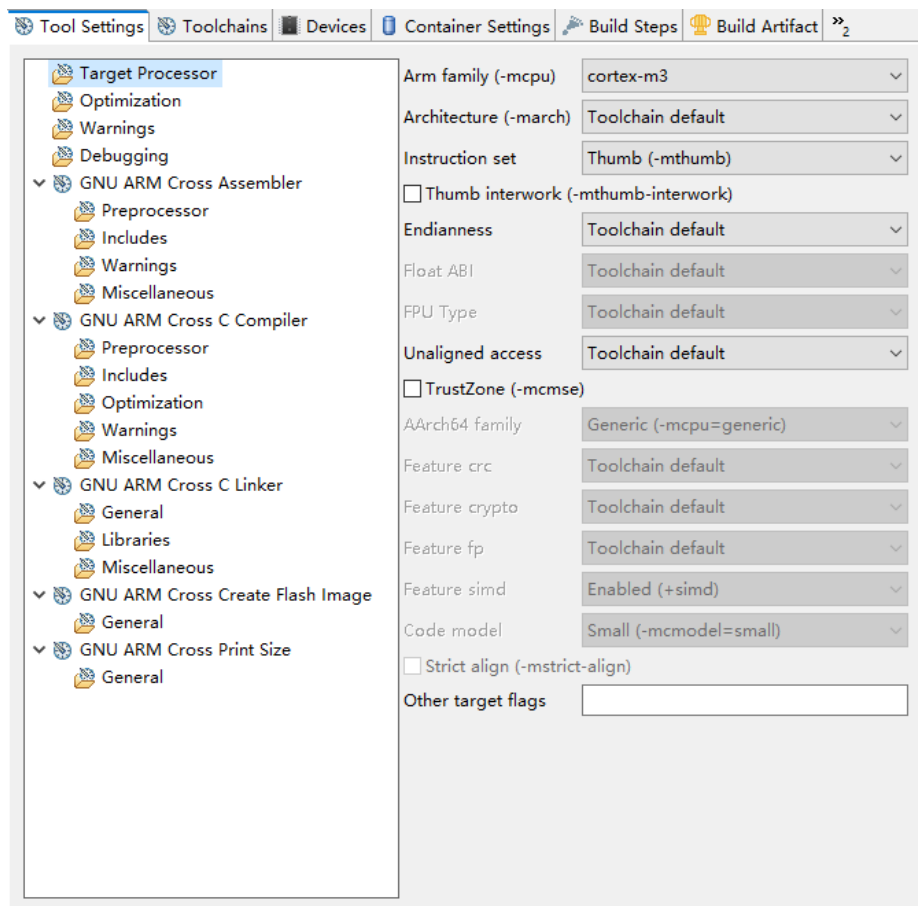
8.2.1 ARM MCU Project Configuration

Take Gowin_EMPU (GW1NS-4C) as an example for description.

Target Processor

Configure the ARM target processor as shown in Figure 8-5.

Figure 8-5 Target Processor



The settings are described in Table 8-1.

Table 8-1 Target Processor

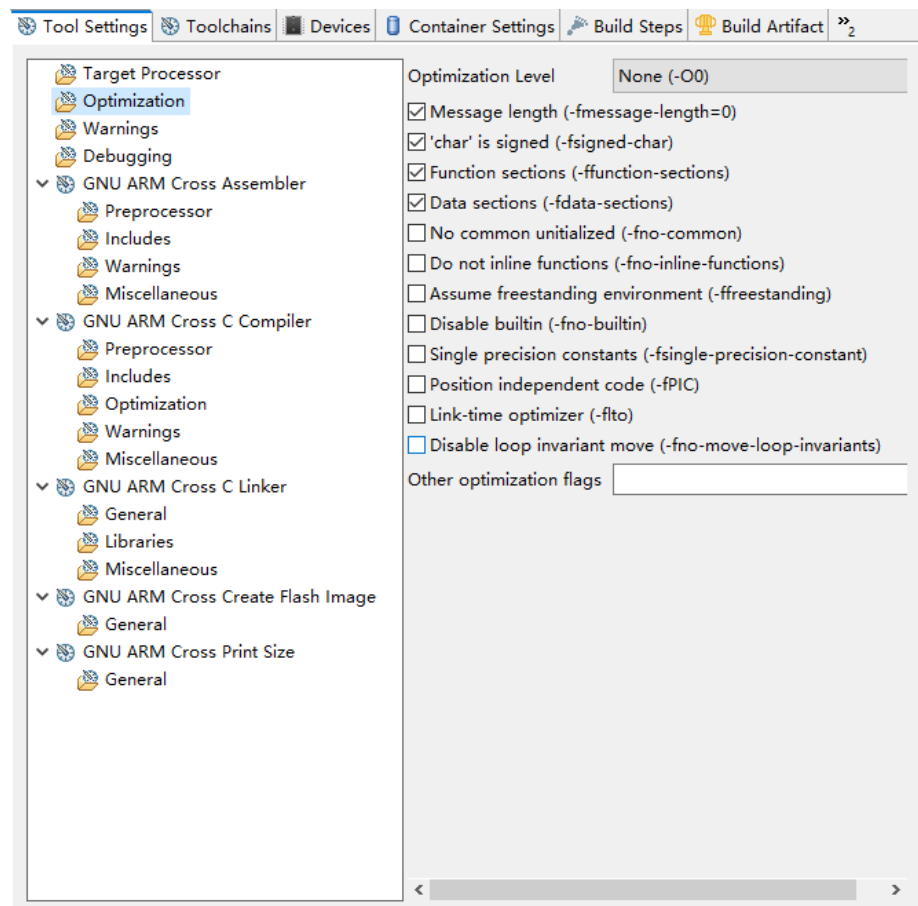
Option	Setting	Description
ARM family (-mcpu)	cortex-m3	Specify the ARM; Cortex-M1/M3/M4 are supported.
Architecture (-march)	Toolchain default	ARM architecture
Instruction set	Thumb (-mthumb)	ARM instruction set
Thumb interwork (-mthumb)	-	Thumb interworking, allowing mixing of 32-bit and 16-bit Thumb instruction sets within the same executable.
Endianness	Toolchain default	Specify the byte order (e.g., Little Endian or Big Endian).
Float ABI	-	Support for a floating-point ABI
FPU Type	-	Support for hardware floating-point
Unaligned access	Toolchain default	Unaligned memory access

Option	Setting	Description
TrustZone (-mcmse)	-	Compiler generates code supporting ARM TrustZone security
Other target flags	-	Additional target-specific compiler flags

Optimization

Set the compiler optimization level, such as -O0, -O1, -O2, -O3, -Os, -Ofast, -Og, or -Oz, to balance build time, performance, and code size, as shown in Figure 8-6.

Figure 8-6 Optimization



The settings are described in Table 8-2.

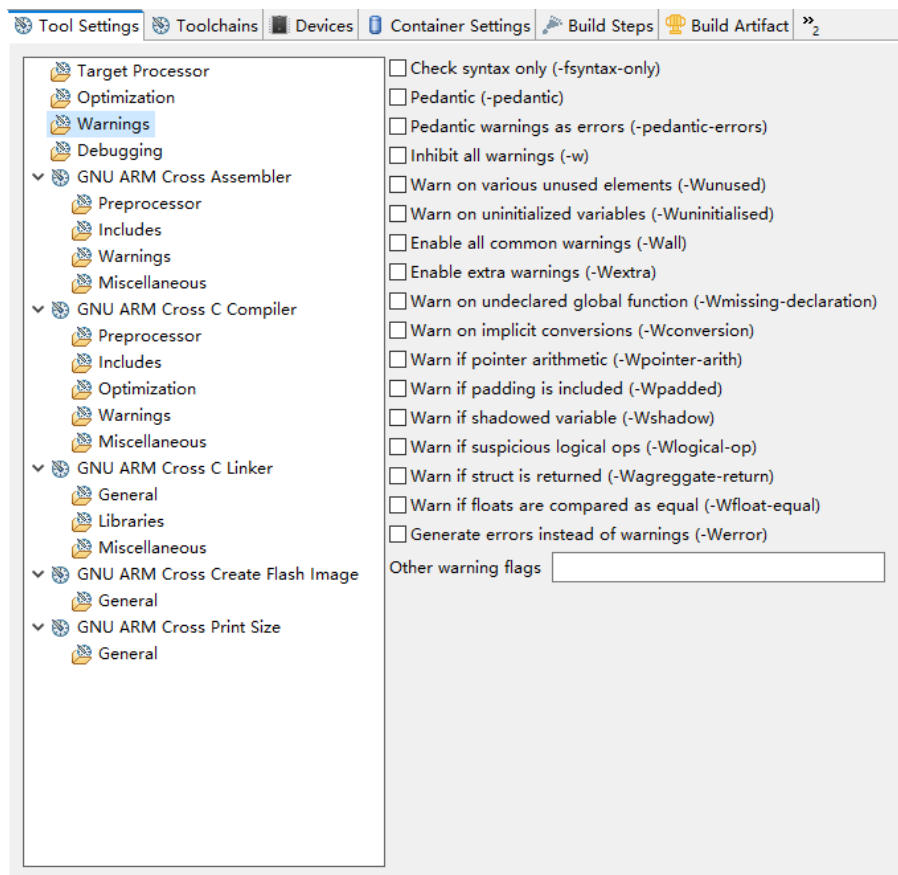
Table 8-2 Optimization

Option	Setting
Optimization Level	None (-O0)
Message length (-fmessage-length = 0)	✓

Option	Setting
'char' is signed (-fsigned-char)	✓
Function sections (-ffunction-sections)	✓
Data sections (-fdata-sections)	✓
No common uninitialized (-fno-common)	-
Do not inline functions (-fno-inline-functions)	-
Assume freestanding environment (-ffreestanding)	-
Disable builtin (-fno-builtin)	-
Single precision constants (-fsingle-precision-constant)	-
Position independent code (-fPIC)	-
Link-time optimizer (-flto)	-
Disable loop invariant move (-fno-move-loop-invariants)	-
Other optimization flags	-

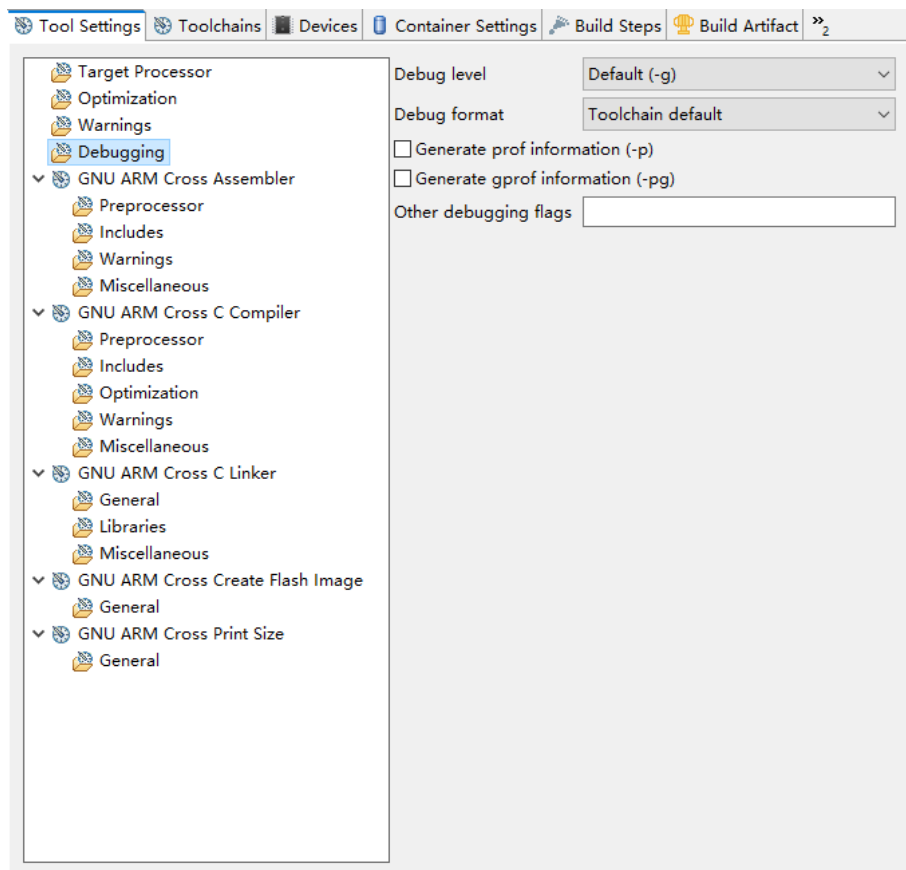
Warnings

Warning messages can be enabled to help developers identify potential issues early. Warnings can also be treated as errors when needed, as shown in Figure 8-7.

Figure 8-7 Warnings

Debugging

Set the debugging level, such as `-g`, `-g1`, `-g3`, to generate different levels of detailed debug information, as shown in Figure 8-8.

Figure 8-8 Debugging

The settings are described in Table 8-3.

Table 8-3 Debugging

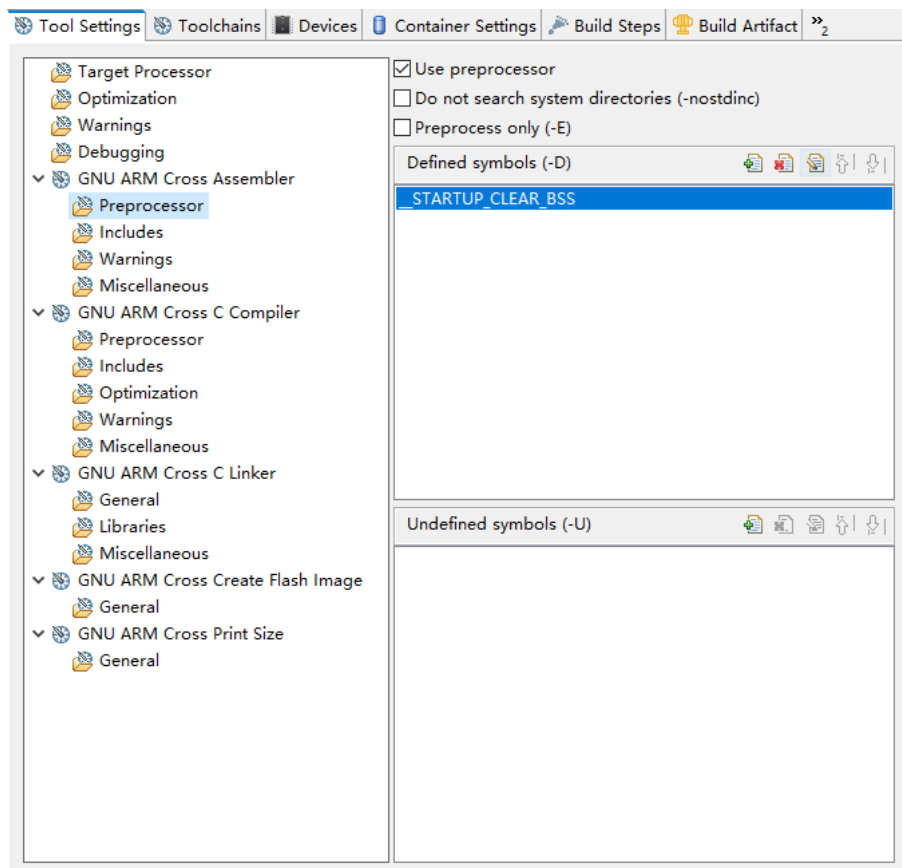
Option	Setting
Debug level	Default (-g)
Debug format	Toolchain default
Generate prof information (-p)	-
Generate gprof information (-pg)	-
Other debugging flags	-

GNU ARM Cross Assembler

Configure the assembler options for the ARM MCU, including settings for Preprocessor, Includes, Warnings, and Miscellaneous.

1. Preprocessor

Configure macro definitions to add project-specific preprocessor macros, as shown in Figure 8-9.

Figure 8-9 GNU ARM Cross Assembler > Preprocessor

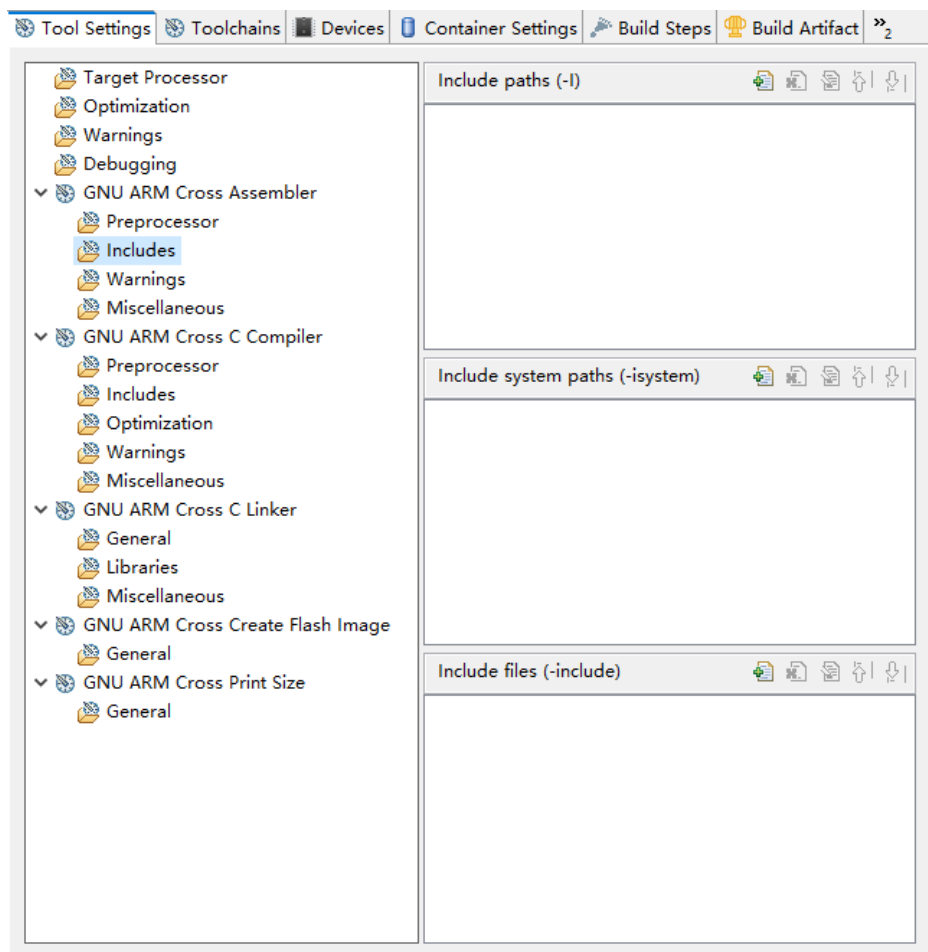
The settings are described in Table 8-4.

Table 8-4 GNU ARM Cross Assembler > Preprocessor

Option	Setting
Use preprocessor	✓
Do not search system directions (-nostdinc)	-
Preprocess only (-E)	-
Defined symbols (-D)	__STARTUP_CLEAR_BSS
Undefined symbols (-U)	-

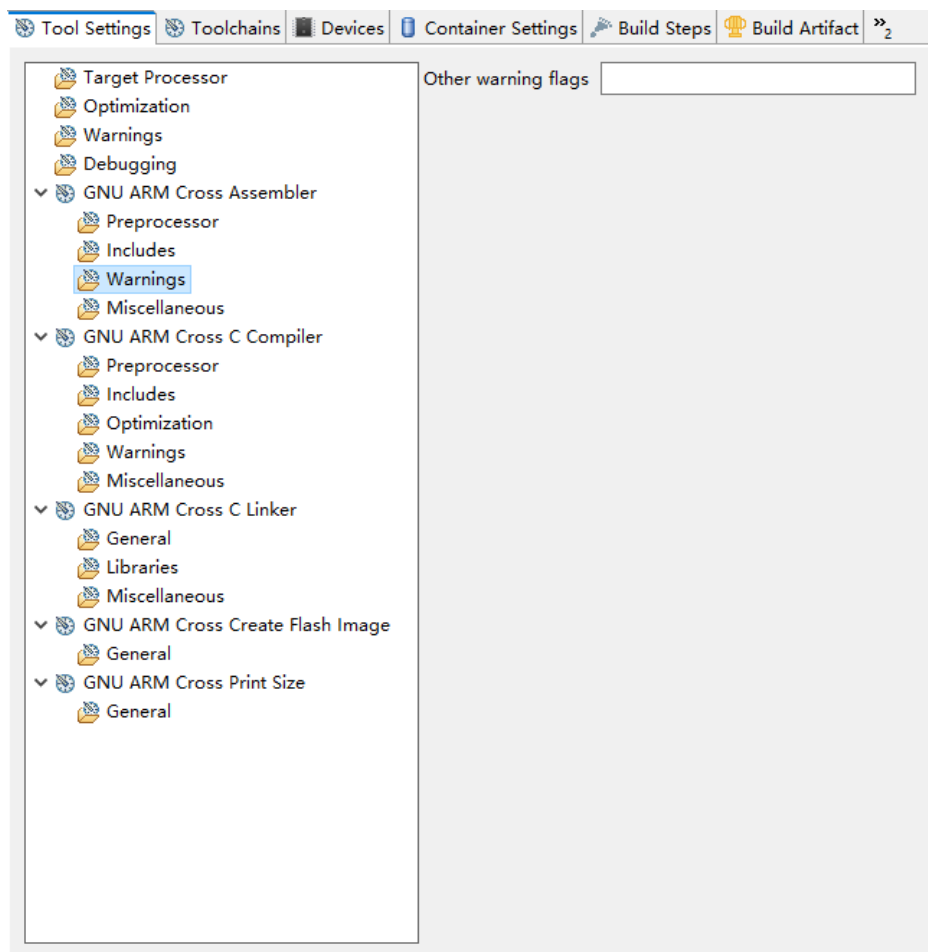
2. Includes

Specify the header file search paths. You can use absolute paths, relative paths, or Eclipse variables (e.g., `${workspace_loc}`), as shown in Figure 8-10.

Figure 8-10 GNU ARM Cross Assembler > Includes

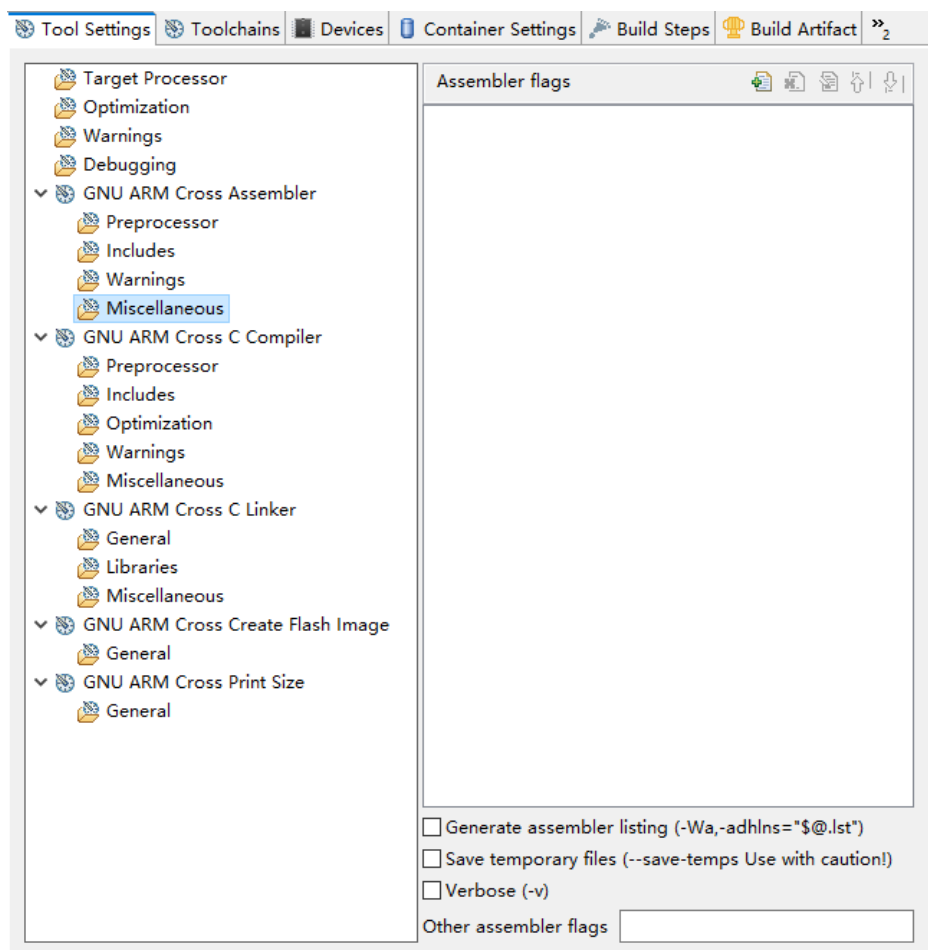
3. Warnings

Additional warning flags can be set as needed, as shown in Figure 8-11.

Figure 8-11 GNU ARM Cross Assembler > Warnings

4. Miscellaneous

Add any extra assembler flags as needed, as shown in Figure 8-12.

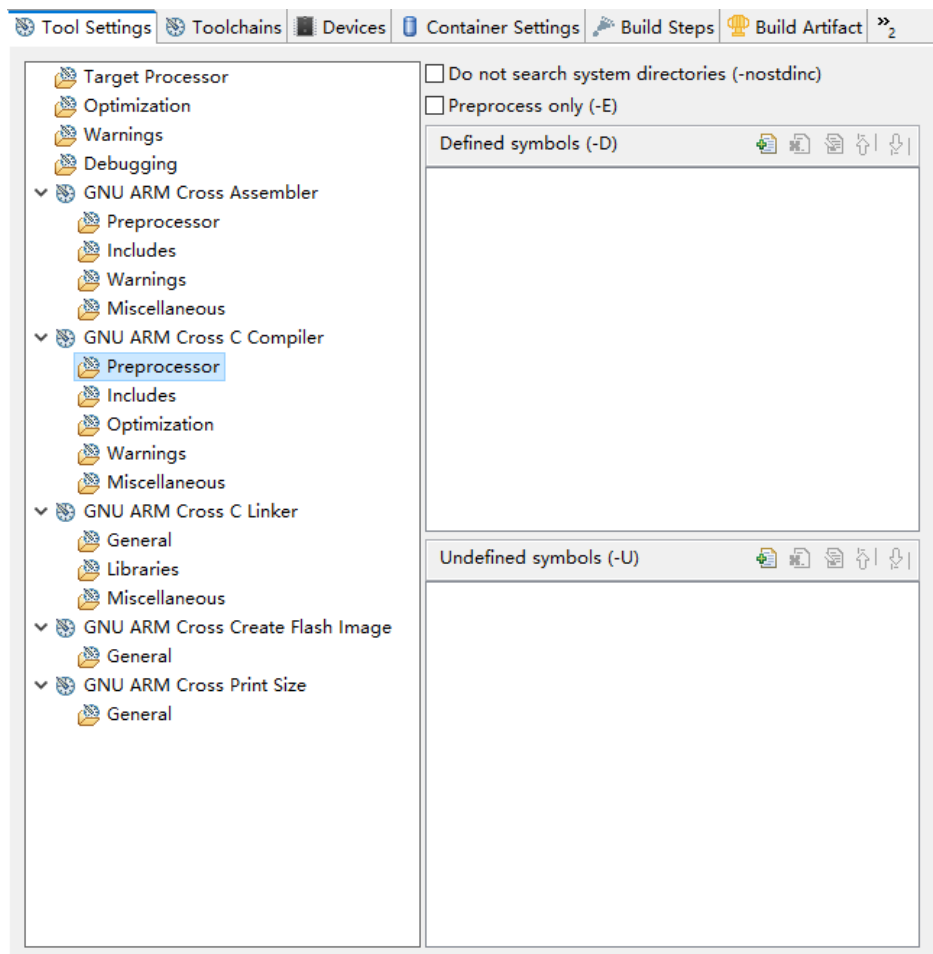
Figure 8-12 GNU ARM Cross Assembler > Miscellaneous

GNU ARM Cross C Compiler

Configure the C compiler options for the ARM MCU project, including settings for Preprocessor, Includes, Optimization, Warnings, and Miscellaneous.

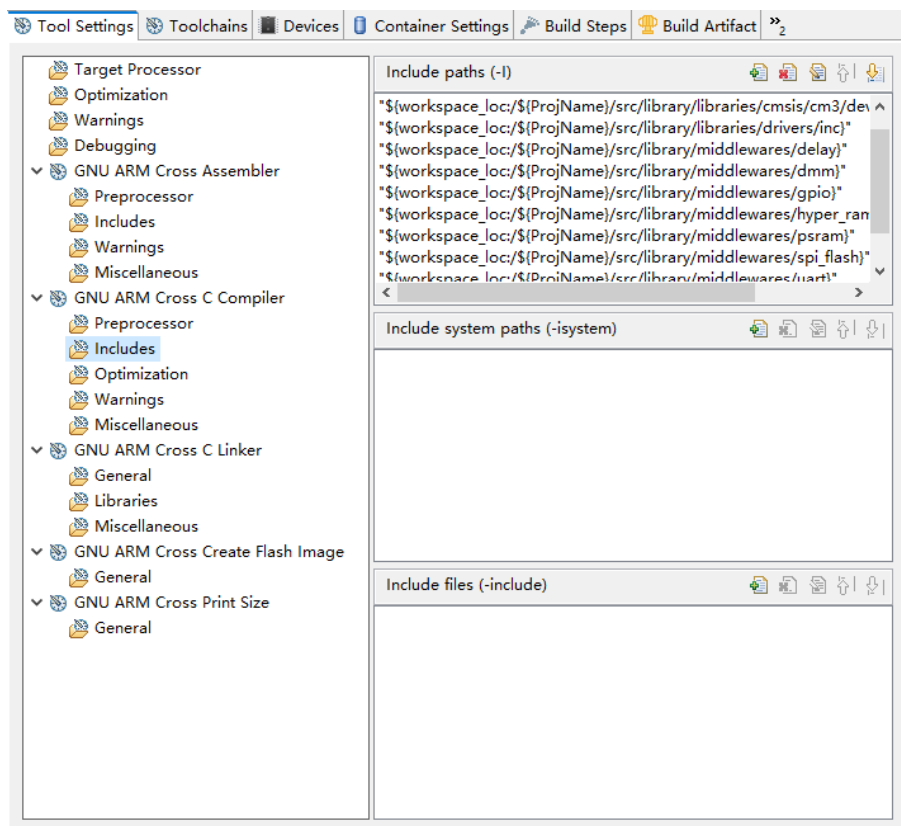
1. Preprocessor

Configure macro definitions to add project-specific preprocessor macros, as shown in Figure 8-13.

Figure 8-13 GNU ARM Cross C Compiler > Preprocessor

2. Includes

Specify the header file search paths. You can use absolute paths, relative paths, or Eclipse variables (e.g., `${workspace_loc}`), as shown in Figure 8-14.

Figure 8-14 GNU ARM Cross C Compiler > Includes

The settings are described in Table 8-5.

Table 8-5 GNU ARM Cross C Compiler > Includes

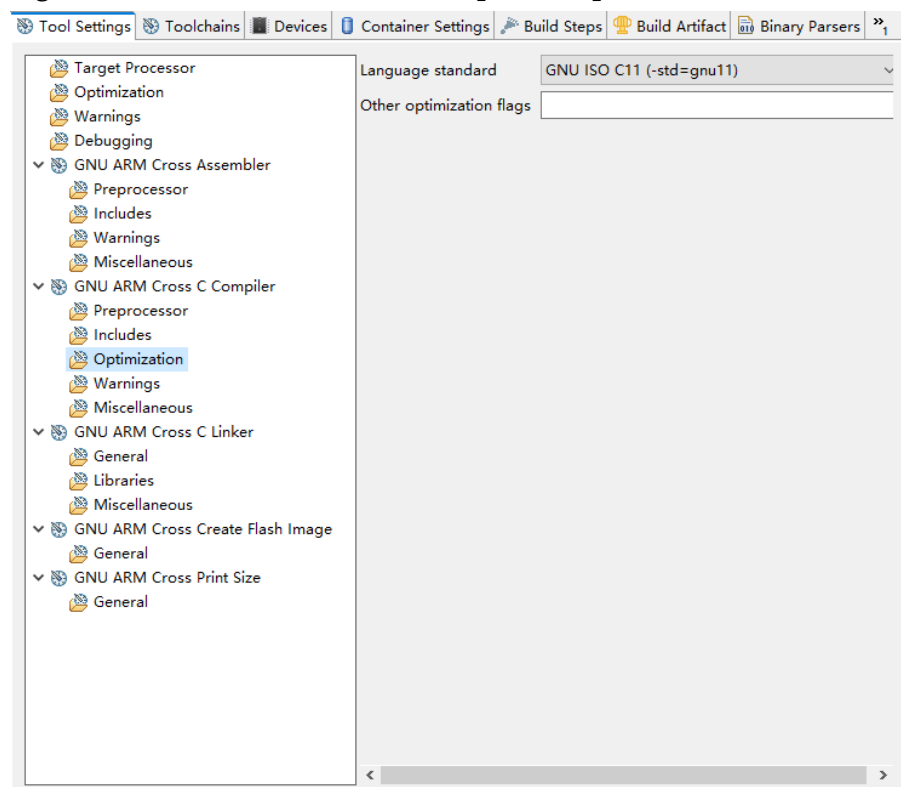
Option	Setting
Include paths (-I)	"\${workspace_loc}/\${ProjName}/src/library/libraries/cmsis/cm3/core_support/gmd}" "\${workspace_loc}/\${ProjName}/src/library/libraries/cmsis/cm3/device_support}" "\${workspace_loc}/\${ProjName}/src/library/libraries/drivers/inc" "\${workspace_loc}/\${ProjName}/src/library/middlewares/delay}" "\${workspace_loc}/\${ProjName}/src/library/middlewares/dmm}" "\${workspace_loc}/\${ProjName}/src/library/middlewares/gpio}" "\${workspace_loc}/\${ProjName}/src/library/middlewares/hyper_ram}" "\${workspace_loc}/\${ProjName}/src/library/middlewares/psram}"

Option	Setting
	<code>"\${workspace_loc}/\${ProjName}/src/library/middlewares/spi_flash"</code> <code>"\${workspace_loc}/\${ProjName}/src/library/middlewares/uart"</code> <code>"\${workspace_loc}/\${ProjName}/src/project"</code>
Include system paths (-isystem)	-
Include files (-include)	-

3. Optimization

Set the C standard and extensions to enable the corresponding language features, as well as additional optimization options as needed, as shown in Figure 8-15.

Figure 8-15 GNU ARM Cross C Compiler > Optimization



The settings are described in Table 8-6.

Table 8-6 GNU ARM Cross C Compiler > Optimization

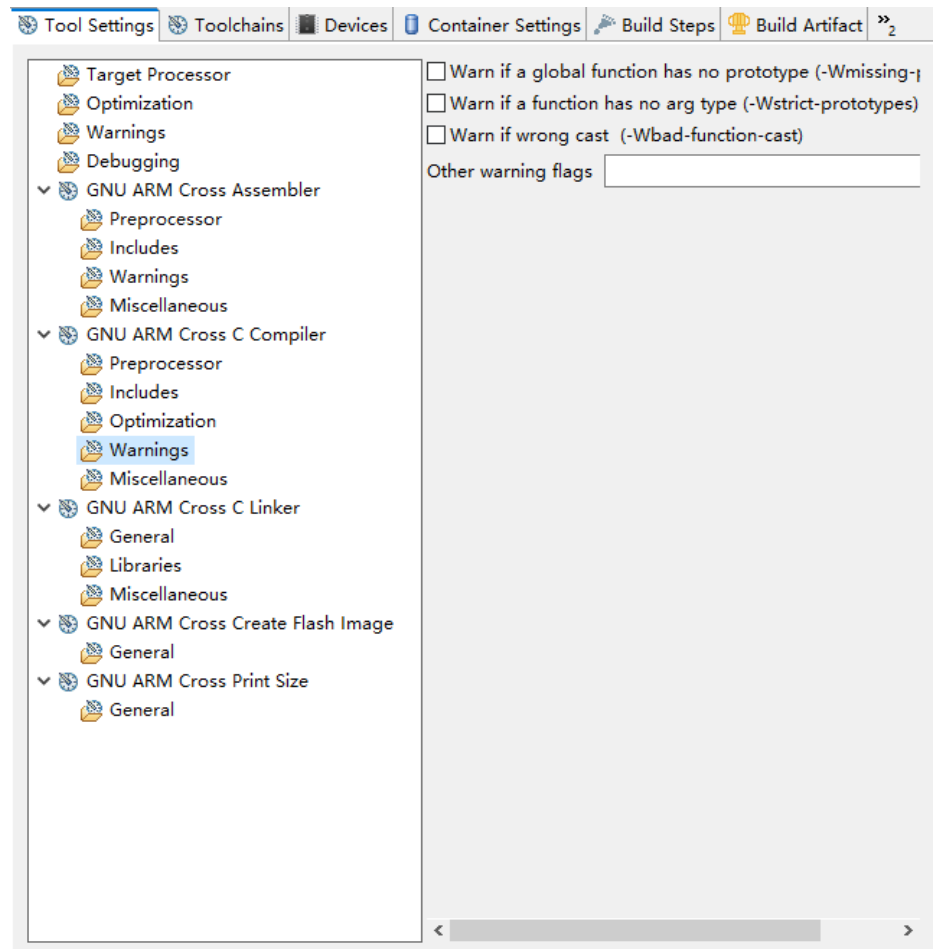
Option	Setting
Language standard	GNU ISO C11 (-std=gnu11)

Option	Setting
Other optimization flags	-

4. Warnings

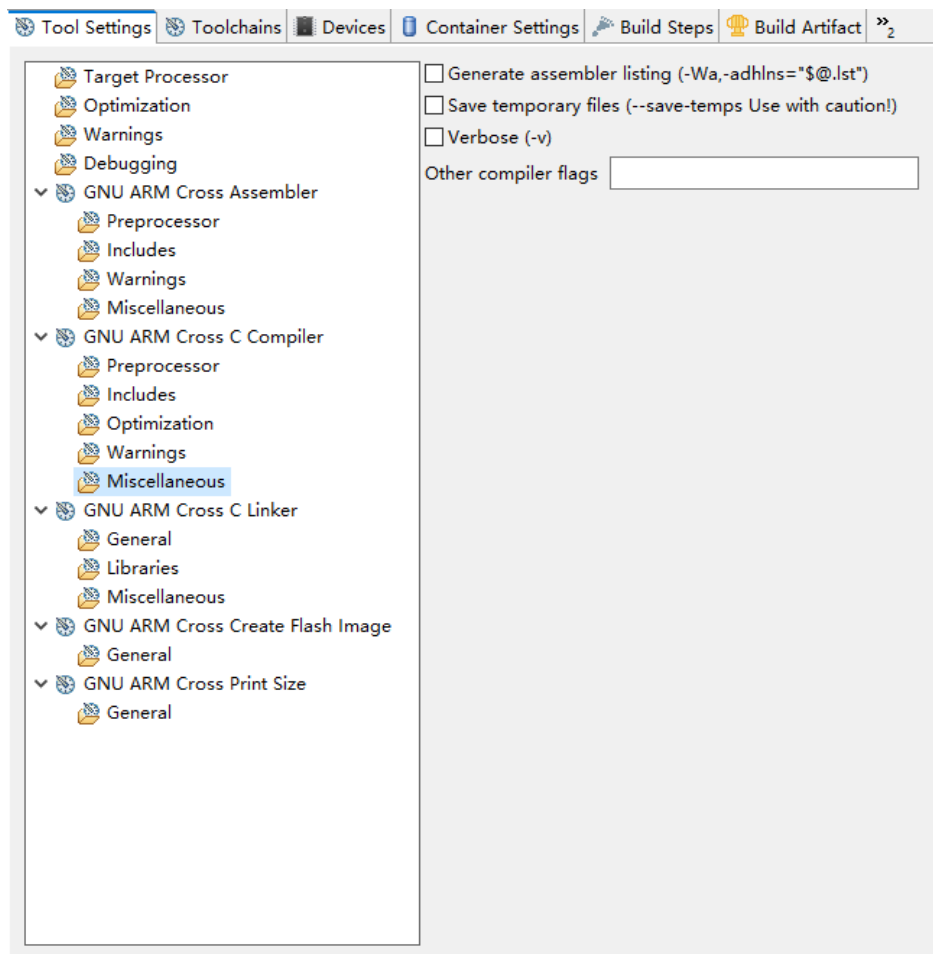
Optional warning flags can be set as needed, as shown in Figure 8-16.

Figure 8-16 GNU ARM Cross C Compiler > Warnings



5. Miscellaneous

Additional compiler flags can be configured if required, as shown in Figure 8-17.

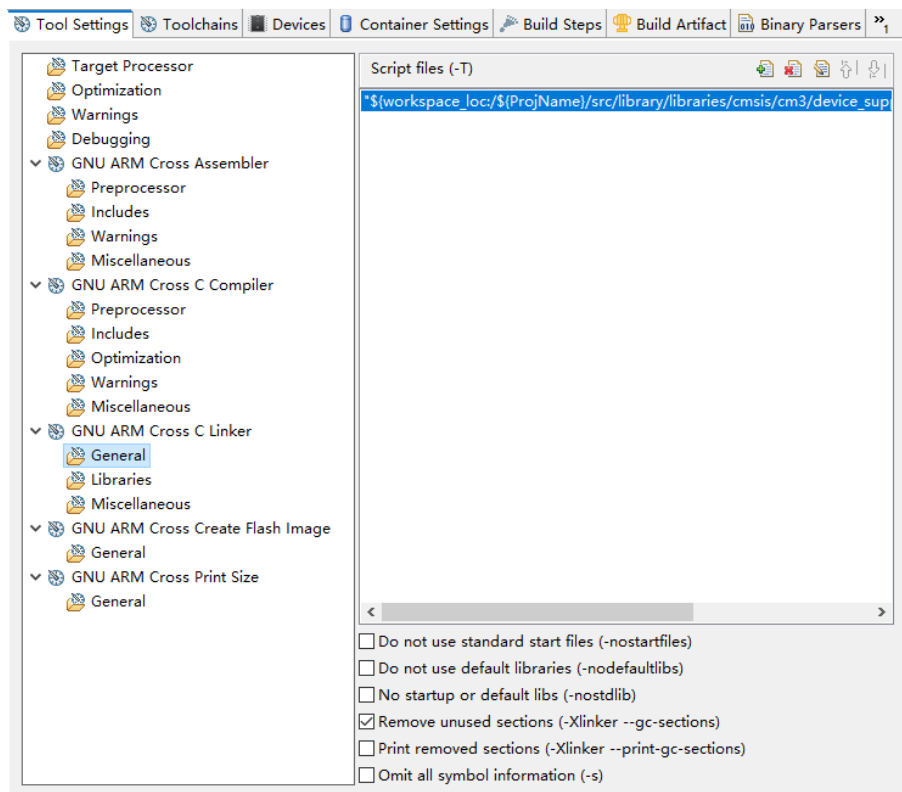
Figure 8-17 GNU ARM Cross C Compiler > Miscellaneous

GNU ARM Cross C Linker

Configure the C linker options for the ARM MCU project, including settings for General, Libraries, and Miscellaneous.

1. General

As shown in Figure 8-18, the Script files (-T) option is used to specify the linker script file.

Figure 8-18 GNU ARM Cross C Linker > General

The settings are described in Table 8-7.

Table 8-7 GNU ARM Cross C Linker > General

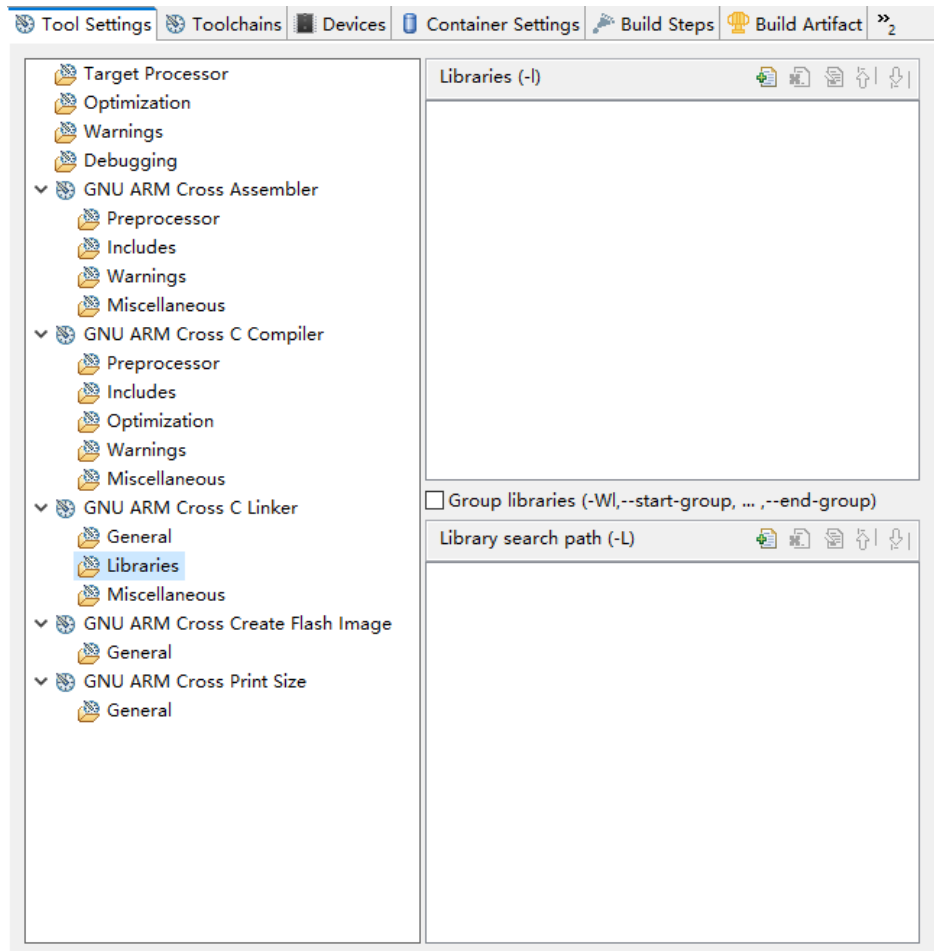
Option	Setting
Script files (-T)	"\${workspace_loc}/\${ProjName}/src/library/libraries/cmsis/cm3/device_support/startup/gmd/linker/gw1ns4c_flash.ld}"
Do not use standard start files (-nostartfiles)	-
Do not use default libraries (-nodefaultlibs)	-
No startup or default libs (-nostdlib)	-
Remove unused sections (-Xlinker --gc-sections)	✓
Print removed sections (-Xlinker --print-gc-sections)	-
Omit all symbol information (-s)	-

2. Libraries

As shown in Figure 8-19, it is used to configure the library linking

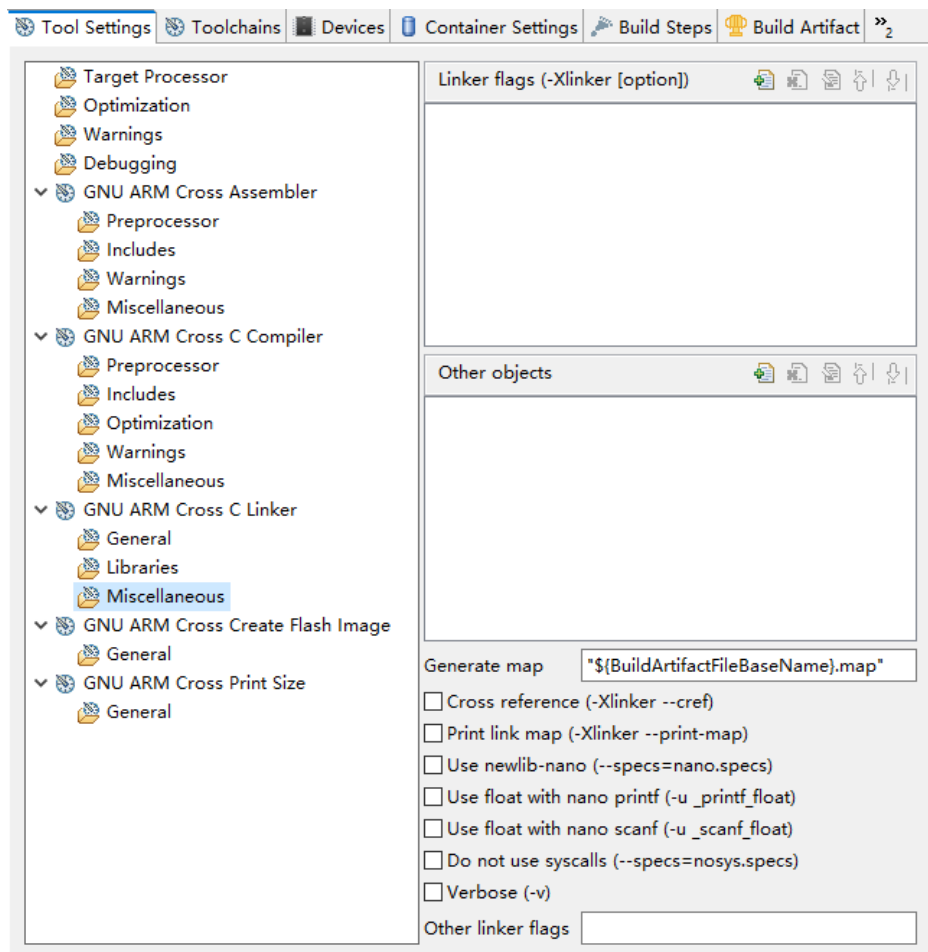
options. Use Libraries (-l) to specify the libraries to be linked, and Library search path (-L) to define the paths to locate static libraries (.a) and shared libraries (.so).

Figure 8-19 GNU ARM Cross C Linker > Libraries



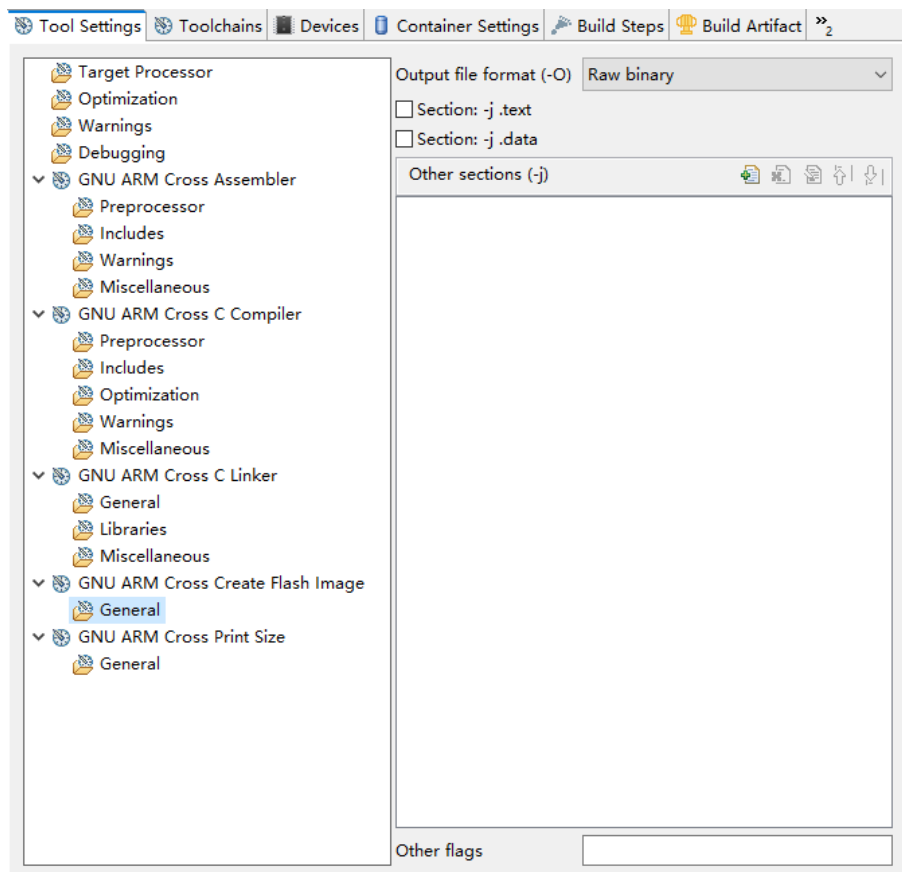
3. Miscellaneous

Additional linker options can be set as needed, as shown in Figure 8-20.

Figure 8-20 GNU ARM Cross C Linker > Miscellaneous

GNU ARM Cross Create Flash Image

It is used to configure the output format for the executable file of an ARM MCU project, such as Raw binary, Intel HEX, etc., as shown in Figure 8-21.

Figure 8-21 GNU ARM Cross Create Flash Image

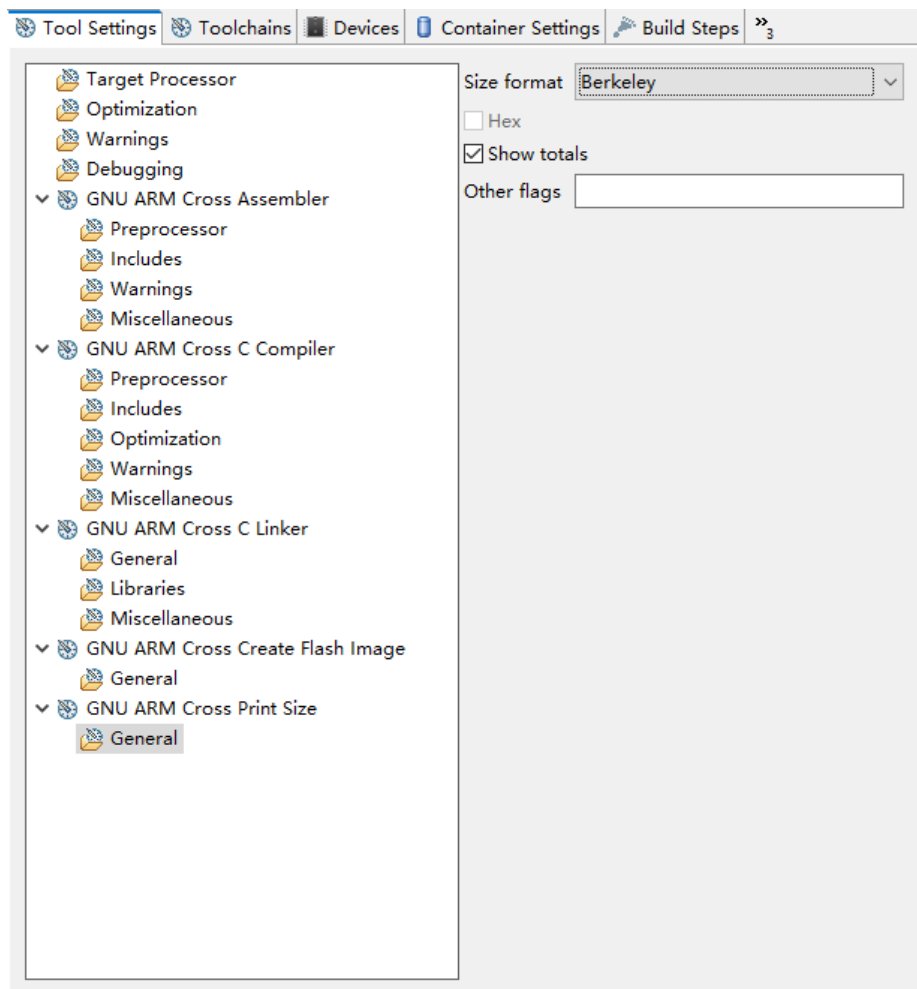
The settings are described in Table 8-8.

Table 8-8 GNU ARM Cross Create Flash Image

Option	Setting
Output file format (-O)	Raw binary
Section: -j .text	-
Section: -j .data	-
Other sections (-j)	-
Other flags	-

GNU ARM Cross Print Size

It is used to generate a report on the size of various sections (.text, .data, .bss) as well as the total size of the executable file for an ARM MCU project. Both Berkeley and SysV formats are supported. The format setting affects only the display style of the output and does not impact the actual size calculation, as shown in Figure 8-22.

Figure 8-22 GNU ARM Cross Print Size

The settings are described in Table 8-9.

Table 8-9 GNU ARM Cross Print Size

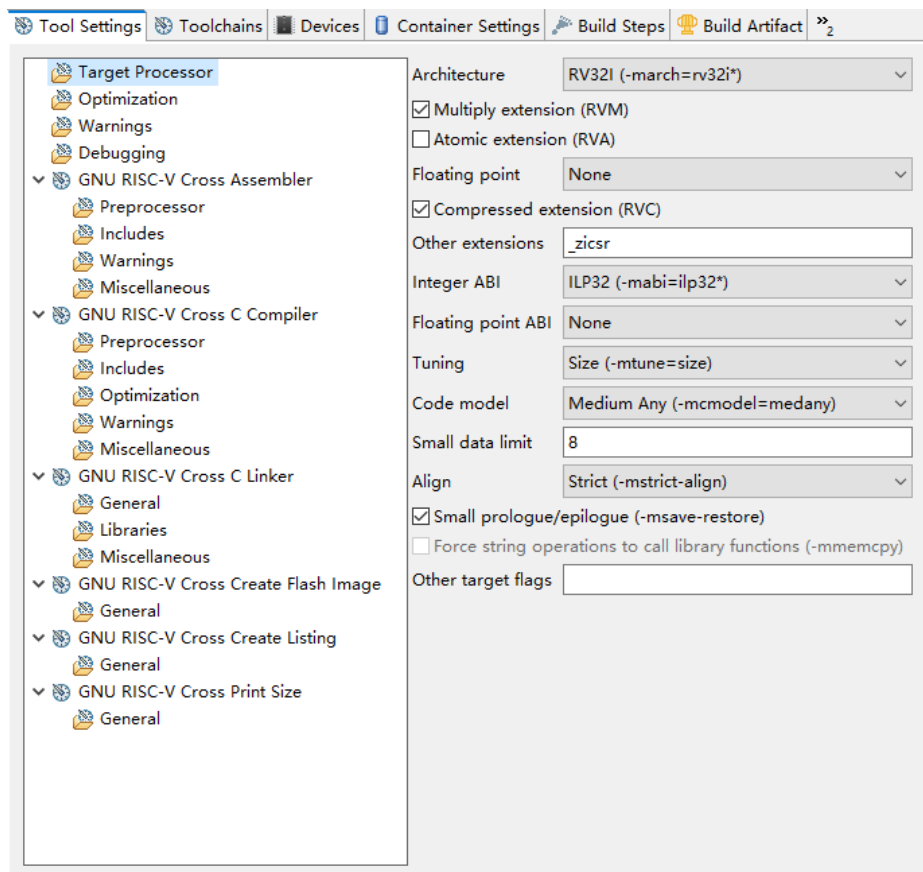
Option	Setting
Size format	Berkeley
Show totals	√
Other flags	-

8.2.2 RISC-V MCU Project Configuration

Take Gowin_PicoRV32 as an example to illustrate the configuration.

Target Processor

Configure the RISC-V target processor as shown in Figure 8-23.

Figure 8-23 Target Processor

The settings are described in Table 8-10.

Table 8-10 Target Processor

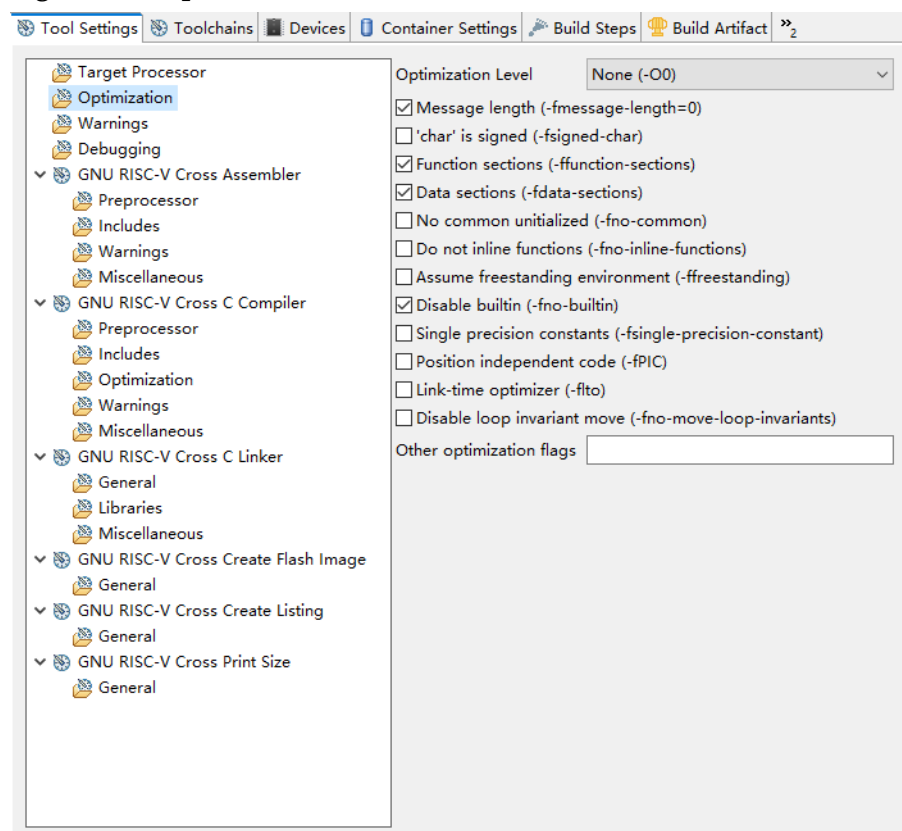
Option	Setting	Description
Architecture	RV32I (-march = rv32i)	RISC-V architecture
Multiply extension (RVM)	✓	Multiply/divide instructions
Atomic extension (RVA)	-	Atomic instructions
Floating point	None	Floating-point instructions
Compressed extension (RVC)	✓	Compressed instruction
Other extensions	_zicsr	Additional extension instruction
Integer ABI	ILP32 (-mabi = ilp32*)	Integer operations ABI
Floating point ABI	None	Floating point ABI
Tuning	Size (-mtune = size)	Optimize code generation for specific processor microarchitectures

Option	Setting	Description
Code model	Medium Any (-mcmmodel = medany)	Code model
Small data limit	8	Size limit of the small data area
Align	Strict (-mstrict-align)	Alignment method
Small prologue/epilogue (-msave-restore)	✓	For the compiler, you can choose to use external functions to save and restore registers
Other target flags	-	Additional target processor build options

Optimization

Set the compiler optimization level, such as -O0, -O1, -O2, -O3, -Os, -Ofast, -Og, or -Oz, to balance building speed, code performance, and binary size, as shown in Figure 8-24.

Figure 8-24 Optimization



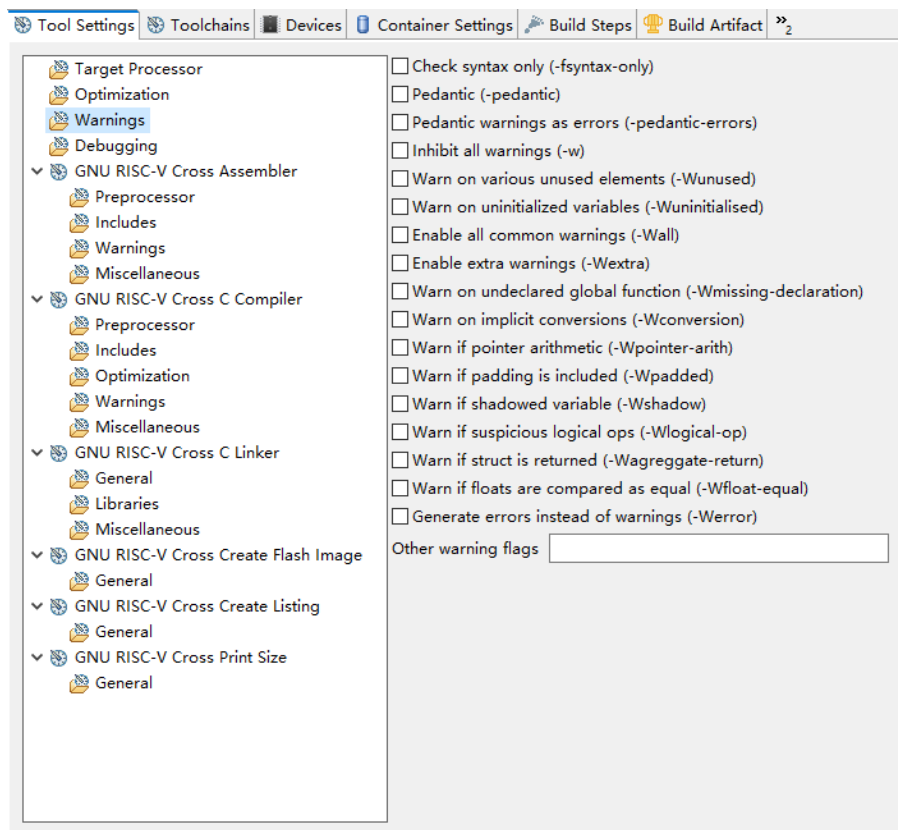
The settings are described in Table 8-11.

Table 8-11 Optimization

Option	Setting
Optimization Level	None (-O0)
Message length (-fmessage-length = 0)	✓
'char' is signed (-fsigned-char)	-
Function sections (-ffunction-sections)	✓
Data sections (-fdata-sections)	✓
No common uninitialized (-fno-common)	-
Do not inline functions (-fno-inline-functions)	-
Assume freestanding environment (-ffreestanding)	-
Disable builtin (-fno-builtin)	✓
Single precision constants (-fsingle-precision-constant)	-
Position independent code (-fPIC)	-
Link-time optimizer (-flto)	-
Disable loop invariant move (-fno-move-loop-invariants)	-
Other optimization flags	-

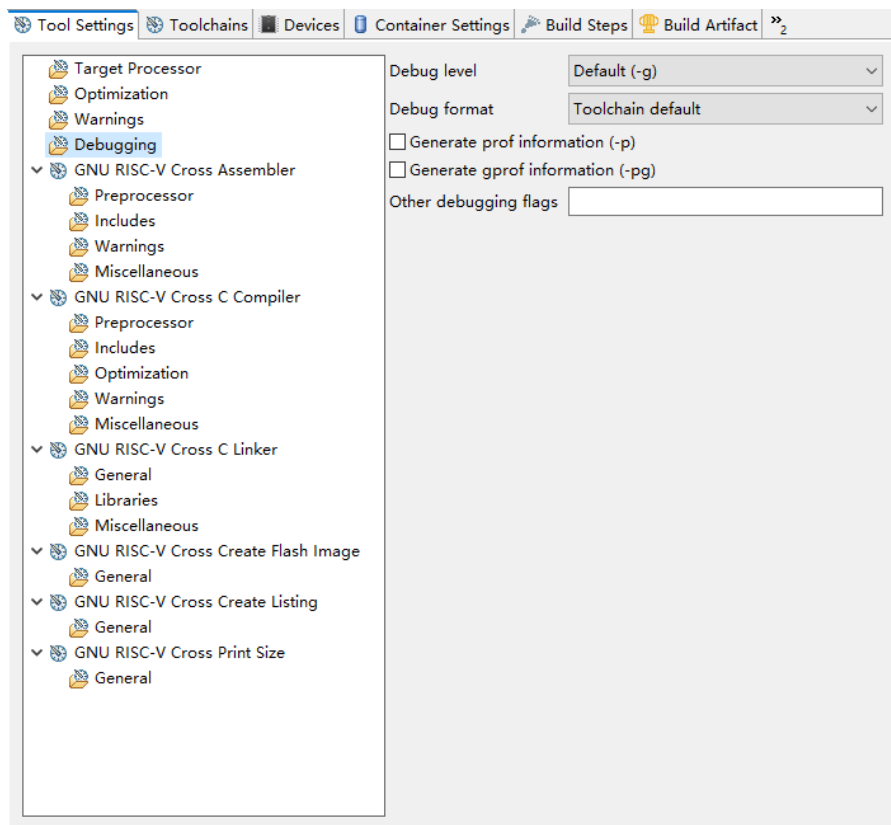
Warnings

It is used to help users detect potential issues early. Warnings can also be treated as errors if needed, as shown in Figure 8-25.

Figure 8-25 Warnings

Debugging

Set the debug level, such as `-g`, `-g1`, or `-g3`, to generate debugging information with varying levels, as shown in Figure 8-26.

Figure 8-26 Debugging

The settings are described in Table 8-12.

Table 8-12 Debugging

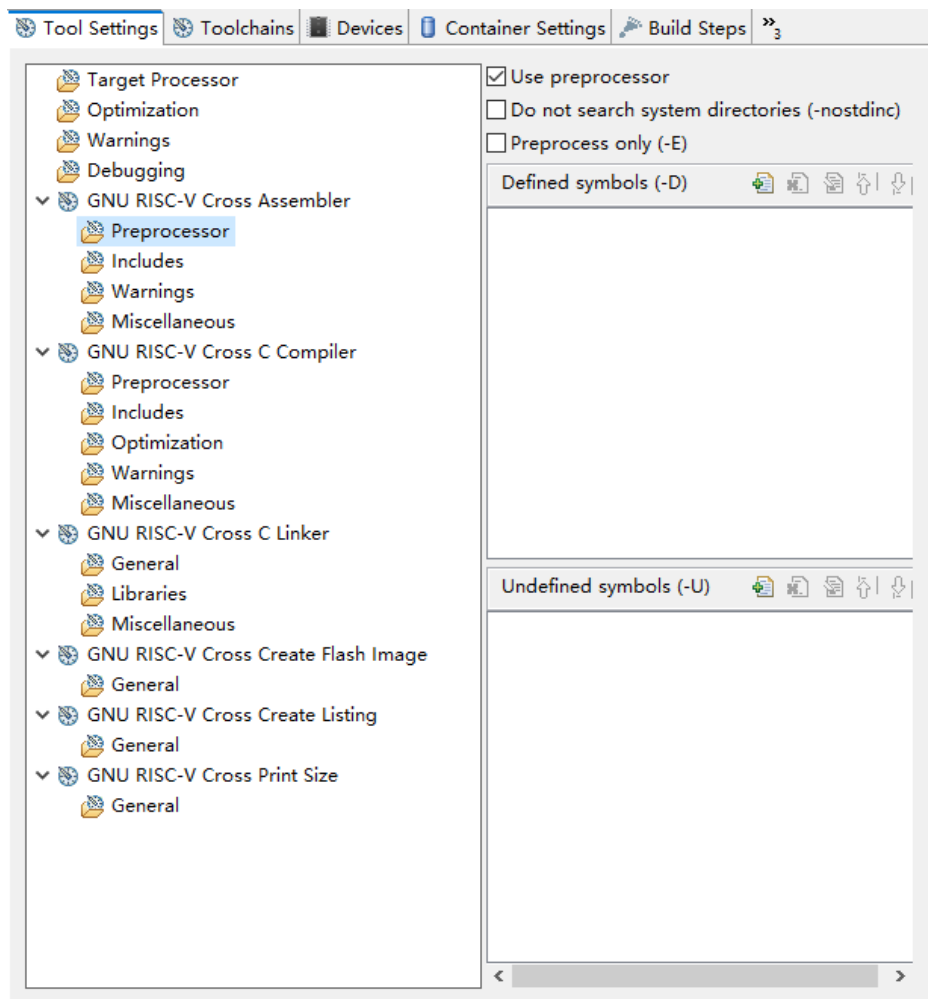
Option	Setting
Debug level	Default (-g)
Debug format	Toolchain default
Generate prof information (-p)	-
Generate gprof information (-pg)	-
Other debugging flags	-

GNU RISC-V Cross Assembler

Configure RISC-V MCU assembler options, including Preprocessor, Includes, Warnings, and Miscellaneous.

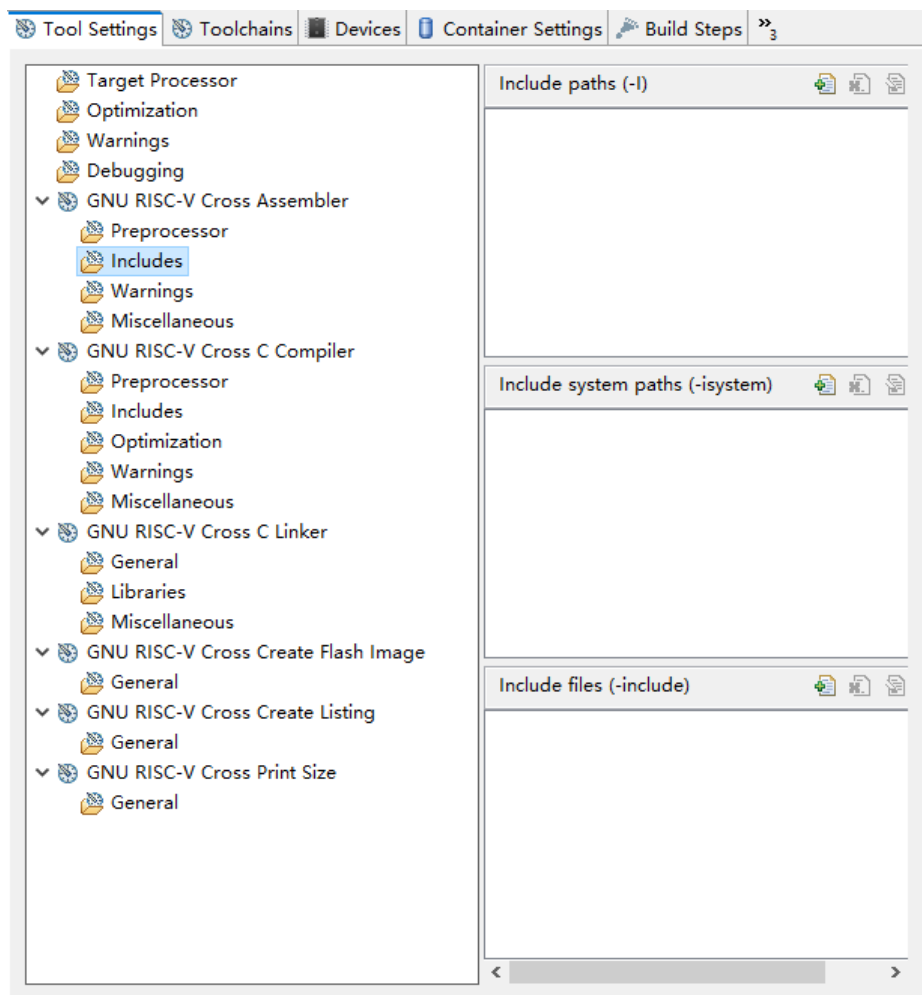
1. Preprocessor

Configure macro definitions to add project-specific preprocessor macros, as shown in Figure 8-27.

Figure 8-27 GNU RISC-V Cross Assembler > Preprocessor

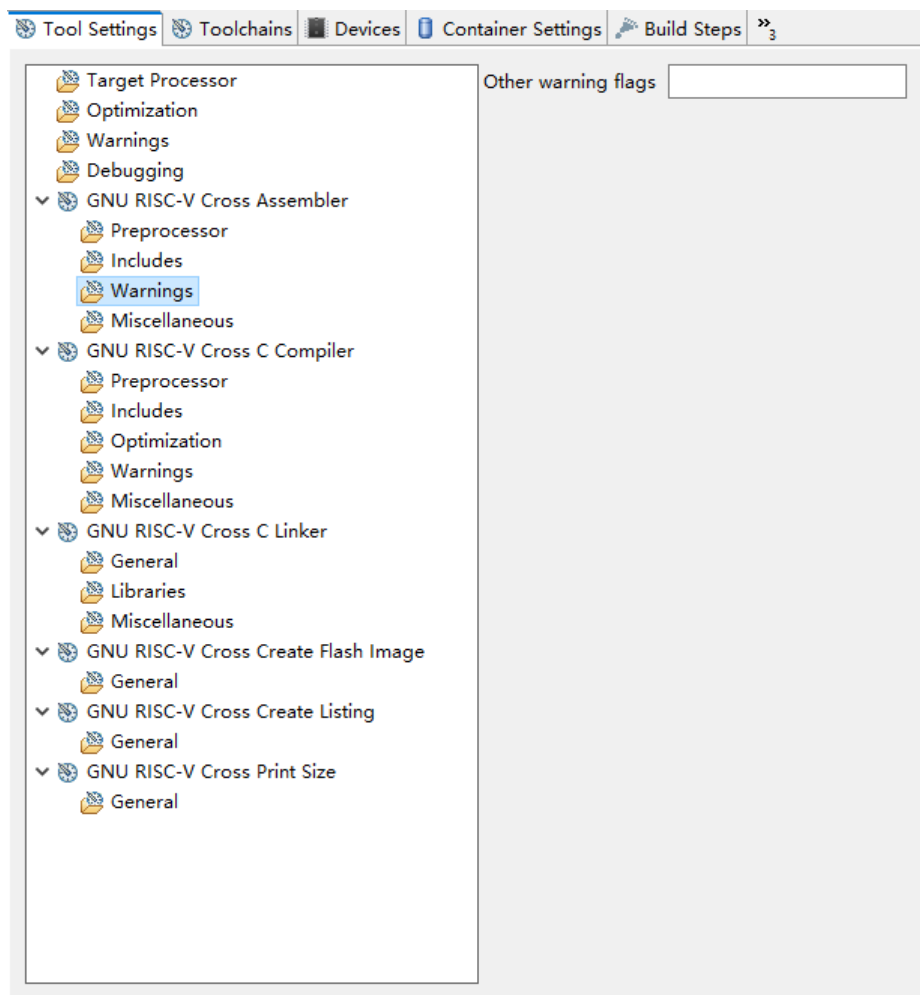
2. Includes

Specify the header file search paths. You can use absolute paths, relative paths, or Eclipse variables (e.g., `${workspace_loc}`), as shown in Figure 8-28.

Figure 8-28 GNU RISC-V Cross Assembler > Includes

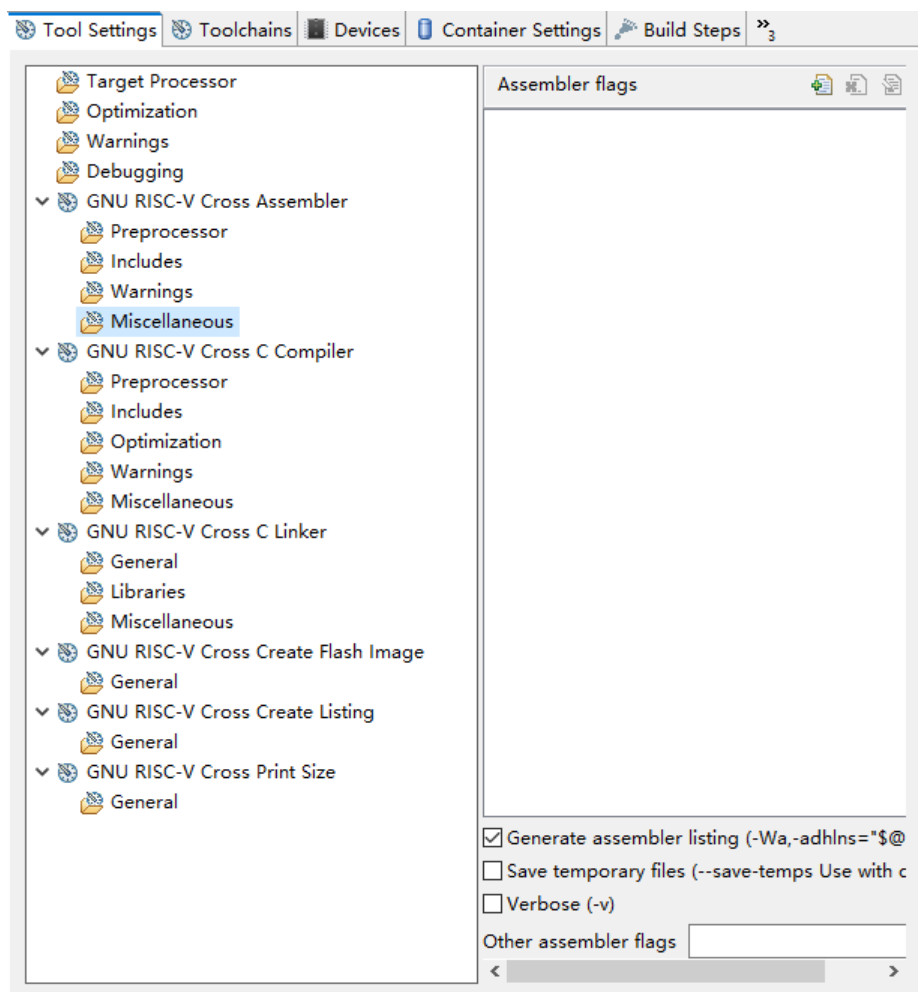
3. Warnings

Additional warning flags can be set as needed, as shown in Figure 8-29.

Figure 8-29 GNU RISC-V Cross Assembler > Warnings

4. Miscellaneous

Add any extra assembler flags as needed, as shown in Figure 8-30.

Figure 8-30 GNU RISC-V Cross Assembler > Miscellaneous

The settings are described in Table 8-13.

Table 8-13 GNU RISC-V Cross Assembler > Miscellaneous

Option	Setting
Assembler flags	-
Generate assembler listing (-Wa,-adhlns=\"\$@.lst\")	✓
Save temporary files (--save-temps Use with caution!)	-
Verbose (-v)	-
Other assembler flags	-

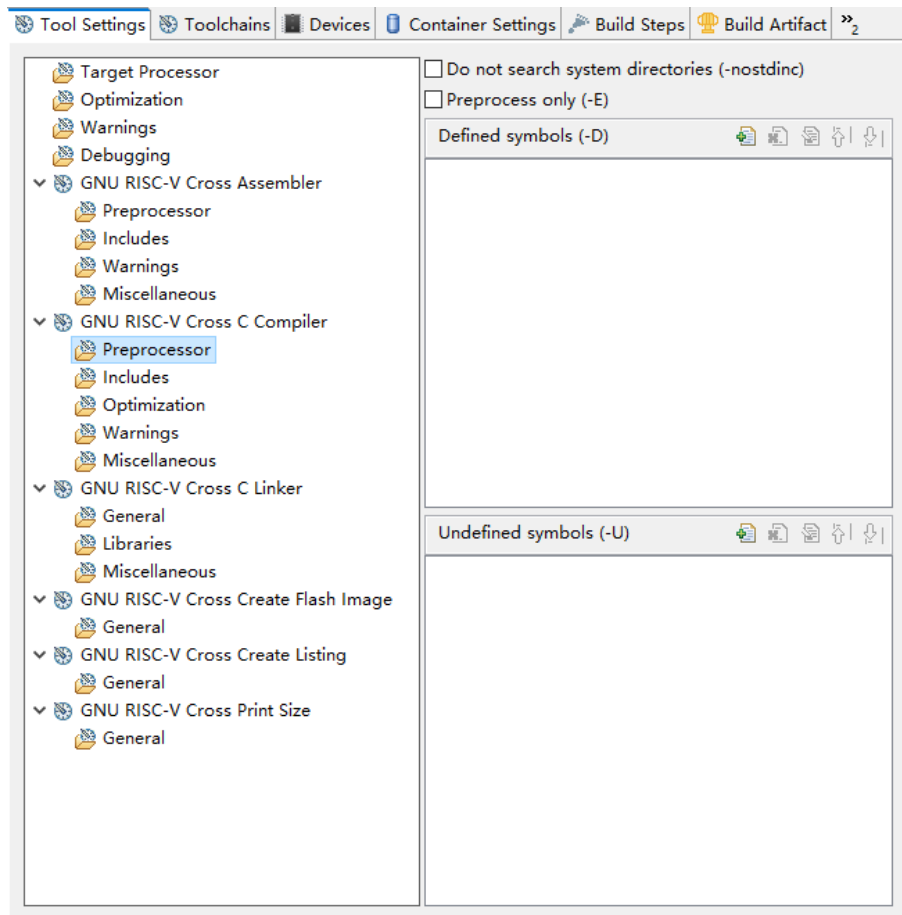
GNU RISC-V Cross C Compiler

Configure RISC-V MCU C compiler options, includes settings for Preprocessor, Includes, Optimization, Warnings, and Miscellaneous.

1. Preprocessor

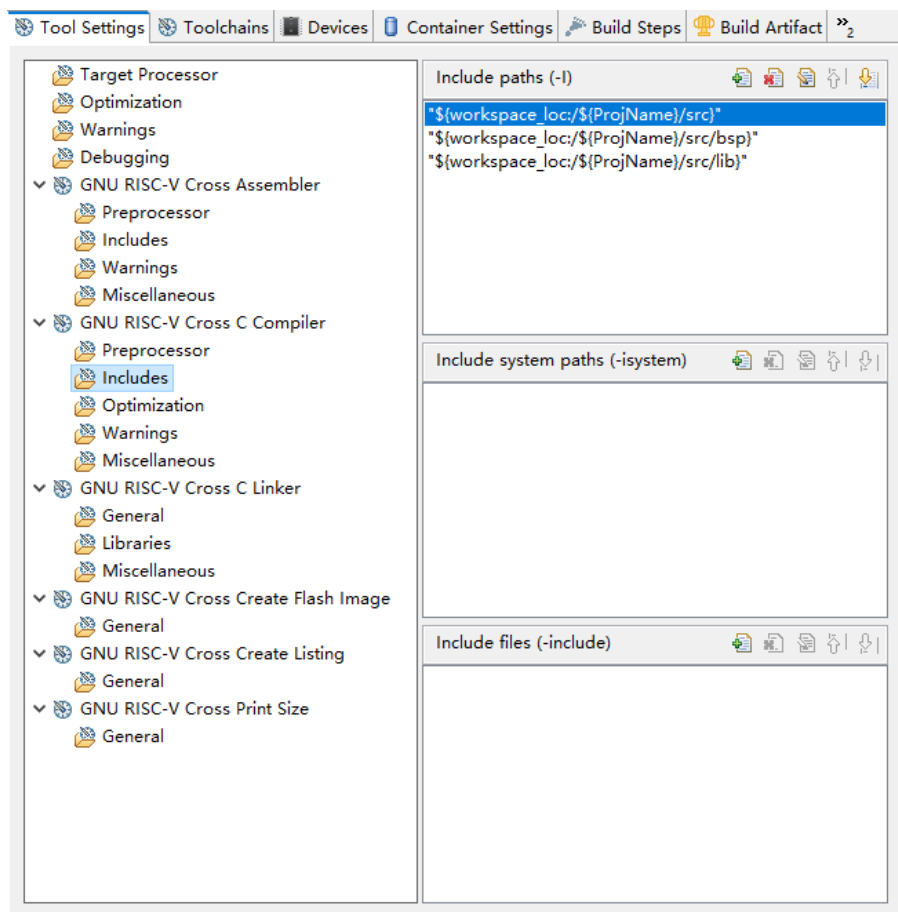
Configure macro definitions to add project-specific preprocessor macros, as shown in Figure 8-31.

Figure 8-31 GNU RISC-V Cross C Compiler > Preprocessor



2. Includes

Specify the header file search paths. You can use absolute paths, relative paths, or Eclipse variables (e.g., `${workspace_loc}`), as shown in Figure 8-32.

Figure 8-32 GNU RISC-V Cross C Compiler > Includes

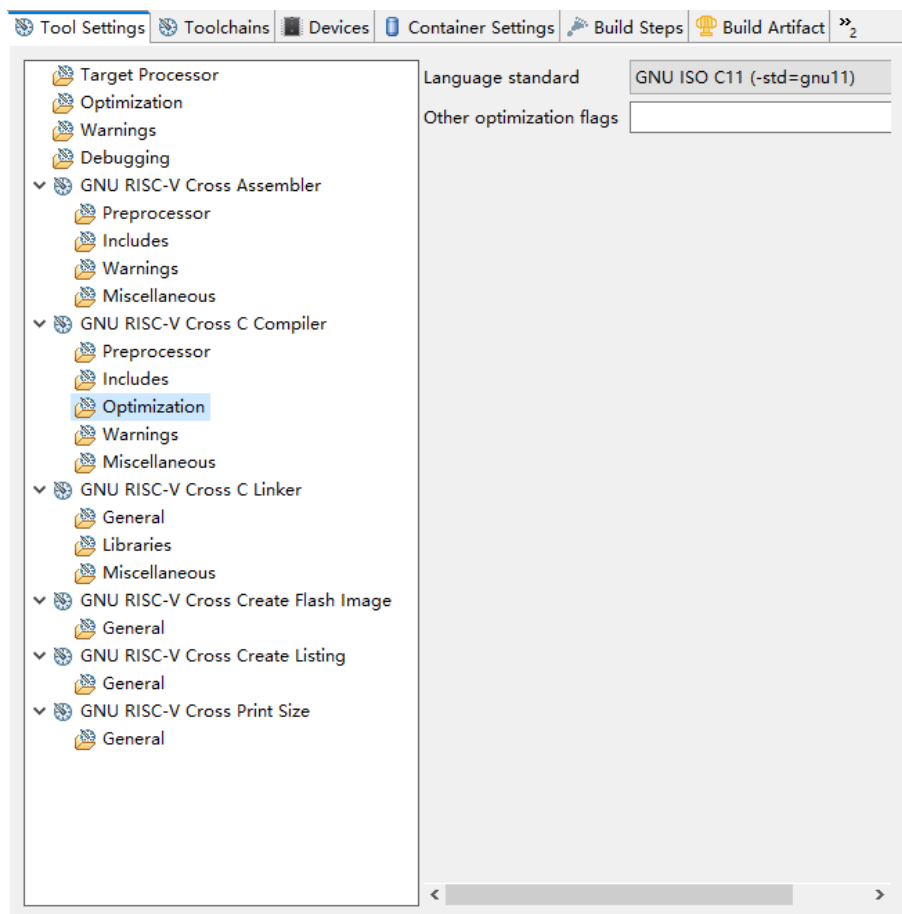
The settings are described in Table 8-14.

Table 8-14 GNU RISC-V Cross C Compiler > Includes

Option	Setting
Include paths (-I)	"\${workspace_loc}/\${ProjName}/src" "\${workspace_loc}/\${ProjName}/src/bsp" "\${workspace_loc}/\${ProjName}/src/lib"
Include system paths (-isystem)	-
Include files (-include)	-

3. Optimization

Set the C standard and extensions to enable the corresponding language features, as well as additional optimization options as needed, as shown in Figure 8-33.

Figure 8-33 GNU RISC-V Cross C Compiler > Optimization

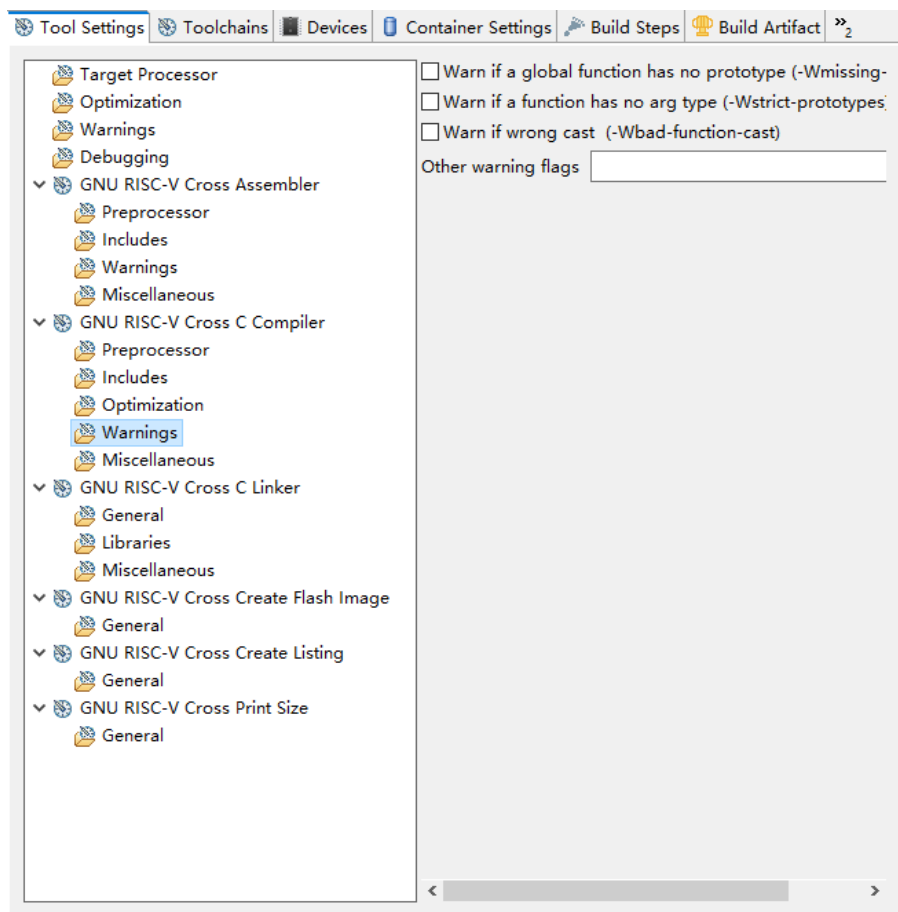
The settings are described in Table 8-15.

Table 8-15 GNU RISC-V Cross C Compiler > Optimization

Option	Setting
Language standard	GNU ISO C11 (-std=gnu11)
Other optimization flags	-

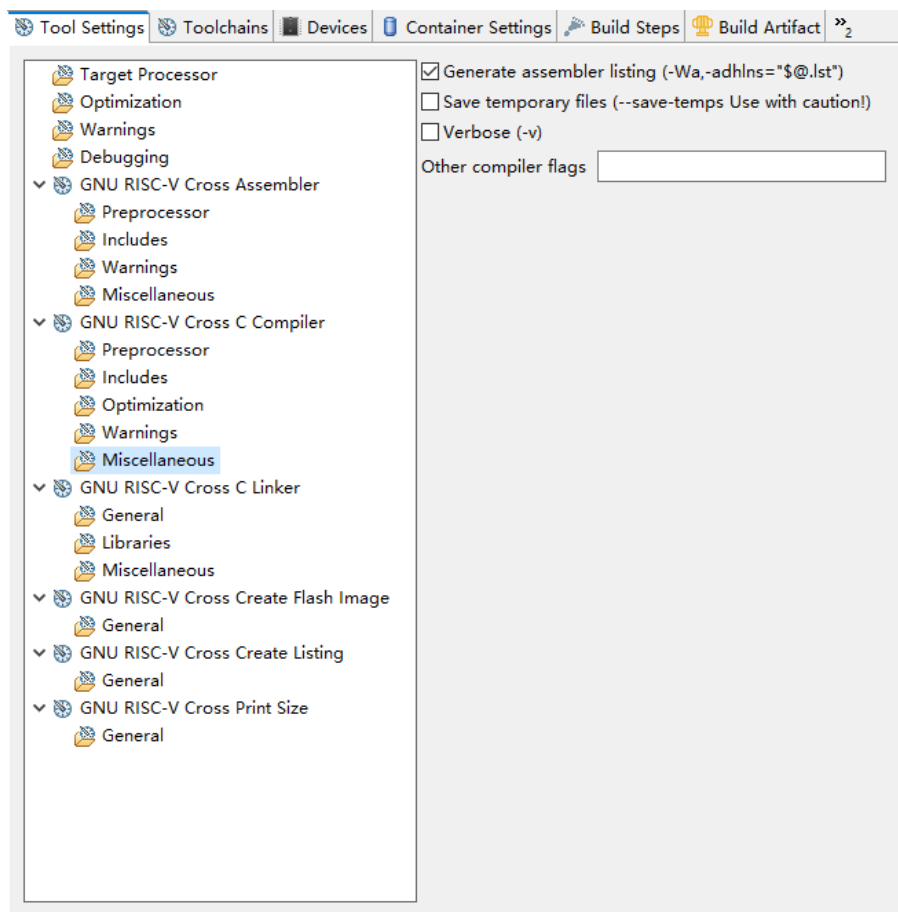
4. Warnings

Additional warning flags can be set as needed, as shown in Figure 8-34.

Figure 8-34 GNU RISC-V Cross C Compiler > Warnings

5. Miscellaneous

Add any extra compiler flags as needed, as shown in Figure 8-35.

Figure 8-35 GNU RISC-V Cross C Compiler > Miscellaneous

The settings are described in Table 8-16.

Table 8-16 GNU RISC-V Cross C Compiler > Miscellaneous

Option	Setting
Generate assembler listing (-Wa,-adhlns="\$@.lst")	✓
Save temporary files (--save-temps Use with caution)	-
Verbose (-v)	-
Other compiler flags	-

GNU RISC-V Cross C Linker

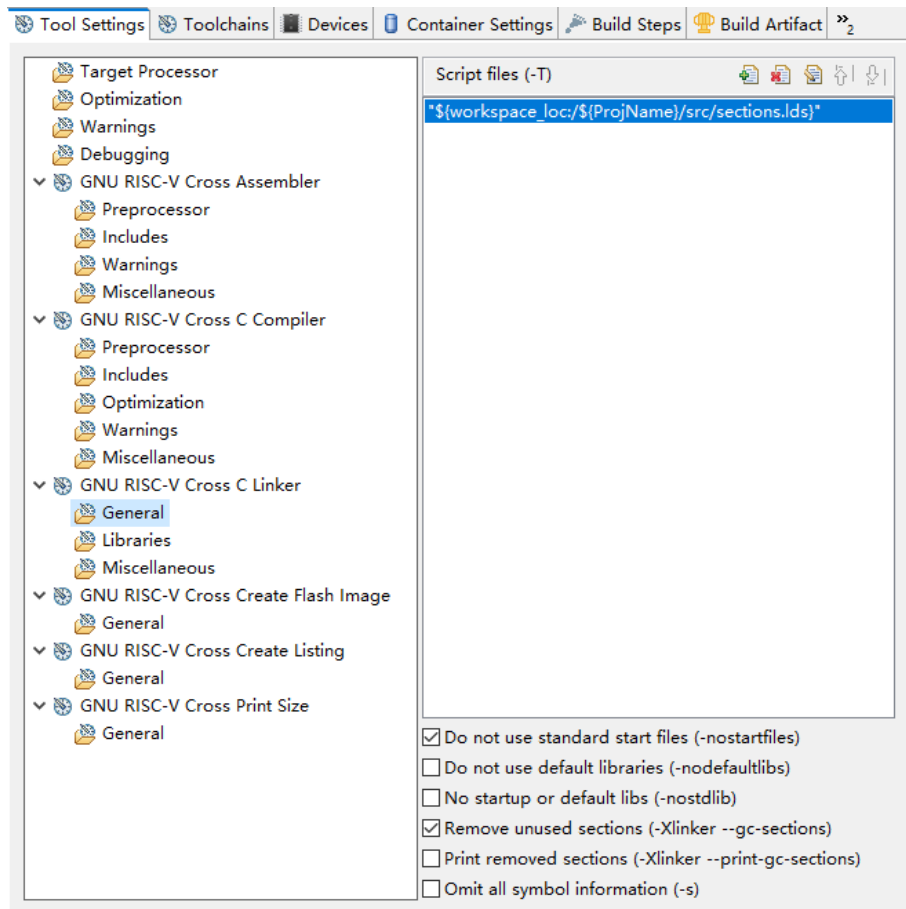
Configure the RISC-V MCU C linker options, including General, Libraries, and Miscellaneous.

1. General

As shown in Figure 8-36, the Script files (-T) option is used to specify

the linker script file.

Figure 8-36 GNU RISC-V Cross C Linker > General



The settings are described in Table 8-17.

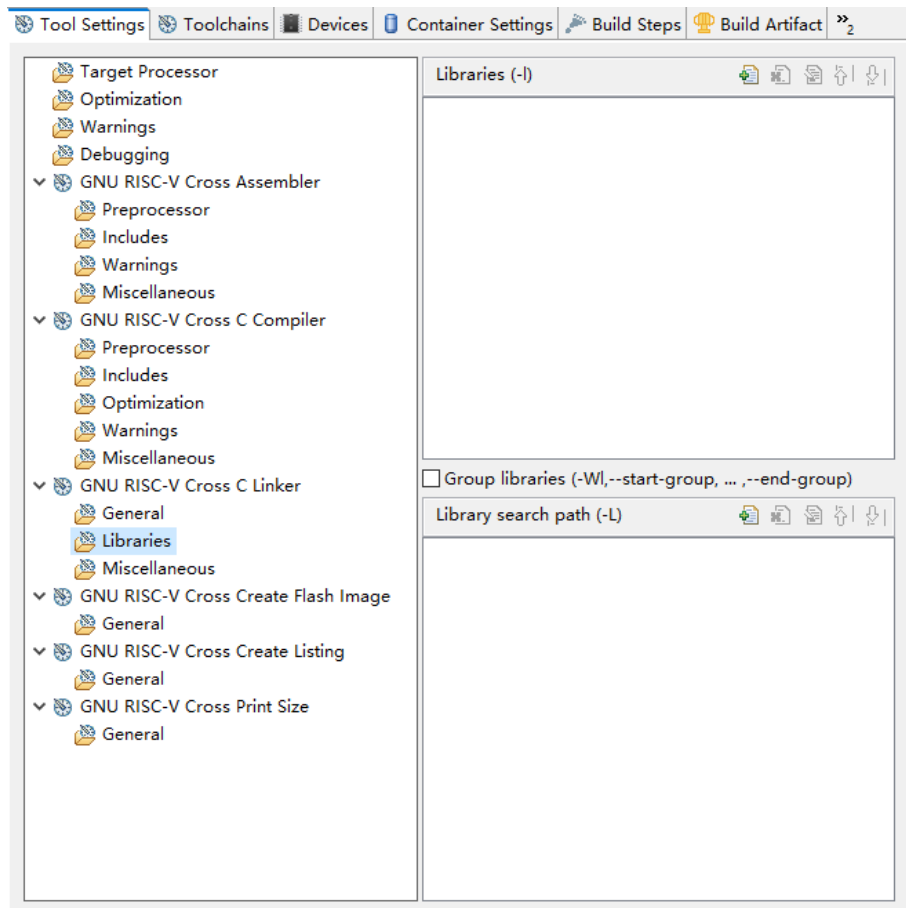
Table 8-17 GNU RISC-V Cross C Linker > General

Option	Setting
Script files (-T)	"\${workspace_loc}/\${ProjName}/src/sections.lds"
Do not use standard start files (-nostartfiles)	✓
Do not use default libraries (-nodefaultlibs)	-
No startup or default libs (-nostdlib)	-
Remove unused sections (-Xlinker --gc-sections)	✓
Print removed sections (-Xlinker --print-gc-sections)	-
Omit all symbol information (-s)	-

2. Libraries

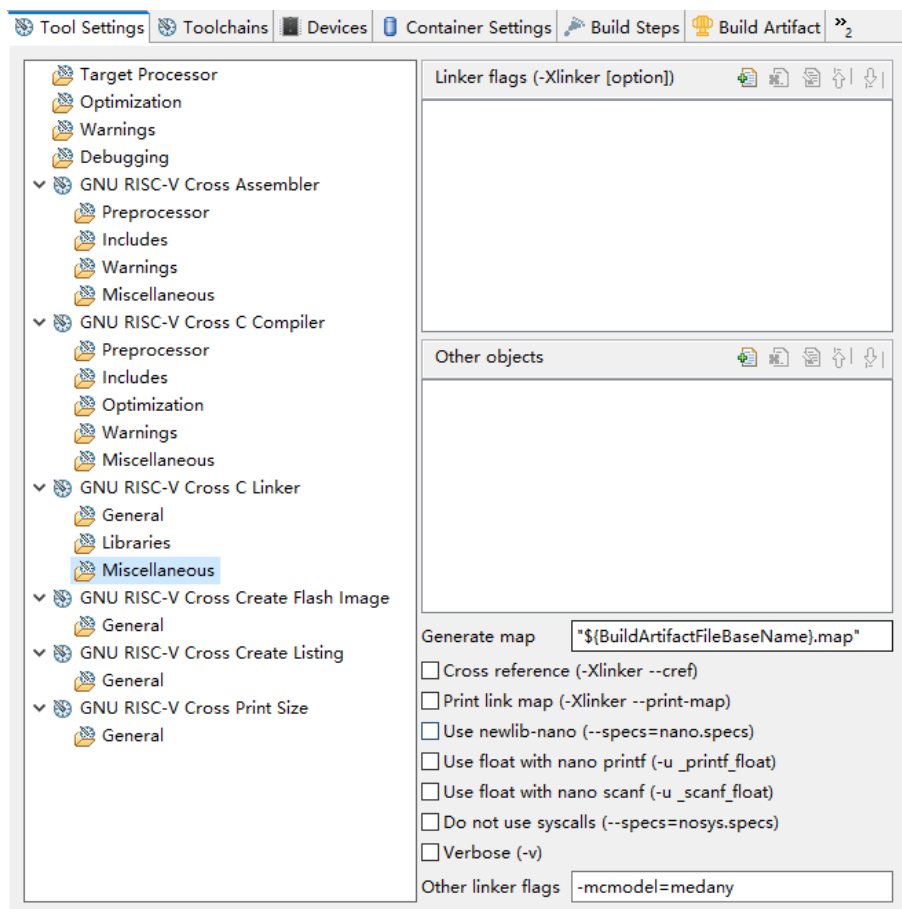
Configure the library linking options as shown in Figure 8-37. Use Libraries (-l) to specify libraries to link, and Library search path (-L) to set search paths for static (.a) and shared (.so) libraries.

Figure 8-37 GNU RISC-V Cross C Linker > Libraries



3. Miscellaneous

Additional linker options can be configured as needed, as shown in Figure 8-38.

Figure 8-38 GNU RISC-V Cross C Linker > Miscellaneous

The settings are described in Table 8-18.

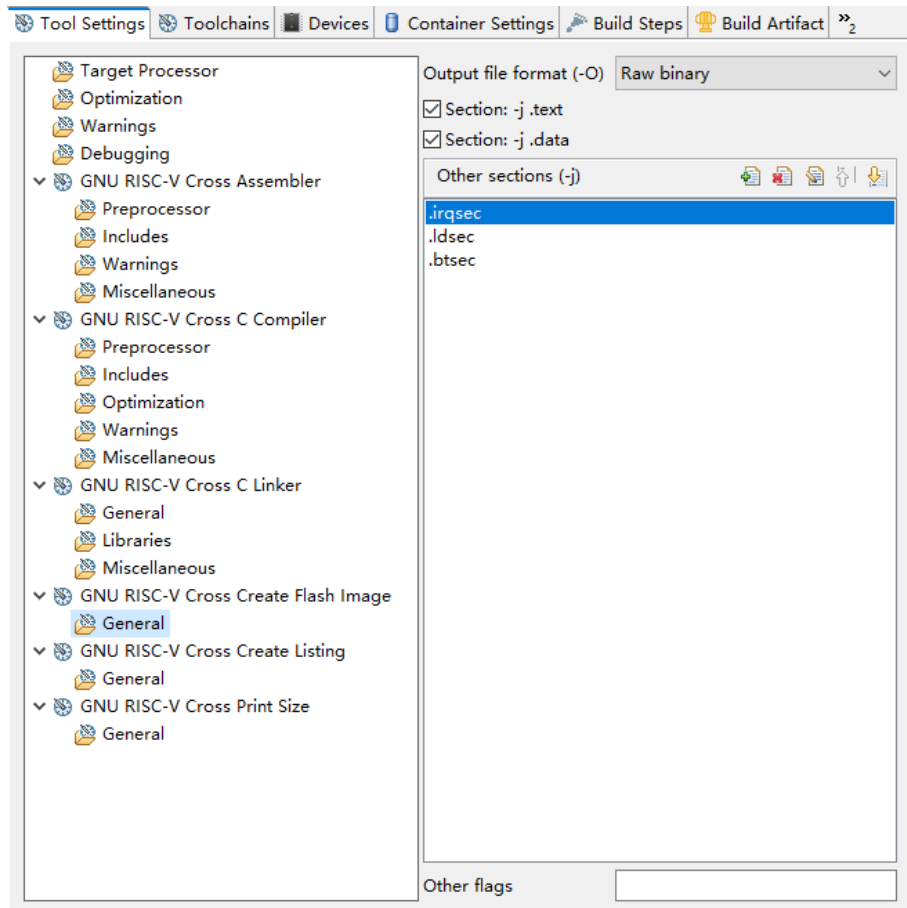
Table 8-18 GNU RISC-V Cross C Linker > Miscellaneous

Option	Setting
Linker flags (-Xlinker [option])	-
Other objects	-
Cross reference (-Xlinker --cref)	-
Print link map (-Xlinker --print-map)	-
Use newlib-nano (--specs=nano.specs)	Optional
Use float with nano printf (-u _printf_float)	Optional
Use float with nano scanf (-u _scanf_float)	Optional
Do not use syscalls (--specs = nosys.specs)	-
Verbose (-v)	-
Other linker flags	-mcmmodel=medany

GNU RISC-V Cross Create Flash Image

Configure the output format for RISC-V MCU executables, such as Raw binary or Intel HEX, as shown in Figure 8-39.

Figure 8-39 GNU RISC-V Cross Create Flash Image



The settings are described in Table 8-19.

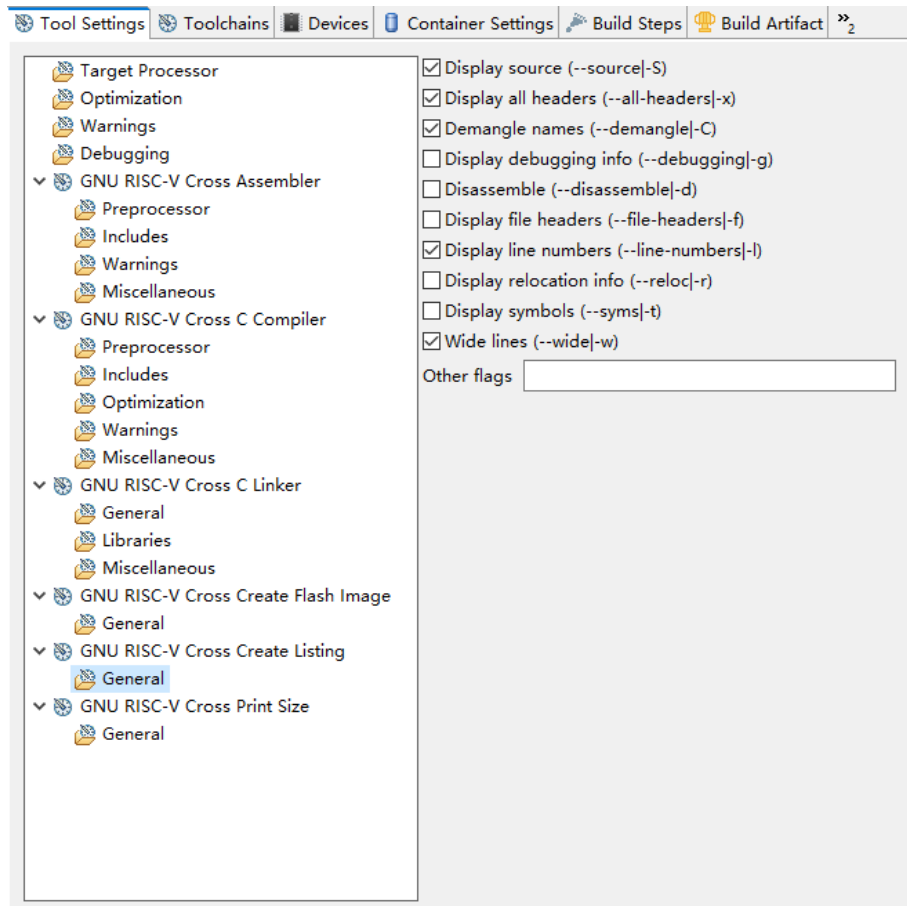
Table 8-19 GNU RISC-V Cross Create Flash Image

Option	Setting
Output file format (-O)	Raw binary
Section: -j .text	✓
Section: -j .data	✓
Other sections (-j)	.irqsec .ldsec .btsec
Other flags	-

GNU RISC-V Cross Create Listing

It is used to generate a listing file (typically .lst) that contains assembly code, symbol information, and debug data. These listing files are useful for debugging, performance profiling, and code review, as shown in Figure 8-40.

Figure 8-40 GNU RISC-V Cross Create Listing



The settings are described in Table 8-20.

Table 8-20 GNU RISC-V Cross Create Listing

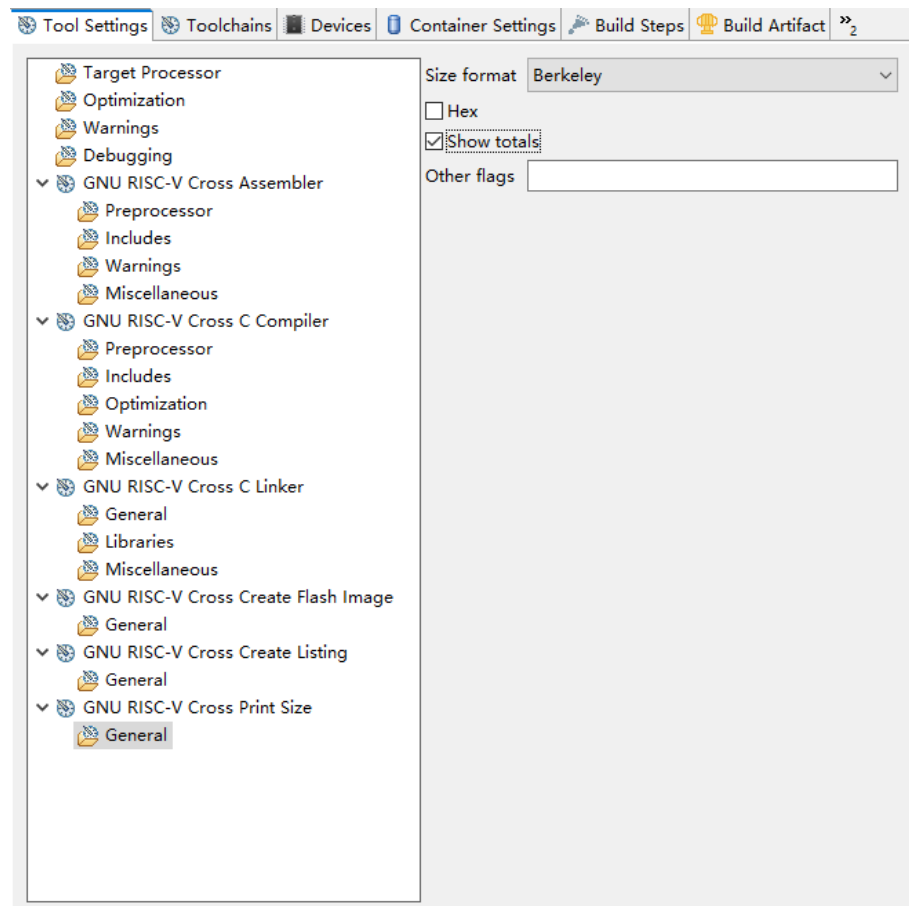
Option	Setting
Display source (--source -S)	✓
Display all headers (--all-headers -x)	✓
Demangle names (--demangle -C)	✓
Display debugging info (--debugging -g)	-
Disassemble (--disassemble -d)	-
Display file headers (--file-headers -f)	-
Display line numbers (--line-numbers -l)	✓

Option	Setting
Display relocation info (--reloc -r)	-
Display symbols (--syms -t)	-
Wide lines (--wide -w)	✓
Other flags	-

GNU RISC-V Cross Print Size

It is used to output the size statistics of each section (e.g., .text, .data, .bss) and the total size of a RISC-V MCU executable file. It supports both Berkeley and SysV formats. The choice of format affects only the display style and does not impact the actual calculation of section sizes, as shown in Figure 8-41.

Figure 8-41 GNU RISC-V Cross Print Size



The settings are described in Table 8-21.

Table 8-21 GNU RISC-V Cross Print Size

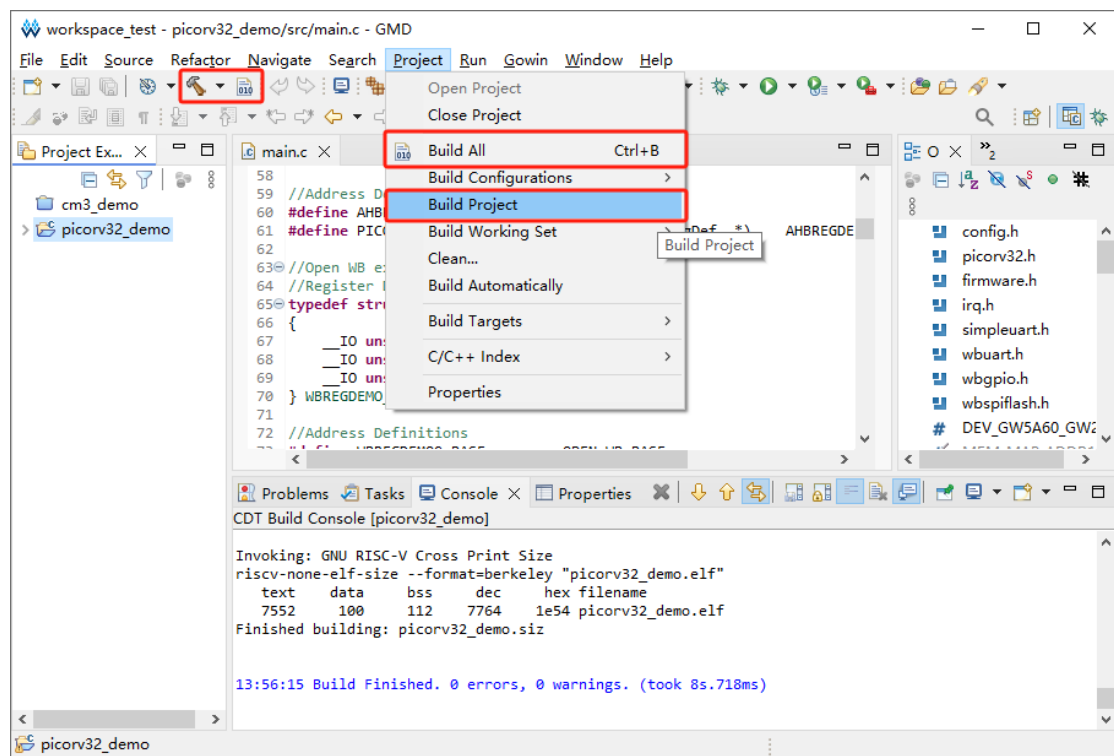
Option	Setting
Size format	Berkeley
Hex	-
Show totals	√
Other flags	-

8.3 Build Projects

8.3.1 Build Methods

There are several methods to build ARM MCU and RISC-V MCU projects, as shown in Figure 8-42.

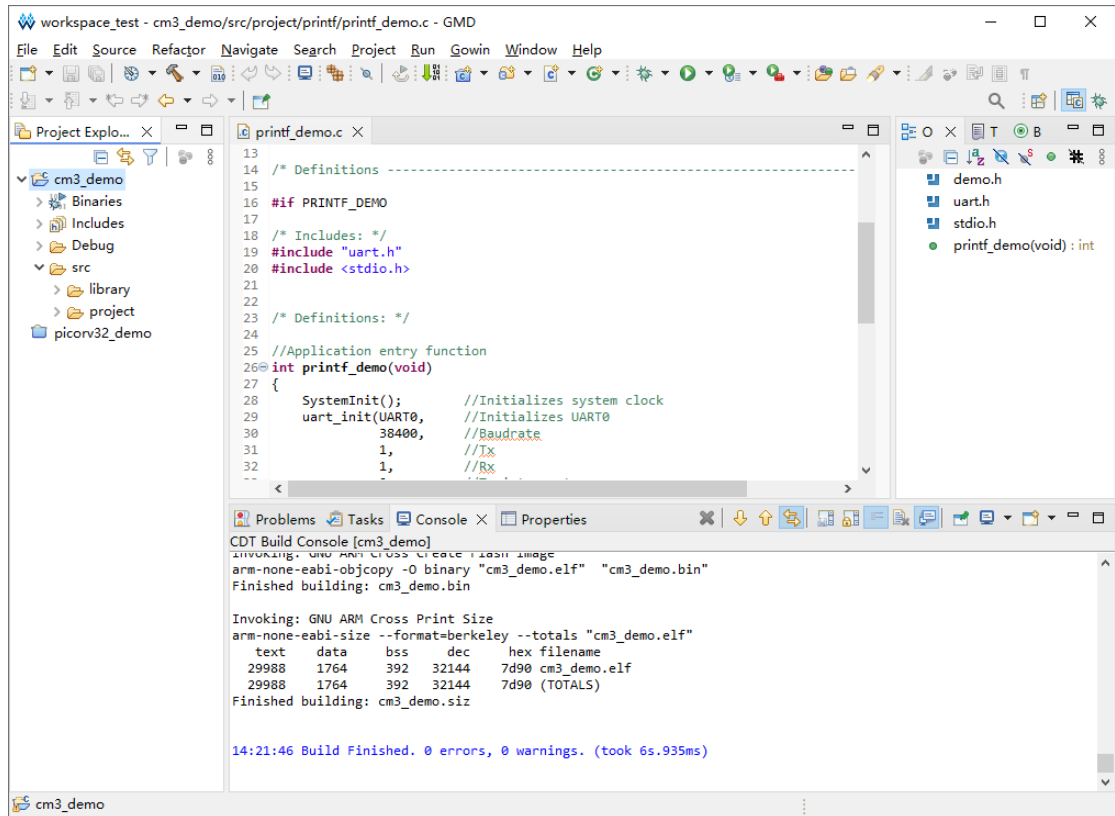
- Right-click the current project and select "Build Project"
- Click the "Build" button in the toolbar ()
- Click the "Build All" button in the toolbar ()
- Select "Project > Build All" from the menu bar
- Select "Project > Build Project" from the menu bar

Figure 8-42 Build

8.3.2 Build the Project

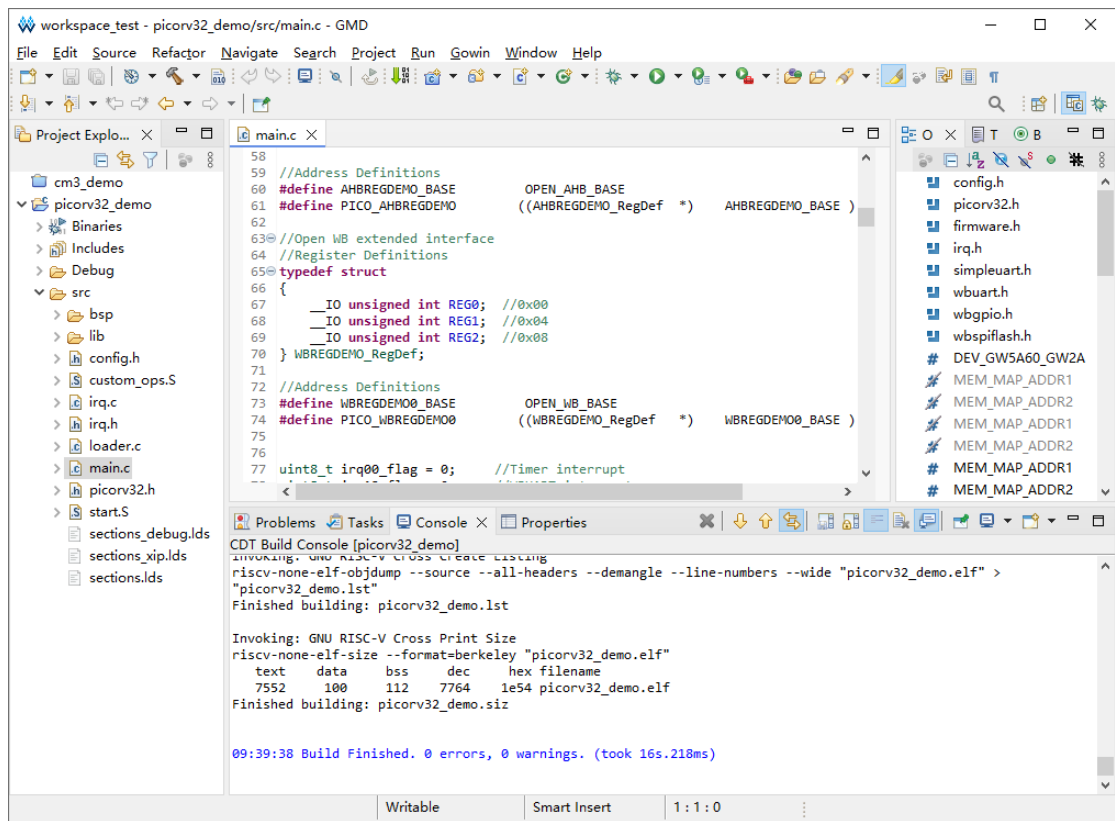
An example of building an ARM MCU project is shown in Figure 8-43.

Figure 8-43 ARM MCU Project Build Example



An example of building a RISC-V MCU project is shown in Figure 8-44.

Figure 8-44 RISC-V MCU Project Build Example

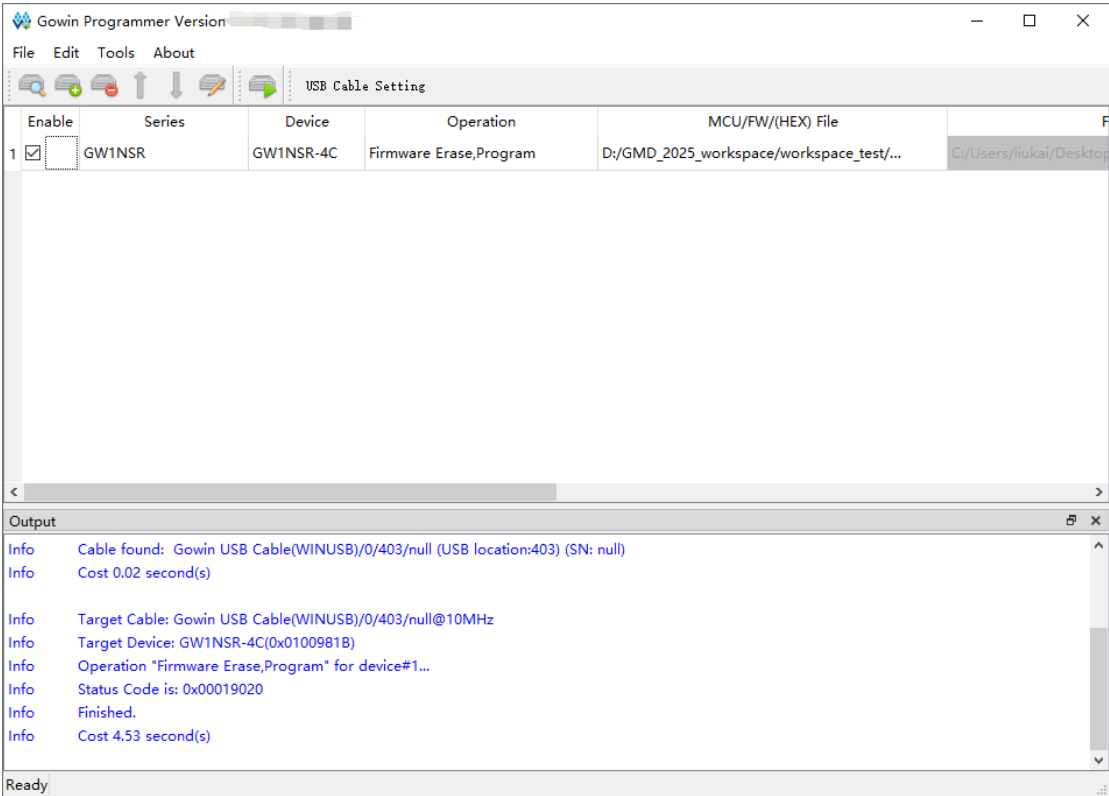


9Download Projects

9.1 Downloading Tool

GMD software integrates Gowin Programmer, which is used to download executable files for ARM MCU and RISC-V MCU, as shown in Figure 9-1.

Figure 9-1 Download Tool

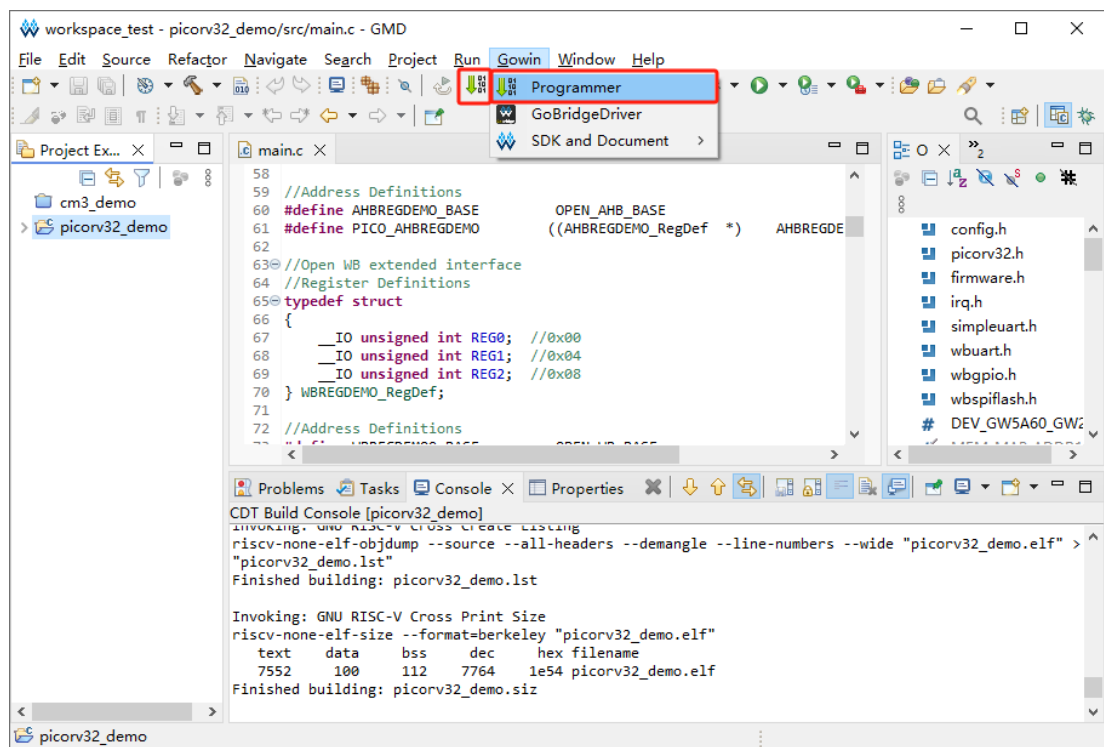


9.2 Downloading Methods

You can launch the Programmer tool using the following methods, as shown in Figure 9-2.

- Click the Programmer icon () in the toolbar.
- Select "Gowin > Programmer" from the menu bar.

Figure 9-2 Programmer



9.3 Download the Project

The download settings for ARM and RISC-V MCU projects depend on the specific FPGA device used. The following are several typical configuration examples.

1. GW1NS/R-4C

Gowin_EMPU (GW1NS-4C) configurations are as shown in Figure 9-3 and Table 9-1.

Figure 9-3 Download Settings

The screenshot shows a 'Device configuration' dialog box with the following settings:

- Device Operation:**
 - Access Mode: MCU Mode
 - Operation: Firmware Erase, Program
 - Text area: Firmware Erase, Program
- Programming Options:**
 - File name: sktop/GMD_test/gowin_empu/cm3_demo/impl/pnr/cm3_demo.fs
 - ☐ Retained User Flash Data
 - ☐ User Flash Initialization
- FW/MCU/Binary Input Options:**
 - Firmware/Binary File: e:/workspace_test/cm3_demo/Debug/cm3_demo.bin

Buttons: Save, Cancel

Table 9-1 Download Settings

Option	Setting
Access Mode	MCU Mode
Operation	Firmware Erase, Program Firmware Erase, Program, Verify
File name	.fs file
Firmware/Binary File	.bin file

2. GW1NSER-4C

The Secure FPGA Gowin_EMPU(GW1NS-4C) configurations are as shown in Figure 9-4 and Table 9-2.

Figure 9-4 Download Settings

The screenshot shows a 'Device configuration' dialog box with the following settings:

- Device Operation:**
 - Access Mode: SecureFPGA Mode
 - Operation: Firmware Erase, Program Securely
 - Description: Firmware Erase, Program, Verify with Security(One Time Programmable For Key Authentication Code)
- Programming Options:**
 - File name: sktop/GMD_test/gowin_empu/cm3_demo/impl/pnr/cm3_demo.fs
 - ☐ Retained User Flash Data
 - ☐ User Flash Initialization
- FW/MCU/Binary Input Options:**
 - Firmware/Binary File: e:/workspace_test/cm3_demo/Debug/cm3_demo.bin

Buttons at the bottom: Save, Cancel.

Table 9-2 Download Settings

Option	Setting
Access Mode	SecureFPGA Mode
Operation	Firmware Erase, Program Securely
File name	.fs file
Firmware/Binary File	.bin file

3. GW1N/R-9 (C)

The Gowin_EMPU_M1 and Gowin_PicoRV32 configurations are as shown in Figure 9-5 and Table 9-3.

Figure 9-5 Download Settings

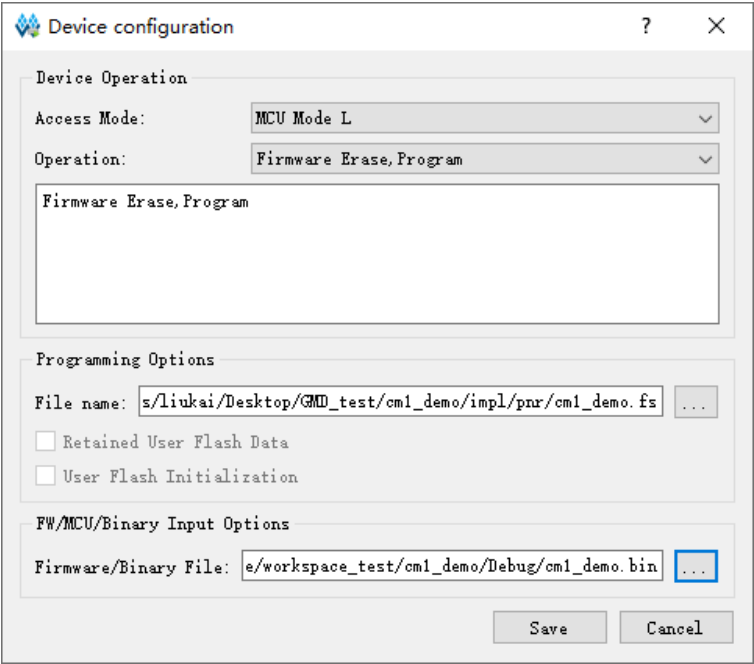


Table 9-3 Download Settings

Option	Setting
Access Mode	MCU Mode L
Operation	Firmware Erase, Program Firmware Erase, Program, Verify
File name	.fs file
Firmware/Binary File	.bin file

4. GW2A/N/R/NR

The Gowin_EMPU_M1 and Gowin_PicoRV32 configurations are as shown in Figure 9-6 and Table 9-4.

Figure 9-6 Download Settings

The screenshot shows a 'Device configuration' dialog box with the following settings:

- Device Operation:**
 - Access Mode: External Flash Mode
 - Operation: exFlash C Bin Erase, Program thru GAO-Bridge
 - Note! When programming exFlash using GAO Bridge in DUAL BOOT mode, please ensure that the "Start Address" is set to 0x000000; otherwise, the FPGA will fail to load data from exflash.
- External Flash Options:**
 - Device: Generic Flash
 - Start Address: 0x400000
- FW/MCU/Binary Input Options:**
 - Firmware/Binary File: e:/workspace_test/cml_demo/Debug/cml_demo.bin

Buttons: Save, Cancel

Table 9-4 Download Settings

Option	Setting
Access Mode	External Flash Mode
Operation	exFlash C Bin Erase, Program thru GAO-Bridge exFlash C Bin Erase, Program, Verify thru GAO-Bridge
Device	Generic Flash
Start Address	GW2A/N/NR/R-18 (C/X): 0x400000 GW2A/N-55 (C): 0x400000
Firmware/Binary File	.bin file

5. GW5A/NT/NRT/R/RT/S/ST/T

The Gowin_EMPU_M1 and Gowin_PicoRV32 configurations are as shown in Figure 9-7 and Table 9-5.

Figure 9-7 Download Settings

The screenshot shows a 'Device configuration' dialog box with the following settings:

- Device Operation:**
 - Access Mode: External Flash Mode 5A
 - Operation: exFlash C Bin Erase, Program 5A
 - exFlash C Bin Erase, Program 5A
- External Flash Options:**
 - Device: Generic Flash
 - Start Address: 0x600000
- FW/MCU/Binary Input Options:**
 - Firmware/Binary File: e:/workspace_test/cm1_demo/Debug/cm1_demo.bin

Buttons: Save, Cancel

Table 9-5 Download Settings

Option	Setting
Access Mode	External Flash Mode 5A
Operation	exFlash C Bin Erase, Program 5A exFlash C Bin Erase, Program, Verify 5A
Device	Generic Flash
Start Address	GW5ANRT/NT/RT/T-15 (A/B): 0x100000 GW5A/R/S-25 (A/B): 0x100000 GW5A/T-60B: 0x400000 GW5AT-75 (B/C): 0x600000 GW5A/S/ST/T-138 (B/C): 0x600000
Firmware/Binary File	.bin file

10 Debug and Run Projects

10.1 Debugging Tools

GMD software integrates Gowin OpenOCD debugging tool, a customized version of OpenOCD developed by Gowin based on the standard OpenOCD version, which is optimized for Gowin's MCU products. OpenOCD is an open-source on-chip debugger designed to provide debugging, system programming, and boundary scan features for embedded systems. It typically works in conjunction with a hardware debugger, which is a small unit that provides the electrical signals for the debug target.

GMD software also supports J-Link debugging. To use it, please visit the SEGGER official website to download and install the SEGGER J-Link driver suitable for your operating system.

Additionally, GMD software integrates Gowin QEMU tool, a customized version of QEMU based on the standard QEMU version, which is optimized for Gowin MCU devices. QEMU is a hosted virtual machine that provides a variety of hardware and peripheral parts, allowing online simulation and emulation without requiring a physical device.

10.2 Notes

All Gowin MCU products do not support automatic downloading during debugging in the GMD software. Before each debugging session, you must first manually download the executable file using the Programmer tool. This ensures that the current project is being debugged.

10.3 Debugging Modes

GMD software supports multiple Gowin MCU products, debugging modes, and debugging emulators, as summarized in Table 10-1.

Table 10-1 Debugging Modes

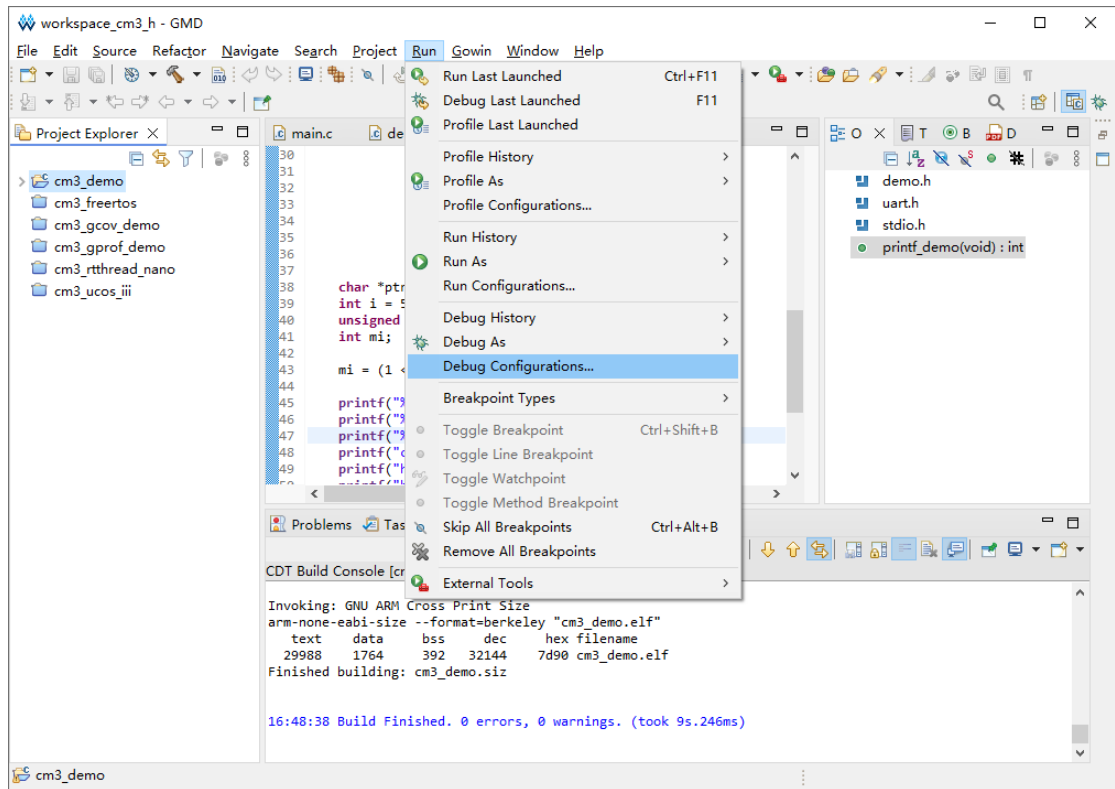
MCU Device	Debugging Mode	Debugging Emulator
Gowin_EMPU(GW1NS-4C)	GDB OpenOCD Debugging	GWU2X
		GWUSB (dual FTDI)
		GWUSB (single FTDI)
		J-Link (WinUSB driver)
	GDB SEGGER J-Link Debugging	J-Link (Segger J-Link driver)
Gowin_EMPU_M1	GDB OpenOCD Debugging	GWU2X
		GWUSB (dual FTDI)
		GWUSB (single FTDI)
		J-Link (WinUSB driver)
	GDB SEGGER J-Link Debugging	J-Link (Segger J-Link driver)
Gowin_EMPU(GW5AS-25)	GDB SEGGER J-Link Debugging	J-Link (Segger J-Link driver)
Gowin_EMPU_M3	GDB SEGGER J-Link Debugging	J-Link (Segger J-Link driver)
Gowin_PicoRV32	GDB OpenOCD Debugging	GWU2X
		GWUSB (dual FTDI)
		GWUSB (single FTDI)
		J-Link (WinUSB driver)
		Olimex
	GDB QEMU riscv32 Debugging	-

10.4 Debugging Methods

The following methods can be used to debug ARM MCU and RISC-V MCU projects, as shown in Figure 10-1.

- Right-click the current project and select "Debug As > Debug Configurations...".
- Click "Debug > Debug Configurations..." from the toolbar ().
- Select "Run > Debug Configurations..." from the menu bar.

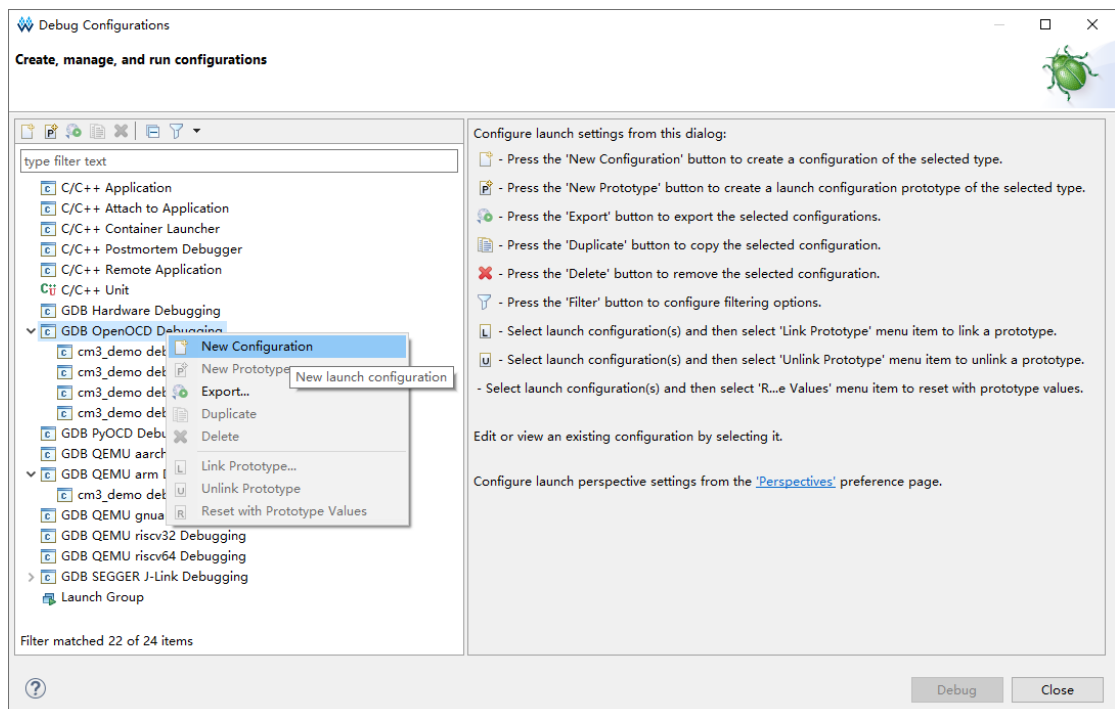
Figure 10-1 Debug



10.5 Debugging Settings

In "Debug Configurations" dialog box, create a debug view for a specific mode and set the debug options, as shown in Figure 10-2.

Figure 10-2 Create a Debug View



Debug settings for both ARM MCU and RISC-V MCU are described separately as follows.

10.5.1 ARM MCU Debugging Setting

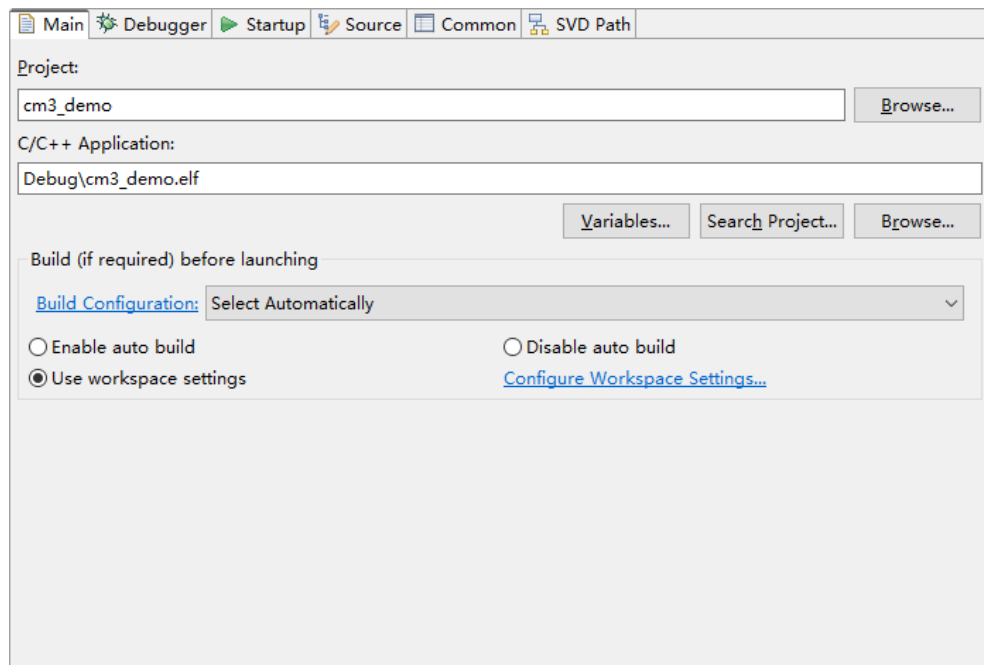
For example, Gowin_EMPU (GW1NS-4C) supports GDB OpenOCD Debugging, GDB SEGGER J-Link Debugging, and GDB QEMU ARM Debugging modes, as shown in Table 10-1.

GDB OpenOCD Debugging

1. Main View

The debugging settings in the Main view are shown in Figure 10-3.

Figure 10-3 Main View



2. Debugger View

The debugging settings in the Debugger view are shown in Figure 10-4 and Table 10-2.

Figure 10-4 Debugger View

The screenshot shows the 'Debugger' tab in an IDE. It contains three main sections: 'OpenOCD Setup', 'GDB Client Setup', and 'Remote Target'.

- OpenOCD Setup:**
 - ☒ Start OpenOCD locally
 - Executable path: `${openocd_path}/${openocd_executable}` (with 'Browse...' and 'Variables...' buttons)
 - Actual executable: `E:\GMD\toolchain\gowin\openocd\bin\openocd.exe`
 - (to change it use the [global](#) or [workspace](#) preferences pages or the [project](#) properties page)
 - GDB port: `3333`
 - Telnet port: `4444`
 - Tcl port: `6666`
 - Config options: `-f board/gowin_u2x_m3.cfg`
 - ☒ Allocate console for OpenOCD
 - ☐ Allocate console for the telnet connection
- GDB Client Setup:**
 - ☒ Start GDB session
 - Executable name: `${cross_prefix}gdb${cross_suffix}` (with 'Browse...' and 'Variables...' buttons)
 - Actual executable: `arm-none-eabi-gdb`
 - Other options: (empty text box)
 - Commands: `set mem inaccessible-by-default off`
- Remote Target:**
 - Host name or IP address: `localhost`
 - Port number: `3333`
 - ☐ Force thread list update on suspend

Table 10-2 Debugger View

Option	Setting
Start OpenOCD locally	✓
GDB port	3333
Config options	-f board/x.cfg The corresponding configuration files (x.cfg) for different ARM MCU devices are listed in Table 10-3.
Start GDB session	✓
Commands	set mem inaccessible-by-default off

Table 10-3 OpenOCD Configuration Files

ARM MCU Device	Debugger Emulator	Configuration File
Gowin_EMPU(GW1NS-4C)	GWU2X	gowin_u2x_m3.cfg
	GWUSB (dual FTDI)	gowin_ftdi_dual_m3.cfg

ARM MCU Device	Debugger Emulator	Configuration File
Gowin_EMPU_M1	GWUSB (single FTDI)	gowin_ftdi_single_m3.cfg
	J-Link (WinUSB driver)	gowin_jlink_m3.cfg
	GWU2X	gowin_u2x_m1.cfg
	GWUSB (dual FTDI)	gowin_ftdi_dual_m1.cfg
	GWUSB (single FTDI)	gowin_ftdi_single_m1.cfg
	J-Link (WinUSB driver)	gowin_jlink_m1.cfg

3. Startup View

The debugging settings in the Startup view are shown in Figure 10-5 and Table 10-4.

Figure 10-5 Startup View

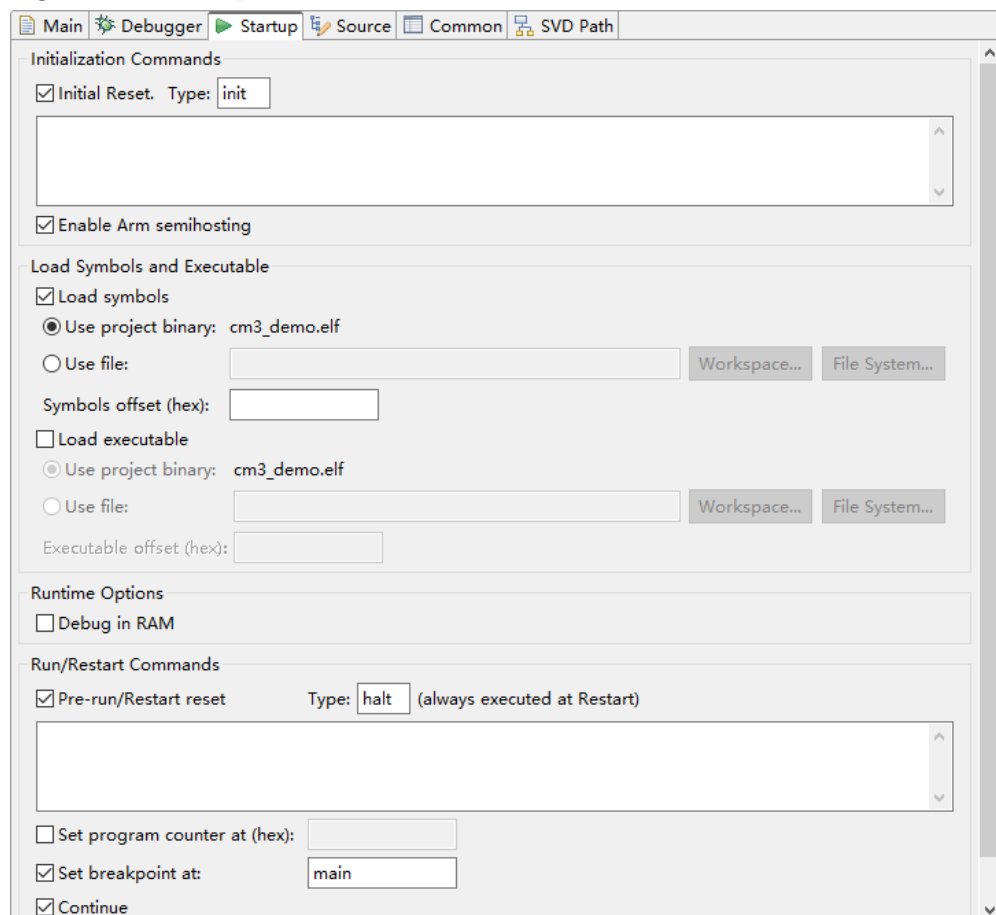


Table 10-4 Startup View

Option	Setting
Initial Reset.	✓

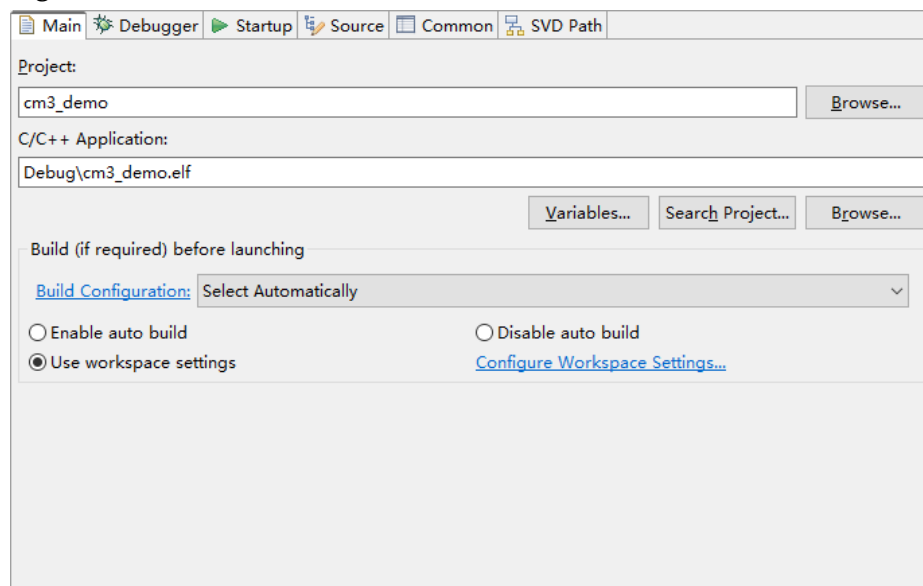
Option	Setting
Type	init
Enable ARM semihosting	✓
Load symbols	✓
Load executable	×
Debug in RAM	×
Pre-run/Restart reset	✓
Type	halt
Set program counter at (hex)	-
Set breakpoint at	main
Continue	✓

GDB SEGGER J-Link Debugging

1. Main View

The debugging settings in the Main view are shown in Figure 10-6.

Figure 10-6 Main View



2. Debugger View

The debugging settings in the Debugger view are shown in Figure 10-7 and Table 10-5.

Figure 10-7 Debugger View

J-Link GDB Server Setup

☒ Start the J-Link GDB server locally ☐ Connect to running target

Executable path:

Actual executable:
 (to change it use the [global](#) or [workspace](#) preferences pages or the [project](#) properties page)

Device name: [Supported device names](#)

Endianness: ☒ Little ☐ Big

Connection: ☒ USB ☐ IP (USB serial or IP name/address)

Interface: ☐ SWD ☒ JTAG

Initial speed: ☐ Auto ☐ Adaptive ☒ Fixed kHz

GDB port:

SWO port: ☒ Verify downloads ☒ Initialize registers on start

Telnet port: ☒ Local host only ☐ Silent

Log file:

Other options:

☒ Allocate console for the GDB server ☒ Allocate console for semihosting and SWO

GDB Client Setup

Executable name:

Actual executable:

Other options:

Commands:

Remote Target

Host name or IP address:

Port number:

☐ Force thread list update on suspend

Table 10-5 Debugger View

Option	Setting
Start the J-Link GDB server locally	✓
Connect to running target	×
Device name	For example, Cortex-M3 Part numbers for ARM MCU devices are listed in Table 10-6.
Endianness	Little
Connection	USB
Interface	JTAG SWD

Option	Setting
Initial speed	Fixed
GDB port	2331
Verify downloads	√
Initialize registers on start	√
Local host only	√
Silent	-
Other options	-singlerun -strict -timeout 0 -nogui
Allocate console for semihosting and SWO	√
Commands	set mem inaccessible-by-default off

Table 10-6 ARM MCU Part Number

ARM MCU Device	Part Number
Gowin_EMPU(GW1NS-4C)	Cortex-M3
Gowin_EMPU_M1	Cortex-M1
Gowin_EMPU(GW5AS-25)	Cortex-M4
Gowin_EMPU_M3	Cortex-M3

3. Startup View

The debugging settings in the Startup view are shown in Figure 10-8 and Table 10-7.

Figure 10-8 Startup View

Main Debugger **Startup** Source Common SVD Path

Initialization Commands
☒ Initial Reset and Halt Type: Low speed: kHz
 JTAG/SWD Speed: ☒ Auto ☐ Adaptive ☐ Fixed kHz
☒ Enable flash breakpoints
☒ Enable semihosting Console routed to: ☒ Telnet ☐ GDB client
☒ Enable SWO CPU freq: Hz. SWO freq: Hz. Port mask:

Load Symbols and Executable
☒ Load symbols
☒ Use project binary: cm3_demo.elf
☐ Use file: Workspace... File System...
 Symbols offset (hex):
☐ Load executable
☒ Use project binary: cm3_demo.elf
☐ Use file: Workspace... File System...
 Executable offset (hex):

Runtime Options
☐ RAM application (reload after each reset/restart)

Run/Restart Commands
☒ Pre-run/Restart reset Type: (always executed at Restart)
☐ Set program counter at (hex):
☒ Set breakpoint at:
☒ Continue

[Restore defaults](#)

Table 10-7 Startup View

Option	Setting
Initial Reset and Halt	✓
JTAG/SWD Speed	Auto
Enable flash breakpoints	✓
Enable semihosting Console routed to	Telnet
Load symbols	✓
Load executable	×
RAM application (reload after each reset/restart)	×

Option	Setting
Pre-run/Restart reset	✓
Set program counter at (hex)	-
Set breakpoint at	main
Continue	✓

GDB QEMU arm Debugging

For QEMU emulation of ARM MCU devices, please refer to Chapter 11 QEMU Emulation & Debugging.

10.5.2 RISC-V MCU Debugging Setting

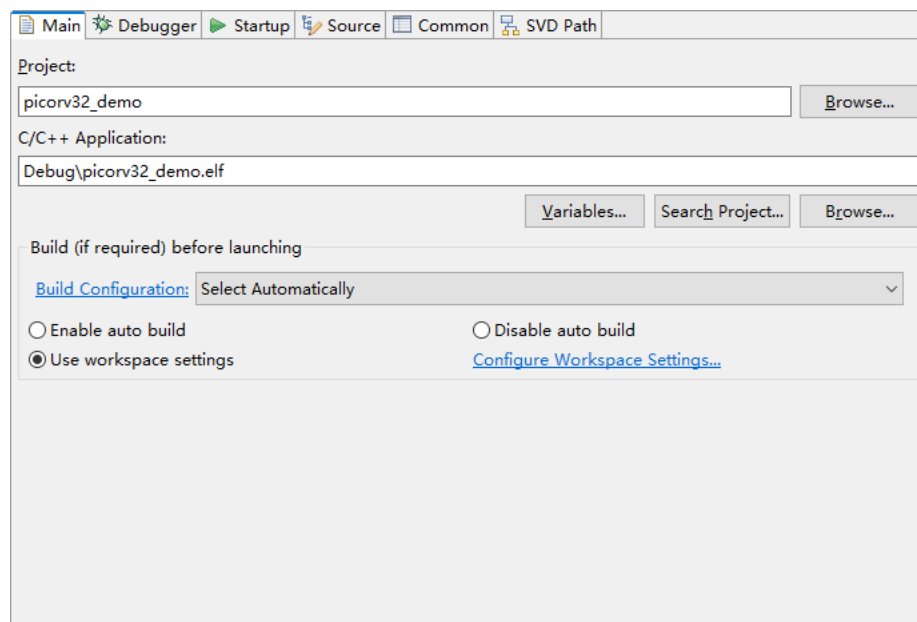
For example, Gowin_PicoRV32 supports GDB OpenOCD Debugging and GDB QEMU riscv32 Debugging modes, as shown in Table 10-1.

GDB OpenOCD Debugging

1. Main View

The debugging settings in the Main view are shown in Figure 10-9.

Figure 10-9 Main View



2. Debugger View

The debugging settings in the Debugger view are shown in Figure 10-10 and Table 10-8.

Figure 10-10 Debugger View

The screenshot shows the 'Debugger' tab in an IDE. It contains two main sections: 'OpenOCD Setup' and 'GDB Client Setup'.

OpenOCD Setup:

- ☒ Start OpenOCD locally
- Executable path: `${openocd_path}/${openocd_executable}` (with 'Browse...' and 'Variables...' buttons)
- Actual executable: `E:\GMD\toolchain\gowin\openocd\bin\openocd.exe`
- (to change it use the [global](#) or [workspace](#) preferences pages or the [project](#) properties page)
- GDB port: `3333`
- Telnet port: `4444`
- Tcl port: `6666`
- Config options: `-f board/gowin_u2x_pico.cfg`
- ☒ Allocate console for OpenOCD ☐ Allocate console for the telnet connection

GDB Client Setup:

- ☒ Start GDB session
- Executable name: `${cross_prefix}gdb${cross_suffix}` (with 'Browse...' and 'Variables...' buttons)
- Actual executable: `riscv-none-elf-gdb`
- Other options: (empty text box)
- Commands: `set mem inaccessible-by-default off`
`set arch riscv:rv32`
`set remotetimeout 250`

Remote Target:

- Host name or IP address: `localhost`
- Port number: `3333`
- ☐ Force thread list update on suspend

Table 10-8 Debugger View

Option	Setting
Start OpenOCD locally	✓
GDB port	3333
Config options	-f board/x.cfg The corresponding x.cfg files for RISC-V MCU devices are listed in Table 10-9.
Start GDB session	✓
Commands	set mem inaccessible-by-default off set arch riscv:rv32 set remotetimeout 250

Table 10-9 OpenOCD Configuration File

RISC-V MCU Device	Debugger Emulator	Configuration File
Gowin_PicoRV32	GWU2X	gowin_u2x_pico.cfg
	GWUSB (dual FTDI)	gowin_ftdi_dual_pico.cfg
	GWUSB (single FTDI)	gowin_ftdi_single_pico.cfg
	J-Link (WinUSB driver)	gowin_jlink_pico.cfg
	Olimex	gowin_olimex_pico.cfg

3. Startup View

The debugging settings in the Startup view are shown in Figure 10-11 and Table 10-10.

Figure 10-11 Startup View

The screenshot shows the 'Startup' tab of the OpenOCD configuration interface. The 'Initialization Commands' section has 'Initial Reset' checked with a type of 'init'. Below it is an empty text box. The 'Enable Arm semihosting' checkbox is unchecked. The 'Load Symbols and Executable' section has 'Load symbols' and 'Load executable' both checked. For both, 'Use project binary' is selected with the file 'picorv32_demo.elf'. There are buttons for 'Workspace...' and 'File System...' next to the file selection. The 'Runtime Options' section has 'Debug in RAM' checked. The 'Run/Restart Commands' section has 'Pre-run/Restart reset' checked with a type of 'halt' and a note '(always executed at Restart)'. Below this is another empty text box. At the bottom, 'Set program counter at (hex):' is unchecked, 'Set breakpoint at:' is checked with the value 'main', and 'Continue' is checked.

Table 10-10 Startup View

Option	Setting
Initial Reset.	√
Type	init
Enable ARM semihosting	×
Load symbols	√
Load executable	×
Debug in RAM	√
Pre-run/Restart reset	√
Type	halt
Set program counter at (hex)	-
Set breakpoint at	main
Continue	√

GDB QEMU riscv32 Debugging

For QEMU emulation of RISC-V MCU devices, please refer to Chapter [11 QEMU Emulation & Debugging](#).

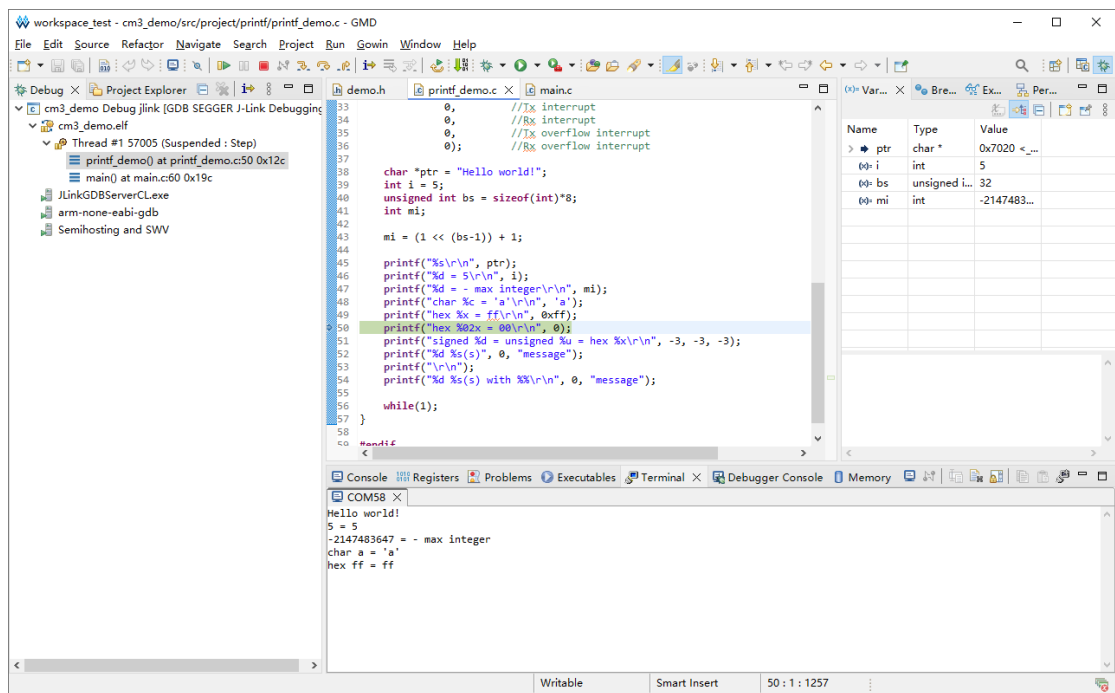
10.6 Run the Project

For example, when running Gowin_EMPU (GW1NS-4C) project, the Segger J-Link debug emulator can be used

10.6.1 Launch Debug View

After completing the debugging settings, click "Debug" to launch the debug view and run the project, as shown in Figure 10-12.

Figure 10-12 Launch Debug View



10.6.2 Debugging Options

The debugging options are listed in Table 10-11.

Table 10-11 Debugging Options

Option	Description
Resume ()	Resume the paused debugging thread
Suspend ()	Suspend the debugging thread
Terminate ()	Stop the debugging thread
Step Into ()	Step Into the current statement of the debugging thread (enters sub-function calling)
Step Over ()	Step over the current statement of the debugging thread (skips sub-function calling)
Step Return ()	Step return from the current statement until returning to the caller
Restart ()	Restart a new debugging thread (stops current thread)
Instruction Stepping Mode ()	Enter disassembly view and step through assembly instructions

10.7 Debug View

After launching the debug thread, GMD software automatically opens the corresponding debug view. To call a specific debug view, you can go to the menu bar and select "Window > Show View > Other...", as shown in Figure 10-13. In the "Show View > Debug" dropdown list, select the debug view you want to display, as shown in Figure 10-14.

Figure 10-13 Window > Show View > Other...

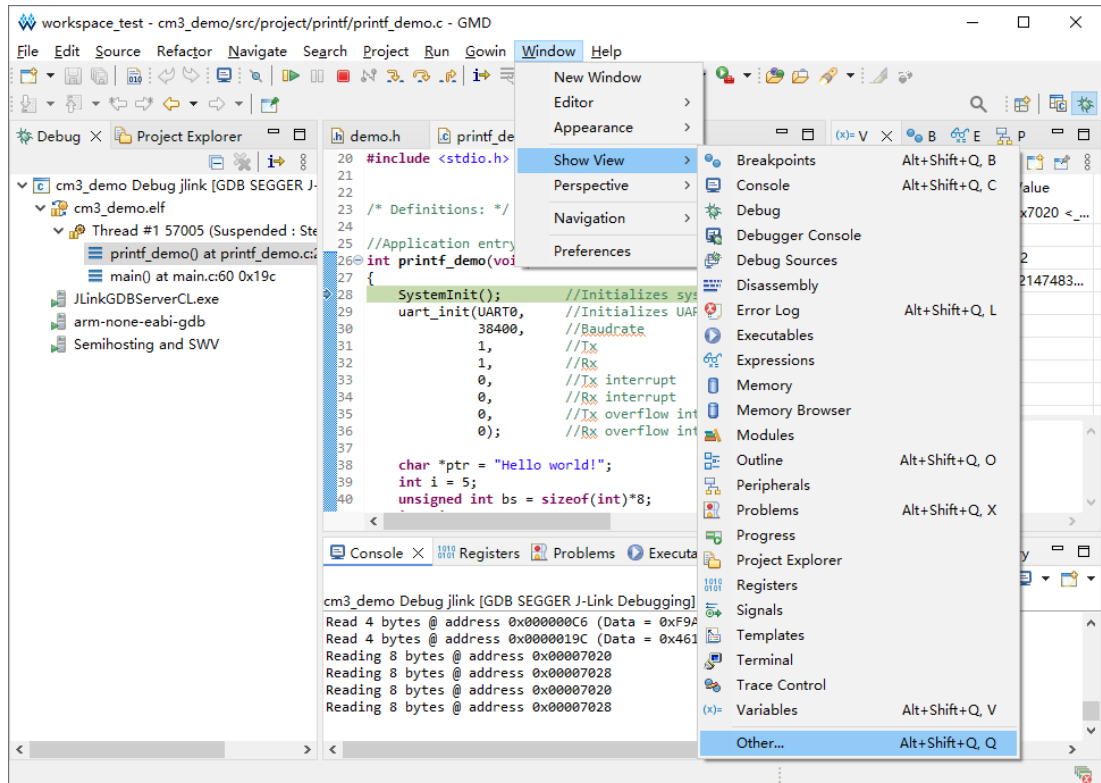
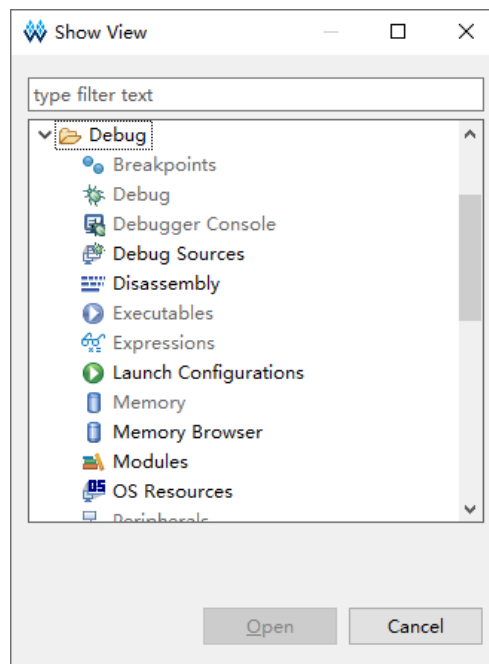


Figure 10-14 Debug View

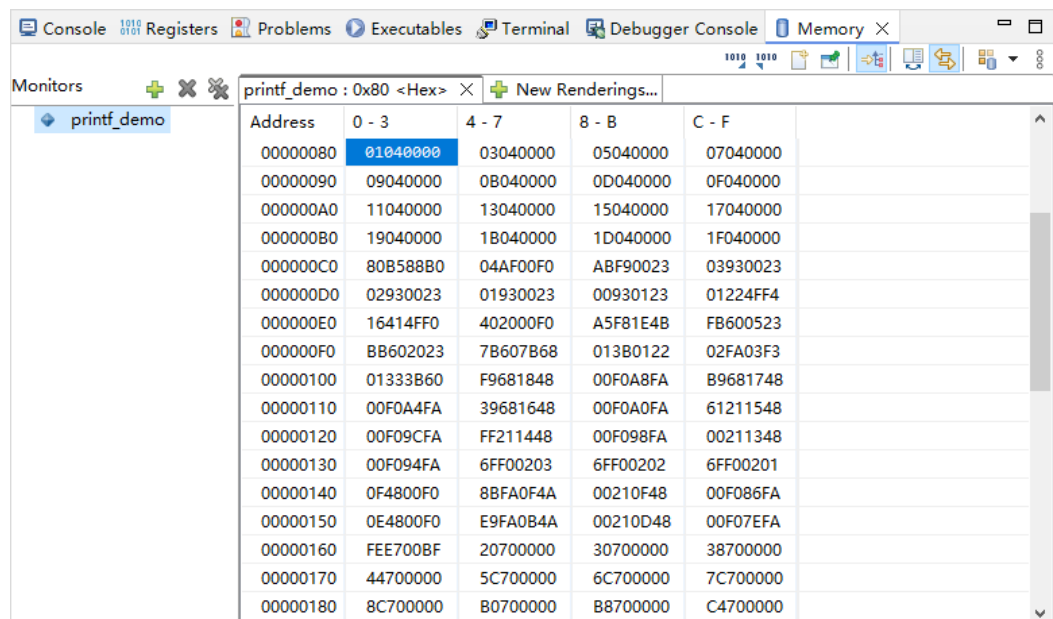


Some commonly used debug views are described below.

10.7.1 Memory View

The Memory view allows you to inspect and modify the program memory through a list of memory monitors, as shown in Figure 10-15. Each memory monitor is associated with a memory segment defined by a specific base address or expression, and the data in each monitor can be displayed in various predefined formats.

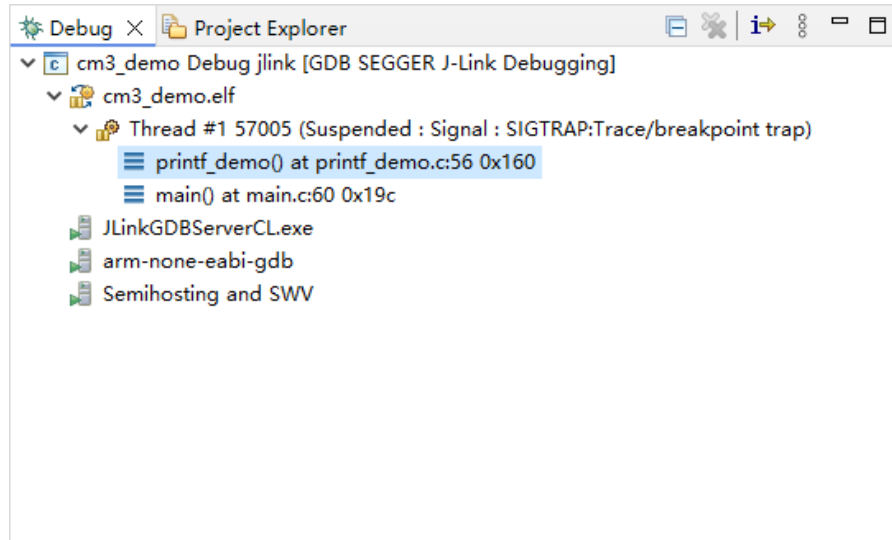
Figure 10-15 Memory View



Create a Memory Monitor

1. After pausing the program, select a thread or stack structure you want to monitor in the debug view, as shown in Figure 10-16.

Figure 10-16 Select a Thread or Stack to Monitor



2. In the Memory view, select "Monitors > Add Memory Monitor" (, as shown in Figure 10-17. In the "Monitor Memory" dialog box, enter the memory address or expression you want to view, as shown in Figure 10-18.

Figure 10-17 Add Memory Monitor

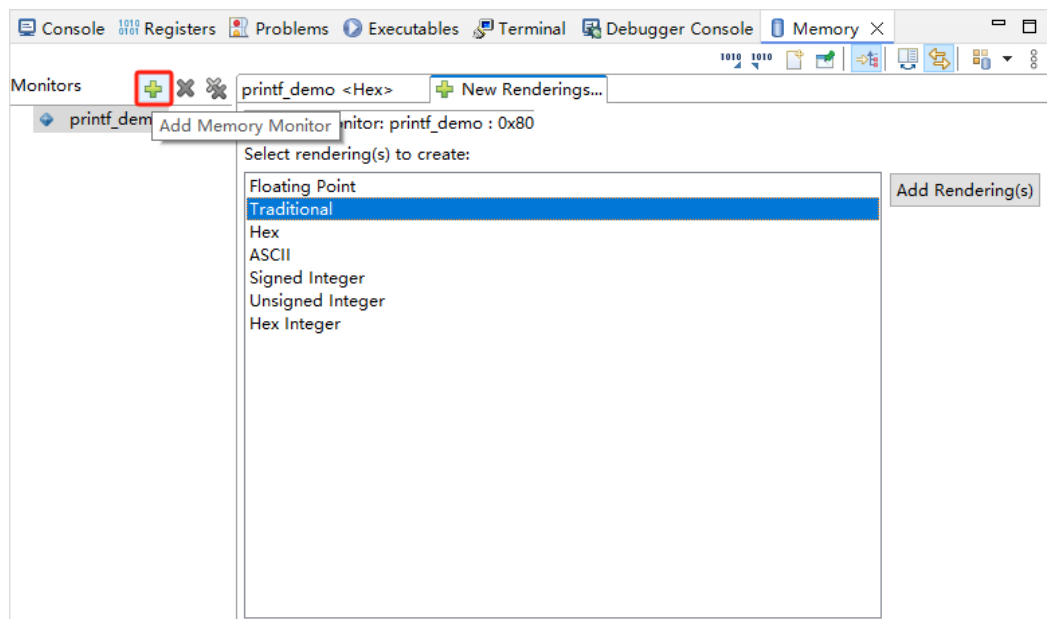
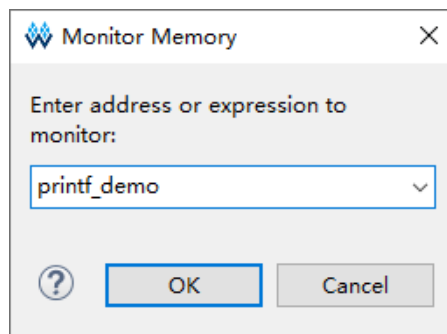
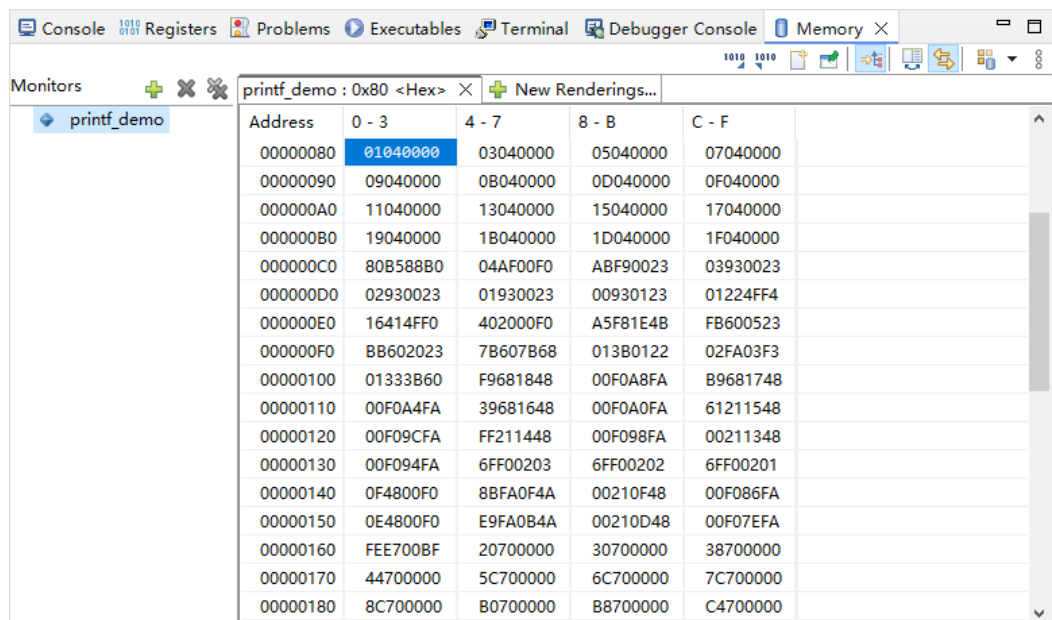


Figure 10-18 Monitor Memory



3. By default, the memory view displays a memory renderer in traditional format, as shown in Figure 10-19.

Figure 10-19 Memory Renderer in Traditional Format



As shown in Figure 10-20, you can also choose "New Renderings... > Floating Point > Add Rendering(s)" to create a memory renderer in floating-point format, as shown in Figure 10-21.

Figure 10-20 Create a Memory Renderer in Floating-Point Format

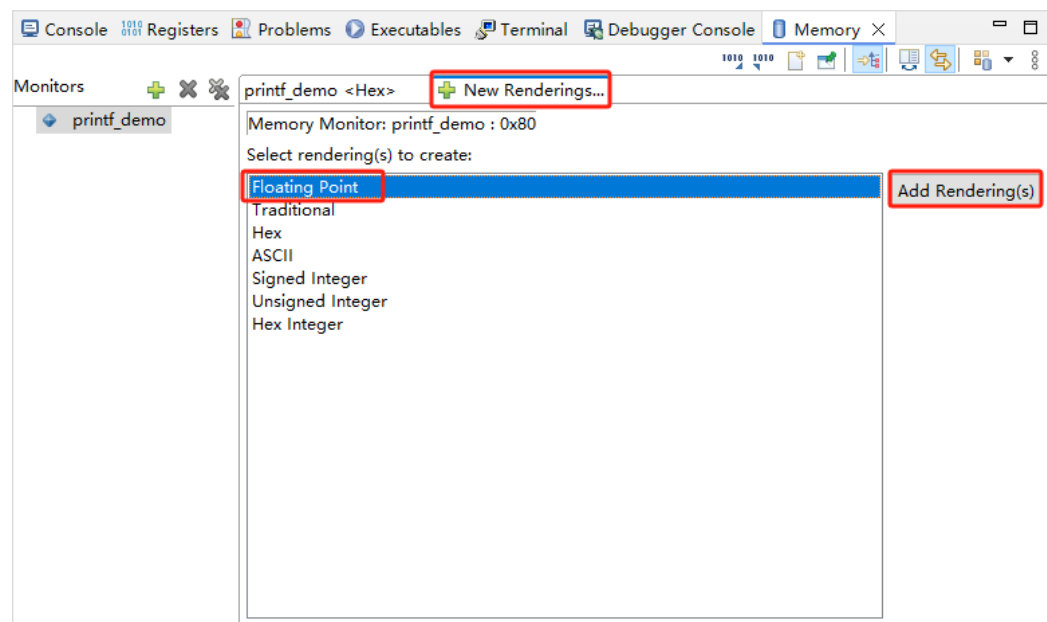
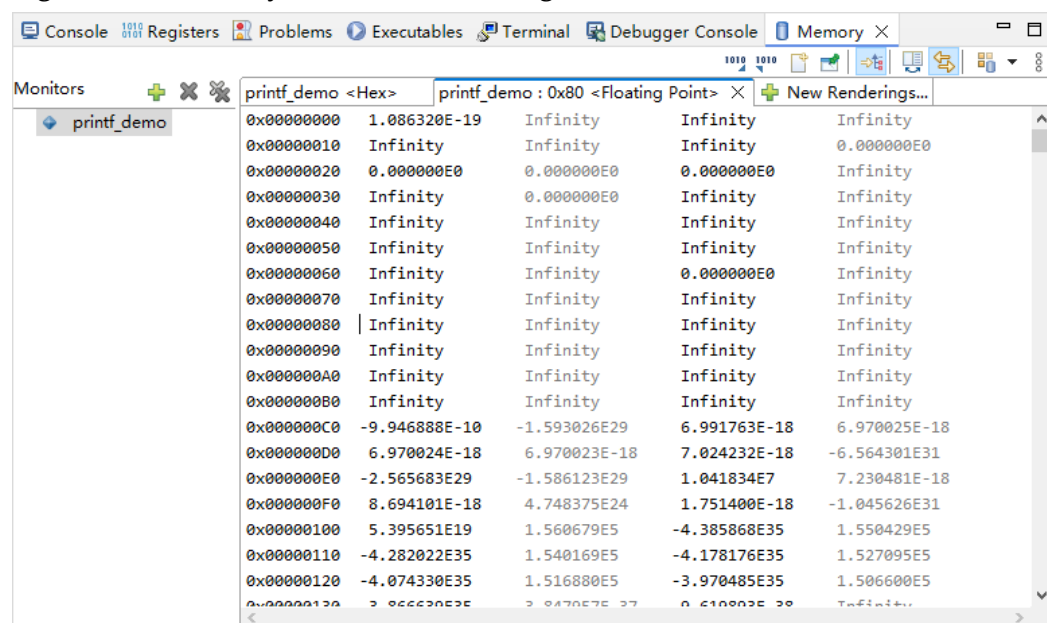
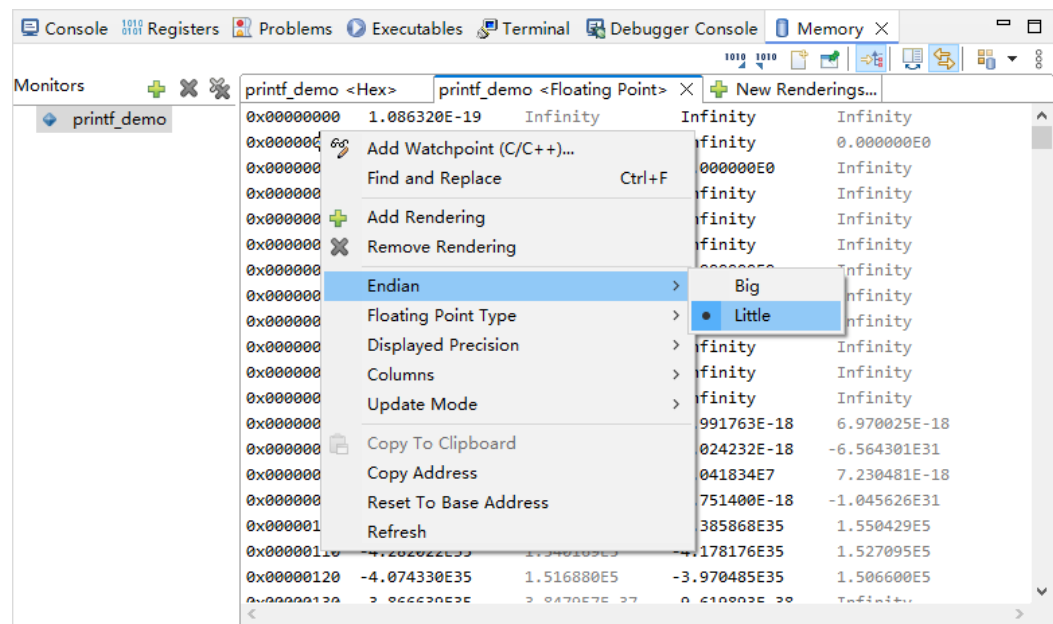


Figure 10-21 Memory Renderer in Floating-Point Format



- Right-click anywhere in the memory renderer, and in the dropdown menu, select "Endian" to modify the display of byte order or addressable unit size for the data, as shown in Figure 10-22.

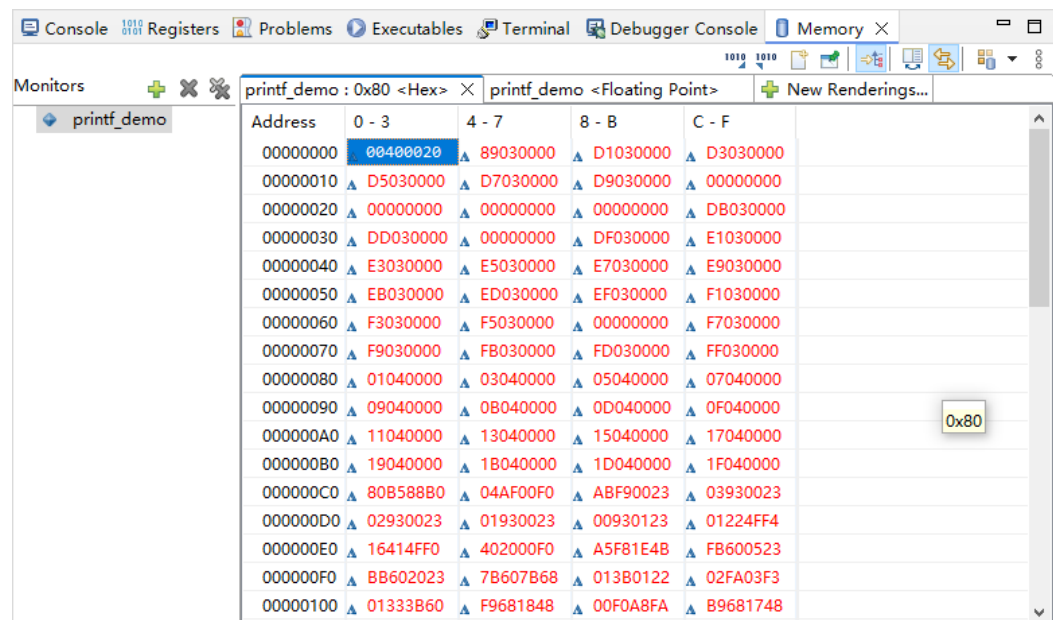
Figure 10-22 Set Byte Order



You can also modify the memory values in each cell. However, entering an invalid value may cause the program to crash.

5. During program running, changes in stored values are highlighted in red, as shown in Figure 10-23.

Figure 10-23 Memory Content Display



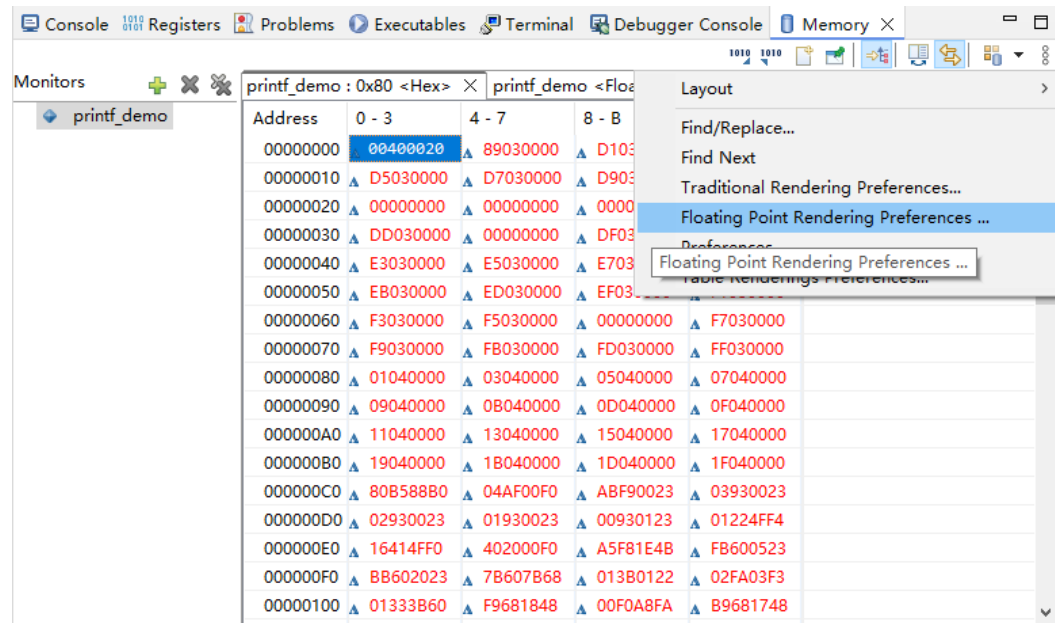
Set Rendering Preferences

The memory renderer uses black text by default. You can follow the steps below to distinguish rendered data and addresses using different

colors.

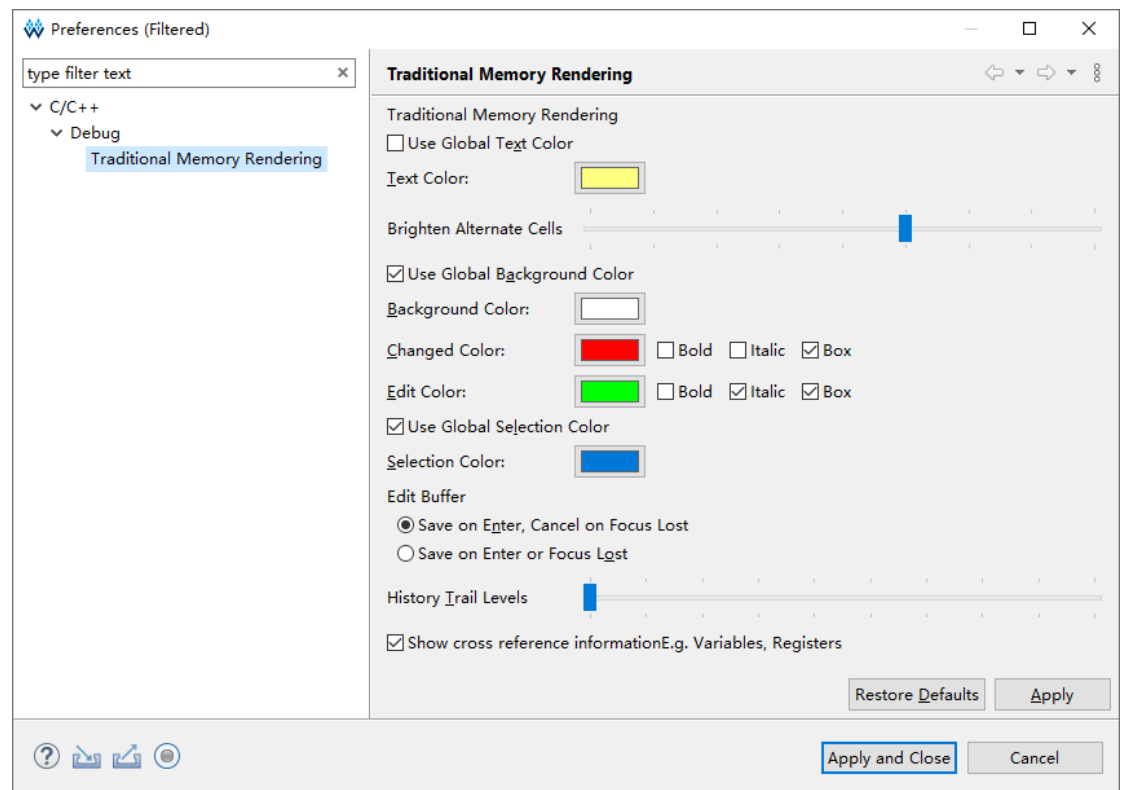
1. In the Memory view, click "View Menu" (). From the dropdown menu, select either "Traditional Rendering Preferences..." or "Floating Point Rendering Preferences..." to configure the rendering as needed, as shown in Figure 10-24.

Figure 10-24 Set Rendering Preferences



2. For example, navigate to "Preferences > C/C++ > Debug > Traditional Memory Rendering", uncheck "Use Global Text Color", and then customize the Text Color, as shown in Figure 10-25.

Figure 10-25 Configure Memory Rendering Appearance

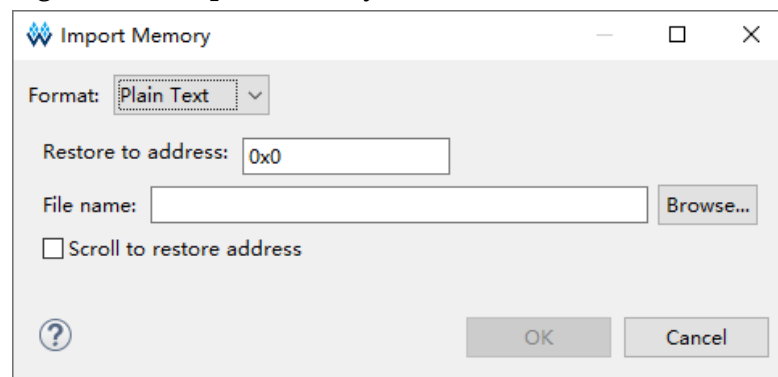


Toolbar in Memory View

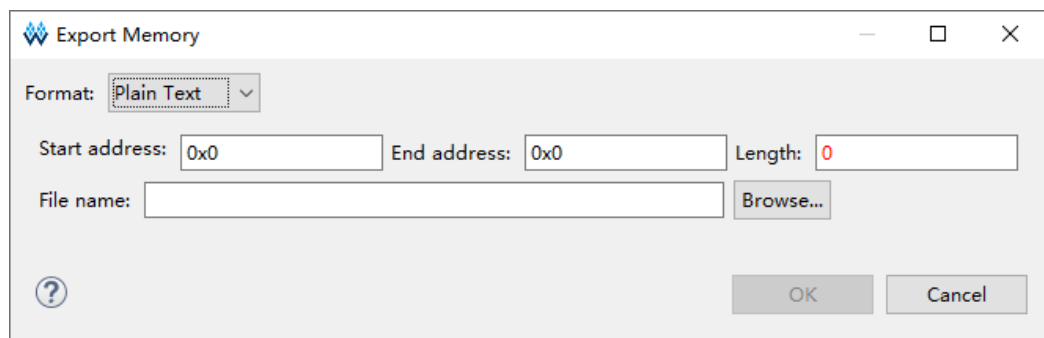
The toolbar in the Memory view includes the following functions:

- Import (): Import a specified set of memory values, as shown in Figure 10-26.

Figure 10-26 Import Memory



- Export (): Export the memory values displayed in the memory renderer to a file, as shown in Figure 10-27.

Figure 10-27 Export Memory

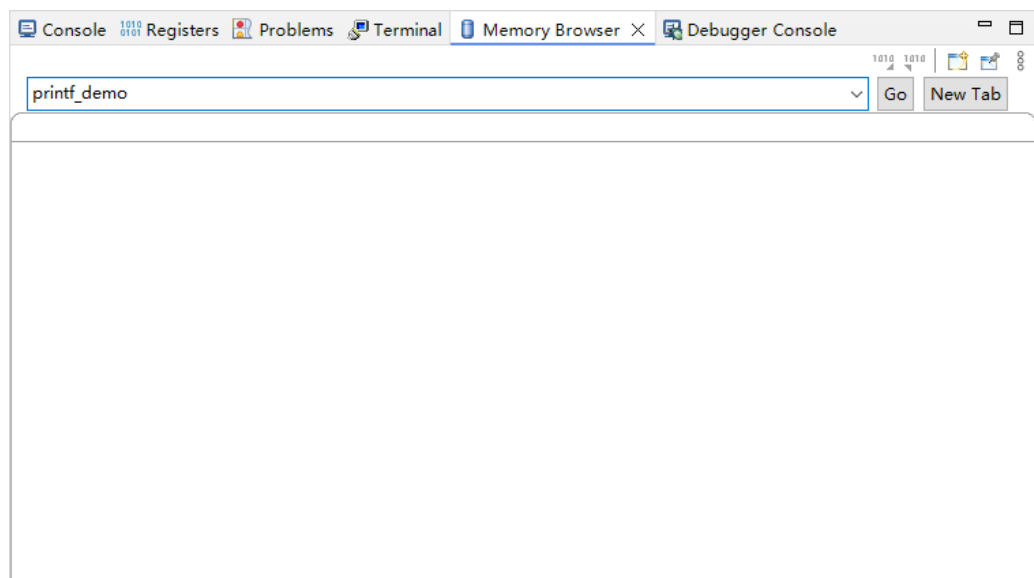
GMD software supports three formats for importing/exporting memory data, including Plain Text, RAW Binary, and SRecord. Plain Text and RAW Binary are more commonly used. Plain Text contains hexadecimal values, with each byte storing two hexadecimal characters. RAW Binary contains binary values.

10.7.2 Memory Browser View

The Memory Browser view is an alternative to the Memory view, used for inspecting and modifying process memory.

Create a Memory Monitor

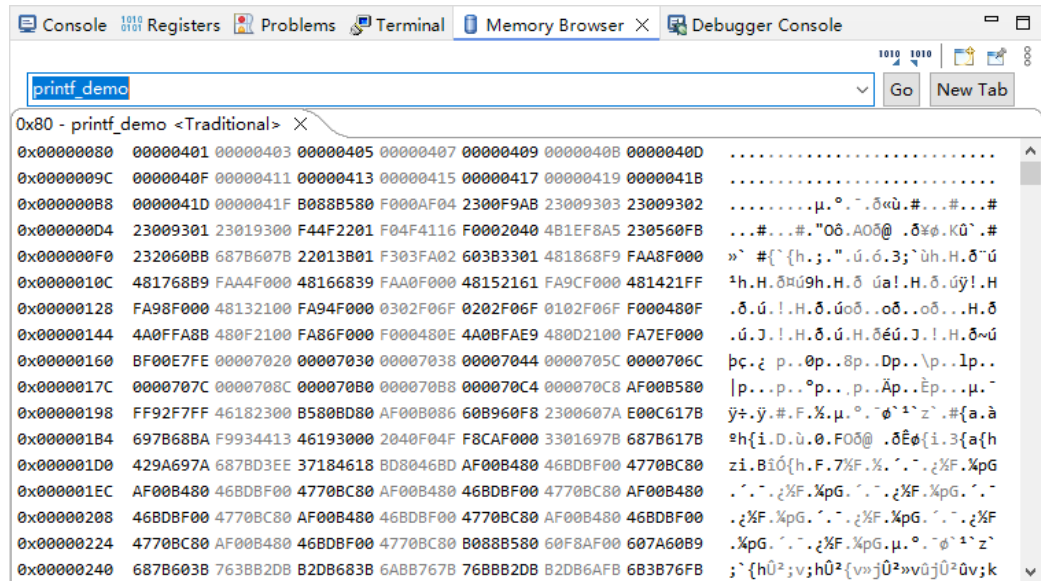
1. After the program is paused, enter the memory address or expression you want to view in the text box of the Memory Browser view and click "Go" to create a memory monitor, as shown in Figure 10-28.

Figure 10-28 Memory Browser View

2. By default, the Memory Browser view displays a memory renderer in

traditional format, as shown in Figure 10-29.

Figure 10-29 Memory Renderer in Traditional Format



You can also create a memory renderer in floating-point format. Click "View Menu" (), and from the dropdown menu, select "Default Rendering > Floating Point > New Tab" to create a memory renderer in floating-point format, as shown in Figure 10-30 and Figure 10-31.

Figure 10-30 Create a Memory Renderer in Floating-Point Format

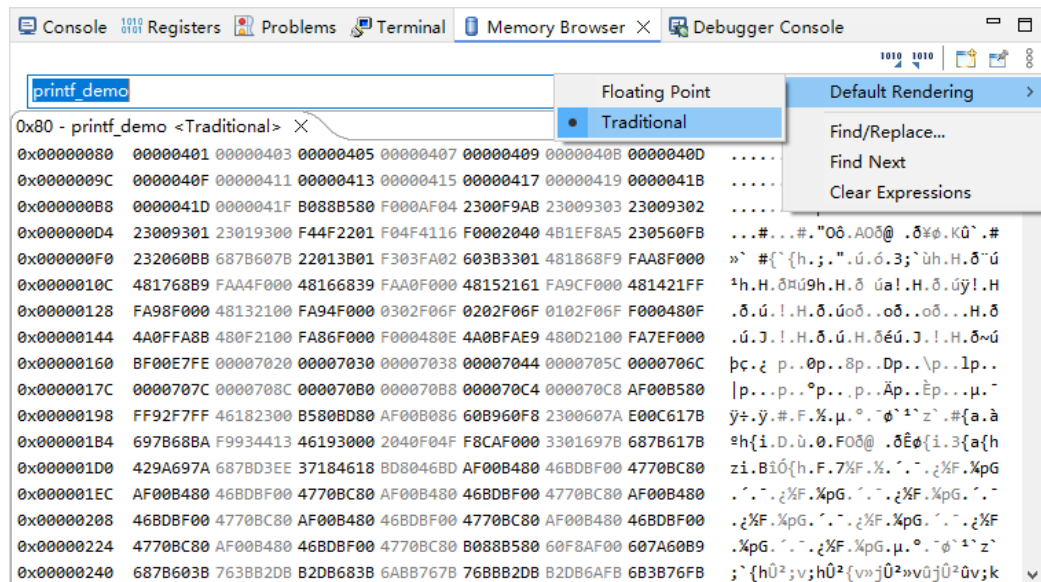
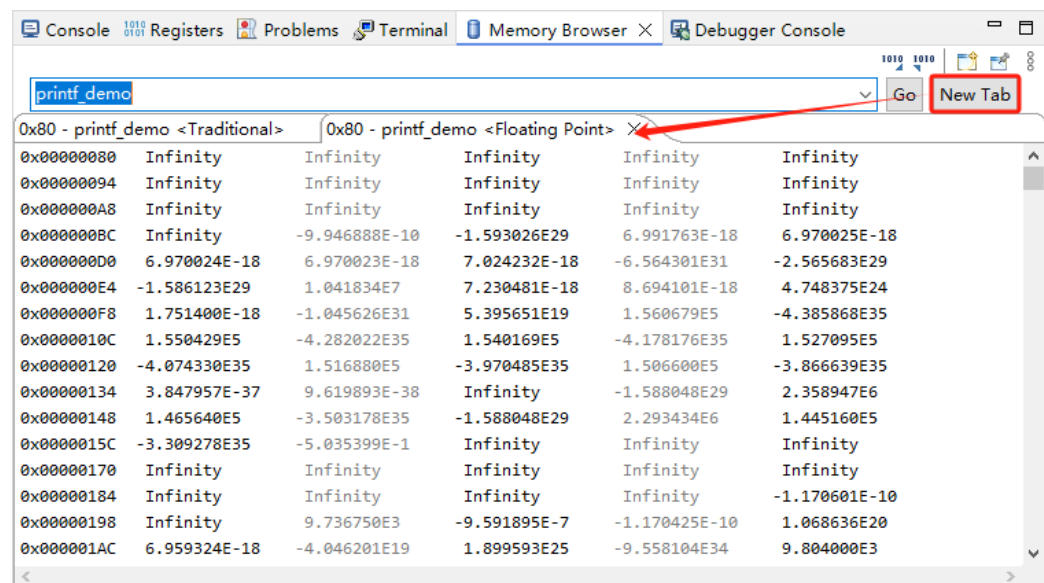
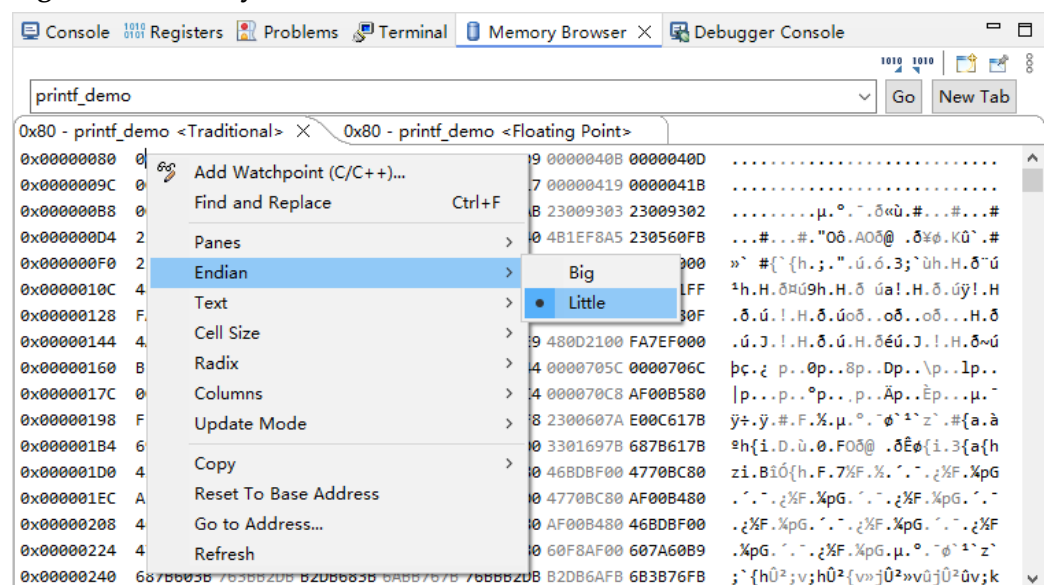


Figure 10-31 Memory Renderer in Floating-Point Format



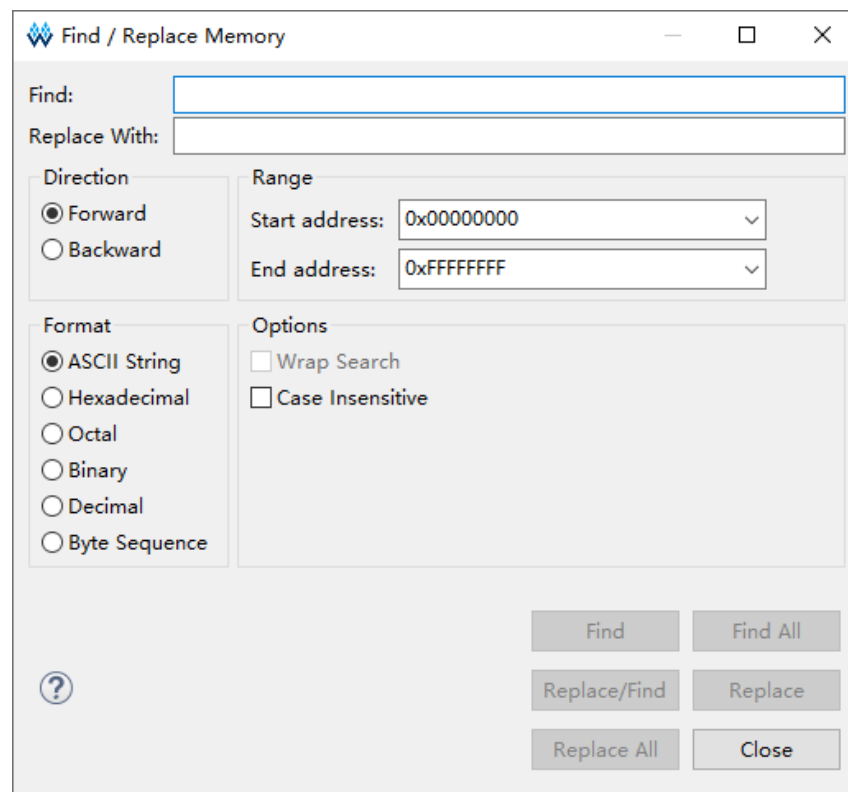
3. Right-click anywhere in the memory renderer and choose "Endian" from the dropdown menu to change the display of the byte order or addressable unit size for the data, as shown in Figure 10-32.

Figure 10-32 Set Byte Order



4. From the dropdown menu of the "View Menu" (), select "Find/Replace..." to locate and replace specific data values in the selected memory monitor, as shown in Figure 10-33.

Figure 10-33 Find/Replace



10.7.3 Registers View

The Registers view displays CPU information at the register bit-field level. You can directly modify the values of registers or bit-fields within the cells, as shown in Figure 10-34.

Figure 10-34 Registers View

Name	Value	Description
General Registers		General Purpose and FPU Register Group
r0	10	
r1	0	
r2	574619648	
r3	301989888	
r4	952682120	
r5	838334808	
r6	3143087189	
r7	536887264	
r8	2374924313	
r9	809339810	
r10	3705169638	
r11	1429610927	
r12	21	
sp	0x20003fd0	
lr	353	

10.7.4 Variables View

The Variables view displays the names and values of all variables within the current scope, as shown in Figure 10-35. When the program is paused, you can inspect and monitor changes in variable values.

Figure 10-35 Variables View

[illegible]

10.7.5 Expressions View

The Expressions view allows you to add and monitor custom expressions, as shown in Figure 10-36. This is useful for evaluating complex expressions or testing code snippets during debugging.

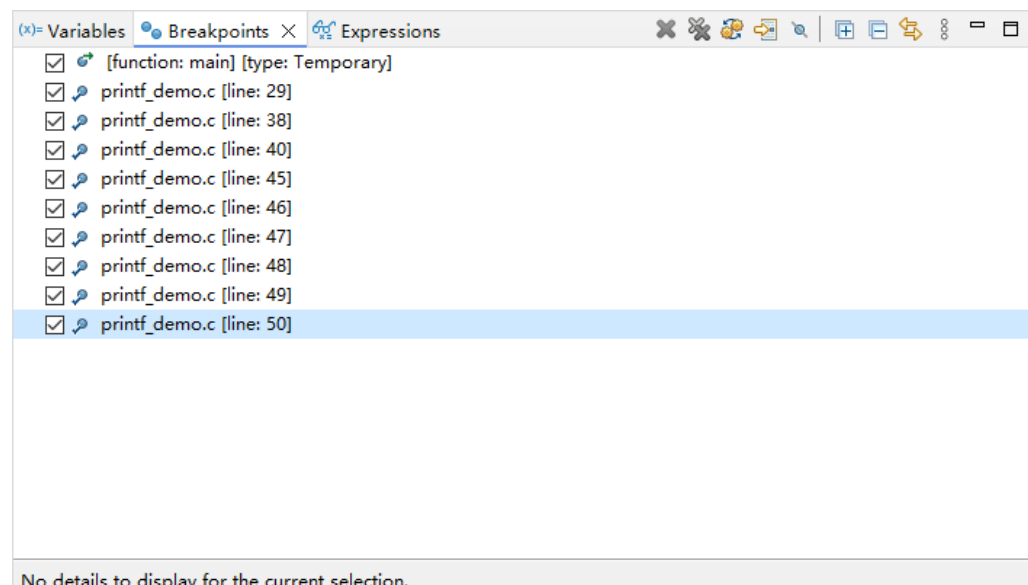
Figure 10-36 Expressions View

[illegible]

10.7.6 Breakpoints View

The Breakpoints view lists all breakpoints set in the software project. Users can manage breakpoints, including enabling, disabling, adding, deleting breakpoints, or setting conditions for breakpoints, as shown in Figure 10-37.

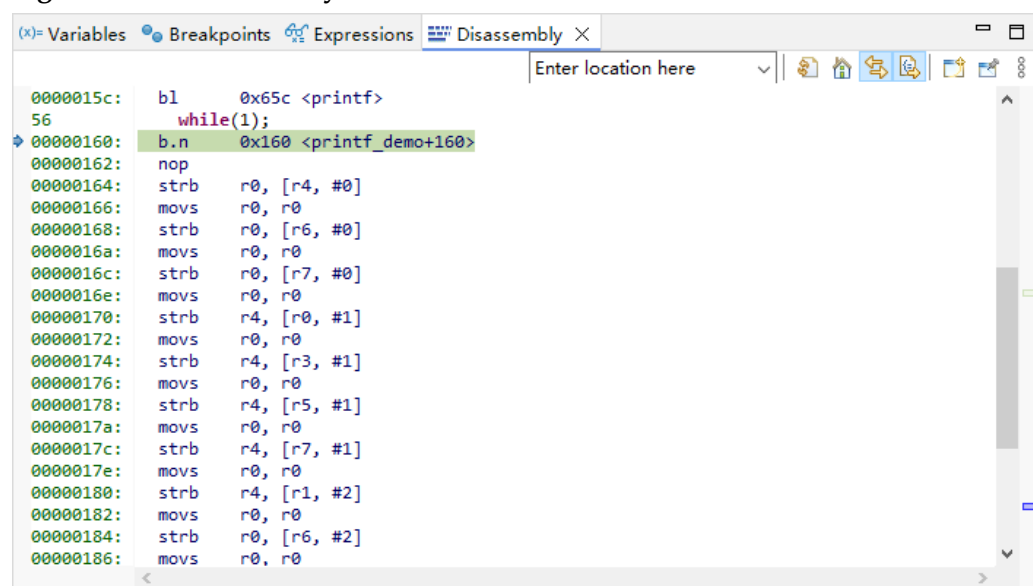
Figure 10-37 Breakpoints View



10.7.7 Disassembly View

The Disassembly view displays the assembly code corresponding to the high-level language source. It is used to troubleshoot instruction-level issues during debugging, as shown in Figure 10-38.

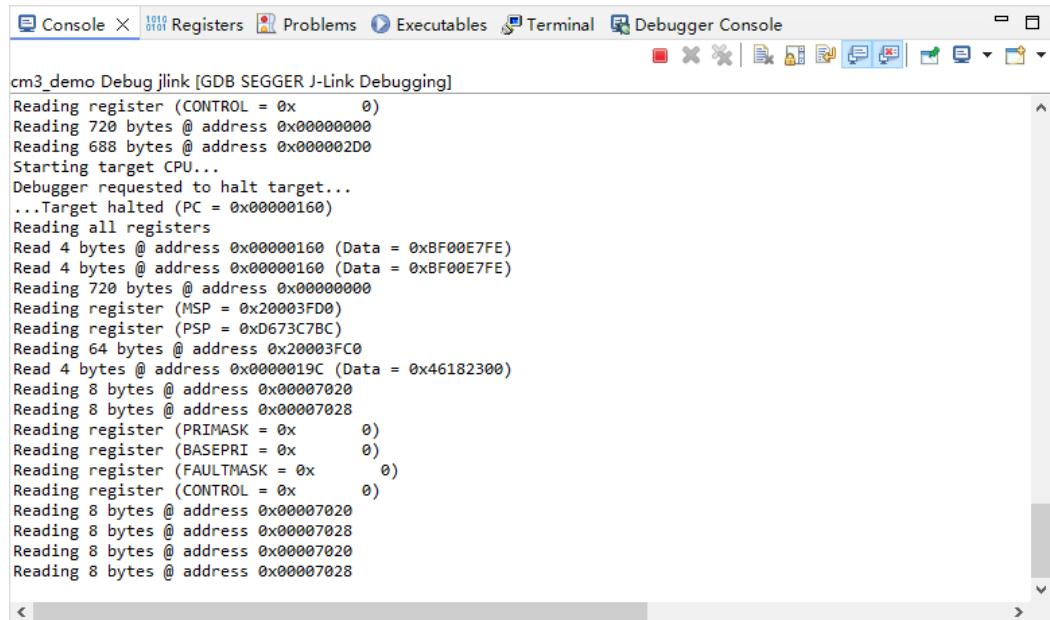
Figure 10-38 Disassembly View



10.7.8 Console View

The Console view displays process output, such as debugging logs, as shown in Figure 10-39.

Figure 10-39 Console View



The screenshot shows the 'Debugger Console' tab in an IDE. The title bar includes 'Console', 'Registers', 'Problems', 'Executables', 'Terminal', and 'Debugger Console'. The main area displays the following text:

```
cm3_demo Debug jlink [GDB SEGGER J-Link Debugging]
Reading register (CONTROL = 0x 0)
Reading 720 bytes @ address 0x00000000
Reading 688 bytes @ address 0x000002D0
Starting target CPU...
Debugger requested to halt target...
...Target halted (PC = 0x00000160)
Reading all registers
Read 4 bytes @ address 0x00000160 (Data = 0xBF00E7FE)
Read 4 bytes @ address 0x00000160 (Data = 0xBF00E7FE)
Reading 720 bytes @ address 0x00000000
Reading register (MSP = 0x20003FD0)
Reading register (PSP = 0xD673C7BC)
Reading 64 bytes @ address 0x20003FC0
Read 4 bytes @ address 0x0000019C (Data = 0x46182300)
Reading 8 bytes @ address 0x00007020
Reading 8 bytes @ address 0x00007028
Reading register (PRIMASK = 0x 0)
Reading register (BASEPRI = 0x 0)
Reading register (FAULTMASK = 0x 0)
Reading register (CONTROL = 0x 0)
Reading 8 bytes @ address 0x00007020
Reading 8 bytes @ address 0x00007028
Reading 8 bytes @ address 0x00007020
Reading 8 bytes @ address 0x00007028
```

11 QEMU Emulation & Debugging

11.1 QEMU Tool

The GMD software integrates the Gowin QEMU tool—a customized development by Gowin based on the standard QEMU version, specifically optimized for the features of Gowin MCU products. QEMU operates as a hosted virtual machine that provides diverse hardware and peripheral models, enabling direct online simulation without requiring a physical hardware platform.

In the GMD 2025.01 release, the QEMU tool now supports both Gowin_EMPU (GW1NS-4C) and Gowin_PicoRV32. Their respective supported peripheral functions are detailed in Table 11-1.

Table 11-1 Supported Peripheral Functions

MCU Product	Supported Peripheral	Description
Gowin_EMPU(GW1NS-4C)	UART	Input and output, interrupt handling
	GPIO	Input and output
	SysTick	Counting, interrupt handling
	Timer	Counting, interrupt handling
Gowin_PicoRV32	UART	Input and output
	GPIO	Input and output

11.2 Use QEMU

There are several methods to launch QEMU for ARM MCU and RISC-V MCU:

- Right-click the current project and select "Debug As > Debug

Configurations...".

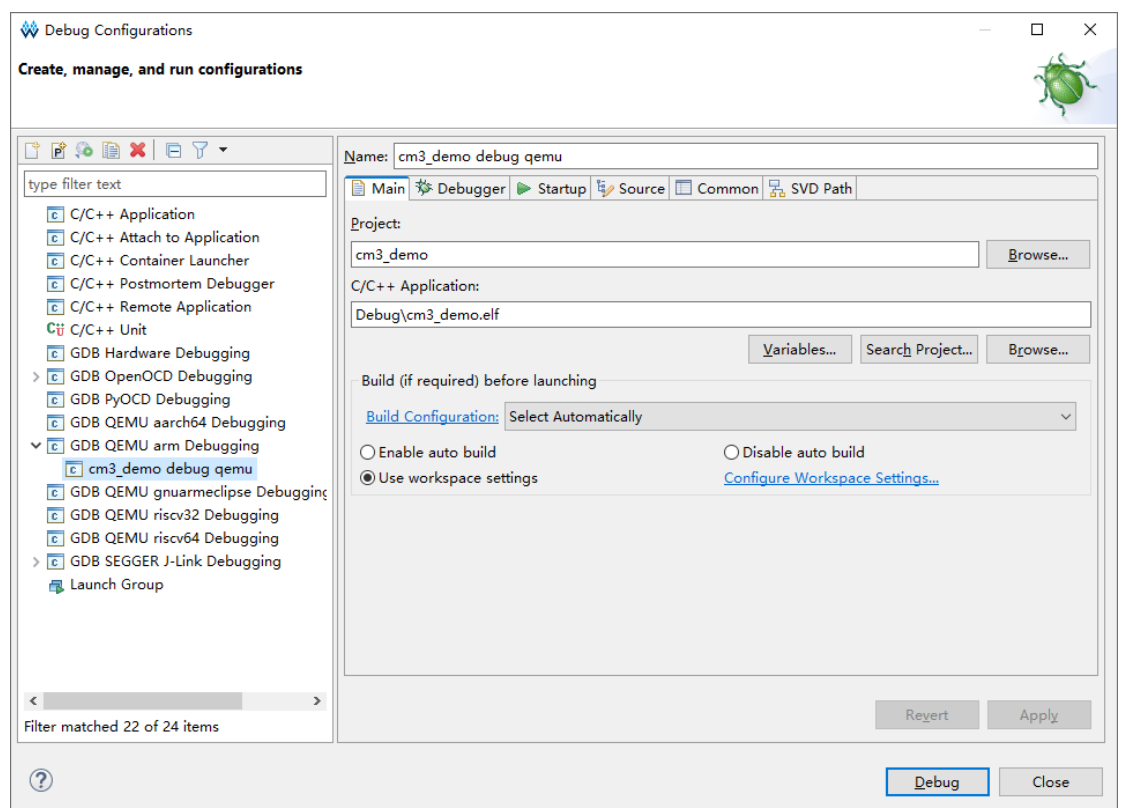
- Select "Debug > Debug Configurations..." in the toolbar ().
- Select "Run > Debug Configurations..." in the menu bar.

ARM MCU and RISC-V MCU use distinct QEMU tools. Configuration methods are detailed below.

11.2.1 ARM MCU QEMU Configuration

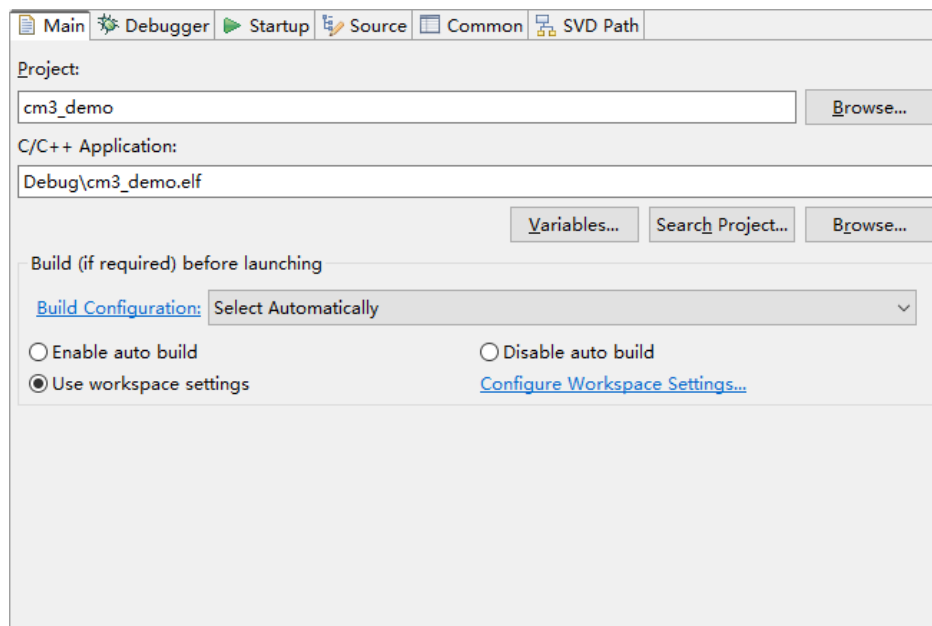
For Gowin_EMPU (GW1NS-4C), you can select "GDB QEMU arm Debugging", as shown in Figure 11-1.

Figure 11-1 GDB QEMU arm Debugging



1. Main View

The QEMU settings in the Main view are shown in Figure 11-2.

Figure 11-2 Main View

2. Debugger View

The QEMU settings in the Debugger view are shown in Figure 11-3 and Table 11-2.

Figure 11-3 Debugger View

The screenshot shows the 'Debugger' tab in an IDE. It contains two main sections: 'QEMU Setup' and 'GDB Client Setup'. In the 'QEMU Setup' section, 'Start QEMU locally' is checked. The 'Executable path' is set to a variable, and the 'Actual executable' is 'E:\GMD\toolchain\gowin\qemu_arm\bin\qemu-system-arm.exe'. The 'QEMU machine name' is 'gowin_cm3', 'QEMU CPU name' is 'cortex-m3', and 'GDB port' is '1234'. 'More options' include '-serial stdio -nodefaults -S'. In the 'GDB Client Setup' section, 'Executable name' is a variable, 'Actual executable' is 'arm-none-eabi-gdb', and 'Commands' include 'set mem inaccessible-by-default off'. The 'Remote Target' section shows 'Host name or IP address' as 'localhost' and 'Port number' as '1234'. 'Force thread list update on suspend' is unchecked.

QEMU Setup

☒ Start QEMU locally

Executable path: Browse... Variables...

☐ Prefer the xpacks/.bin path

Actual executable:
 (to change it use the [global](#) or [workspace](#) preferences pages or the [project](#) properties page)

QEMU machine name:

QEMU CPU name:

GDB port:

More options:

☒ Enable Arm semihosting ☒ Do not open graphic windows

Command line:

☒ Allocate console for QEMU

Delay (seconds):

GDB Client Setup

Executable name: Browse... Variables...

Actual executable:

Other options:

Commands:

Remote Target

Host name or IP address:

Port number:

☐ Force thread list update on suspend

Table 11-2 Debugger View

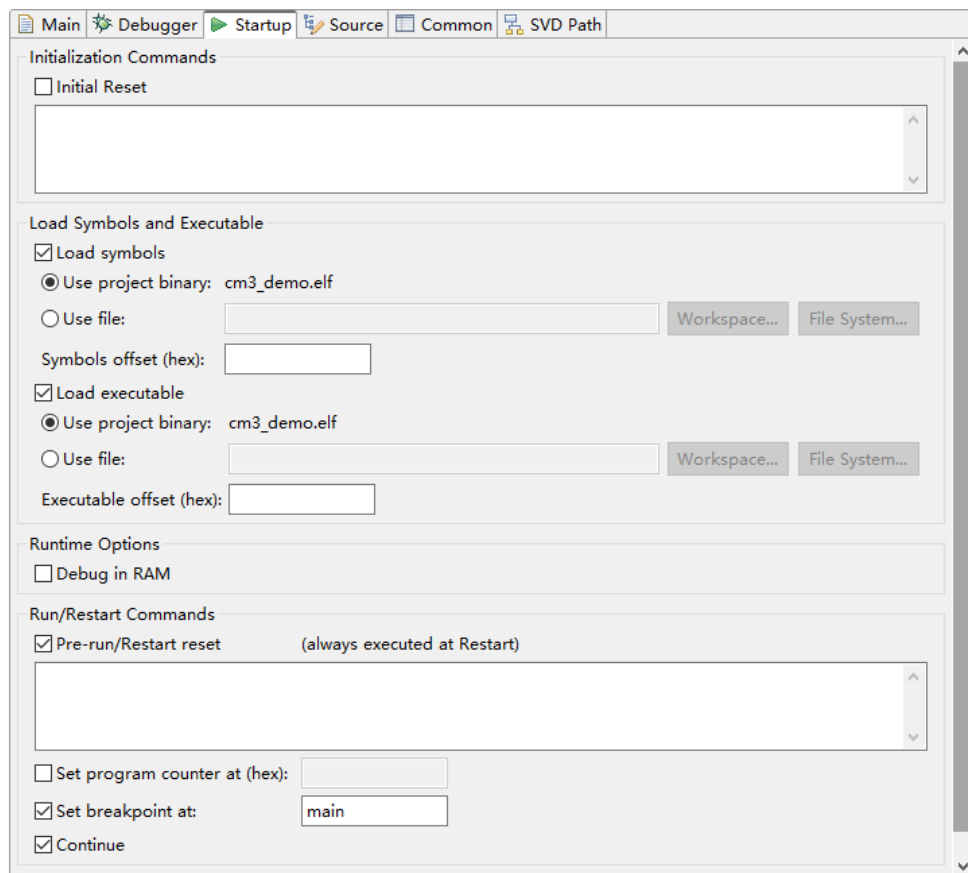
Option	Setting
Start QEMU locally	✓
QEMU machine name	See Table 11-3
QEMU CPU name	See Table 11-3
GDB port	1234
More options	-serial stdio -nodefaults -S
Enable ARM semihosting	✓
Do not open graphic windows	✓
Allocate console for QEMU	✓
Delay (second)	0
Commands	set mem inaccessible-by-default off

Table 11-3 ARM MCU QEMU

ARM MCU	Machine name	CPU name
Gowin_EMPU(GW1NS-4C)	gowin_cm3	cortex-m3

3. Startup View

The QEMU settings in the Startup view are shown in Figure 11-4 and Table 11-4.

Figure 11-4 Startup View**Table 11-4 Startup View**

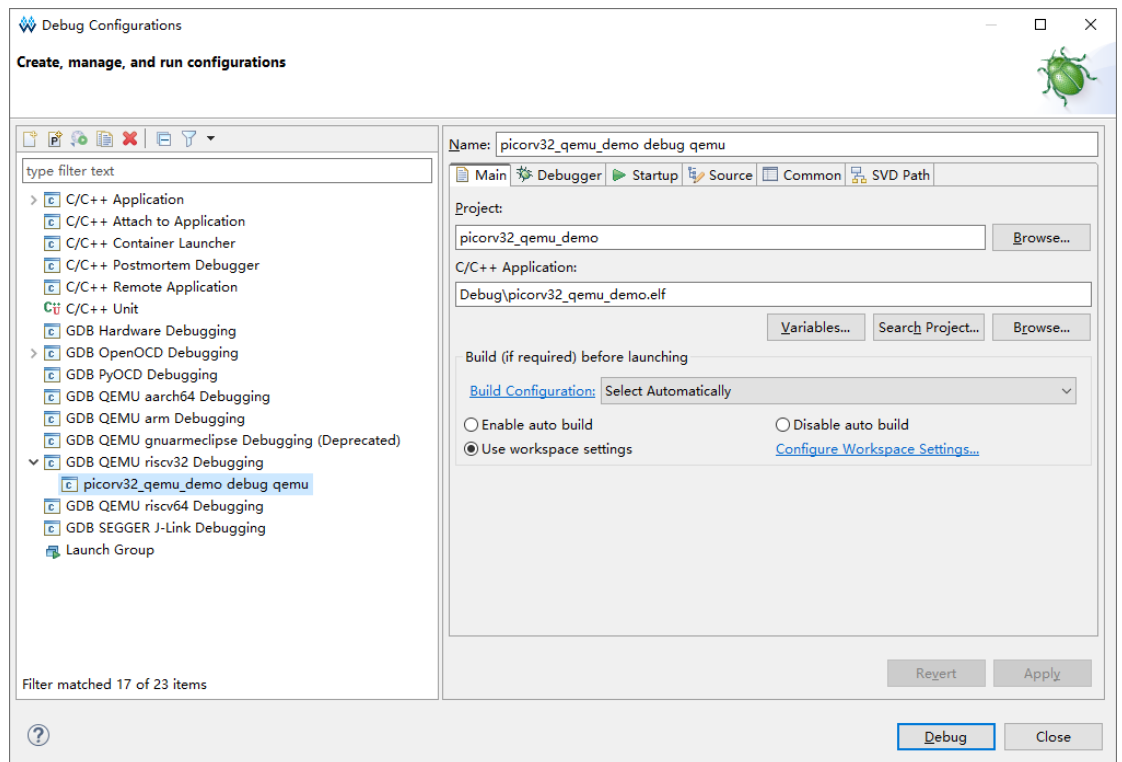
Option	Setting
Initial Reset.	×
Load symbols	✓
Load executable	✓
Debug in RAM	×
Pre-run/Restart reset	✓
Set program counter at (hex)	-

Option	Setting
Set breakpoint at	main
Continue	✓

11.2.2 RISC-V MCU QEMU

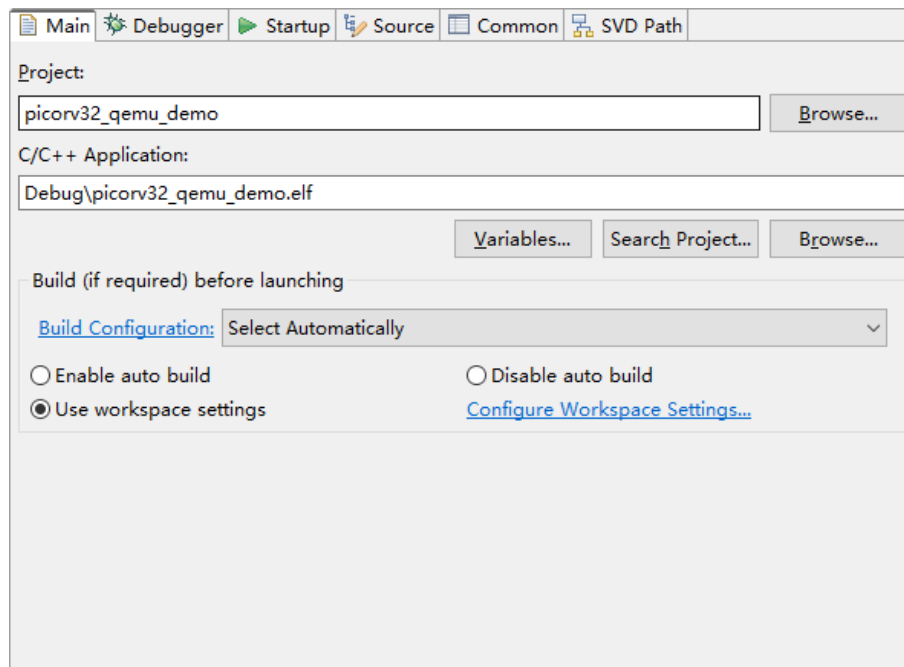
For Gowin_PicoRV32, you can select "GDB QEMU riscv32 Debugging", as shown in Figure 11-5.

Figure 11-5 GDB QEMU riscv32 Debugging



1. Main View

The QEMU settings in the Main view are shown in Figure 11-6.

Figure 11-6 Main View

2. Debugger View

The QEMU settings in the Debugger view are shown in Figure 11-7 and Table 11-5.

Figure 11-7 Debugger View

The screenshot shows the 'Debugger' tab in an IDE. It contains two main sections: 'QEMU Setup' and 'GDB Client Setup'.

QEMU Setup:

- ☒ Start QEMU locally
- Executable path: `${qemu_riscv32_path}/${qemu_riscv32_executable}` (with a 'Browse...' button)
- ☐ Prefer the xpacks/.bin path
- Actual executable: `E:\GMD\toolchain\gowin\qemu_riscv\bin\qemu-system-riscv32.exe` (with a note: "(to change it use the [global](#) or [workspace](#) preferences pages or the [project](#) properties page)")
- QEMU machine name: `gowin_pico`
- QEMU CPU name: `gowin-picorv32`
- GDB port: `1234`
- More options: `-serial stdio -nodefaults -S`
- ☒ Enable Arm semihosting
- ☒ Do not open graphic windows
- Command line: `picorv32_qemu_demo`
- ☒ Allocate console for QEMU
- Delay (seconds): `0`

GDB Client Setup:

- Executable name: `${cross_prefix}gdb${cross_suffix}` (with a 'Browse...' button)
- Actual executable: `riscv-none-elf-gdb`
- Other options: (empty field)
- Commands: `set mem inaccessible-by-default off`

Remote Target:

- ☐ Force thread list update on suspend
- Host name or IP address: `localhost`
- Port number: `1234`

Table 11-5 Debugger View

Option	Setting
Start QEMU locally	✓
QEMU machine name	See Table 11-6
QEMU CPU name	See Table 11-6
GDB port	1234
More options	-serial stdio -nodefaults -S
Enable ARM semihosting	✓
Do not open graphic windows	✓
Allocate console for QEMU	✓
Delay (second)	0

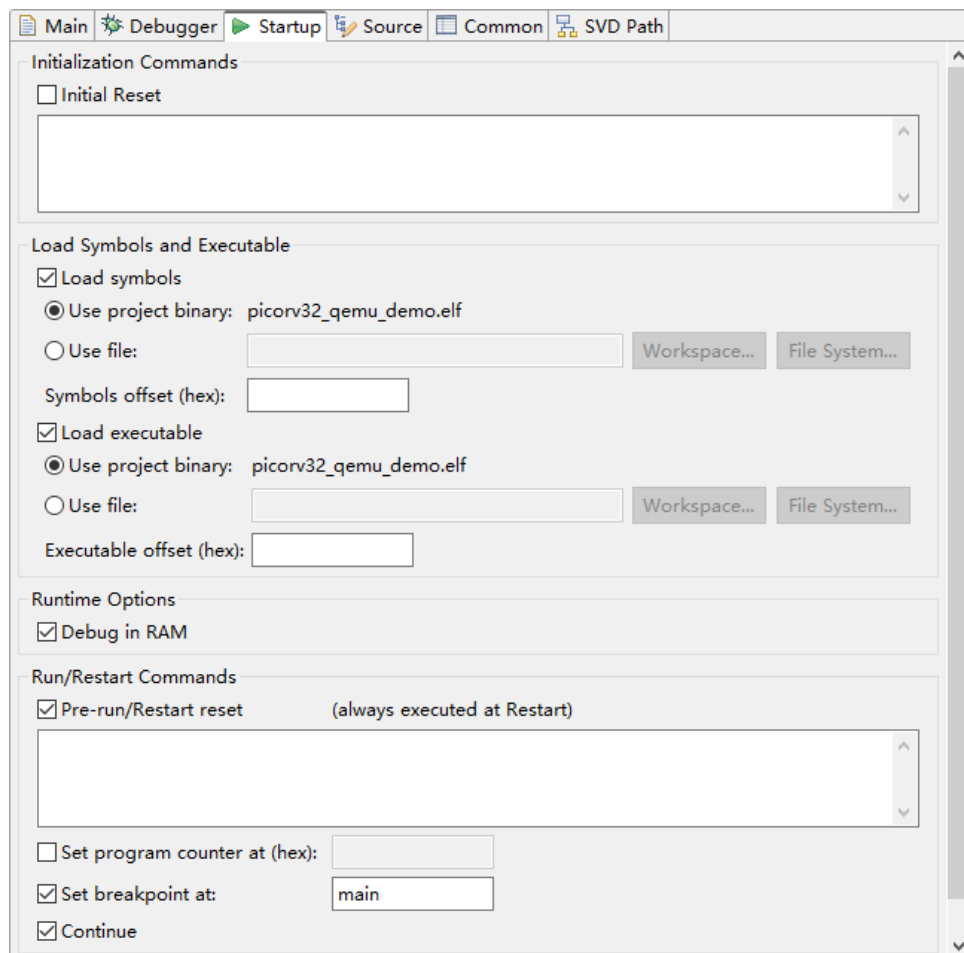
Option	Setting
Commands	set mem inaccessible-by-default off

Table 11-6 RISC-V MCU QEMU

RISC-V MCU	Machine name	CPU name
Gowin_PicoRV32	gowin_pico	gowin-picorv32

3. Startup View

The QEMU settings in the Startup view are shown in Figure 11-8 and Table 11-7.

Figure 11-8 Startup View**Table 11-7 Startup View**

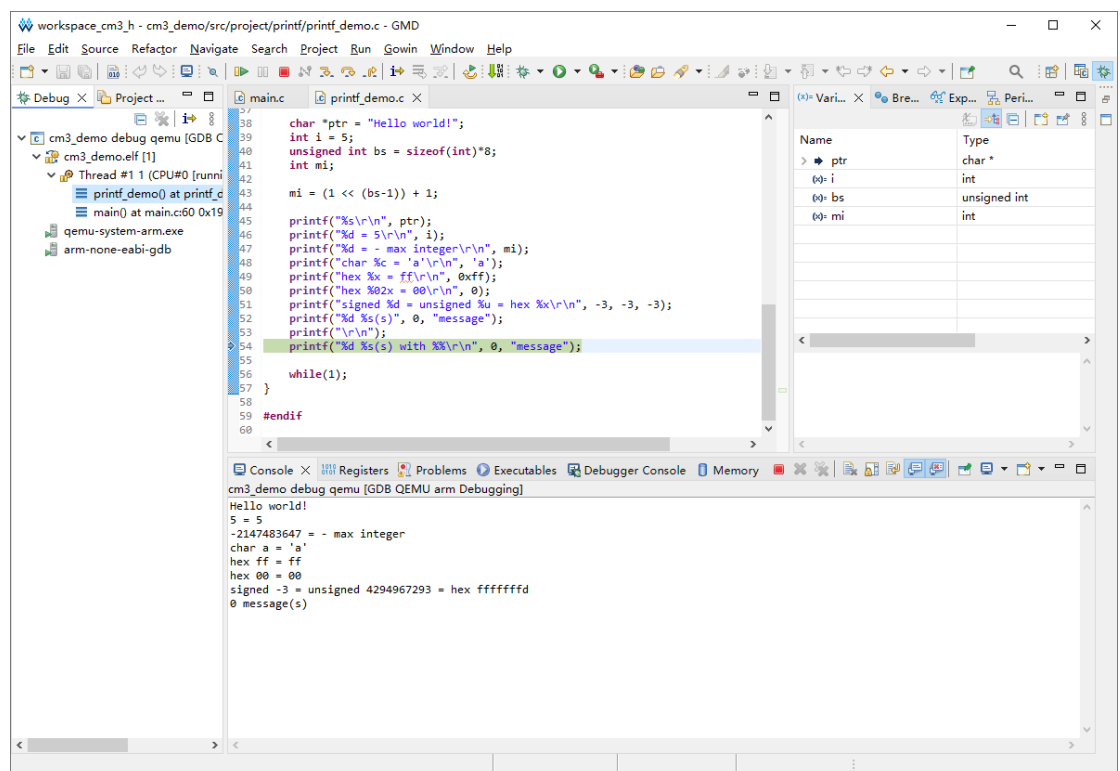
Option	Setting
Initial Reset	×

Option	Setting
Load symbols	✓
Load executable	✓
Debug in RAM	✓
Pre-run/Restart reset	✓
Set program counter at (hex)	-
Set breakpoint at	main
Continue	✓

11.3 Emulation Debugging

Take Gowin_EMPU(GW1NS-4C) as an example. After completing QEMU configuration, click "Debug" to launch the debug view and run the project, as shown in Figure 11-9.

Figure 11-9 Launch QEMU Emulation Debugging



12Code Coverage

12.1 About Code Coverage

GMD software supports Code Coverage function, which utilizes the Gcov tool provided by the GCC compiler to inspect the code coverage of specified source files. This helps developers determine whether their test cases are redundant and whether all branches and paths of the tested code have been covered.

Gcov is a tool for analyzing code coverage in C/C++, used in conjunction with GCC to test statement coverage, function coverage, and branch coverage of C/C++ files.

In GMD software, by building specified source code files with specific -coverage build option, you can either run on a development board and collect the required coverage files (gcda/gcno files) in conjunction with the semihost function, or directly emulate and run on QEMU. With the cooperation of the Gcov tool, the results can be displayed in a graphical manner.

- The .gcno file is generated when the source code is built with the -ftest-coverage option of GCC. It contains information for reconstructing the basic block diagrams and allocation of source code line numbers to blocks.
- The .gcda file is generated when running an object file built with the -fprofile-arcs option of the GCC compiler. Each object file built with this option will generate a corresponding .gcda file that includes arc transition counts, value distribution counts, and summary information.

Generally, using the -coverage option directly can instruct the compiler to generate both of the above files. Note that the .gcda file is only generated during execution.

When using the Code Coverage function, it is recommended to build the code with the -O0 optimization level to ensure the accuracy of the

coverage data.

12.2 Use Code Coverage

For example, emulate and run Gowin_EMPU (GW1NS-4C) on the QEMU platform.

12.2.1 Create a Project

Create a software test project for the Code Coverage function. Compared to a regular software project, this project requires several special code segments:

- Add an additional gcov.c file
- Add and call the gcov_init function to initialize the Gcov function
- Call the gcov_collect function to collect relevant profiling information
- Modify the startup.S assembly file and the flash.ld linker script in conjunction with the Code Coverage

For specific implementation, please refer to the relevant reference design, such as the cm3_gcov_demo reference design for Gowin_EMPU (GW1NS-4C).

12.2.2 Set Relevant Options

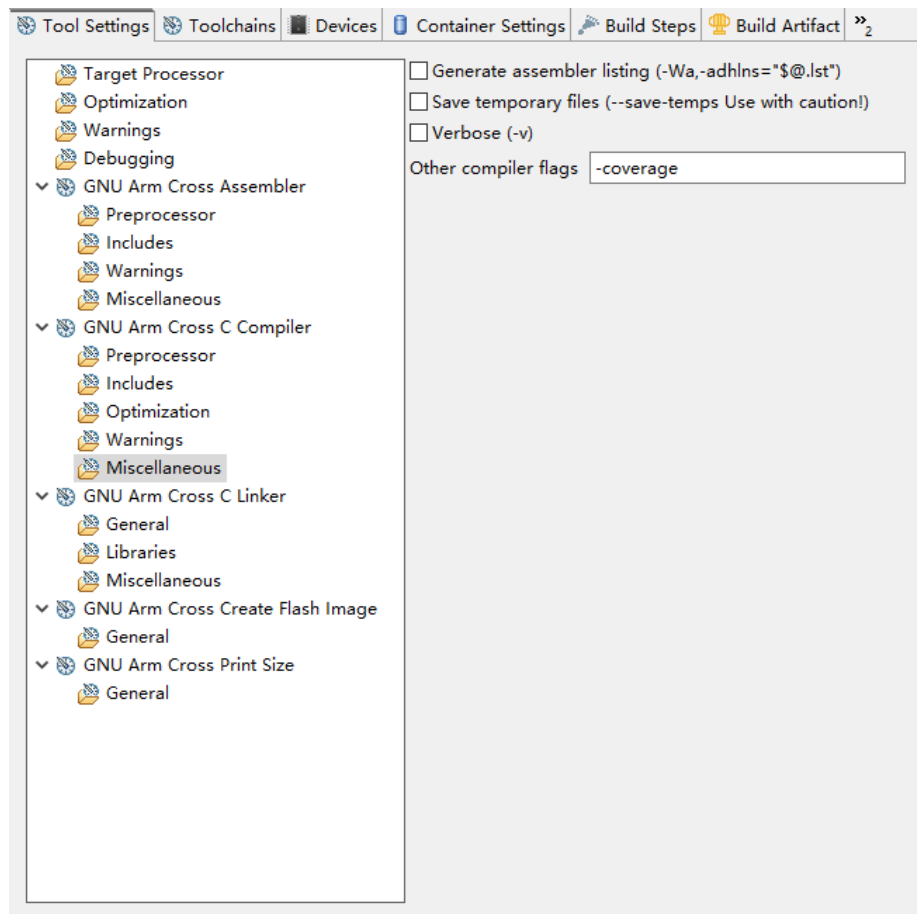
Set Build Options

After creating the project, add the -coverage build option to the target files or folders for code coverage analysis, and then build the project.

- You can apply it to all source files in the project.
- Or apply it to specific source file(s).

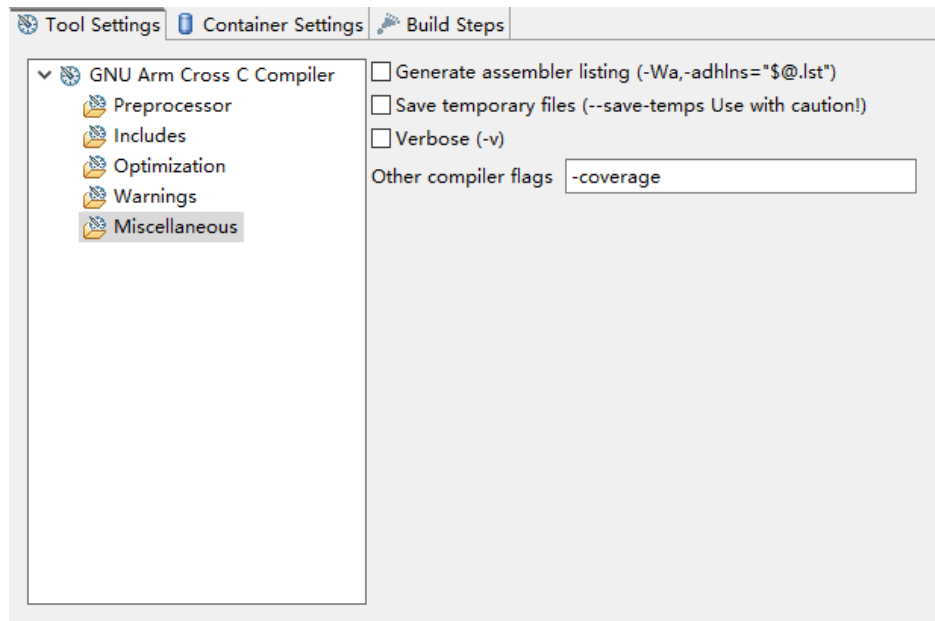
1. Apply it to All Source Files

Select the current project, right-click and choose "Properties > C/C++ Build > Settings > Tool Settings > GNU ARM Cross C Compiler > Miscellaneous > Other compiler flags", then add -coverage, as shown in Figure 12-1.

Figure 12-1 Apply it to All Source Files

2. Apply it to a Specific Source File

Select a specific file, right-click and choose "Properties > C/C++ Build > Settings > GNU ARM Cross C Compiler > Other compiler flags", then add -coverage, as shown in Figure 12-2.

Figure 12-2 Apply it to a Specific Source File

Set Linux Tools Path

When using the Code Coverage analysis, the Parse HexDump Data function needs to be invoked. Due to differences in user environments, it is recommended to first specify the compiler toolchain in the "Linux Tools Path".

On Windows, manually specify the toolchain path and prefix, such as `\${eclipse_home}\toolchain\gowin\gcc_arm\bin` and `arm-none-eabi-`, as shown in Figure 12-3.

On Linux, use the system's default environment path by selecting "Use the System Environment PATH", as shown in Figure 12-4.

Right-click on the current project and choose "Properties > Linux Tools Path", then specify the toolchain and its path.

Figure 12-3 Set Linux Tools Path

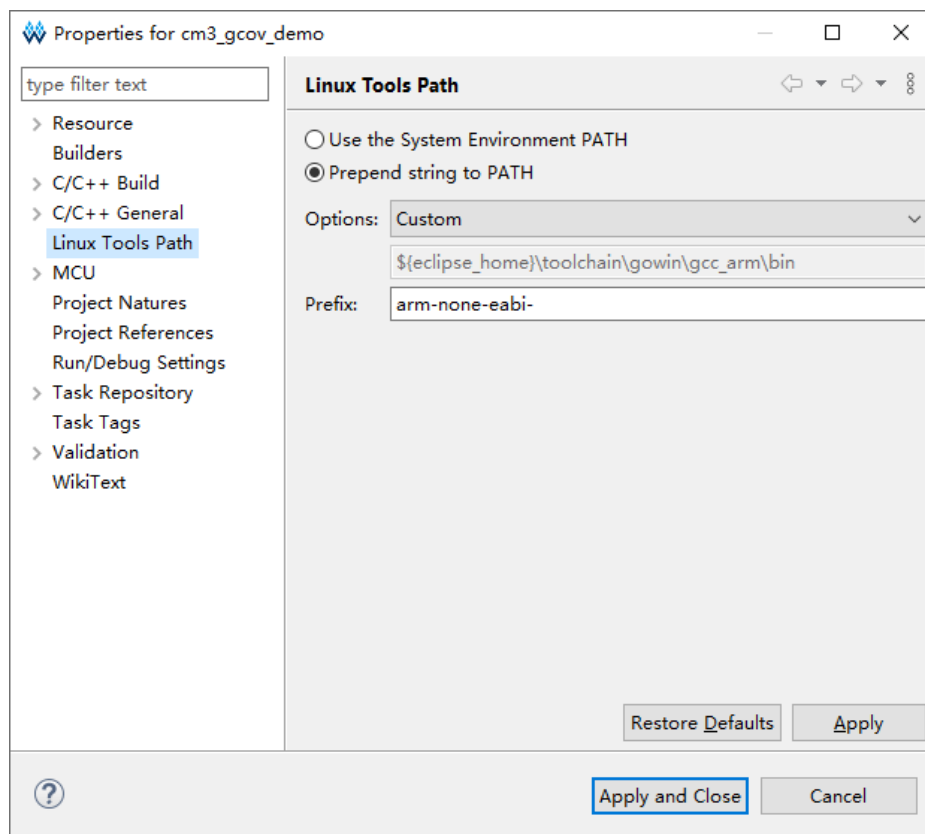
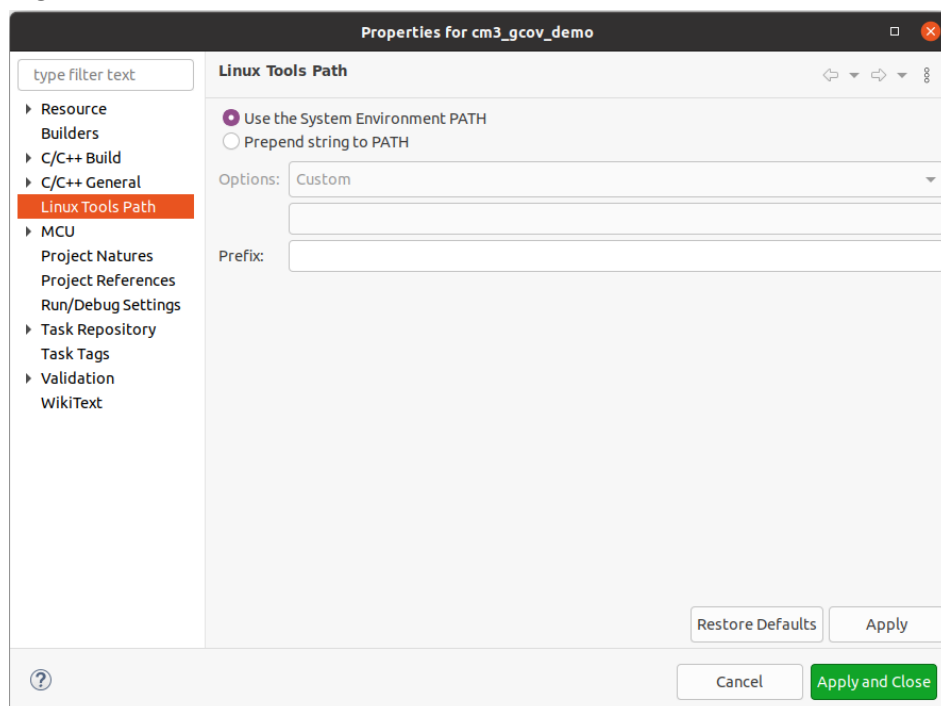


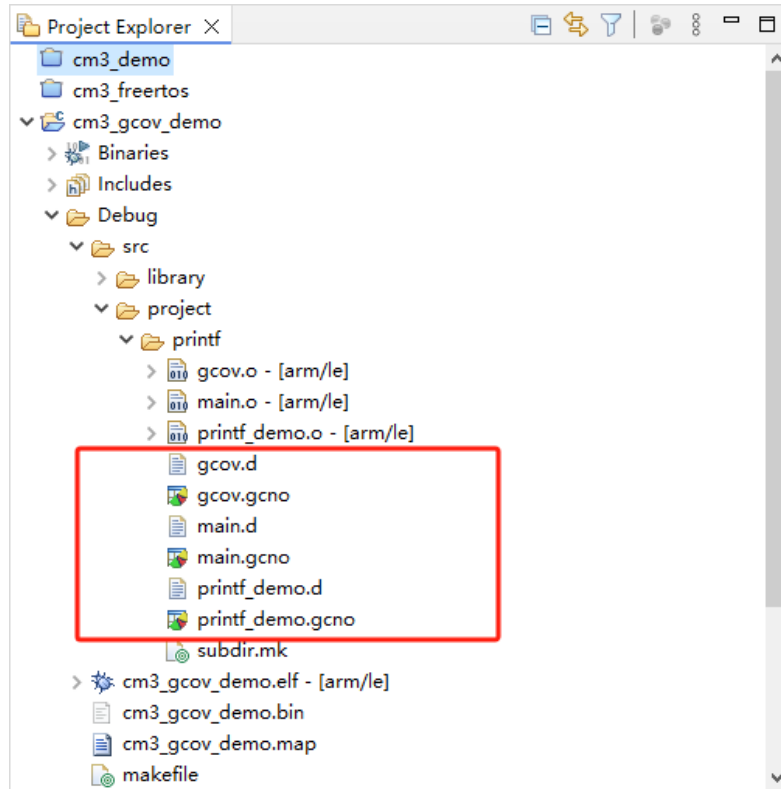
Figure 12-4 Set Linux Tools Path



12.2.3 Build the Project

After successfully building the project, several .gcno files will be generated in the Debug folder, as shown in Figure 12-5.

Figure 12-5 Build the Project



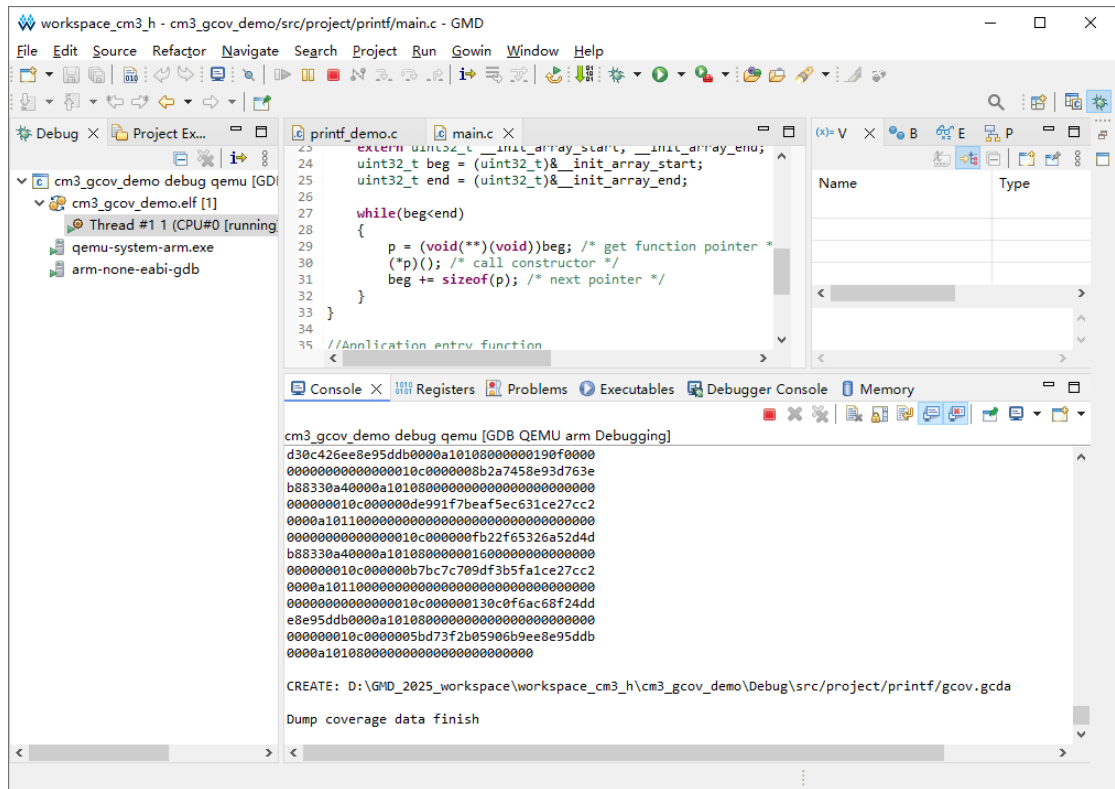
12.2.4 Run the Project and Use Code Coverage

After building the project, you can run or debug it using either the QEMU or a physical development board. For example, when running on QEMU, the Profiling information will be displayed in the GMD console. When debugging on a development board, the Profiling information will be displayed through the serial port.

You can follow the steps below to run the project and use the Code Coverage.

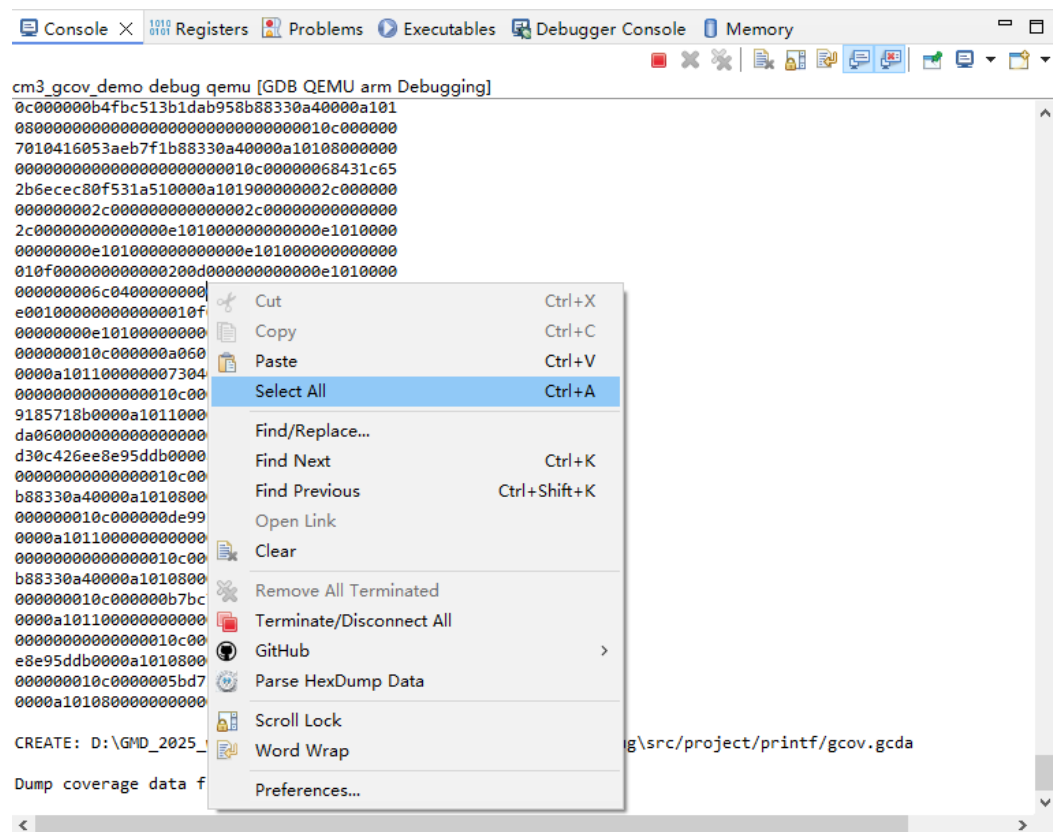
1. Navigate to "Debug As > Debug Configurations... > GDB QEMU arm Debugging" to launch the project in QEMU. The console will display the Profiling information during execution, as shown in Figure 12-6.

Figure 12-6 Run the Project and View Profiling Information



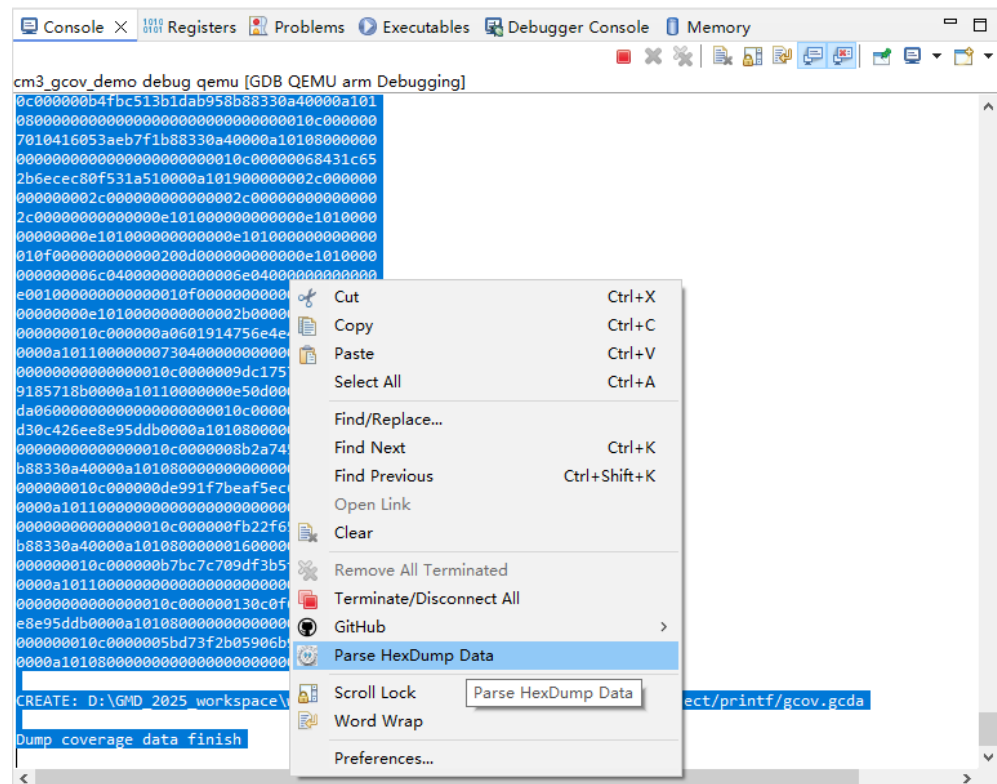
2. The output profiling data requires parsing before GMD can process it correctly. Right-click anywhere in the console and select "Select All" from the dropdown menu to select all output content, as shown in Figure 12-7.

Figure 12-7 Select All



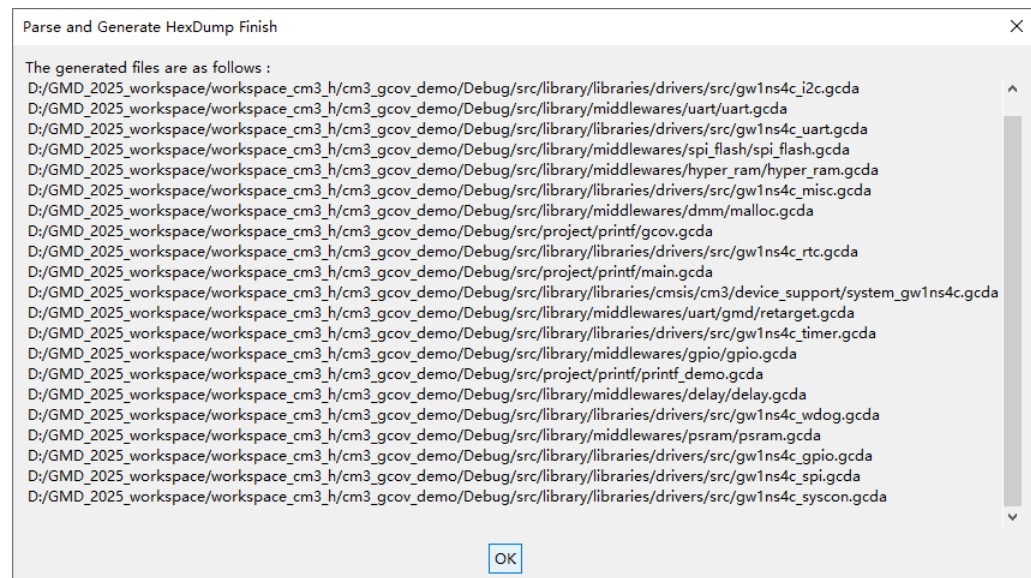
3. Then right-click anywhere in the console again and select "Parse HexDump Data", as shown in Figure 12-8.

Figure 12-8 Parse HexDump Data



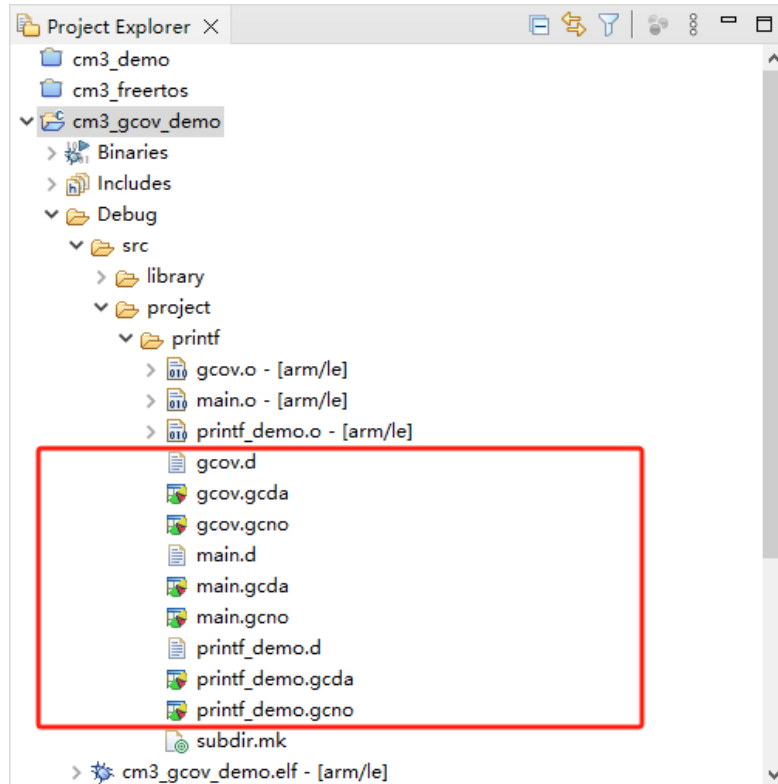
4. GMD software will analyze the output files and save the results in the corresponding files, as shown in Figure 12-9.

Figure 12-9 Parse Profiling Data and Generate Files

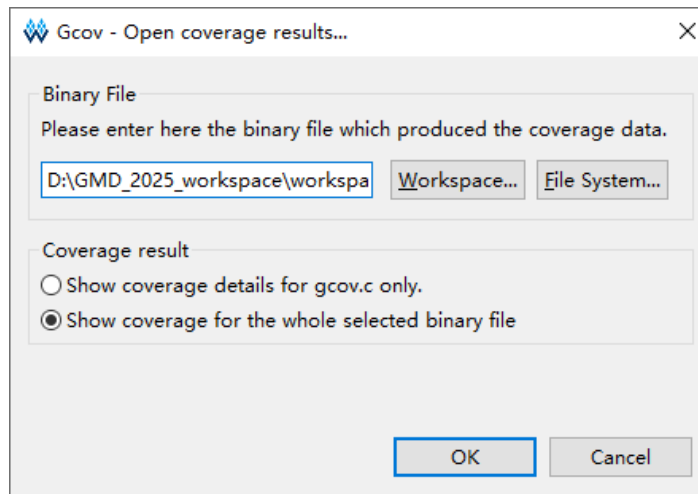


5. Check the project's Debug folder again. You will find that the corresponding .gcda files have been generated, as shown in Figure

12-10.

Figure 12-10 View Generated Files

6. Double-click the .gcda file you want to view. This will launch the Gcov tool, as shown in Figure 12-11.

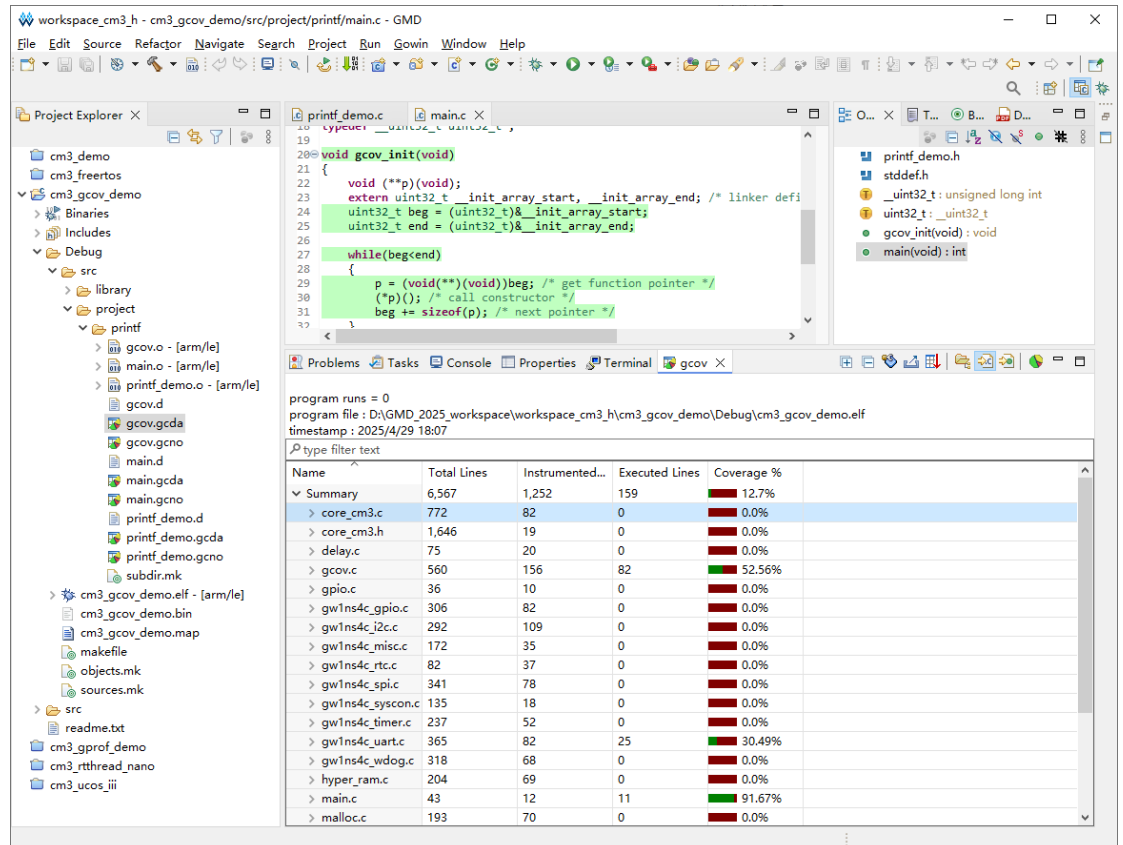
Figure 12-11 Open .gcda File

7. You can then view the analysis results of the application. The results show whether a file or method is executed during program runtime along with quantitative metrics such as code coverage percentages.

By double-clicking on a line in Gcov, GMD software will automatically open the corresponding file and color the code within it. Green indicates that the code is executed during the program runtime, and red indicates that it is not executed, as shown in Figure 12-12.

Developers can refer to the results from Gcov and make corresponding optimizations to the code.

Figure 12-12 Gcov



The available functions in the Gcov tool are shown in Table 12-1.

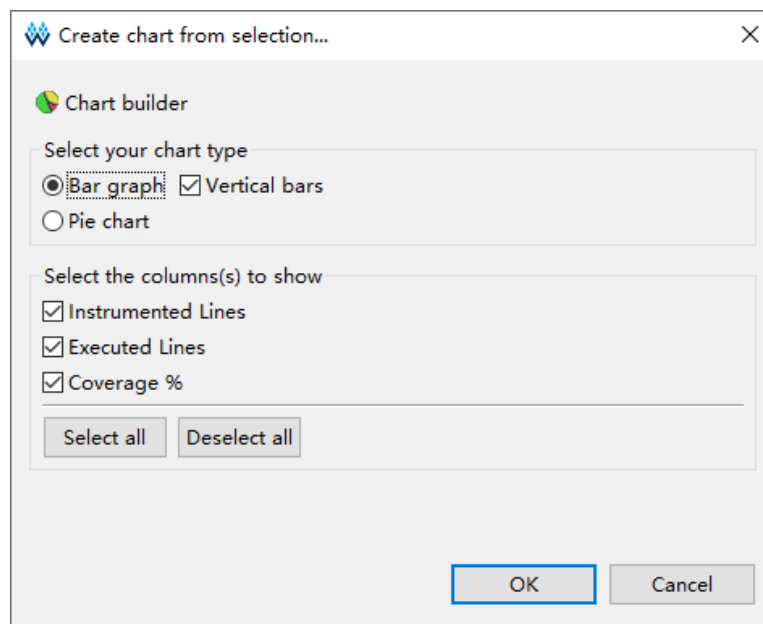
Table 12-1 Gcov Functions

Option	Description
Show/Hide columns ()	Select columns to display
Export to CSV ()	Export analysis data to a .csv file
Sorting ()	Sort analysis data
Sort coverage per file ()	Sort data by source file

Option	Description
Sort coverage per function ()	Sort data by function
Create chat... ()	Create a bar or pie chart using the specified analysis data

8. You can generate bar or pie charts. In the Gcov tool, select the file for which you want to create a chart, and choose "Create chart..." (), as shown in Figure 12-13.

Figure 12-13 Create chat...



A bar chart is as shown in Figure 12-14, and a pie chart is as shown in Figure 12-15.

Figure 12-14 Bar Chart

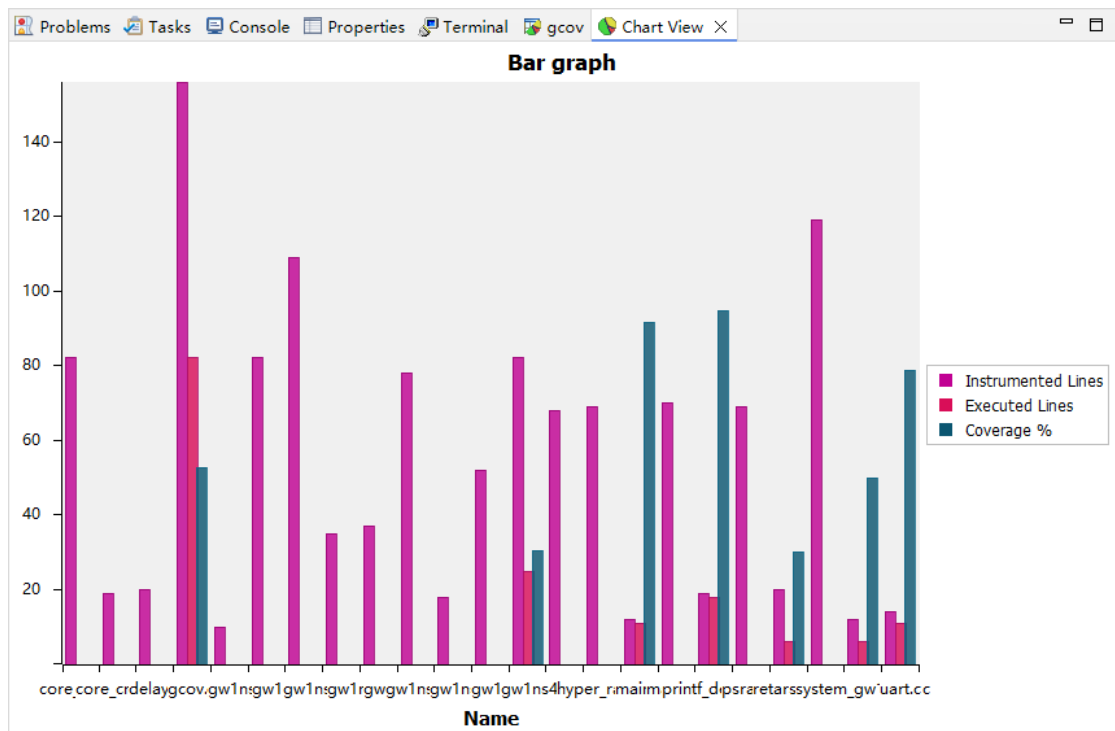
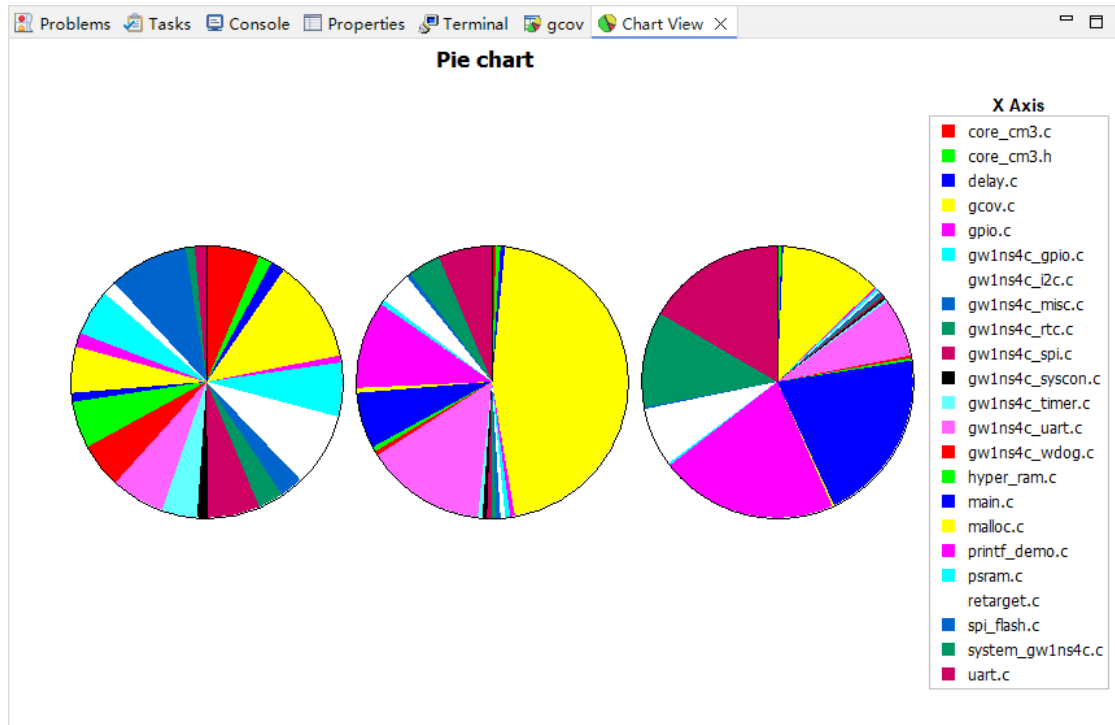


Figure 12-15 Pie Chart



13 Profiling

13.1 About Profiling

GMD software supports Profiling, which uses the Gprof tool from GCC Binutils to analyze the execution time, invocation counts, and call relationships within specified files, as well as the call relationships between functions. Gprof is useful for identifying performance bottlenecks in a program, enabling developers to optimize accordingly. During program execution, Gprof collects runtime data and generates a comprehensive report detailing each function's CPU time percentage and call hierarchy relationships.

As a performance analysis tool, Gprof helps developers understand the runtime behavior of C/C++ programs. It provides information on the invocation counts, execution time, and other details for each function in the program. This information is crucial for optimizing the program and improving its runtime efficiency.

In GMD software, by building specified source code files with specific -pg compilation option, you can either run on a development board and collect the required gmon.out files in conjunction with the semihost function, or directly emulate and run on QEMU. With the cooperation of the Gcov tool, the results can be displayed in a graphical manner.

The generated gmon.out file needs to be used in conjunction with the compiler, and must be run either on an actual hardware board or emulated using QEMU. Additionally, the runtime environment must support file reading in order to perform effective Profiling.

13.2 Use Profiling

For example, emulate and run Gowin_EMPU (GW1NS-4C) on the QEMU platform.

13.2.1 Create a Project

Create a software test project for the Profiling function. Compared to a regular software project, this project requires several special code segments:

- Add the additional files: gmon.c, gmon.h, profile.c, and profile.h
- Call functions related to gmon
- Modify the startup.S assembly file and the flash.ld linker script to support profiling

For detailed implementation, refer to the relevant reference design, such as cm3_gprof_demo for Gowin_EMPU (GW1NS-4C).

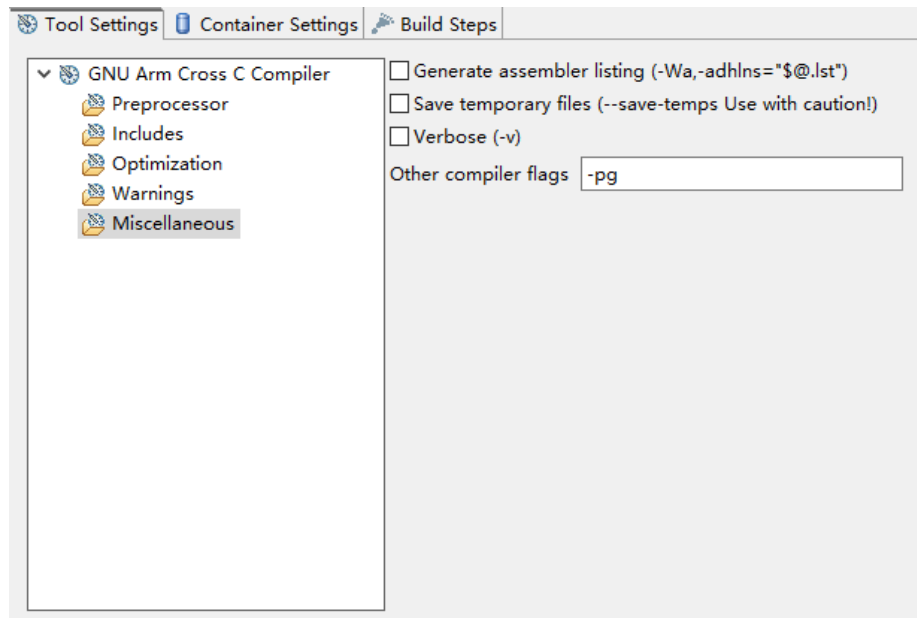
13.2.2 Set Relevant Options

Set Build Options

After creating the project, add the -pg build option to the target files or folders, and then build the project.

Right-click on a file, and select "Properties > C/C++ Build > Settings > GNU ARM Cross C Compiler > Other compiler flags", then add -pg, as shown in Figure 13-1.

Figure 13-1 Set Build Options



Set Linux Tools Path

When using the Profiling, the Parse HexDump Data function needs to

be invoked. Due to differences in user environments, it is recommended to first specify the compiler toolchain in the "Linux Tools Path".

On Windows, manually specify the toolchain path and prefix, such as "\${eclipse_home}\toolchain\gowin\gcc_arm\bin" and "arm-none-eabi-", as shown in Figure 13-2.

On Linux, use the system's default environment path by selecting "Use the System Environment PATH", as shown in Figure 13-3.

Right-click on the current project and choose "Properties > Linux Tools Path", then specify the toolchain and its path.

Figure 13-2 Set Linux Tools Path

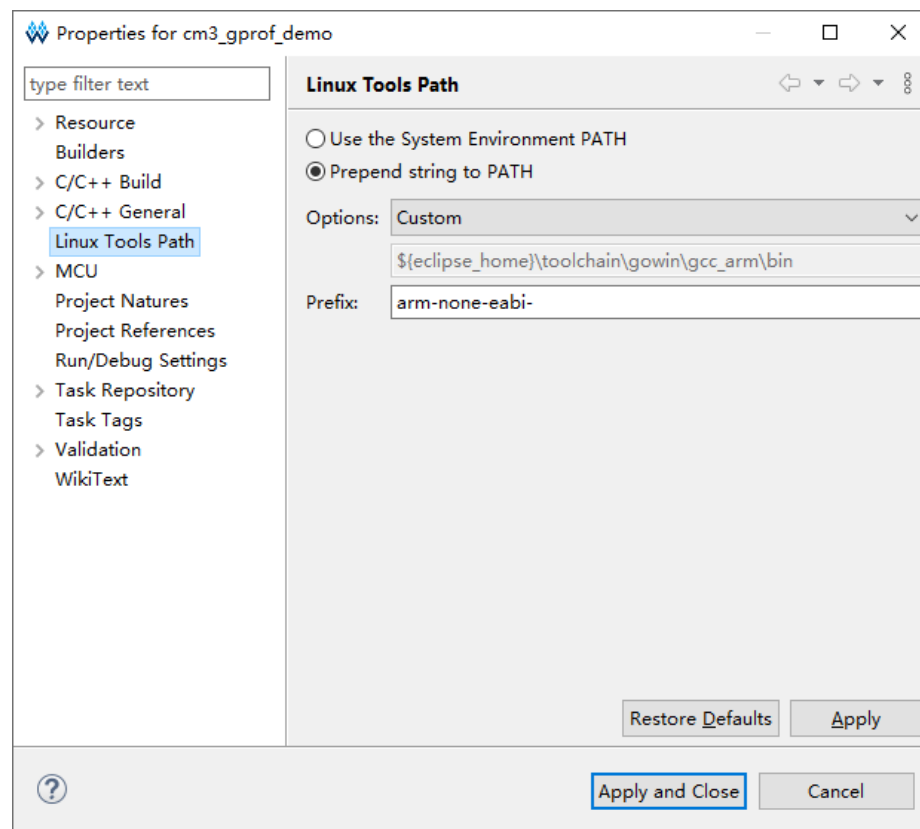
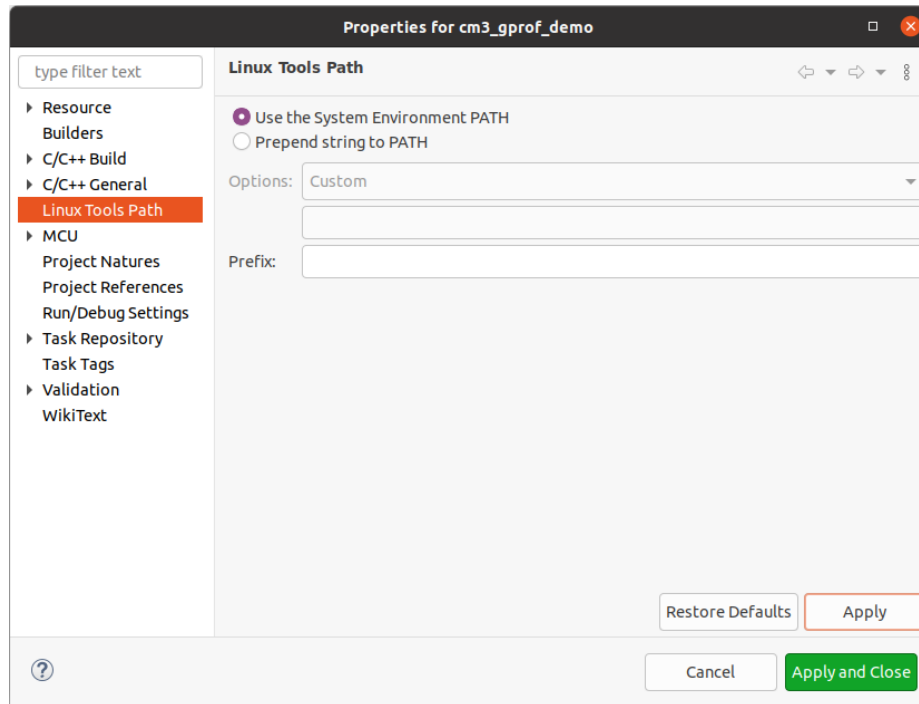


Figure 13-3 Set Linux Tools Path

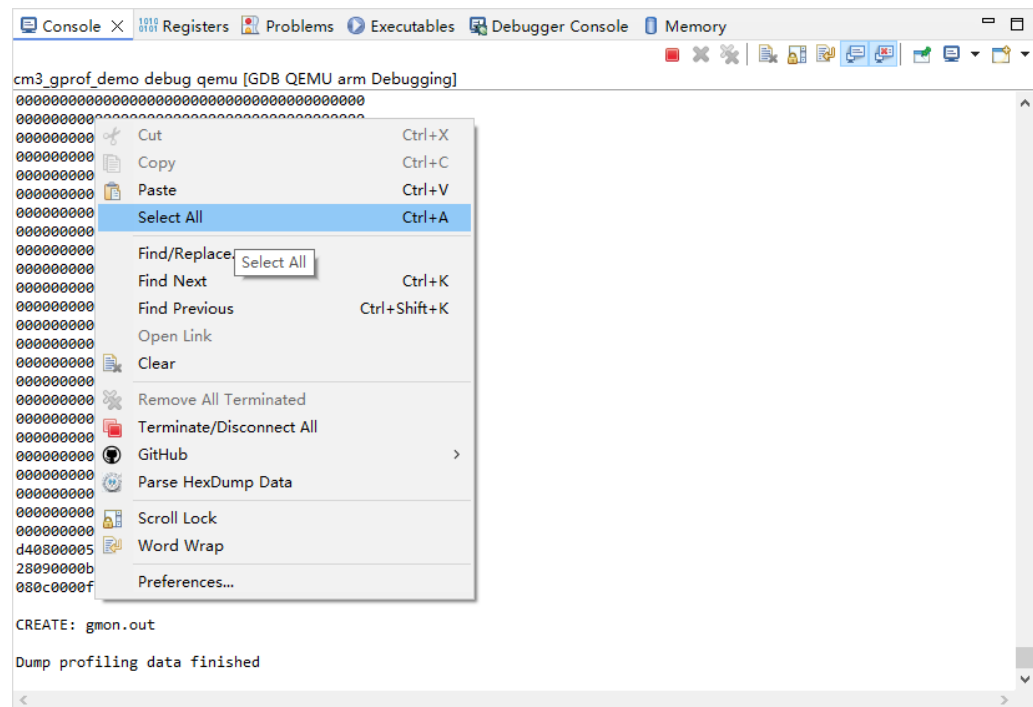
13.2.3 Run the Project and Use Profiling

After building the project, you can run or debug it using either the QEMU or a physical development board. For example, when running on QEMU, the Profiling information will be displayed in the GMD console. When debugging on a development board, the Profiling information will be displayed through the serial port.

You can follow the steps below to run the project and use the Code Coverage.

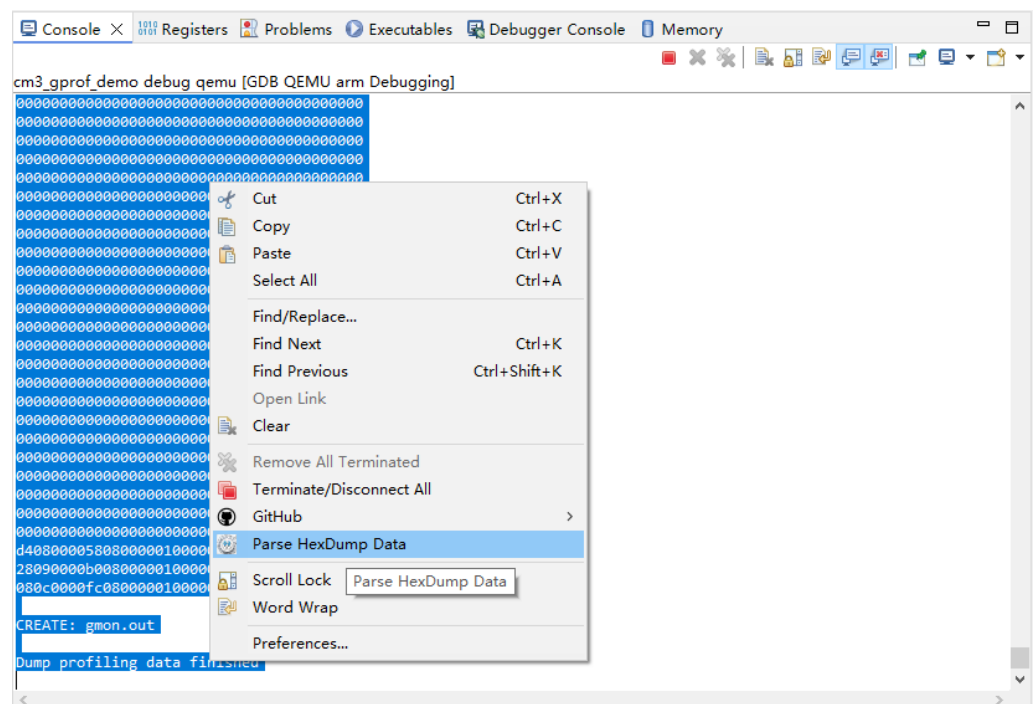
1. Navigate to "Debug As > Debug Configurations... > GDB QEMU arm Debugging" to launch the project in QEMU. The console will display the Profiling information during execution, as shown in Figure 13-4.

Figure 13-5 Select All



3. Then right-click anywhere in the console again and select "Parse HexDump Data", as shown in Figure 13-6.

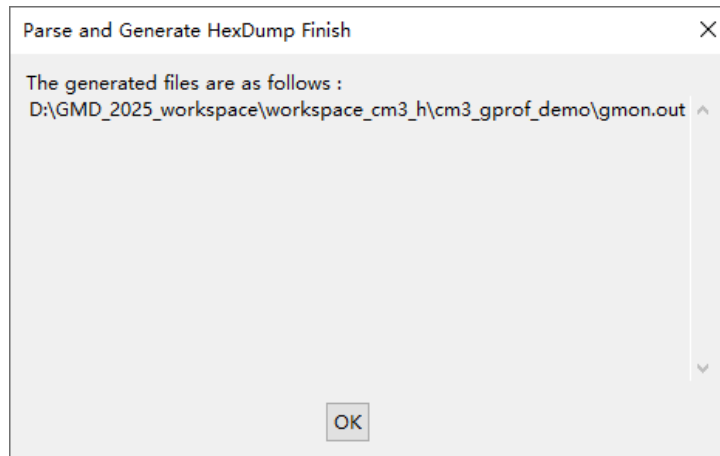
Figure 13-6 Parse HexDump Data



4. GMD software will analyze the output files and save the results in the

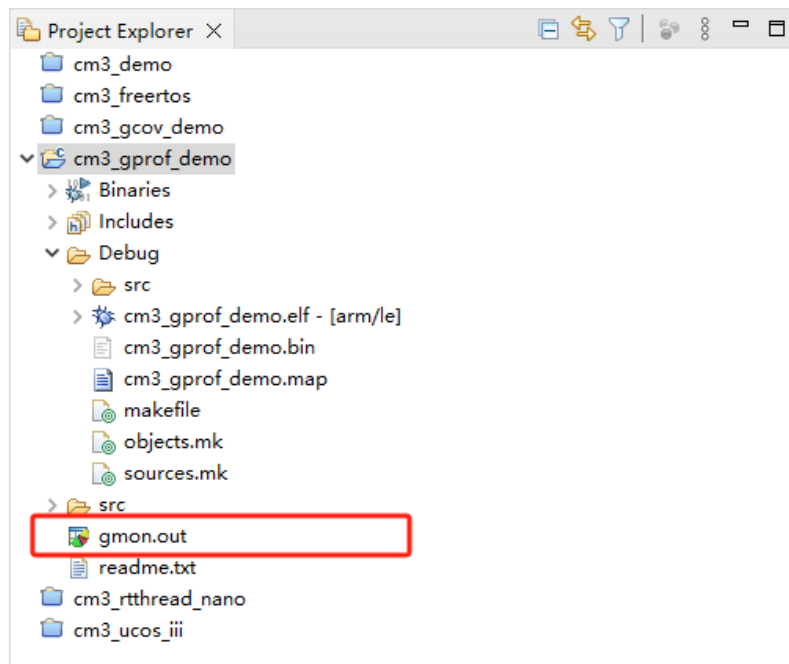
corresponding files, as shown in Figure 13-7.

Figure 13-7 Parse Profiling Data and Generate Files



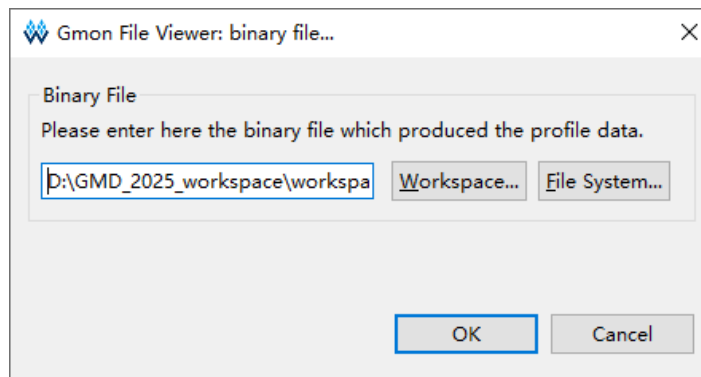
5. Check the project's Debug folder again. You will find that the corresponding gmon.out files have been generated, as shown in Figure 13-8.

Figure 13-8 View Generated gmon.out File



6. Double-click the gmon.out file you want to view. This will launch the Gcov tool, as shown in Figure 13-9.

Figure 13-9 Open gmon.out File



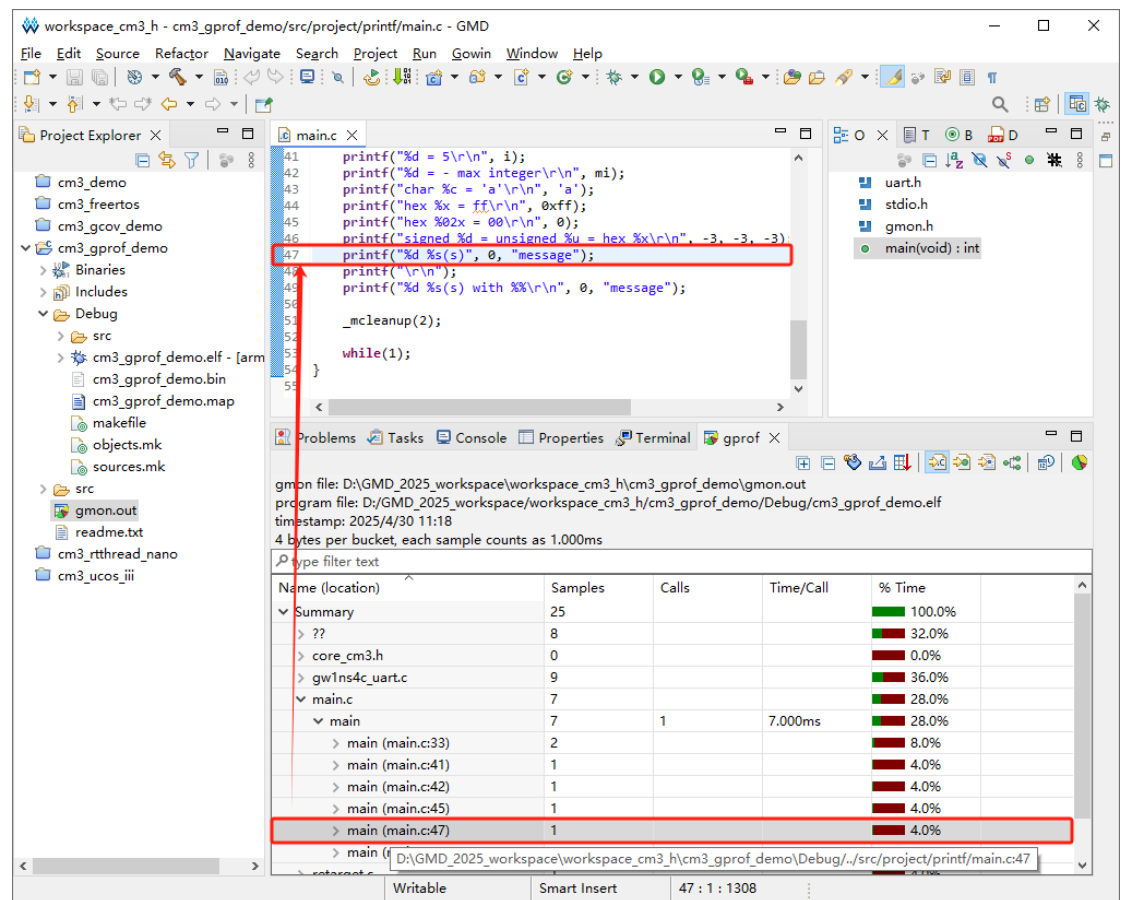
7. Once the Gprof tool is launched, you can view the analysis results of the application, which display the call relationships between files and methods, as shown in Figure 13-10.

Figure 13-10 Gprof

Name (location)	Samples	Calls	Time/Call	% Time
Summary	25			100.0%
> ??	8			32.0%
> core_cm3.h	0			0.0%
> gw1ns4c_uart.c	9			36.0%
main.c	7			28.0%
main	7	1	7.000ms	28.0%
main (main.c:33)	2			8.0%
main (main.c:41)	1			4.0%
main (main.c:42)	1			4.0%
main (main.c:45)	1			4.0%
main (main.c:47)	1			4.0%
main (main.c:49)	1			4.0%
retarget.c	1			4.0%
startup_gw1ns4c.S	0			0.0%

8. Double-click on a line in Gprof, and the GMD software will automatically open the corresponding source file and navigate to the specific line, as shown in Figure 13-11.

Figure 13-11 View Profiling Information



The available functions in the Gprof tool are shown in Table 13-1.

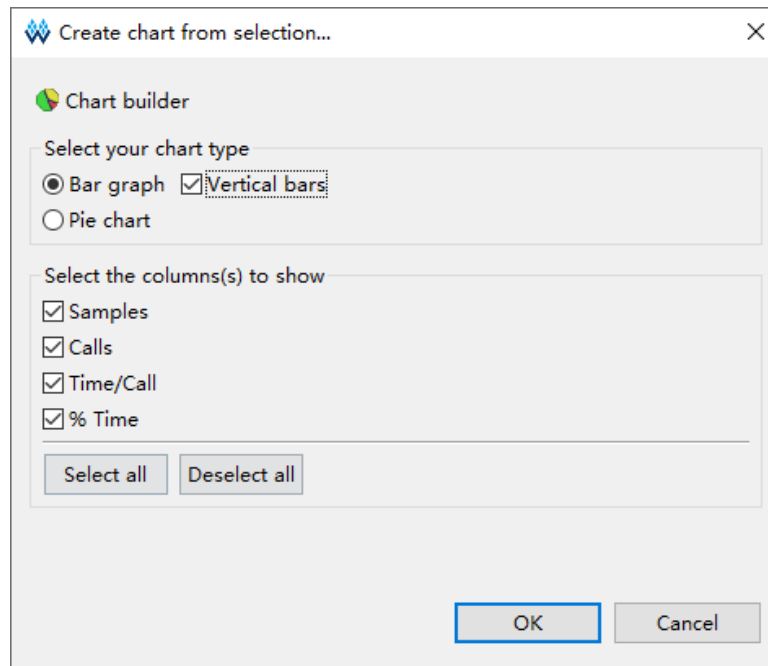
Table 13-1 Gprof Functions

Option	Description
Show/Hide columns ()	Select columns to display
Export to CSV ()	Export analysis data to a .csv file
Sorting ()	Sort analysis data
Sort samples per file ()	Sort data by source file
Sort samples per function ()	Sort data by function
Sort samples per line ()	Sort data by line

Option	Description
Display function call graph ()	View the function call graph
Switch sample/time ()	Switch sample/time view
Create chat... ()	Create a bar or pie chart using the specified analysis data

9. You can generate bar or pie charts. In the Gcof tool, select the file for which you want to create a chart, and choose "Create chart..." (), as shown in Figure 13-12.

Figure 13-12 Create chat...



A bar chart is as shown in Figure 13-13, and a pie chart is as shown in Figure 13-14.

Figure 13-13 Bar Chart

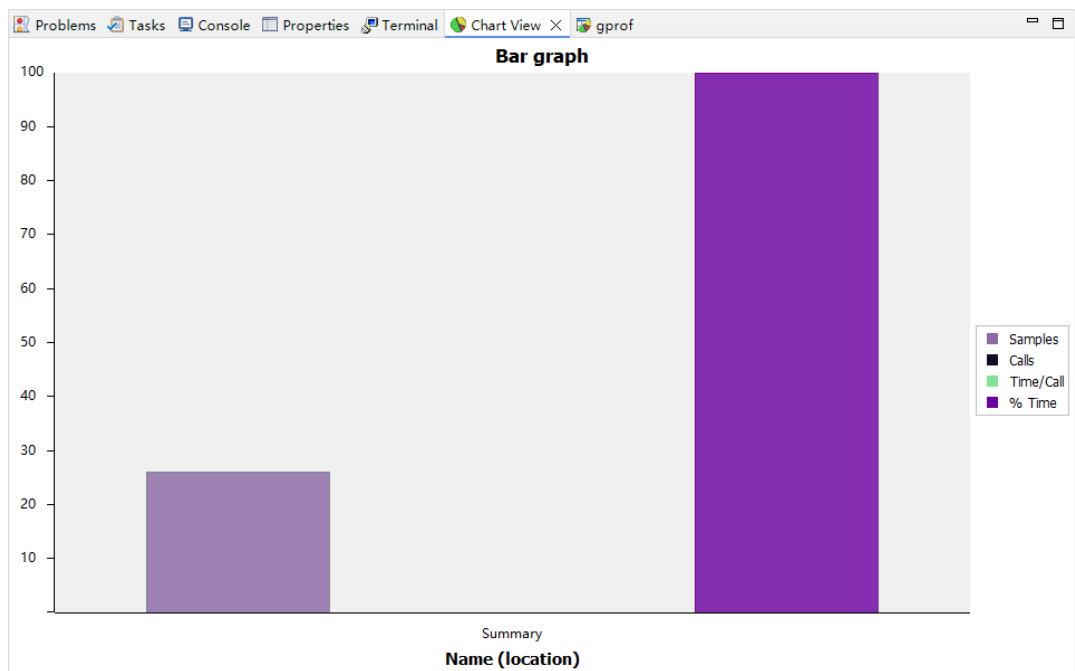


Figure 13-14 Pie Chart

